Confiança em escala.

Engenharia que transforma complexidade em confiança.

Uma proposta de contribuição para o Nubank.
Da experiência no app às decisões de arquitetura, execução e liderança.

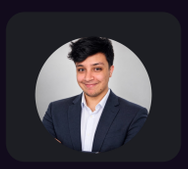


320
PÁGINAS

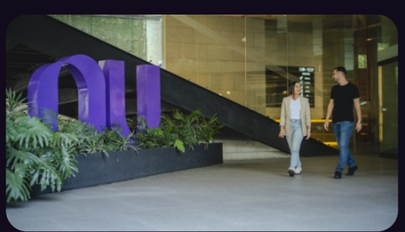
20
CASOS

86
BLOCOS DE CÓDIGO

216
TESTES APROVADOS



Wesley Vieira
Engineering Manager
Staff Software Engineer



ÍNDICE

Uma ideia. Vinte contextos.

Clique em um capítulo ou use o menu no rodapé. Marcadores abrem os subtópicos.

Parte 1 / 2 | Parte 2 / 2

Wesley Vieira · uma proposta para o Nubank 4-17

Experiência, evidência e intenção de contribuição

Fundamentos e tecnologias 18-34

Código, contratos e limites de cada escolha

Arquiteturas que conectam os casos 35-38

Fronteiras, autoridade e recuperação

Caso 01 / Pix falha e a ajuda falha junto 39-51

Idempotência, isolamento de falhas e recuperação de resultado

Caso 02 / Recuperar o acesso quando a câmera frontal não funciona 52-64

Recuperação assistida, validade de decisão e inclusão digital

Caso 03 / Uma notificação deve ajudar a decidir 65-77

Contratos de comunicação, privacidade tardia e utilidade

Caso 04 / Open Finance precisa mostrar o que ainda não está completo 78-90

Proveniência, consentimento e contratos de completude

Caso 05 / Próximo vencimento, sem procurar entre meses 91-103

Agenda por data civil, preferências e revalidação de jobs

Caso 06 / Modo Rua: explicar a permissão sem enfraquecer a proteção 104-116

Política contextual, permissões e fronteiras de confiança

Caso 07 / Um centavo pode quebrar uma integração inteira 117-129

Serialização monetária, identidade e compatibilidade de extratos

Caso 08 / A experiência digital também termina na entrega do cartão 130-142

Da evidência pública ao contrato verificável

Caso 09 / Evoluir a fatura sem perder a tarefa principal 143-155

Da evidência pública ao contrato verificável

Caso 10 / Comprovante no contexto da compra, com significado correto 156-168

Da evidência pública ao contrato verificável

ÍNDICE

Da decisão à execução.

Clique em um capítulo ou use o menu no rodapé. Marcadores abrem os subtópicos.

Parte 1 / 2 | Parte 2 / 2

Caso 11 / Pagamento rápido, sem trocar correção por velocidade 169-181
Da evidência pública ao contrato verificável

Caso 12 / Capacidade cloud como compromisso de produto 182-194
Da evidência pública ao contrato verificável

Caso 13 / Ownership que sobrevive à reorganização 195-207
Da evidência pública ao contrato verificável

Caso 14 / BDC: velocidade com contrato de experiência 208-220
Da evidência pública ao contrato verificável

Caso 15 / Release mobile guiado pela jornada real 221-233
App · Observabilidade

Caso 16 / Precog: chegar ao especialista certo 234-246
IA · Atendimento

Caso 17 / Antifraude que responde dentro da jornada 247-259
Segurança · Machine Learning

Caso 18 / A mesma feature no treino e na decisão 260-272
Dados · Qualidade de ML

Caso 19 / Dark Mode: consistência que inclui legibilidade 273-285
App · Design System

Caso 20 / NuBraille: autonomia de ponta a ponta 286-298
Acessibilidade · Jornada

Liderança, valor e execução 299-313
Como transformar hipóteses em decisões e resultados

Fontes e créditos 314-320
Referências públicas e proveniência visual

ABERTURA

O trabalho que quero construir.



Fotografia publicada pelo Nubank: ambiente de trabalho com a identidade da empresa.

Quero contribuir para um Nubank em que simplicidade na experiência seja sustentada por rigor técnico. Minha proposta começa em jornadas concretas: pagar sem ambiguidade, recuperar acesso com segurança, compreender uma decisão e conseguir ajuda quando a operação sai do caminho esperado. Cada uma exige produto, arquitetura e organização funcionando como um sistema.

Este livro torna minha candidatura examinável. São vinte estudos de sinais públicos, acompanhados de código, contratos, desenhos de arquitetura, testes e decisões de execução. A profundidade está nos limites: onde uma garantia termina, quem pode decidir, como recuperar uma falha e qual evidência autoriza expandir uma mudança.

Wesley Vieira

Candidatura a Engineering Manager ou Staff Software Engineer. Estudo independente; não é uma publicação oficial do Nubank.

Nubank: About Nu

CARTA DE APRESENTAÇÃO

Time Nubank,



Fotografia editorial publicada pelo Nubank: profissionais em uma sala de reunião.

Quero fazer parte do Nubank porque me interessa o trabalho que existe entre uma experiência simples e a engenharia que precisa sustentá-la. Para quem usa o aplicativo, um Pix, uma contestação ou a recuperação de acesso não são apenas fluxos. São dinheiro, tempo e confiança. Quero contribuir para que essas jornadas sejam compreensíveis quando tudo funciona e continuem oferecendo um próximo passo seguro quando algo dá errado.

Hoje, no Itaú, sou responsável pela direção técnica de duas squads, com mais de 15 engenheiros e mais de 20 sistemas e serviços de prevenção a fraudes. Minha trajetória também inclui liderança de engenharia em crédito no Bradesco e gestão de produto técnico no Itaú. Essas experiências me ensinaram a aproximar arquitetura, contexto de negócio e desenvolvimento de pessoas, sem perder de vista quem depende do sistema.

Minha base prática passa por Java/JVM, AWS, mensageria, integração de sistemas e modernização de legado. Também trabalho com dados, segurança e IA aplicada. Mais do que transportar uma lista de tecnologias, quero trazer capacidade de formular problemas, discutir alternativas e construir soluções que caibam no contexto e na capacidade de operação do time.

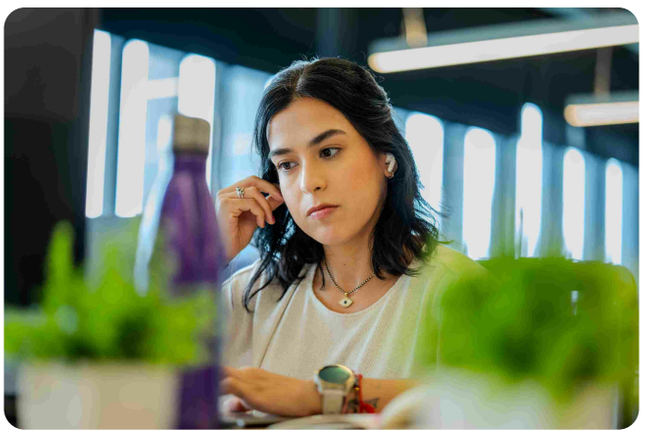
Minha proposta

Combinar profundidade em sistemas financeiros, desenvolvimento de pessoas e responsabilidade pelo resultado da jornada.

Portfólio de Wesley Vieira
Nubank: fotografia editorial

CARTA DE APRESENTAÇÃO

Uma conversa técnica antes da entrevista.



Fotografia editorial publicada pelo Nubank: ambiente de trabalho. A imagem não retrata o candidato.

Preparei este material para tornar essa intenção examinável. Parti de informações públicas sobre o Nubank e de relatos de clientes para organizar vinte estudos. Cada um distingue o que foi observado, o que ainda é hipótese e o que eu proporia validar. Não conheço o backlog interno e espero encontrar controles já existentes, prioridades diferentes e boas razões para mudar minhas primeiras ideias.

Como Engineering Manager, quero ajudar pessoas a crescer, assumir responsabilidade e entregar com clareza. Como Staff Software Engineer, quero melhorar decisões técnicas que atravessam equipes, tornando contratos, riscos e caminhos de evolução mais explícitos. Em ambas as funções, meu compromisso é com colaboração, qualidade e resultado.

Este livro é uma amostra de como eu começaria uma conversa: ouvir a dor, tornar a decisão concreta, testar a hipótese e aprender com o resultado. Gostaria de construir a próxima versão junto ao time do Nubank.

Não parto da premissa de que o Nubank precisa reconstruir o que já faz bem. Parto da disposição de aprender com as capacidades publicadas, formular uma hipótese útil e testá-la com o time. O julgamento relevante não é quantas tecnologias cabem no desenho; é quanta incerteza conseguimos retirar da próxima decisão.

Como ler minha candidatura

As evidências de carreira sustentam meu repertório. Os laboratórios mostram raciocínio executável. Os casos mostram como esse repertório poderia servir ao contexto do Nubank.

Portfólio de Wesley Vieira
Nubank: fotografia editorial

APRESENTAÇÃO

Sou Wesley Vieira.

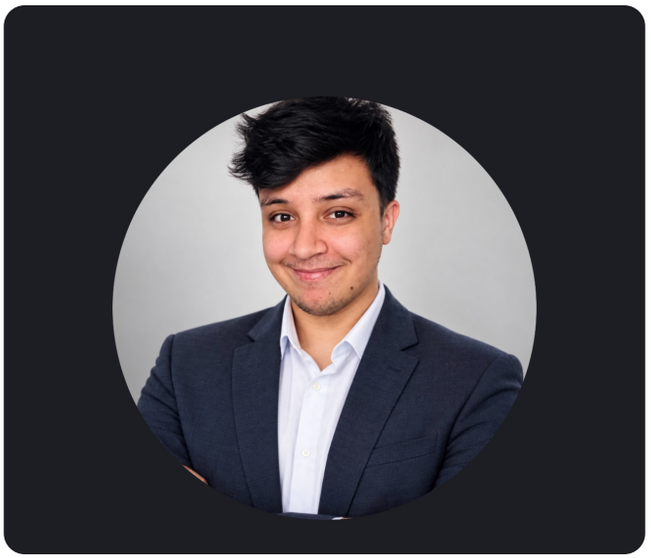


Foto profissional disponibilizada no meu portfólio.

Sou Lead Software Engineer com atuação em serviços financeiros, prevenção a fraudes, crédito e produto técnico. Hoje, no Itaú, conduzo a direção técnica de duas squads, com mais de quinze engenheiros e mais de vinte sistemas e serviços. Minha trajetória inclui liderança de engenharia no Bradesco e gestão de produto técnico no Itaú.

Minha base prática combina Java/JVM, AWS, mensageria, integração de sistemas e modernização de legado. Uso esse repertório para aproximar o comportamento do software da necessidade de negócio: o que precisa continuar correto, que risco aceitamos e como as pessoas que operam a solução vão perceber uma falha.

O que levo para a conversa

Experiência em jornadas financeiras críticas, capacidade de atravessar fronteiras entre times e interesse em formar pessoas que tomem decisões melhores com autonomia.

Histórico e portfólio

EXPERIÊNCIA

Trajetória com escopos claros.

Período / contexto	Responsabilidade	Contribuição relevante
Itaú · desde jun/2026	Lead Software Engineer	Direção técnica de 2 squads e 20+ sistemas antifraude.
Bradesco · fev/2025-mai/2026	Lead Software Engineer	Arquitetura e entrega em 3 squads de crédito e canais.
Itaú · nov/2023-out/2024	Senior Technical Product Manager	Liderança multidisciplinar e prioridades de proteção.

Esses escopos se complementam. Em prevenção a fraudes, decisões técnicas afetam perdas e fricção. Em crédito, políticas, canais e integrações precisam permanecer coerentes ao longo de mudanças. Em produto técnico, o desafio é organizar o trabalho para que equipes diferentes entendam o mesmo resultado e consigam negociar compromissos.

Direção técnica e gestão de pessoas são responsabilidades distintas. Os quinze ou mais engenheiros do escopo atual não são apresentados como subordinados diretos. Na atuação anterior em produto técnico, meu perfil descreve liderança de squads multidisciplinares, descoberta, prioridades e alinhamento de stakeholders, sem uma contagem confirmada de subordinados diretos. Essa precisão é parte do modo como quero liderar: expectativa clara, responsabilidade explícita e evidência que sustenta a afirmação.

Modernização

Meu portfólio descreve a evolução de um ambiente legado de aproximadamente 1.900 arquivos. O número representa escopo; não implica migração concluída nem economia auditada.

Histórico declarado pelo candidato

EVIDÊNCIA DE CARREIRA

Reconhecimento é trabalho coletivo.



Registro do 21º Prêmio Walther Moreira Salles, disponibilizado no portfólio do candidato.

Meu portfólio registra reconhecimento no 21º Prêmio Walther Moreira Salles, em dezembro de 2023, relacionado ao projeto de alertas de fraude por WhatsApp. A fotografia mostra uma conquista de equipe. Para uma candidatura de liderança, o aspecto mais relevante é a capacidade de aproximar tecnologia, comunicação e proteção ao cliente em um problema real.

O repertório que trago dessa experiência não é uma receita para copiar um canal. É a disciplina de perguntar se a mensagem chega a tempo, se a pessoa consegue reconhecer a intenção legítima, se o retorno alimenta a decisão de risco e se existe recuperação quando a automação não resolve. Comunicação também tem contrato, observabilidade e responsabilidade operacional.

Evidência e alcance

A imagem documenta o reconhecimento apresentado no portfólio. Ela não comprova isoladamente os valores financeiros relatados pelo candidato.

Fotos, reconhecimentos e projetos

HISTÓRICO DE IMPACTO

Impacto com atribuição honesta.

Resultado declarado	Natureza	Como discutir na entrevista
R$ 350 milhões	Mitigação de fraude associada a alertas por WhatsApp	Esclarecer janela, baseline, participação do time e cálculo.
R$ 300 milhões	Mitigação de perdas em iniciativa com ML	Separar contrafactual, prevenção e recuperação.
R$ 150 milhões+	Receita associada a iniciativa de crédito	Distinguir receita, margem e causalidade.

Esses valores vêm do meu histórico declarado. Não são resultados do Nubank, não foram auditados neste trabalho e não devem ser somados como se fossem uma única economia. Iniciativas podem compartilhar períodos, populações ou mecanismos de contribuição. Uma boa conversa de contratação precisa explorar o papel exercido, a decisão tomada e a forma de medir.

Eu apresentaria cada projeto com contexto anterior, responsabilidade pessoal, contribuição de outras áreas, alternativas descartadas e evidência disponível. Se a medida veio de negócio, explicaria quem a produziu e o que ela não cobre. Se um resultado dependeu de rollout ou mudança de política, mostraria a sequência para evitar atribuir tudo ao software.

Princípio de trabalho

Um número forte deve vir acompanhado de uma explicação igualmente forte. O rigor que aplico à candidatura é o mesmo que proponho para decisões de produto.

Resultados autodeclarados no portfólio

ENGINEERING MANAGER

Por que me contratar como EM.



Fotografia editorial do Nubank sobre colaboração entre profissionais.

Como Engineering Manager, minha contribuição é criar um ambiente em que pessoas entendam o problema, tenham autonomia proporcional ao risco e consigam concluir trabalho com qualidade. Isso exige mais que acompanhar entregas: é preciso tornar a capacidade visível, reconhecer sobrecarga, desenvolver liderança técnica e resolver dependências antes que se transformem em atraso permanente.

Minha experiência em produto técnico e engenharia permite conectar objetivos de negócio a decisões de implementação sem centralizar tudo em mim. Quero trabalhar com tech leads fortes, estabelecer expectativas por nível, oferecer feedback específico e reservar espaço para aprendizagem. A cadência de gestão precisa ajudar o time a decidir; se apenas produz relatório, precisa mudar.

Responsabilidade	Evidência neste livro	Resultado a perseguir
Pessoas	Delegação, mentoria e capacidade	Mais autonomia com responsabilidade.
Execução	Pilotos, critérios de parada e dependências	Menos trabalho iniciado sem conclusão.
Produto	Métricas de jornada e valor	Melhoria verificável para o cliente.

Engineering Manager: descrição oficial

STAFF SOFTWARE ENGINEER

Por que me contratar como Staff.

Como Staff Software Engineer, quero melhorar decisões que atravessam equipes: a semântica de um evento, a autoridade de um dado, o limite de uma transação, a estratégia de migração ou a forma de observar uma jornada. Meu papel é tornar o problema claro o suficiente para que outras pessoas consigam agir com segurança, sem depender da minha presença em cada revisão.

Minha base em Java/JVM e sistemas financeiros é transferível para o raciocínio funcional demonstrado neste livro. Ao mesmo tempo, não apresento estudo em Clojure ou Dart como experiência profissional já adquirida no Nubank. Mostro código autoral, testes executados e perguntas de integração para que a discussão técnica tenha algo concreto a examinar.

Âmbito	Decisão típica	Artefato que deixo
Domínio	Qual estado é autoritativo?	Invariantes e máquina de estados.
Entre equipes	Quem muda o contrato e como?	ADR, versão e plano de compatibilidade.
Operação	Como detectar e recuperar?	SLO, teste de falha e procedimento.

A vaga oficial de Staff consultada menciona Clojure, Kafka, AWS, Datomic/DynamoDB, Prometheus e Kubernetes. Os capítulos usam esse contexto como ponto de partida. Ferramentas complementares aparecem quando ajudam a testar uma hipótese; não são apresentadas como requisitos universais da empresa.

Staff Software Engineer: descrição oficial

MÉTODO DE PESQUISA

Como reconhecer cada tipo de evidência.

Rótulo	O que significa	O que não significa
OBSERVADO	Fonte pública identificada, com link e contexto.	Incidente reproduzido ou frequência estimada.
IMPLEMENTADO	Laboratório local com código e testes executados.	Integração com sistemas reais do Nubank.
PROPOSTO	Contrato, desenho ou decisão formulada neste livro.	Roadmap aprovado ou arquitetura interna.
MODELADO	Cenário sintético com premissas explícitas.	Receita, economia ou SLA medido na empresa.

Os dez primeiros casos tratam relatos ou avaliações públicas de clientes. Eles são sinais para investigação, não amostra representativa de toda a base. Alguns relatos vêm do mesmo autor; os casos de avaliações da loja compartilham a mesma página de origem. O livro não transforma diversidade de problemas em uma taxa estatística de falhas.

Os dez casos seguintes partem de publicações do Nubank sobre tecnologia, produto e acessibilidade. Neles, o ponto inicial pode ser uma capacidade bem-sucedida. A proposta é discutir como raciocinar sobre extensão, operação e limites. Quando uma fonte relata ganho, o valor permanece atribuído à publicação original e ao contexto descrito por ela.

Data de consulta

23 de outubro de 2026. Páginas públicas, descrições de vagas e avaliações podem mudar depois desta edição.

PRODUTO E EVIDÊNCIA

O app: promessa pública e experiência.



Material oficial de divulgação do aplicativo Nubank no Google Play.

A promessa pública de simplicidade ajuda a definir a unidade de análise: uma pessoa tentando concluir uma tarefa financeira. Neste livro, o app não é apenas uma camada de apresentação. Ele participa de estados, decisões de recuperação, acessibilidade, comunicação de risco e percepção de tempo.

As imagens oficiais mostram como o produto se apresenta; não são capturas de uma conta testada pelo autor. Prints de clientes, quando usados nos casos, permanecem identificados como material publicado por terceiros. Nenhuma tela conceitual é apresentada como evidência de que uma função existe ou falta no aplicativo.

Nubank no Google Play

MAPA DE LEITURA

Vinte perguntas, uma disciplina.

A primeira metade do livro começa onde a experiência fica ambígua: um Pix sem confirmação, uma câmera que impede recuperação, uma notificação incompleta, uma integração que parece omitir saldo. A segunda acompanha capacidades públicas do Nubank: desempenho, plataformas, observabilidade, suporte, modelos e inclusão.

Cada estudo avança do sinal para uma hipótese, desenha fronteiras, implementa um núcleo de decisão e define como verificar seu comportamento. Os laboratórios são deliberadamente menores que um sistema de produção. Isso permite testar uma propriedade sem esconder a discussão em milhares de linhas de infraestrutura. Depois, o capítulo explicita o trabalho adicional necessário para integração, concorrência distribuída e operação.

Percurso	Casos	Foco
Confiança na jornada	01-07	Pagamentos, acesso, contexto e dados.
Operação e plataforma	08-14	Entrega, documentos, desempenho e contratos.
Inteligência e inclusão	15-20	Observabilidade, suporte, ML e acessibilidade.

A pergunta que atravessa o livro

Que evidência permite afirmar que a experiência melhorou sem deslocar o risco para outro cliente, outro time ou outra etapa?

TECNOLOGIAS

Stack documentada, escolhas justificadas.

 clojure	kafka kafka	aws aws	Datomic datomic
 prometheus	 kubernetes	 flutter	 dart

A descrição oficial de Staff consultada destaca Clojure, Kafka, AWS, Datomic/DynamoDB, Prometheus e Kubernetes. Publicações técnicas do Nubank também documentam Flutter e a importância de padronizar escolhas. Esse conjunto orienta a linguagem dos exemplos e a estrutura das propostas.

Java/JVM aparece como ponte com minha experiência profissional. Python é usado para laboratórios reproduzíveis com pouca instalação. PostgreSQL, Redis, OpenTelemetry, Grafana, Terraform e TypeScript aparecem como opções ou instrumentos didáticos, com suas razões e limites explicitados. A presença de um logo indica a tecnologia discutida, não uma afirmação de uso universal pelo Nubank.

Não há aqui ranking estatístico das tecnologias “mais pedidas”. Há aderência a fontes oficiais identificadas. Em uma equipe real, eu confirmaria versões, bibliotecas canônicas, plataformas internas e restrições antes de introduzir componentes. Variedade técnica só produz valor quando responde a uma necessidade concreta.

Stack na vaga oficial
Nubank: The value of canonicity

PLANO DE ADAPTAÇÃO

Do meu repertório à stack do time.

Experiência de partida	Aplicação no contexto	Evidência que eu buscaria
Java/JVM e AWS	Serviços Clojure e operação cloud	Revisão pareada de código e incidentes.
Mensageria e integração	Contratos Kafka e consumidores	Replays controlados e compatibilidade.
Fraude e crédito	Autoridade, risco e dinheiro	Casos limites e revisão de políticas.
Produto e gestão	Experimentos e execução	Baseline, dono e critério de decisão.

Entrar em uma nova organização exige aprender como ela decide. Eu começaria por um fluxo completo: do gesto no app ao serviço, ao evento, à persistência e ao atendimento. A partir daí, identificaria o que é padrão, onde há exceções e quais compromissos foram assumidos para sustentar a operação atual.

O laboratório funcional incluído nesta edição é uma demonstração dessa postura. Ele executa em Clojure e separa transformação de estado de efeitos externos. Isso não substitui experiência com a plataforma interna; reduz a distância entre dizer que aprendo uma linguagem e mostrar um artefato que pode ser revisado. O mesmo critério vale para gestão: explicitar como eu aprenderia com o time antes de prometer reorganização.

Primeira entrega desejada

Uma melhoria pequena, observável e reversível, construída junto a quem conhece o domínio. O objetivo inicial é ganhar contexto e confiança por meio do trabalho.

FUNDAMENTOS · CLOJURE

Clojure: decisões como dados.

O núcleo funcional recebe um estado e devolve outro. Uma transição inválida produz erro explícito; não aciona rede, fila ou banco. Essa separação permite discutir o contrato antes de escolher a infraestrutura. A versão impede que uma intenção antiga altere um estado mais recente dentro do modelo.

CLOJURE / PROPOSTA AUTORAL

```clojure
(def transitions
  {:received #{:pending :rejected}
   :pending #{:settled :rejected}
   :settled #{}
   :rejected #{}})

(defn advance [db key expected-version next-status]
  (let [{:keys [status version] :as current} (get db key)]
    (when-not current
      (throw (ex-info "unknown operation" {:reason :not-found})))
    (when-not (= expected-version version)
      (throw (ex-info "stale writer" {:reason :version-conflict})))
    (when-not (contains? (get transitions status #{}) next-status)
      (throw (ex-info "illegal transition"
                      {:reason :illegal-transition})))
    (assoc db key (assoc current :status next-status
                         :version (inc version)))))
```

Trecho do laboratório decision_core.clj; execução local verificada. Modelo em memória, sem rede.

O trecho integra decision_core.clj, executado em Clojure 1.12.0 sobre Java 17. São 11 testes e 22 assertions no arquivo completo. A imutabilidade do valor não oferece persistência durável nem exclusão distribuída: ao integrar um banco, a comparação de versão e a escrita precisam compartilhar a mesma fronteira atômica.

Documentação oficial da tecnologia

FUNDAMENTOS · CLOJURE

Clojure: repetição não cria outra intenção.

A chave de idempotência identifica uma intenção; o fingerprint identifica seu conteúdo relevante. Repetir a mesma chave com o mesmo valor mantém o estado. Reutilizá-la com outro valor é conflito, não uma nova compra. No exemplo sintético, o escopo da chave é o banco em memória; produção precisaria incluir cliente, operação e política de retenção.

CLOJURE / PROPOSTA AUTORAL

```clojure
(defn register [db {:keys [key amount currency] :as cmd}]
  (when-not (valid-command? cmd)
    (throw (ex-info "invalid command"
                    {:reason :invalid-command})))
  (let [fingerprint [amount currency]
        existing (get db key)]
    (cond
      (nil? existing)
      (assoc db key {:fingerprint fingerprint
                     :status :received :version 0})
      (= fingerprint (:fingerprint existing)) db
      :else
      (throw (ex-info "key reused with different payload"
                      {:reason :key-conflict})))))

;; Em um único processo: atualização atômica do modelo.
;; Nunca adicionar efeitos externos à função de swap!.
(def state (atom {}))
(swap! state register
       {:key "synthetic-001" :amount 1250 :currency :BRL})
```

Trecho do laboratório local; valid-command? está definido no arquivo completo. Teste concorrente executado.

O teste concorrente dispara oitenta registros iguais contra um atom e verifica uma única entrada. Isso exercita atomicidade local, não demonstra processamento exatamente uma vez entre serviços. O caminho de produção ainda precisa de identidade autenticada, persistência, controle de expiração, reconciliação e resposta consistente quando a conexão cai depois do commit.

Documentação oficial da tecnologia

FUNDAMENTOS · DATOMIC

Datomic: identidade e histórico de decisão.

O modelo separa identidade da operação e evolução de estado. A restrição de identidade evita representar a mesma operação como duas entidades. A comparação e troca da versão acompanha a mudança de status na transação. Esse desenho é uma proposta didática; não descreve o schema usado pelo Nubank.

CLOJURE / PROPOSTA AUTORAL

```clojure
;; Schema proposto para um exercício de domínio.
[{:db/ident :operation/id
  :db/valueType :db.type/uuid
  :db/cardinality :db.cardinality/one
  :db/unique :db.unique/identity}
 {:db/ident :operation/status
  :db/valueType :db.type/keyword
  :db/cardinality :db.cardinality/one}
 {:db/ident :operation/version
  :db/valueType :db.type/long
  :db/cardinality :db.cardinality/one}]

;; Uma transação propõe a mudança de versão e status.
;; entity-id e expected-version são obtidos pelo adapter.
[[:db/cas entity-id :operation/version
  expected-version (inc expected-version)]
 [:db/add entity-id :operation/status :operation/pending]]
```

Exemplo autoral de referência; não executado contra infraestrutura real.

Uma versão correta não garante, sozinha, uma transição de negócio válida. O adapter deve validar a máquina de estados e assegurar que a condição observada pertence à transação autorizada. Histórico ajuda a explicar a decisão, mas exige política de retenção e cuidado para não persistir dados sensíveis desnecessários. A semântica exata depende da edição e da API adotadas pelo time.

Documentação oficial da tecnologia

FUNDAMENTOS · KAFKA

Kafka: contrato antes de tópico.

O envelope proposto carrega identidade do evento, identidade do agregado e versão de domínio. Cada campo resolve uma pergunta diferente: já processei esta entrega, a qual operação pertence e onde se encaixa na sequência? O consumidor não deve confundir offset de transporte com versão de negócio.

JSON / PROPOSTA AUTORAL

```json
{
  "event_id": "synthetic-event-001",
  "event_type": "operation.status_changed",
  "schema_version": 1,
  "aggregate_id": "synthetic-operation-001",
  "aggregate_version": 2,
  "occurred_at": "2026-10-01T12:00:00Z",
  "producer": "proposed-operation-service",
  "correlation_id": "synthetic-correlation-001",
  "payload": {
    "previous_status": "pending",
    "status": "settled",
    "currency": "BRL",
    "amount_minor": 1250
  }
}
```

Contrato JSON proposto, com identificadores sintéticos; não representa um evento interno do Nubank.

A chave de partição precisa refletir a unidade para a qual a ordem importa. Reprocessar uma mensagem exige idempotência no efeito e não apenas no produtor. Uma transação Kafka não transforma automaticamente uma chamada externa ou escrita em outro banco em uma única transação global. O teste relevante interrompe o consumidor entre persistência e confirmação e observa a recuperação.

Documentação oficial da tecnologia

FUNDAMENTOS · AWS

AWS / DynamoDB: arbitrar a primeira escrita.

A reserva condicional torna a criação da chave uma disputa resolvida pelo armazenamento. O serviço não deve fazer “consultar, depois inserir” como duas operações independentes e assumir exclusividade. O escopo evita que uma chave de um cliente se confunda com outra intenção.

PYTHON / PROPOSTA AUTORAL

```python
# Exemplo de adapter com boto3; cliente e tabela são externos.
# Nomes e valores são sintéticos.
def reserve(table, scope, request_key, fingerprint):
    pk = f"IDEMPOTENCY#{scope}#{request_key}"
    return table.put_item(
        Item={
            "pk": pk,
            "fingerprint": fingerprint,
            "status": "reserved",
            "version": 0,
        },
        ConditionExpression="attribute_not_exists(pk)",
    )

# Uma ConditionalCheckFailedException exige consultar
# o registro: replay compatível ou conflito de payload.
# Não converter todo erro em 'já processado'.
# A expiração da chave precisa respeitar a janela de replay.
```

Exemplo autoral de referência; não executado contra infraestrutura real.

Reserva não equivale a liquidação. Se o processo cair depois de reservar e antes de enviar a operação, é preciso distinguir trabalho pendente de trabalho concluído. O estado reservado precisa de recuperação com ownership e limite temporal. A política de expiração não pode permitir que um retry tardio execute novamente um efeito financeiro já realizado.

Documentação oficial da tecnologia

FUNDAMENTOS · KUBERNETES

Kubernetes: disponibilidade durante mudança.

O exemplo reserva capacidade para uma réplica adicional enquanto remove gradualmente a versão anterior. A readiness deve indicar se a instância pode receber trabalho; não pode oscilar a cada falha de uma dependência compartilhada e retirar todo o serviço ao mesmo tempo. Os números são premissas didáticas.

YAML / PROPOSTA AUTORAL

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: proposed-journey-reader
spec:
  replicas: 3
  selector:
    matchLabels: {app: journey-reader}
  strategy:
    rollingUpdate: {maxUnavailable: 0, maxSurge: 1}
  template:
    metadata:
      labels: {app: journey-reader}
    spec:
      containers:
        - name: reader
          image: example.invalid/reader:replace-with-reviewed-digest
          ports: [{containerPort: 8080}]
          readinessProbe:
            httpGet: {path: /ready, port: 8080}
            periodSeconds: 5
          resources:
            requests: {cpu: "250m", memory: "256Mi"}
            limits: {memory: "512Mi"}
```

Manifesto de referência, deliberadamente sem imagem executável. Não aplicado a cluster.

Antes de aplicar, seriam necessários imagem revisada, identidade de serviço, políticas de rede, graceful shutdown, limites de concorrência e distribuição entre zonas. Três réplicas no mesmo domínio de falha não oferecem três falhas independentes. O rollout precisa observar conclusão da jornada e saturação, além de contar pods prontos.

Documentação oficial da tecnologia

FUNDAMENTOS · FLUTTER

Flutter: o estado dirige a interface.

O componente recebe estado de domínio, não deduz conclusão pela ausência de spinner. Uma mensagem pendente precisa continuar verdadeira quando a rede oscila. O desenho evita oferecer “tentar novamente” enquanto o resultado financeiro permanece desconhecido; a ação seguinte depende do contrato de reconciliação.

DART / PROPOSTA AUTORAL

```dart
// Fragmento didático: imports e modelos do app são omitidos.
Widget statusPanel(JourneyState state) {
  final String label;
  switch (state) {
    case JourneyState.pending:
      label = 'Estamos confirmando a operação';
      break;
    case JourneyState.settled:
      label = 'Operação concluída';
      break;
    case JourneyState.rejected:
      label = 'Operação não concluída';
      break;
  }
  return Semantics(
    liveRegion: true,
    child: Padding(
      padding: const EdgeInsets.all(16),
      child: Text(label),
    ),
  );
}
```

Fragmento Flutter/Dart de referência; não compilado nesta edição.

O exemplo precisa de testes de widget, leitor de tela, escala de texto e navegação real. Uma região viva pode informar mudança sem deslocar foco, mas anúncios excessivos também prejudicam a experiência. O texto final deve ser trabalhado com Design e conteúdo, incluindo tempo esperado, ação segura e canal de ajuda quando o prazo deixa de ser normal.

Documentação oficial da tecnologia

FUNDAMENTOS · DART

Dart: estados que não se contradizem.

Uma coleção de booleanos pode representar combinações impossíveis: carregando e concluído, sem recibo e disponível. Um conjunto fechado de estados torna essas possibilidades mais visíveis. Aqui, recibo confirmado exige identificador próprio; indisponibilidade da consulta não é apresentada como falha da operação.

DART / PROPOSTA AUTORAL

```dart
sealed class ReceiptState {
  const ReceiptState();
}
final class AwaitingConfirmation extends ReceiptState {
  const AwaitingConfirmation(this.operationId);
  final String operationId;
}
final class Confirmed extends ReceiptState {
  const Confirmed(this.operationId, this.receiptId);
  final String operationId;
  final String receiptId;
}
final class Unavailable extends ReceiptState {
  const Unavailable(this.operationId);
  final String operationId;
}

String actionLabel(ReceiptState state) => switch (state) {
  AwaitingConfirmation() => 'Consultar andamento',
  Confirmed() => 'Abrir comprovante',
  Unavailable() => 'Tentar consultar novamente',
};
```

Exemplo didático em Dart 3; não compilado nesta edição.

A tipagem ajuda na implementação, mas o servidor continua sendo a autoridade do estado financeiro. O cliente precisa lidar com versões antigas do contrato e respostas desconhecidas. O fallback deve preservar dados já confirmados sem inventar um novo estado de sucesso. A proposta deve ser validada com a versão de Dart e os padrões de modelagem usados pela equipe.

Documentação oficial da tecnologia

FUNDAMENTOS · PROMETHEUS

Prometheus: medir uma promessa.

O numerador e o denominador precisam observar a mesma população. Se jornadas abandonadas desaparecem da instrumentação, uma métrica de latência pode parecer melhor enquanto a experiência piora. O contrato de medição deve definir início, conclusão, timeout, cancelamento legítimo e eventos ausentes.

PROMQL / PROPOSTA AUTORAL

```promql
# Proporção de jornadas com resultado confirmado em até 2s.
# Métricas e rótulos propostos; janela ilustrativa.
sum(rate(journey_confirmation_seconds_bucket{
  journey="payment", le="2"
}[30m]))
/
sum(rate(journey_confirmation_seconds_count{
  journey="payment"
}[30m]))

# Percentil de cauda: analisar por versão do app/serviço.
histogram_quantile(
  0.99,
  sum by (le, release) (
    rate(journey_confirmation_seconds_bucket[5m])
  )
)
```

Consultas PromQL propostas; não executadas em servidor Prometheus.

Percentis agregados exigem cuidado com buckets e distribuição. Um percentil não revela sozinho volume ou taxa de sucesso; deve ser lido ao lado de contagem, erros e segmentação. Os rótulos precisam de cardinalidade limitada. Identificadores de conta, operação ou cliente pertencem a trilhas controladas de diagnóstico, não a labels de uma série temporal.

Documentação oficial da tecnologia

FUNDAMENTOS · OPENTELEMETRY

OpenTelemetry: seguir a causalidade.

Uma trilha deve explicar a relação entre operações, não apenas enumerar chamadas. A confirmação de um pagamento pode atravessar etapas síncronas e assíncronas, retries e reconciliação. A amostragem precisa preservar exemplos de falha e latência longa suficientes para investigação.

PYTHON / PROPOSTA AUTORAL

```python
# Fragmento de instrumentação; tracer fornecido pelo runtime.
def confirm(command, repository, tracer):
    with tracer.start_as_current_span("journey.confirm") as span:
        span.set_attribute("journey.kind", "payment")
        span.set_attribute("contract.version", 1)
        try:
            result = repository.confirm(command)
            span.set_attribute("journey.outcome", result.kind)
            return result
        except TimeoutError:
            span.set_attribute("journey.outcome", "unknown")
            raise

# Não registrar CPF, token, saldo ou payload integral.
# Propagar contexto entre produtores e consumidores.
# Para processamento assíncrono, modelar links/parentesco
# de acordo com a semântica real do trabalho.
```

Exemplo autoral de referência; não executado contra infraestrutura real.

O trace não substitui um registro financeiro durável. Pode ser amostrado, sofrer atraso ou faltar justamente durante um incidente. O diagnóstico deve conseguir relacionar uma operação a uma trilha por acesso controlado, mantendo dados pessoais fora de atributos comuns. O objetivo é reduzir tempo de investigação sem criar um novo repositório indiscriminado de informações sensíveis.

Documentação oficial da tecnologia

FUNDAMENTOS · JAVA

Java / JVM: orçamentos explícitos.

Um timeout por dependência pode somar mais tempo do que a jornada comporta. Um orçamento compartilhado força cada etapa a considerar o tempo já consumido. O relógio monotônico é adequado para duração dentro do processo; timestamps distribuídos exigem outro contrato e tolerância a diferenças de relógio.

JAVA / PROPOSTA AUTORAL

```java
// Fragmento autoral: todas as durações são ilustrativas.
record Budget(long deadlineNanos) {
    static Budget start(java.time.Duration timeout) {
        return new Budget(System.nanoTime() + timeout.toNanos());
    }
    long remainingNanos() {
        return Math.max(0L, deadlineNanos - System.nanoTime());
    }
    boolean exhausted() {
        return remainingNanos() == 0L;
    }
}

// O adapter repassa apenas o orçamento restante.
// O relógio monotônico mede intervalo dentro do processo.
// Não serializar deadlineNanos para outro host.
var budget = Budget.start(java.time.Duration.ofMillis(800));
if (budget.exhausted()) {
    throw new java.util.concurrent.TimeoutException();
}
```

Fragmento Java de referência; não compilado como programa nesta edição.

Cancelar a espera não desfaz um efeito remoto. Depois de um timeout, a resposta pode continuar desconhecida e exigir reconciliação. Esse é o ponto em que minha experiência JVM se conecta ao domínio financeiro: controlar threads e conexões é necessário, mas a decisão sobre retry depende de idempotência, autoridade e estado. O exemplo não constitui um cliente HTTP completo.

Documentação oficial da tecnologia

FUNDAMENTOS · PYTHON

Python: um experimento deve poder falhar.

Python permite materializar rapidamente uma hipótese de comportamento. Um teste útil descreve uma consequência observável, como bloquear expansão quando falta amostra. O código acima é um exemplo de gate determinístico, não um teste estatístico de significância. O limiar precisa vir de uma política aprovada para a jornada.

PYTHON / PROPOSTA AUTORAL

```python
import unittest

def can_release(samples, failures, min_samples=1000):
    if type(samples) is not int or type(failures) is not int:
        raise ValueError("counts must be integers")
    if type(min_samples) is not int or min_samples <= 0:
        raise ValueError("invalid sample policy")
    if not 0 <= failures <= samples:
        raise ValueError("invalid counts")
    if samples < min_samples:
        return False
    return failures / samples <= 0.001

class ReleaseGateTests(unittest.TestCase):
    def test_small_sample_does_not_pass(self):
        self.assertFalse(can_release(50, 0))

    def test_error_budget_is_respected(self):
        self.assertTrue(can_release(2000, 2))
        self.assertFalse(can_release(2000, 3))

    def test_invalid_evidence_is_rejected(self):
        with self.assertRaises(ValueError):
            can_release(2000, 2001)
```

Exemplo didático separado dos vinte laboratórios executados; limiar sintético.

Os vinte laboratórios de casos usam dados sintéticos e testes locais. Eles ajudam a detectar regressões de contrato, estados ilegais e erros de representação. Não medem a infraestrutura do Nubank, não estimam carga real e não certificam controles de produção. Cada capítulo registra exatamente o que o laboratório cobre e o que precisa de validação adicional.

Documentação oficial da tecnologia

FUNDAMENTOS · POSTGRESQL

PostgreSQL: integridade no armazenamento.

O schema torna algumas invariantes impossíveis de ignorar por acidente: valor positivo, moeda suportada e uma chave única por escopo. A atualização condiciona a transição ao estado e à versão. Se nenhuma linha retornar, o serviço precisa distinguir conflito, repetição e identificador inexistente.

SQL / PROPOSTA AUTORAL

```sql
CREATE TABLE proposed_operation (
  operation_id UUID PRIMARY KEY,
  customer_scope TEXT NOT NULL,
  request_key TEXT NOT NULL,
  amount_minor BIGINT NOT NULL CHECK (amount_minor > 0),
  currency CHAR(3) NOT NULL CHECK (currency = 'BRL'),
  version BIGINT NOT NULL DEFAULT 0 CHECK (version >= 0),
  status TEXT NOT NULL CHECK (
    status IN ('received','pending','settled','rejected')
  ),
  UNIQUE (customer_scope, request_key)
);

UPDATE proposed_operation
SET status = 'pending', version = version + 1
WHERE operation_id = $1
  AND version = $2
  AND status = 'received'
RETURNING operation_id, version, status;
```

Exemplo autoral de referência; não executado contra infraestrutura real.

PostgreSQL é uma opção didática neste livro, não uma inferência sobre o banco de cada domínio do Nubank. Um constraint de status limita o vocabulário, mas não expressa toda a máquina de estados. A transação, os índices, o isolamento e a estratégia de outbox precisam ser avaliados juntos para que persistência e publicação se recuperem de falhas coerentemente.

Documentação oficial da tecnologia

FUNDAMENTOS · REDIS

Redis: cache com validade de domínio.

A política de cache começa com uma pergunta de produto: quanto atraso esse dado tolera? Uma projeção informativa pode aceitar validade curta; uma autorização financeira pode exigir a fonte autoritativa. A mesma ferramenta pode ser apropriada para um caminho e inadequada para outro.

PYTHON / PROPOSTA AUTORAL

```python
# Interface proposta; não é um cliente Redis completo.
def read_projection(key, cache, source, clock):
    cached = cache.get(key)
    if cached and cached["valid_until"] > clock():
        return {
            "data": cached["data"],
            "origin": "cache",
            "version": cached["version"],
        }
    current = source.read(key)
    cache.set(key, {
        "data": current.data,
        "version": current.version,
        "valid_until": clock() + 15,
    }, ttl_seconds=15)
    return {"data": current.data, "origin": "source",
            "version": current.version}
```

Exemplo autoral de referência; não executado contra infraestrutura real.

TTL limita duração, mas não impede uma escrita tardia de reinstalar uma versão antiga. A implementação distribuída precisa de comparação de versão, invalidação ou outra estratégia coerente com a carga. O exemplo omite esse mecanismo de propósito e o identifica como trabalho restante. Redis entra como alternativa de proposta, não como afirmação sobre o desenho interno do Nubank.

Documentação oficial da tecnologia

FUNDAMENTOS · TERRAFORM

Terraform: mudanças revisáveis.

Infraestrutura como código deixa decisões disponíveis para revisão: ownership, recuperação, identidade do recurso e proteção contra remoção acidental. O plano precisa ser interpretado por alguém capaz de avaliar impacto, não apenas aprovado porque o arquivo passou por formatação.

HCL / PROPOSTA AUTORAL

```hcl
# Fragmento de referência. Provider e backend são externos.
resource "aws_dynamodb_table" "operation_keys" {
  name         = "proposed-operation-keys"
  billing_mode = "PAY_PER_REQUEST"
  hash_key     = "pk"

  attribute {
    name = "pk"
    type = "S"
  }
  point_in_time_recovery {
    enabled = true
  }
  server_side_encryption {
    enabled = true
  }
  lifecycle {
    prevent_destroy = true
  }
  tags = {
    owner       = "proposed-journey-team"
    environment = "example"
  }
}
```

Exemplo autoral de referência; não executado contra infraestrutura real.

prevent_destroy não substitui controle de acesso nem política de backup. Remover o bloco de configuração ou modificar a estratégia de gestão exige cuidado adicional. O exemplo não foi aplicado e não inclui backend remoto, versão de provider, chave gerenciada ou política IAM. Essas omissões são explícitas para impedir que um fragmento editorial pareça uma configuração pronta para produção.

Documentação oficial da tecnologia

FUNDAMENTOS · TYPESCRIPT

TypeScript: validar a fronteira.

Um tipo estático não valida o JSON recebido em runtime. O parser proposto trata a fronteira como desconhecida e rejeita estados fora do contrato. Em produto, essa rejeição precisa virar uma experiência segura, com consulta alternativa ou mensagem de incompatibilidade, sem inventar confirmação financeira.

TYPESCRIPT / PROPOSTA AUTORAL

```ts
type Status = "pending" | "settled" | "rejected";
type Projection = { version: number; status: Status };

function parseProjection(input: unknown): Projection {
  if (typeof input !== "object" || input === null) {
    throw new Error("invalid object");
  }
  const x = input as Record<string, unknown>;
  if (typeof x.version !== "number" ||
      !Number.isSafeInteger(x.version) || x.version < 0) {
    throw new Error("invalid version");
  }
  if (typeof x.status !== "string" ||
      !["pending", "settled", "rejected"].includes(x.status)) {
    throw new Error("unsupported status");
  }
  return { version: x.version as number,
           status: x.status as Status };
}
```

Exemplo autoral de referência; não executado contra infraestrutura real.

TypeScript é usado aqui como ferramenta de contrato e visualização, não como requisito geral do Nubank. Para um SDK real, eu geraria tipos a partir de schema versionado, manteria fixtures de consumidores antigos e definiria como campos adicionais são tratados. A evolução segura permite expandir o contrato sem obrigar todos os clientes a atualizar no mesmo instante.

Documentação oficial da tecnologia

FUNDAMENTOS · GRAFANA

Grafana: um painel que orienta ação.

O painel começa com a decisão que alguém precisa tomar. Conclusão confirmada, estado desconhecido e idade da reconciliação descrevem aspectos diferentes da confiança. CPU e memória ajudam a diagnosticar, mas não demonstram sozinhas que o cliente conseguiu concluir a tarefa.

JSON / PROPOSTA AUTORAL

```json
{
  "title": "Proposta: confiança na jornada",
  "panels": [
    {"title": "Conclusão confirmada", "unit": "percent"},
    {"title": "Estado desconhecido", "unit": "short"},
    {"title": "Tempo de confirmação P99", "unit": "ms"},
    {"title": "Reconciliação mais antiga", "unit": "s"},
    {"title": "Contatos por 1.000 jornadas", "unit": "short"}
  ],
  "dimensions": ["journey", "release", "cohort"],
  "decision": {
    "owner": "proposed-journey-team",
    "action": "hold rollout when guardrails fail",
    "runbook": "referência local do serviço"
  }
}
```

Especificação conceitual de painel; não é o schema de importação do Grafana.

O JSON é uma especificação editorial, não um dashboard importável do Grafana. Cada indicador precisa de definição, denominador, janela, proprietário e vínculo com o procedimento de resposta. Cor vermelha sem ação combinada gera ruído. Um bom painel também mostra dados ausentes e baixa amostra, para que silêncio da telemetria não seja interpretado como sucesso.

Documentação oficial da tecnologia

ARQUITETURA TRANSVERSAL · PROPOSTA EXTERNA

Uma jornada, duas autoridades.



Desenho autoral de referência. Não representa a arquitetura interna do Nubank.

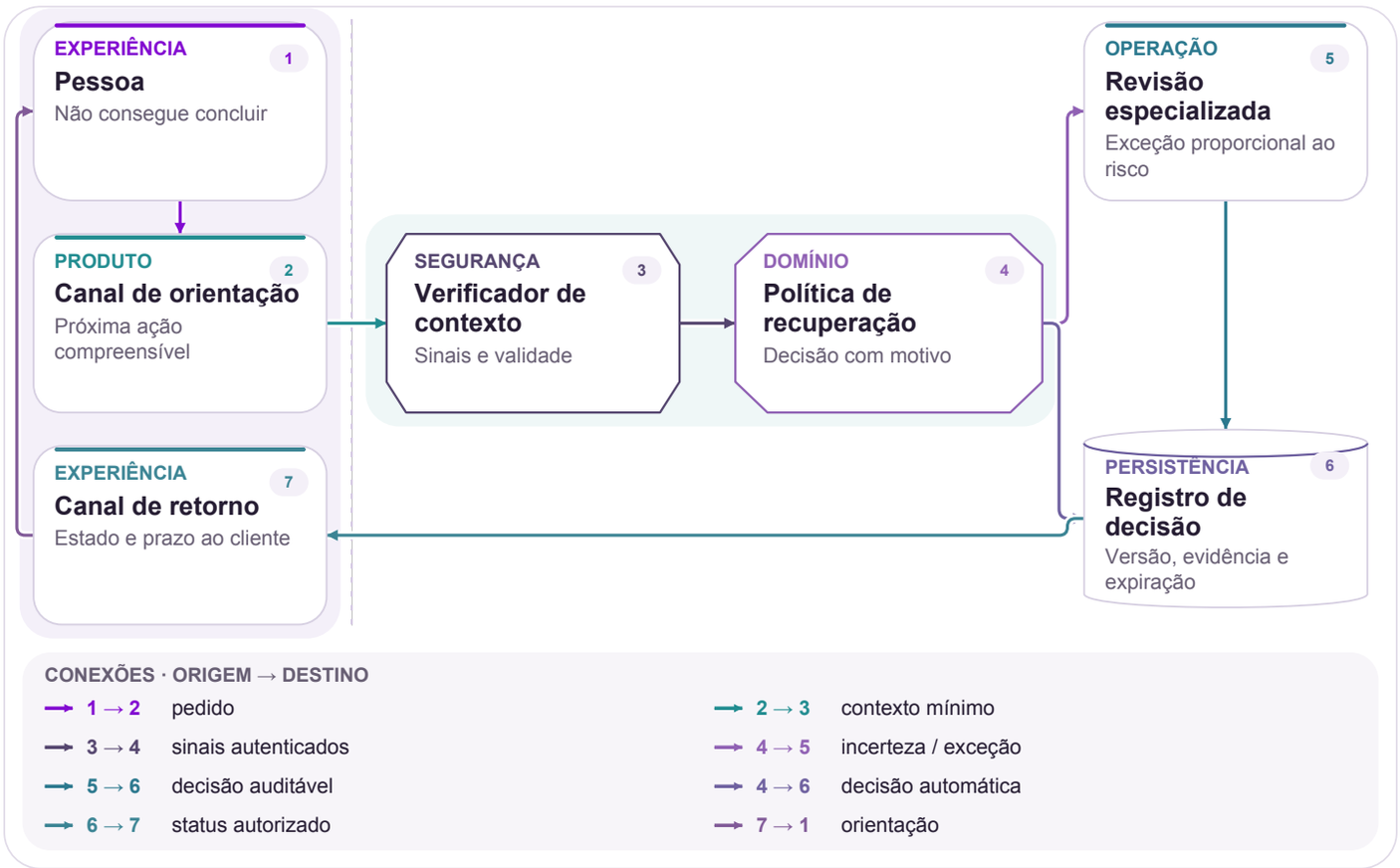
A arquitetura separa a autoridade que muda o estado da autoridade que o apresenta. O núcleo de domínio e seu armazenamento decidem se uma operação foi aceita. A projeção facilita leitura, mas carrega versão e atraso. Uma tela não pode converter ausência de atualização em rejeição, nem um atendimento pode prometer reversão sem consultar o estado responsável pelo efeito.

O outbox representa intenção de publicação na mesma fronteira da escrita de domínio. Um publicador pode repetir mensagens; por isso o consumidor precisa deduplicar efeitos. O desenho admite falhas entre essas etapas e torna a recuperação parte do contrato. A escolha entre Datomic, DynamoDB ou outra persistência depende do domínio e da plataforma existente; os três não são adicionados simultaneamente por estética.

O laboratório Clojure demonstra apenas a decisão em memória. Os laboratórios de casos exploram propriedades específicas. Integração distribuída, autorização, retenção e capacidade ainda exigem validação no contexto do time.

ARQUITETURA TRANSVERSAL · PROPOSTA EXTERNA

Recuperação como parte do produto.



Desenho autoral de referência. Não representa a arquitetura interna do Nubank.

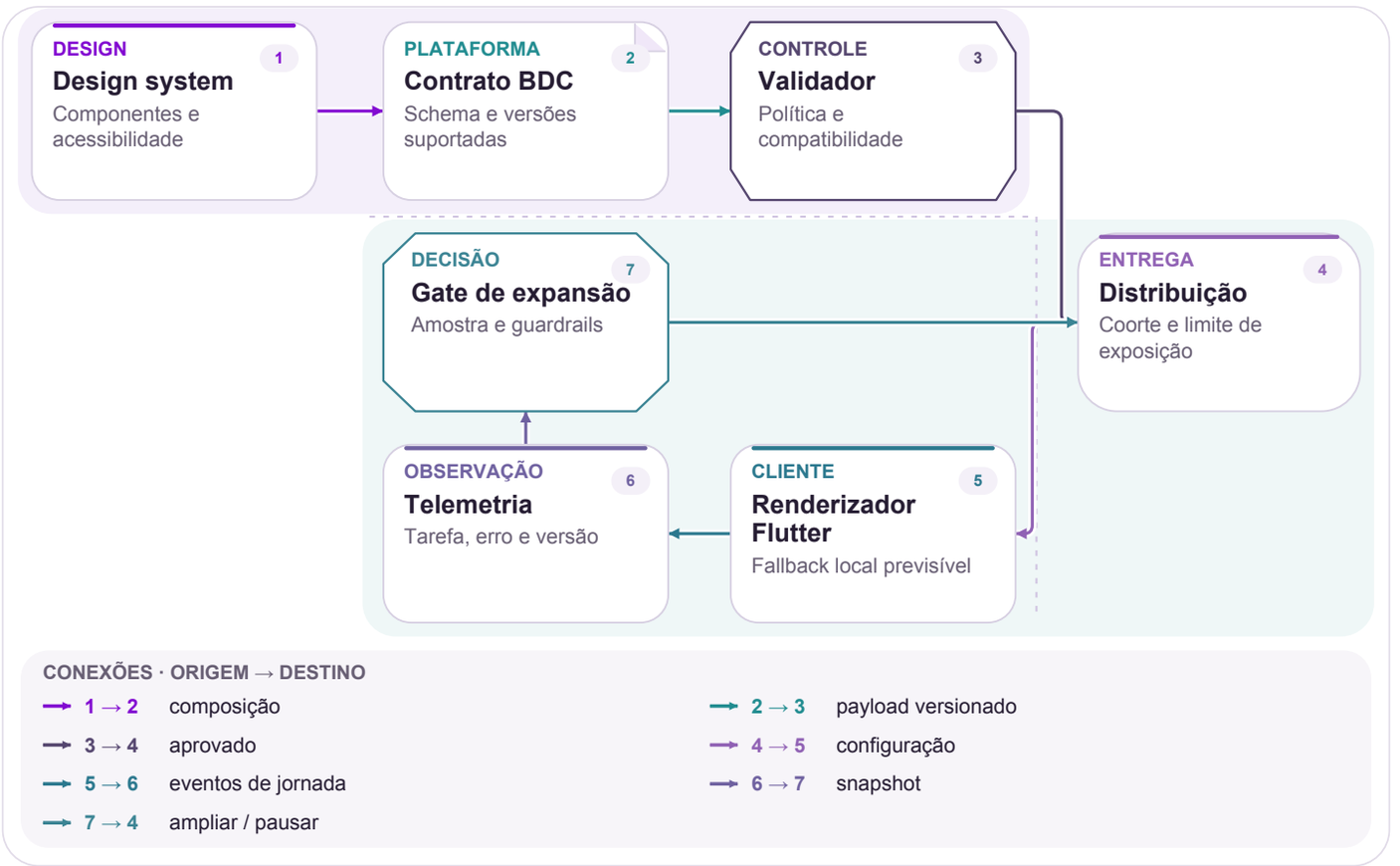
Recuperação não deve ser uma coleção de exceções sem dono. O fluxo precisa distinguir incapacidade técnica, suspeita de fraude e falta de evidência. Cada situação pode exigir um caminho diferente. A revisão humana aparece onde adiciona julgamento, não como saída ilimitada para qualquer erro de integração.

O registro associa decisão, política e validade. Um sinal antigo não deve autorizar uma ação sensível indefinidamente. Quando uma alternativa é recusada, o produto precisa explicar o próximo passo possível sem divulgar detalhes que ajudem a contornar controles. Essa linguagem exige colaboração entre Segurança, Produto, conteúdo e Atendimento.

A medida de sucesso combina solução legítima, tempo de recuperação e proteção. Reduzir fila rejeitando mais pessoas pode melhorar uma métrica operacional e piorar o resultado. Como EM, eu garantiria capacidade e responsabilidades; como Staff, definiria autoridade, expiração e mecanismos de revisão.

ARQUITETURA TRANSVERSAL · PROPOSTA EXTERNA

Plataforma de experiência com contratos.



Desenho autoral de referência. Não representa a arquitetura interna do Nubank.

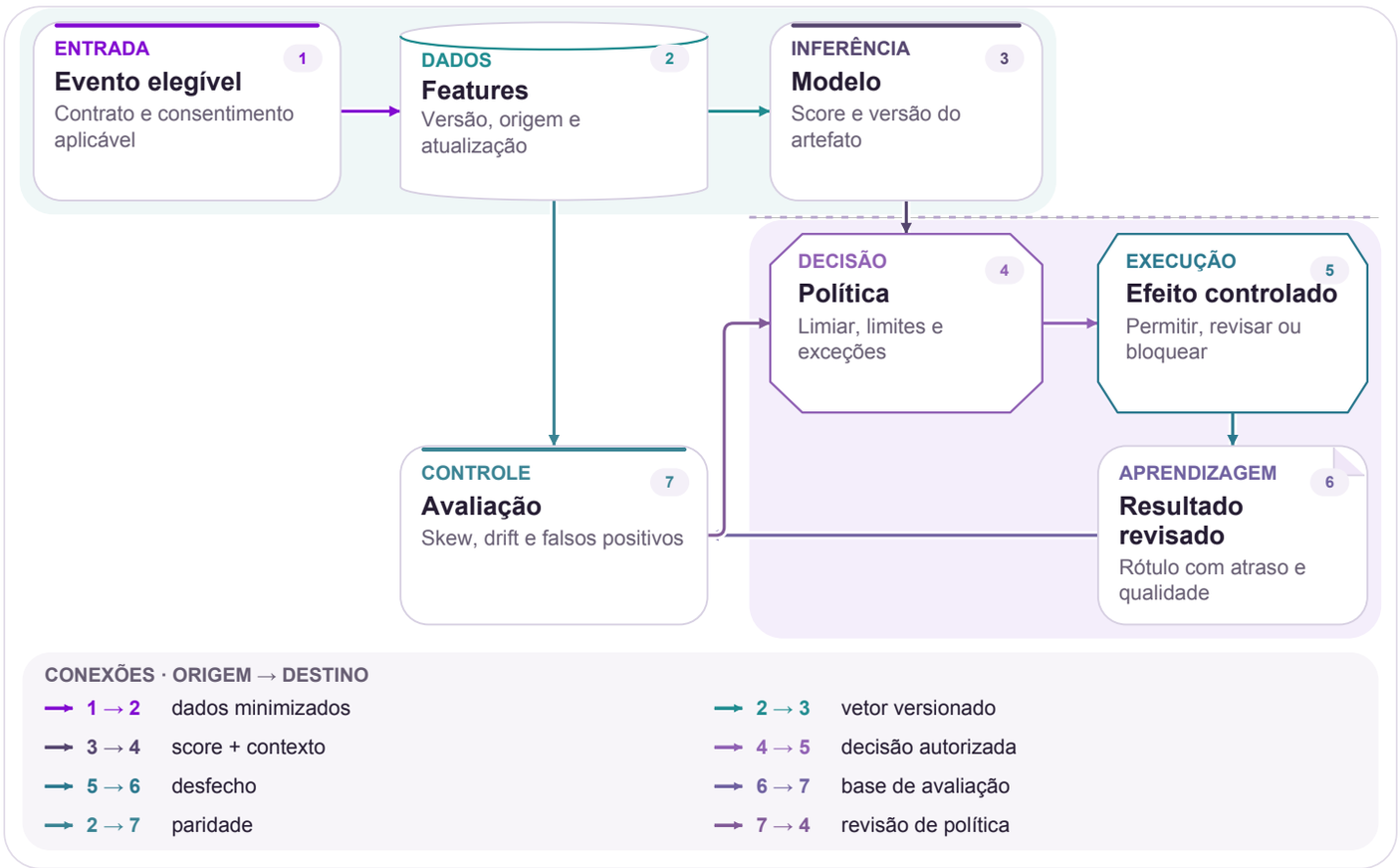
Uma plataforma de interface dirigida pelo servidor amplia velocidade de composição, mas também amplia a importância do contrato. O cliente instalado pode estar atrás da versão do servidor. O validador precisa conhecer a capacidade mínima do renderizador e impedir que uma configuração exija um comportamento inexistente.

O fallback é um componente de produto: deve preservar a tarefa e a acessibilidade, não apenas evitar crash. A telemetria liga configuração, versão do app e resultado da jornada. O gate usa evidência suficiente antes de ampliar exposição. A decisão continua tendo dono; não é entregue a uma única métrica sem contexto.

O Nubank publica trabalho sobre BDC; este desenho é uma proposta didática para discutir governança e evolução. Eu investigaria quais controles já existem e buscaria a menor extensão necessária. A plataforma produz valor quando reduz esforço das equipes consumidoras mantendo qualidade e capacidade de recuperação.

ARQUITETURA TRANSVERSAL · PROPOSTA EXTERNA

ML: separar previsão de decisão.



Desenho autoral de referência. Não representa a arquitetura interna do Nubank.

Um score não é uma decisão de negócio. A política combina a previsão com limites, contexto e vias de revisão. Separar essas responsabilidades permite mudar um limiar sem fingir que o modelo foi retreinado e permite investigar se o problema está nos dados, na inferência ou no uso do resultado.

Features precisam de identidade e tempo: o que estava disponível quando a decisão foi tomada pode diferir do que sabemos hoje. Avaliar usando informação futura cria uma impressão de qualidade que não existiria em produção. O contrato de treino e serving precisa impedir esse vazamento e medir diferenças relevantes.

A base de verdade pode chegar tarde e ser incompleta. Os experimentos precisam preservar trilhas de versão e tratar falsos positivos como pessoas legítimas prejudicadas por uma decisão. O valor proposto é melhorar proteção e recuperação de forma conjunta, com especialistas responsáveis por política e critérios de exposição.

ANÁLISE E PROPOSTA EXTERNA

O problema começa quando o estado fica invisível

01



Pix fora do AR novamente?

Captura da fonte pública consultada em 23/10/2026. Relato não equivale a incidente reproduzido.

O registro público de 15 de julho de 2026 descreve acesso ao aplicativo, seguido de erro em transferências, pagamentos Pix e no canal de suporte. Isso sustenta uma pergunta de engenharia, não um diagnóstico: como preservar orientação quando a operação principal encontra uma falha? O relato não permite concluir que toda a base foi afetada, que houve débito indevido ou que os componentes compartilham infraestrutura. A análise parte dessa fronteira e propõe uma investigação verificável.

Eu separaria três dificuldades que podem parecer iguais na tela: indisponibilidade antes de aceitar o comando, resultado desconhecido depois de aceitá-lo e informação de status indisponível. A primeira admite uma nova tentativa quando seguro; a segunda exige consultar ou reconciliar; a terceira pede uma mensagem que preserve a incerteza. Um único botão “tente novamente” pode esconder essas diferenças e levar o cliente a criar operações novas sem necessidade.

O objetivo de produto seria permitir que a pessoa entenda se seu dinheiro se moveu, qual ação pode tomar e como obter ajuda. O objetivo técnico correspondente seria manter identidade da operação, decisão financeira e comunicação consistentes. O laboratório deste capítulo testa a parte local dessa promessa: um débito confirmado possui recibo durável e não reaparece como outro débito quando a resposta se perde.

NuCommunity - Pix fora do AR novamente?

ANÁLISE E PROPOSTA EXTERNA

A operação é uma entidade; a tentativa é um evento



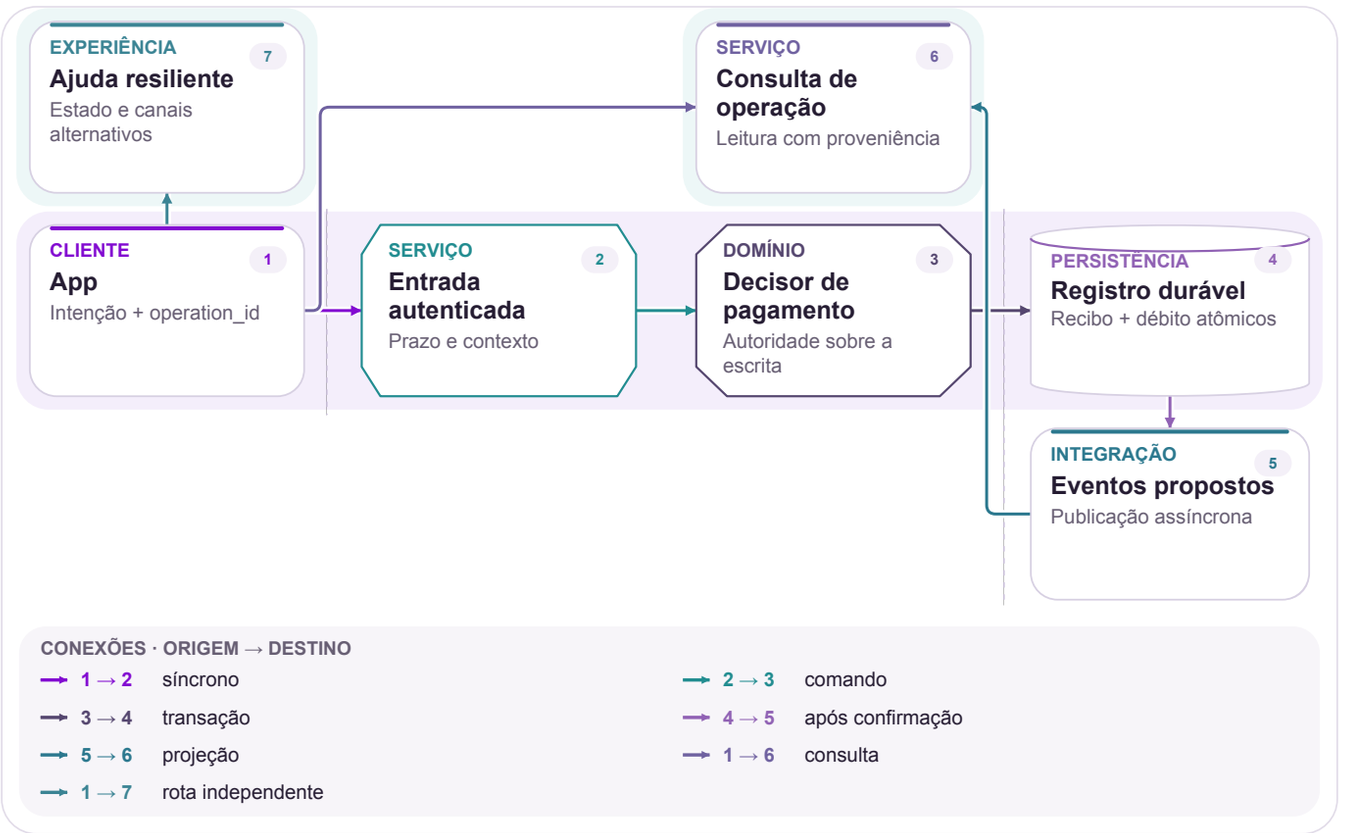
O domínio proposto distingue operation_id de request_id. A operação representa a intenção estável: uma conta, um favorecido e um valor. A tentativa representa uma interação de transporte, com seu próprio instante, prazo e resultado de rede. Confundir os dois identificadores faz cada repetição parecer uma intenção nova. No contrato sugerido, o aplicativo mantém operation_id enquanto tenta descobrir o resultado daquela mesma intenção; alterar valor ou favorecido exige uma nova confirmação e outro identificador.

Os estados úteis seriam recebido, rejeitado, confirmado e resultado pendente de esclarecimento. “Timeout” descreve a tentativa e não deve, sozinho, colocar a operação em rejeitado. O estado confirmado precisa referenciar uma decisão durável. Uma consulta indisponível tampouco pode apagar uma confirmação já conhecida. Para a interface, isso significa conservar recibo e contexto durante transições de conectividade, distinguindo informação local da confirmação obtida no serviço.

As invariantes são concretas: saldo não negativo, uma identidade de operação associada a um único payload e correspondência entre débito, lançamento e recibo. A recuperação de sessão adiciona outra condição: uma sessão antiga não pode emitir novos comandos após a barreira de revogação. Essas regras são testáveis sem conhecer a arquitetura interna do banco e permitem avaliar qualquer implementação candidata antes de discutir linguagem ou infraestrutura.

ARQUITETURA PROPOSTA • FRONTEIRAS EXPLÍCITAS

Separar o caminho financeiro do caminho de orientação



Desenho autoral proposto; não representa a arquitetura interna do Nubank.

O desenho propõe duas rotas observáveis: executar uma intenção financeira e esclarecer uma intenção existente. A autoridade sobre o débito permanece no decisor e em seu registro transacional. A projeção de consulta pode ajudar a escalar leituras, mas precisa declarar atraso e não pode inventar confirmação. Se ela estiver defasada, a consulta deve procurar a fonte autoritativa ou comunicar que ainda não há resposta confiável.

A ajuda aparece como domínio separado porque seu conteúdo mínimo deve sobreviver à degradação do caminho financeiro. Isso não exige, automaticamente, outro cluster: primeiro seria necessário identificar dependências comuns, autenticação, configurações e limites de capacidade. Uma página estática de orientação pode ser suficiente para parte do problema, enquanto informações de conta continuam protegidas.

Kafka é uma opção para propagar eventos confirmados; não transforma uma escrita externa em atomicidade distribuída. A implementação proposta precisaria de publicação recuperável, consumidor idempotente e reconciliação. O laboratório demonstra somente a transação local, deixando essa fronteira explícita.

Apache Kafka - Design

ENGENHARIA EXPLICADA

O recibo anterior vence a repetição

PYTHON / PROPOSTA AUTORAL

```python
def pay(self, *, account_id, operation_id, amount_cents,
        payee, recovery_epoch, fault=None):
    # Recorte: validação e payload/digest estão no arquivo completo.
    with self._write() as connection:
        account = connection.execute(
            "SELECT balance_cents, recovery_epoch FROM accounts "
            "WHERE account_id=?", (account_id,)
        ).fetchone()
        if account is None:
            raise UnknownAccount(account_id)
        # A sessão é validada antes inclusive de retornar um recibo já existente.
        if recovery_epoch != account["recovery_epoch"]:
            raise StaleSession("Comando de sessão anterior à recuperação")
        previous = connection.execute(
            "SELECT payload_json, payload_hash, receipt_json "
            "FROM payments WHERE operation_id=?",
            (operation_id,),
        ).fetchone()
        if previous is not None:
            # Compara também o conteúdo canônico, sem depender apenas do hash.
            if (previous["payload_hash"] != digest
                    or previous["payload_json"] != payload):
                raise IdempotencyConflict(
                    "operation_id reutilizado com outro payload")
            return json.loads(previous["receipt_json"])
        if account["balance_cents"] < amount_cents:
            raise InsufficientFunds("Saldo insuficiente no cenário sintético")
```

Recorte do método pay; preâmbulo e gravação omitidos. Arquivo completo executado e verificado.

O trecho consulta a geração de sessão antes de revelar qualquer resultado. Depois procura a identidade da operação e compara o conteúdo canônico, além de seu hash. Uma repetição válida retorna o recibo armazenado; uma reutilização com outro payload produz conflito. Só uma intenção ainda inexistente chega à verificação de saldo.

Essa ordem importa: o saldo posterior à primeira compra pode ser menor que o valor original, mas isso não deveria transformar o replay em “saldo insuficiente”. O bloco executa dentro de BEGIN IMMEDIATE no modelo SQLite. A serialização é uma escolha didática local, não uma recomendação de escala distribuída.

Python - sqlite3

ENGENHARIA EXPLICADA

Clojure: separar decisão pura de efeito transacional

CLOJURE / PROPOSTA AUTORAL

```clojure
(defn payment-decision [account previous command]
  ;; Contrato proposto; não é código interno do Nubank.
  (let [{:keys [epoch balance-cents]} account
        {:keys [session-epoch payload]} command
        amount (:amount-cents payload)]
    (cond
      (not= epoch session-epoch)
      {:type :reject :reason :stale-session}

      (and previous (not= payload (:payload previous)))
      {:type :reject :reason :idempotency-conflict}

      previous
      {:type :replay :receipt (:receipt previous)}

      (or (not (integer? amount)) (not (pos? amount)))
      {:type :reject :reason :invalid-amount}

      (< balance-cents amount)
      {:type :reject :reason :insufficient-funds}

      :else
      {:type :commit
       :new-balance (- balance-cents amount)
       :payload payload})))
```

Função didática proposta; este trecho Clojure não foi executado.

A função proposta retorna uma decisão em dados. Ela não grava, não chama serviços e não garante concorrência por si só. Sua utilidade é tornar as regras legíveis e permitir explorar combinações de estado sem depender da rede. A camada de persistência deve executar a decisão sobre uma visão válida e confirmar suas precondições atomicamente.

Em uma implementação com Datomic, eu examinaria identidade única, comparação de versão e transação apropriada antes de mapear esse resultado para dados transacionais. Ler um saldo e escrever depois, sem proteção, manteria a corrida mesmo com uma função pura correta. O exemplo é ilustrativo e não foi executado em Clojure.

Datomic - Transaction Data

ANÁLISE E PROPOSTA EXTERNA

Contrato persistente: o que precisa sobreviver ao processo



O registro de idempotência não pode morar somente na memória do processo que recebeu a primeira tentativa. Reinício, escalonamento e troca de instância fariam a repetição perder sua história. No modelo local, a tabela payments possui operation_id único, conteúdo canônico e recibo. A tabela ledger exige uma identidade de operação única e referencia o pagamento. A atualização de saldo acontece na mesma transação que as duas inserções, formando uma unidade de recuperação.

O contrato deve declarar por quanto tempo essa identidade é reconhecida. Um prazo curto pode permitir que um cliente desconectado reapresente uma intenção antiga como nova; retenção ilimitada aumenta custo e obrigações de tratamento de dados. Eu decidiria a janela com dados de repetição e necessidades operacionais, preservando identificadores suficientes para esclarecer disputas. O livro não estabelece um prazo real para o Nubank.

O recibo registra a confirmação histórica, inclusive o saldo observado após aquela decisão. Ele não deve ser usado como saldo atual. Para uma integração real, haveria ainda a identidade do participante externo e estados de liquidação que o laboratório não implementa. Garantir um lançamento local não garante que um sistema remoto recebeu a mensagem uma única vez. Essa diferença precisa estar no contrato, no runbook e na explicação entregue ao cliente.

ENGENHARIA EXPLICADA

A concorrência deve atacar a mesma invariante

PYTHON / PROPOSTA AUTORAL

```python
def test_concurrent_distinct_payments_cannot_overdraw(self):
    barrier = threading.Barrier(2)
    def attempt(operation_id):
        barrier.wait(timeout=10)
        try:
            return self.bank.pay(**self.command(operation_id=operation_id))
        except InsufficientFunds:
            return "insufficient"
    with ThreadPoolExecutor(max_workers=2) as pool:
        results = list(pool.map(attempt, ("parallel-a", "parallel-b")))
    self.assertEqual(sum(isinstance(value, dict) for value in results), 1)
    self.assertEqual(results.count("insufficient"), 1)
    self.assert_invariants(balance=300, payments=1)

def test_concurrent_same_operation_has_one_receipt_and_one_debit(self):
    barrier = threading.Barrier(8)
    def retry(_):
        barrier.wait(timeout=10)
        return self.bank.pay(**self.command())
    with ThreadPoolExecutor(max_workers=8) as pool:
        results = list(pool.map(retry, range(8)))
    self.assertTrue(all(receipt == results[0] for receipt in results))
    self.assert_invariants(balance=300, payments=1)
```

Trecho do laboratório sintético; execução verificada no arquivo completo.

O primeiro teste libera duas threads simultaneamente e usa conexões independentes para disputar um saldo que só comporta uma compra. O resultado aceitável é exatamente uma confirmação e uma rejeição por saldo. No segundo, oito tentativas representam a mesma intenção e devem convergir para um recibo único.

A barreira aumenta a chance de sobreposição; ela não enumera todas as interleavings possíveis. O valor de 1.000 centavos e os pagamentos de 700 são fixtures sintéticas. A verificação posterior confere saldo, cardinalidade dos registros e soma dos débitos. Assim o teste observa efeito financeiro, não apenas códigos de retorno.

ENGENHARIA EXPLICADA

Matar o processo é diferente de lançar uma exceção

PYTHON / PROPOSTA AUTORAL

```python
def run_crash_worker(self, fault: str, expected_exit: int):
    result = subprocess.run(
        [sys.executable, str(Path(__file__).resolve()),
         "--crash-worker", str(self.database), fault],
        capture_output=True, text=True, timeout=20,
    )
    self.assertEqual(result.returncode, expected_exit, result.stderr)
    self.bank = SyntheticPayments(self.database)

def test_process_crash_before_commit_rolls_back_and_resume_executes_once(self):
    self.run_crash_worker("crash_before_commit", 72)
    self.assert_invariants(balance=1000, payments=0)
    receipt = self.bank.pay(**self.command())
    self.assertEqual(self.bank.pay(**self.command()), receipt)
    self.assert_invariants(balance=300, payments=1)

def test_process_crash_after_commit_preserves_receipt_and_resume_replays(self):
    self.run_crash_worker("crash_after_commit", 73)
    self.assert_invariants(balance=300, payments=1)
    stored = json.loads(self.bank.inspect("alice")["payments"][0]["receipt_json"])
    self.assertEqual(self.bank.pay(**self.command()), stored)
    self.assert_invariants(balance=300, payments=1)
```

Trecho do laboratório sintético; execução verificada no arquivo completo.

Os testes iniciam outro processo e o encerram com os._exit em pontos definidos. Antes do commit, a reabertura deve encontrar saldo intacto; depois do commit, deve encontrar o recibo persistido. Essa distinção exercita recuperação sem depender do finally de uma exceção comum.

O arquivo completo passou em 12 testes, incluindo esses cenários e o timeout após confirmação. O resultado não comprova resistência a perda de energia, corrupção de disco ou falha de um provedor externo. A evidência é mais estreita e mais útil: neste modelo local, terminar o processo nesses dois pontos não cria débito duplicado na retomada.

ANÁLISE E PROPOSTA EXTERNA

Observar incerteza, não apenas sucesso HTTP



Eu proporia três famílias de sinais. A primeira mede tentativas: duração, erros de transporte e saturação. A segunda mede operações: confirmações, conflitos de identidade e tempo até esclarecer resultado desconhecido. A terceira mede experiência: consulta bem-sucedida, ajuda acessível e contatos associados ao incidente. Uma resposta HTTP rápida não compensa uma operação que permanece indefinida, e um erro de transporte não significa necessariamente perda da decisão financeira.

As métricas propostas poderiam incluir payment_attempts_total por resultado limitado, payment_resolution_seconds por jornada e payment_unknown_operations como quantidade agregada. Identificadores de conta, operation_id e favorecido não virariam rótulos Prometheus. Eles pertencem, quando necessários e autorizados, a investigação controlada com correlação e retenção adequadas. A decisão evita cardinalidade explosiva e reduz exposição desnecessária de dados.

O painel de incidente começaria pelo impacto observável: quantas intenções aguardam esclarecimento, há crescimento contínuo e o canal de ajuda responde? Alertas deveriam ter um responsável e uma ação possível, como reduzir novas admissões, acionar reconciliação ou publicar orientação. Eu acompanharia percentis e coortes de versão do app, mas não misturaria o tempo de fila de reconciliação com a latência do comando. São problemas diferentes e precisam de limites separados.

Prometheus - Metric and label naming

ANÁLISE E PROPOSTA EXTERNA

Lançar com uma rota de recuo que preserve recibos



A implantação começaria por instrumentação e leitura comparativa, sem alterar a decisão financeira. Um conjunto de intenções sintéticas exercitaria timeout e consulta, enquanto o time conferiria se identificadores atravessam as camadas. A primeira mudança visível poderia ser a explicação de estado e a rota de ajuda, desde que a informação apresentada fosse fiel. Isso permite reduzir confusão antes de substituir mecanismos críticos de escrita.

Para a parte transacional, eu exigiria compatibilidade entre versões durante a migração. A versão antiga precisa reconhecer uma operação escrita pela nova ou ser impedida de executá-la novamente. Feature flag de interface não resolve essa compatibilidade. Uma estratégia segura manteria um registro compartilhado e contratos de leitura estáveis, com critérios claros para interromper a expansão se aumentarem conflitos inesperados, pendências ou divergências de saldo.

Em Kubernetes, a distribuição de tráfego e o rollback seriam controles operacionais, não garantias de correção do domínio. Reverter o binário não deve apagar registros de idempotência nem voltar um schema incompatível. Eu prepararia um exercício em que o processo novo confirma, falha antes da resposta e o processo anterior atende o retry. O lançamento só avançaria quando a continuidade do recibo estivesse demonstrada e o atendimento conseguisse explicar esse caminho.

ANÁLISE E PROPOSTA EXTERNA

Limitar trabalho repetido sem esconder o resultado



Uma tempestade de retries pode transformar uma indisponibilidade parcial em saturação. Eu adotaria prazos explícitos, orçamento de tentativas e espera com variação aleatória para consultas repetidas, separando esses controles da identidade financeira. O cliente pode esperar mais para consultar, mas não deve gerar outra operação apenas porque terminou seu orçamento de rede. A resposta de conflito também precisa ser distinguível de um erro transitório, pois repetir o mesmo payload inválido não ajuda.

O endpoint de consulta requer autorização equivalente à sensibilidade do recibo. Conhecer um operation_id não pode bastar para ler dados de outra conta. A geração de sessão no laboratório demonstra a ordem da barreira, mas o programa recebe os parâmetros diretamente e não implementa autenticação real. Em um serviço, a identidade e a geração viriam de credenciais verificadas, e a consulta aplicaria escopo por titular.

Para desempenho, eu mediria a contenção antes de escolher particionamento. O laboratório serializa escritores no arquivo SQLite, o que simplifica a prova local e limita throughput. Um desenho distribuído teria de escolher a fronteira de consistência e evitar concorrência indevida sobre a mesma conta. O objetivo Staff seria retirar trabalho desnecessário do caminho crítico sem mover a autoridade financeira para uma projeção potencialmente atrasada.

ANÁLISE E PROPOSTA EXTERNA

ADR: recibo durável e publicação recuperável



A decisão proposta é tornar a confirmação local um fato durável e usar essa identidade para responder a repetições. A alternativa de guardar apenas um cache de respostas seria mais simples no início, mas deixaria uma lacuna quando o cache expirasse ou uma instância reiniciasse. A alternativa de bloquear toda repetição evitaria parte das duplicidades, porém impediria recuperar respostas perdidas e transferiria a incerteza para o atendimento.

Para distribuir o resultado, eu compararia publicação recuperável associada à transação, leitura do log de mudanças e integração síncrona com o consumidor. A escolha dependeria das garantias do armazenamento e da operação disponível. Em qualquer opção, o consumidor precisa reconhecer a identidade do evento e tornar seu efeito repetível. Um tópico Kafka proposto não deve ser descrito como prova de efeito único em serviços externos.

A consequência aceita é maior cuidado com estados, retenção e reconciliação. O benefício pretendido é uma explicação determinística para o cliente e para quem investiga o incidente. Eu reabriria essa decisão se a fronteira de liquidação exigisse reserva e confirmação separadas, se houvesse múltiplas moedas ou se a pressão de throughput mudasse a unidade de serialização. O ADR registraria essas condições e os testes que precisam continuar passando após qualquer migração.

ANÁLISE E PROPOSTA EXTERNA

Como eu lideraria a entrega e demonstraria valor



Eu organizaria a iniciativa em torno de uma jornada, não de uma lista de componentes. Um responsável de pagamentos definiria a verdade financeira; mobile validaria como a intenção sobrevive à conectividade; atendimento definiria a informação necessária para resolver o caso; plataforma apoiaria isolamento e operação. Meu papel como EM seria transformar essas dependências em entregas verificáveis e evitar que a responsabilidade pela incerteza terminasse entre equipes.

A primeira revisão exigiria exemplos de falha e estados permitidos. A segunda mostraria os testes de concorrência e recuperação. A terceira avaliaria uma simulação com suporte e comunicação, incluindo como informar um resultado ainda desconhecido. Como Staff, eu revisaria a fronteira transacional, os contratos de repetição e as decisões de migração, documentando o que o laboratório prova e o que permanece fora dele. Nenhuma dessas atividades depende de atribuir ao banco uma arquitetura não observada.

O valor seria estimado por contatos evitados multiplicados pelo custo marginal de resolução, descontando manutenção e infraestrutura. Tempo de esclarecimento e confiança percebida entrariam como resultados próprios, sem serem convertidos automaticamente em receita. A pergunta de entrevista que eu levaria é concreta: “Qual componente pode afirmar que uma intenção foi confirmada quando a resposta original se perdeu, e como demonstramos que todas as versões do cliente conseguem recuperar essa verdade?”

ANÁLISE E PROPOSTA EXTERNA

A segurança também precisa de uma saída legítima

02

Sem acesso a minha conta , pede reconhecimento facial

Captura da fonte pública consultada em 23/10/2026. Relato não equivale a incidente reproduzido.

O participante da NuCommunity informa que sua câmera frontal estava quebrada, que a traseira funcionava e que não conseguia ativar a conta pelo reconhecimento facial. O trecho público não revela o desfecho nem as alternativas oferecidas posteriormente. Portanto, ele não prova falha biométrica. Ele oferece um caso concreto para examinar como um requisito de segurança se comporta quando o dispositivo não possui a capacidade esperada.

Uma recuperação bem desenhada precisa proteger duas pessoas: o titular legítimo que perdeu acesso e o titular que alguém tenta personificar. Simplificar o caminho ignorando evidências de identidade poderia resolver o primeiro contato e criar um risco maior. Minha proposta seria tornar o tratamento de exceções previsível, com pré-verificação de capacidades, encaminhamento assistido e estado acompanhável, preservando a avaliação de risco exigida para restaurar acesso.

O problema de engenharia vai além da câmera. Há tentativas repetidas, prazos, revisão humana, mudança de aparelho e comandos emitidos por sessões antigas. O capítulo modela essas fronteiras com uma máquina de estados persistente e uma geração de recuperação. O laboratório não realiza prova de vida, comparação de rosto ou avaliação antifraude: demonstra como impedir pular etapas, reaplicar uma decisão vencida ou restaurar duas vezes o mesmo processo. Essa delimitação torna o trabalho revisável e evita apresentar uma simulação como sistema de identidade.

NuCommunity - Sem acesso a minha conta, pede reconhecimento facial

ANÁLISE E PROPOSTA EXTERNA

Capacidade do aparelho não é identidade do titular







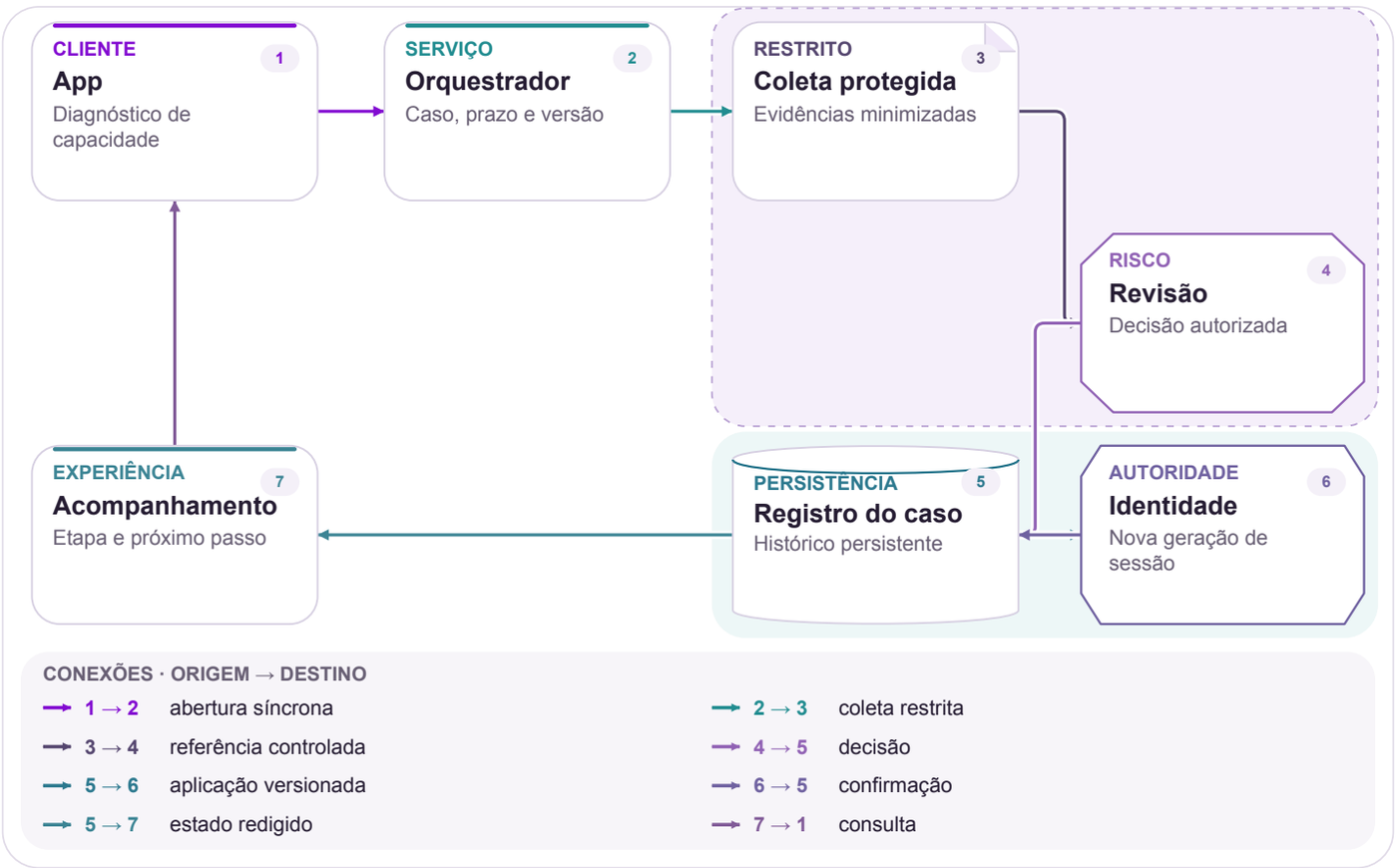
Eu separaria o diagnóstico técnico da avaliação de identidade. O primeiro responde se a câmera existe, se a permissão foi concedida e se a captura consegue começar. O segundo decide se as evidências disponíveis são suficientes para uma ação sensível. Uma câmera indisponível pode encaminhar o caso para outra jornada; não pode, por si só, autorizar o acesso. Essa divisão evita que um código de erro do dispositivo se transforme inadvertidamente em uma exceção de segurança.

Os estados propostos seriam diagnóstico, revisão, verificado, restaurado, expirado e encerrado. Cada transição possui precondições, responsável e versão. O histórico preservaria por que uma etapa ocorreu, sem exigir que imagens biométricas circulem por todos os serviços. “Verificado” significa que uma avaliação autorizada concluiu sua etapa; “restaurado” significa que a alteração de acesso foi efetivada. Separá-los permite detectar uma aprovação que ainda aguarda aplicação.

O modelo local usa dois revisores distintos para tornar a separação de responsabilidade explícita. Isso é uma política sintética, não uma recomendação universal de operação manual. Em produção, a combinação de automação e revisão dependeria do risco e da governança. O princípio transferível é que a aplicação da decisão deve verificar seu escopo, validade e versão no momento em que restaura o acesso, inclusive quando mensagens chegam atrasadas.

ARQUITETURA PROPOSTA • FRONTEIRAS EXPLÍCITAS

Recuperação com fronteira entre evidência e autorização



Desenho autoral proposto; não representa a arquitetura interna do Nubank.

A proposta mantém evidências sensíveis em uma fronteira restrita e usa referências para acompanhar o caso. O atendimento precisa conhecer a etapa e a ação seguinte, mas nem todo operador precisa acessar material biométrico. A restauração passa pela autoridade de identidade, que registra a geração de sessão e confirma a aplicação. A tela de acompanhamento recebe apenas os dados necessários para orientar o titular.

O ponto delicado é a distância entre aprovar e efetivar. Se uma decisão expira ou é revogada enquanto aguarda processamento, o consumidor deve rejeitá-la. Uma mensagem duplicada deve recuperar o resultado existente. A versão do caso atua como precondição, enquanto a geração de sessão impede que comandos antigos continuem válidos após a recuperação.

Uma implementação real exigiria autenticação dos revisores, autorização por função, armazenamento protegido e auditoria de acesso. O laboratório demonstra as transições com nomes fictícios e SQLite; não oferece essas garantias de identidade ou operação.

ENGENHARIA EXPLICADA

A transição valida versão, prazo e responsável

PYTHON / PROPOSTA AUTORAL

```python
def transition(self, case_id, expected, action, actor, now):
    if type(now) is not int or type(expected) is not int:
        raise ValueError("integer_clock_and_revision_required")
    with sqlite3.connect(self.path, isolation_level=None) as db:
        db.execute("BEGIN IMMEDIATE")
        row = db.execute("SELECT * FROM cases WHERE id=?", (case_id,)).fetchone()
        if row is None:
            raise KeyError(case_id)
        _, state, revision, expires, owner, verifier, epoch = row
        if expected != revision:
            raise ValueError("stale_revision")
        if now >= expires:
            raise ValueError("expired")
        if action == "verify" and state == "review" and actor != owner:
            state, verifier = "verified", actor
        elif (action == "restore" and state == "verified"
              and actor not in (owner, verifier)):
            state, epoch = "restored", epoch + 1
        else:
            raise ValueError("invalid_transition_or_actor")
        db.execute("UPDATE cases SET state=?, revision=?, verifier=?, epoch=? "
                   "WHERE id=?", (state, revision + 1, verifier, epoch, case_id))
        db.commit()
    return self.read(case_id)
```

Trecho do laboratório sintético; execução verificada no arquivo completo.

A leitura e a atualização ocorrem sob uma transação de escrita. A revisão esperada evita que dois operadores apliquem decisões sobre a mesma versão. O prazo é conferido antes de qualquer transição, inclusive restauração. A identidade do ator precisa ser diferente do solicitante e, na aplicação final, também do revisor anterior.

No laboratório esses atores são strings fornecidas pelo teste, portanto não constituem autenticação. O valor está na regra de domínio, que permanece visível e verificável. Em uma implementação real, ator e permissões seriam derivados de uma credencial confiável, e cada decisão deixaria auditoria apropriada. A função não valida biometria.

ENGENHARIA EXPLICADA

Datomic: a versão como precondição explícita

CLOJURE / PROPOSTA AUTORAL

```clojure
(defn restore-tx [case-id expected-version now decision]
  ;; Esboço proposto de dados transacionais.
  ;; O chamador autenticado precisa validar a autorização.
  (when-not (= :verified (:state decision))
    (throw (ex-info "case-not-verified" {})))
  (when (>= now (:expires-at decision))
    (throw (ex-info "decision-expired" {})))
  (let [new-version (inc expected-version)
        identity-id (:identity-id decision)
        old-epoch (:recovery-epoch decision)]
    [[:db/cas case-id
      :recovery/version expected-version new-version]
     [:db/cas case-id
      :recovery/state :verified :restored]
     [:db/cas identity-id
      :identity/recovery-epoch old-epoch (inc old-epoch)]
     {:db/id "audit-event"
      :audit/kind :recovery-restored
      :audit/case case-id
      :audit/decision-id (:decision-id decision)
      :audit/actor-id (:actor-id decision)
      :audit/at now}]))
```

Esboço Datomic/Clojure, atributos propostos; não executado.

O esboço usa comparações transacionais para impedir aplicar uma decisão sobre versão, estado ou geração já alterados. O formato de dados e os atributos são propostos. O objetivo é discutir atomicidade entre mudança do caso, mudança da identidade e registro de auditoria, sem alegar que o Nubank utiliza esse schema.

A checagem de prazo feita antes de enviar a transação não basta se houver uma espera longa. A implementação definitiva precisaria garantir a validade no ponto de aplicação, conforme as capacidades e o modelo transacional escolhidos. O trecho não foi executado em Datomic; essa lacuna é um requisito de revisão, não uma garantia implícita.

Datomic - Transaction Data

ANÁLISE E PROPOSTA EXTERNA

Evidências têm ciclo de vida diferente do caso



O identificador de um caso pode ser necessário para atendimento por mais tempo que uma imagem coletada para uma verificação específica. Eu modelaria referências e políticas de retenção separadamente, evitando copiar material sensível para logs, eventos gerais ou ferramentas de acompanhamento. Um evento de etapa concluída deveria carregar a decisão, sua proveniência e o escopo autorizado, não um pacote de documentos pessoais anexado indiscriminadamente.

O contrato proposto incluiria case_id, revision, status, expires_at e reason_category, com vocabulário controlado. Detalhes destinados ao usuário seriam diferentes dos detalhes de investigação. Uma categoria compreensível, como “precisamos confirmar uma informação”, pode orientar sem revelar limiares ou sinais usados para detectar abuso. O contrato não deve prometer um prazo que a operação ainda não consegue cumprir; deve distinguir estimativa, compromisso e ausência de previsão.

Reabrir um caso também exige decisão explícita. Reutilizar uma aprovação antiga pode ser inseguro após troca de aparelho ou mudança de risco. Criar uma nova revisão preserva o vínculo com o histórico sem tratar a evidência anterior como autorização permanente. No laboratório, expiração rejeita novas transições; não existe exclusão automática nem política regulatória. Essas capacidades seriam definidas com responsáveis por identidade, privacidade e atendimento antes de qualquer uso com pessoas reais.

ENGENHARIA EXPLICADA

Testar a regra que um atalho tentaria contornar

PYTHON / PROPOSTA AUTORAL

```python
# Validação dos limites de autoridade e validade.
def test_restore_requires_two_distinct_reviewers(self):
    verified = self.store.transition("case", 0, "verify", "reviewer-a", 101)
    self.assertEqual(verified["state"], "verified")
    with self.assertRaises(ValueError):
        self.store.transition("case", 1, "restore", "reviewer-a", 102)
    result = self.store.transition("case", 1, "restore", "reviewer-b", 102)
    self.assertEqual((result["state"], result["epoch"]), ("restored", 1))

def test_owner_cannot_verify(self):
    with self.assertRaises(ValueError):
        self.store.transition("case", 0, "verify", "customer", 101)
    self.assertEqual(self.store.read("case")["revision"], 0)

def test_expired_case_cannot_restore(self):
    self.store.transition("case", 0, "verify", "a", 101)
    with self.assertRaisesRegex(ValueError, "expired"):
        self.store.transition("case", 1, "restore", "b", 400)
```

Trecho do laboratório sintético; execução verificada no arquivo completo.

Os testes atacam três atalhos: o mesmo revisor aprovar e restaurar, o solicitante verificar a própria identidade e uma decisão ser aplicada após o prazo. Cada rejeição deve manter o estado anterior. O teste não pergunta se a tela mostrou uma mensagem; verifica que a mutação proibida não aconteceu.

Essas restrições são fixtures de governança para a simulação. Uma equipe real poderia adotar outra política de aprovação, desde que a regra fosse explícita e testada. O aspecto importante para uma revisão Staff é identificar quem possui autoridade para cada transição e como essa autoridade permanece válida diante de repetição e atraso.

ENGENHARIA EXPLICADA

Retomar um caso não pode reaplicar a restauração

PYTHON / PROPOSTA AUTORAL

```python
def test_stale_revision_cannot_repeat_verification(self):
    self.store.transition("case", 0, "verify", "a", 101)
    with self.assertRaisesRegex(ValueError, "stale_revision"):
        self.store.transition("case", 0, "verify", "b", 102)

def test_reopen_preserves_case_and_epoch(self):
    self.store.transition("case", 0, "verify", "a", 101)
    reopened = Recovery(self.path)
    self.assertEqual(reopened.read("case")["revision"], 1)
    reopened.transition("case", 1, "restore", "b", 102)
    self.assertEqual(Recovery(self.path).read("case")["epoch"], 1)

def test_cannot_skip_review_or_restore_twice(self):
    with self.assertRaises(ValueError):
        self.store.transition("case", 0, "restore", "b", 101)
    self.store.transition("case", 0, "verify", "a", 101)
    self.store.transition("case", 1, "restore", "b", 102)
    with self.assertRaises(ValueError):
        self.store.transition("case", 2, "restore", "c", 103)
    self.assertEqual(self.store.read("case")["epoch"], 1)
```

Trecho do laboratório sintético; execução verificada no arquivo completo.

A reabertura do objeto Recovery utiliza o mesmo arquivo persistido, comprovando que versão e geração não dependem da memória da instância anterior. Os outros testes rejeitam uma revisão vencida, uma tentativa de pular a verificação e uma segunda restauração.

Os sete testes do arquivo passaram, incluindo relógio inválido e tipo da revisão. Eles cobrem a máquina local e suas precondições, mas não executam concorrência com operadores autenticados, filas reais ou serviços de identidade. Também não medem qualidade biométrica. O próximo experimento de uma implementação seria injetar entrega duplicada e revogação entre aprovação e aplicação, mantendo o mesmo critério: nenhuma alteração de acesso sem decisão válida.

ANÁLISE E PROPOSTA EXTERNA

Medir a fila sem expor quem está nela



A métrica central seria o tempo até uma recuperação legítima concluída, dividido por etapa e por motivo técnico de encaminhamento. A média isolada esconderia pessoas que permanecem presas durante dias enquanto casos simples terminam rapidamente. Eu acompanharia cauda de espera, idade do caso mais antigo e taxa de recontato, além da proporção de solicitações que voltam à mesma etapa sem nova informação útil.

Uma segunda dimensão mede proteção: restaurações posteriormente questionadas, uso de decisões expiradas rejeitado e comandos de geração antiga bloqueados. Esses números não devem ser usados para recompensar o time por negar tudo, nem a velocidade deve incentivar aprovar casos sem evidência suficiente. O painel precisa mostrar os dois lados da decisão e permitir revisão qualitativa de amostras, com acesso controlado aos detalhes.

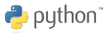
Eu evitaria identificar pessoas, dispositivos específicos ou revisores em rótulos de métricas agregadas. Investigações individuais ficariam em trilhas de auditoria com finalidade e permissão. Para acessibilidade, a segmentação útil seria por barreira observada, como câmera indisponível ou dificuldade de leitura, sem transformar a coleta em classificação invasiva. O resultado esperado é localizar o gargalo operacional e técnico com informação suficiente para agir, preservando o mínimo necessário sobre cada titular.

ANÁLISE E PROPOSTA EXTERNA

Implantar exceções com atendimento preparado







A primeira entrega não seria um novo algoritmo de identidade. Seria detectar a falha técnica mais cedo e impedir uma sequência inútil de tentativas de captura. Uma mensagem poderia explicar que o componente não está disponível e encaminhar para a rota assistida existente. Esse comportamento deve ser validado com pessoas usando câmera quebrada, permissão negada e aparelhos de capacidade limitada, incluindo leitura por tecnologia assistiva.

Depois, eu implantaria o acompanhamento do caso em uma coorte controlada. O atendimento precisaria de acesso ao mesmo estado que o cliente vê, com diferença apenas no nível de detalhe permitido. Uma migração que crie mais estados sem preparar os operadores pode aumentar transferências entre equipes. Por isso, cada novo estado teria uma ação operacional, um responsável e um critério de saída compreensível.

A restauração automatizada ou a integração com identidade viria somente após validar autorização, expiração e revogação em ambiente de teste. O recuo manteria casos abertos consultáveis e impediria reaplicar decisões já confirmadas. Eu definiria um limite operacional de entrada para não lançar uma jornada que coleta solicitações mais rápido do que a equipe consegue revisá-las. A experiência precisa ser sustentável para o cliente e para quem resolve a exceção.

ANÁLISE E PROPOSTA EXTERNA

Acessibilidade é parte da superfície de segurança



Instruções ambíguas podem fazer uma pessoa repetir movimentos, procurar ambientes inadequados ou compartilhar informações em canais errados. Eu trataria a clareza da jornada como um controle de segurança: explicar o que será solicitado, onde a informação é enviada e qual etapa vem depois. Isso não significa revelar regras antifraude. Significa reduzir a diferença entre uma ação legítima e uma improvisação que expõe o cliente.

Na interface proposta em Flutter, os estados precisariam de semântica acessível, ordem de foco estável e mensagens anunciadas sem depender exclusivamente de cor ou animação. Uma falha da câmera não deveria desaparecer tão rápido que um leitor de tela não conseguisse apresentá-la. A validação incluiria texto ampliado e interrupção do app, porque a recuperação costuma acontecer em condições menos confortáveis do que o fluxo ideal de cadastro.

Quanto ao desempenho, eu reduziria coleta repetida e carregamentos que não mudam a decisão. O cliente poderia consultar estado com frequência moderada, enquanto atualizações relevantes seriam propagadas de forma recuperável. A prioridade não é acelerar a aprovação a qualquer custo, mas eliminar espera que não agrega verificação. Se uma etapa demorar por dependência humana, o app deve admitir isso e fornecer acompanhamento fiel, evitando um carregamento infinito que parece falha técnica.

Flutter - Accessibility

ANÁLISE E PROPOSTA EXTERNA

ADR: processo de recuperação explícito, com autorização limitada







A decisão proposta é representar a recuperação como processo de domínio, separado do login cotidiano. Uma alternativa seria embutir todas as exceções no mesmo fluxo de captura. Isso simplificaria a navegação inicial, mas tornaria difícil compreender tentativas que atravessam atendimento, troca de dispositivo e revisão. Outra alternativa seria resolver tudo manualmente por mensagens, com flexibilidade alta e pouca consistência de estado, autorização e acompanhamento.

Eu escolheria um processo explícito porque ele permite declarar a validade de cada decisão e limitar o que uma etapa autoriza. Verificar um documento não precisa significar liberar todas as operações financeiras. O acesso restaurado pode exigir outras verificações de risco conforme a política aplicável, sem transformar a máquina de estados em um atalho universal. Essas políticas seriam decisões de produto e segurança, não deduções do relato público.

O custo aceito é manter contratos entre identidade, revisão e atendimento. O desenho também precisa evitar burocracia: estados só devem existir quando mudam responsabilidade, autorização ou informação útil. Reabriria o ADR se a maioria das exceções pudesse ser resolvida por diagnóstico local confiável, se a operação manual se tornasse o gargalo dominante ou se novas capacidades de dispositivo reduzissem a necessidade de coleta. A escolha seria revisada por evidência de resolução legítima e proteção, não por quantidade de etapas.

ANÁLISE E PROPOSTA EXTERNA

Execução EM: capacidade de resolver, não apenas receber casos







Como EM, eu começaria com uma oficina de casos reais já autorizados para análise, envolvendo atendimento, identidade, risco, mobile e acessibilidade. A saída seria uma matriz de barreiras, responsáveis e caminhos existentes. O relato da câmera quebrada orientaria uma categoria, sem ser usado para afirmar prevalência. Eu separaria mudanças de clareza, mudanças de processo e mudanças de autorização, porque elas possuem riscos e dependências diferentes.

O plano teria entregas pequenas: diagnóstico compreensível, encaminhamento com contexto, estado compartilhado e integração transacional com identidade. Cada entrega precisaria de demonstração com falhas, revisão operacional e capacidade de acompanhamento. Como Staff, eu me concentraria na validade da decisão ao atravessar fronteiras, na geração de sessão e na persistência das transições. O laboratório serve como artefato de conversa técnica, não como substituto da revisão de segurança.

O valor poderia ser calculado por minutos de atendimento evitados, redução de recontato e diminuição do tempo de indisponibilidade para titulares legítimos. Qualquer ganho de retenção exigiria comparação apropriada, e custos de fraude ou revisão adicional seriam descontados. A pergunta de liderança seria: “Estamos removendo uma barreira que não agrega segurança ou apenas deslocando trabalho para outra equipe?” A resposta precisa aparecer na resolução do caso e nos indicadores de proteção, não somente na conversão da tela.

ANÁLISE E PROPOSTA EXTERNA

Informação suficiente, exposição deliberada

03

Notificações Push do App Nubank esquece do valor (sugestões)

Captura da fonte pública consultada em 23/10/2026. Relato não equivale a incidente reproduzido.



Print publicado por cliente na NuCommunity; contexto e limitações na fonte.

A sugestão pública reúne exemplos de notificações de proventos, cashback e boletos que, segundo o autor, não informavam os valores. As imagens anexadas dão contexto à reclamação, mas não revelam todas as preferências de privacidade nem a versão atualmente disponível. A oportunidade não é concluir que qualquer notificação deveria mostrar dinheiro. É construir uma política de conteúdo que ajude a decidir e respeite o contexto de exposição escolhido pelo cliente.

Eu começaria perguntando qual tarefa a mensagem permite concluir. Um aviso pode confirmar que algo aconteceu, pedir uma ação ou alertar sobre um prazo. Cada finalidade exige campos diferentes. Um valor pode reduzir uma abertura desnecessária do aplicativo; em uma tela bloqueada compartilhada com outras pessoas, o mesmo valor pode revelar informação que o titular preferia manter privada. A experiência precisa fazer essa escolha de forma compreensível, não acidental.

A proposta deste capítulo combina contrato de evento, catálogo de mensagens, preferência versionada e aplicação tardia da política de privacidade. O laboratório enfileira fatos sintéticos, altera a preferência antes do envio e verifica o conteúdo final. Ele não se conecta a provedores de push. Seu objetivo é mostrar que o momento de avaliar a política faz parte da correção do sistema, tanto quanto a frase que aparece na tela.

NuCommunity - Notificações Push do App Nubank esquece do valor (sugestões)

ANÁLISE E PROPOSTA EXTERNA

O fato financeiro não é o texto da notificação



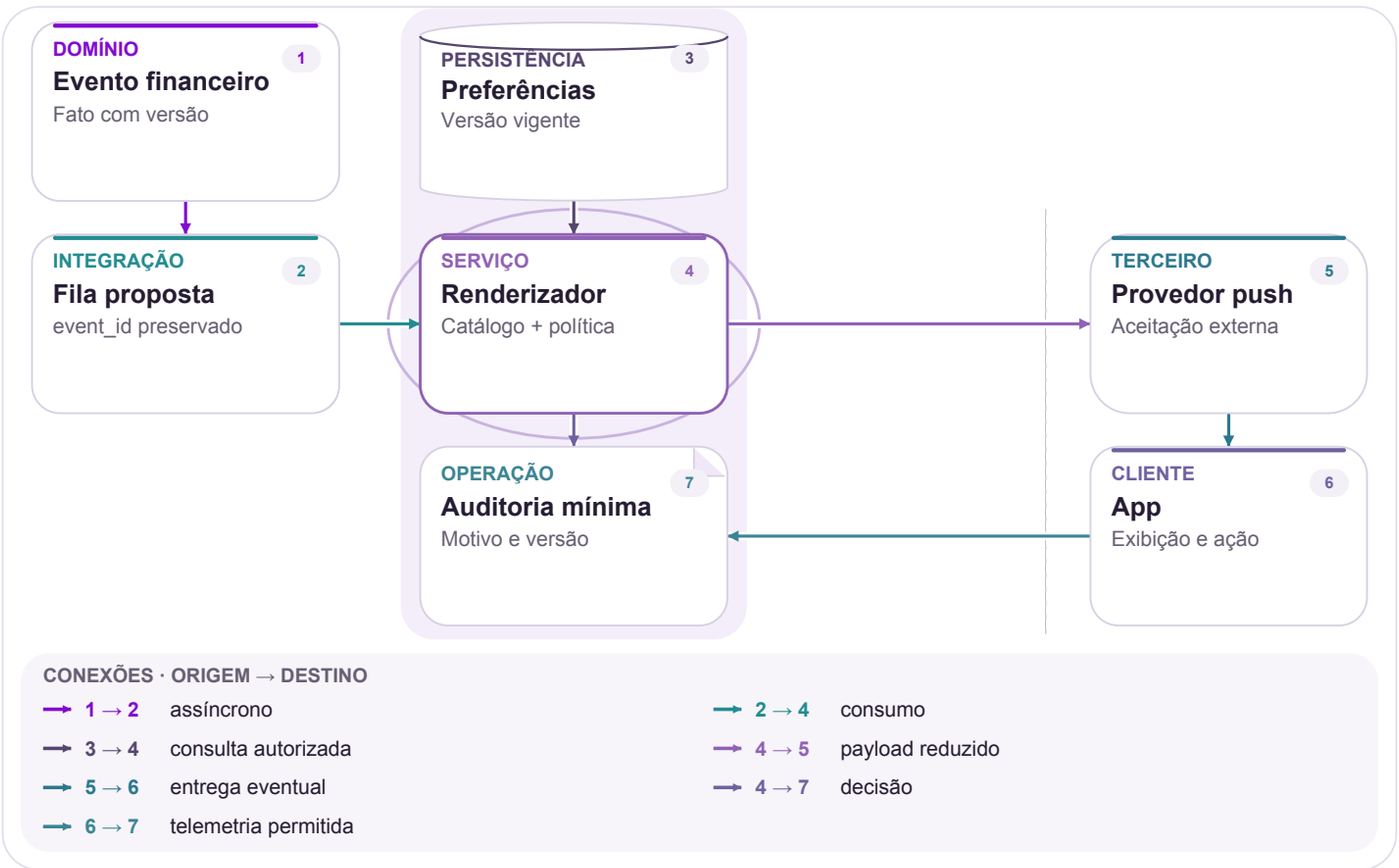
Um evento de domínio deveria descrever o que aconteceu sem incorporar uma frase pronta para determinado idioma ou dispositivo. O catálogo de comunicação interpreta esse fato, decide se existe mensagem relevante e aplica a política de apresentação. Separar essas responsabilidades permite corrigir um texto sem mudar a semântica do evento e evoluir privacidade sem exigir que todos os produtores financeiros conheçam as particularidades de cada plataforma móvel.

O contrato proposto contém event_id, versão do schema, tipo de evento, titular e dados estritamente necessários. A preferência pertence a outro ciclo de vida: ela pode mudar enquanto a mensagem aguarda na fila. Por isso, guardar o texto definitivo com valor exposto no instante de produzir o evento seria insuficiente. A proposta consulta a política vigente perto da entrega e inclui sua versão na evidência operacional.

Há pelo menos quatro estados de comunicação: elegível, suprimida, aceita pelo provedor e exibida ou aberta quando essa informação estiver disponível. Aceitação não comprova leitura, e ausência de abertura não comprova falha. O laboratório modela apenas a aceitação local e a supressão. Uma decisão de produto precisa evitar tratar “enviado” como certeza de que o cliente tomou conhecimento de um fato financeiro ou recebeu oportunidade efetiva de agir.

ARQUITETURA PROPOSTA • FRONTEIRAS EXPLÍCITAS

Renderizar perto da entrega e guardar a decisão



Desenho autoral proposto; não representa a arquitetura interna do Nubank.

O renderizador é a fronteira em que um fato financeiro se transforma em texto potencialmente visível fora do aplicativo. Ele precisa conhecer a preferência atual e a versão do catálogo. Se a consulta de preferência estiver indisponível, a política proposta é degradar para conteúdo sem valor ou suprimir, conforme a finalidade. Não se presume consentimento porque o serviço de preferências está fora do ar.

A fila preserva identidade e versão do evento; duplicatas não devem gerar mensagens repetidas indefinidamente. A auditoria registra a decisão de comunicação e seus parâmetros de política, evitando armazenar o corpo sensível quando uma referência controlada for suficiente. O provedor externo permanece uma fronteira importante: depois de aceitar um payload, pode não ser possível retirá-lo.

Kafka seria uma opção de transporte assíncrono, mas não substituiria a deduplicação do efeito junto ao provedor. O laboratório mantém fila e deduplicação em memória para isolar as regras de conteúdo, sem alegar durabilidade ou entrega real.

ENGENHARIA EXPLICADA

Aplicar privacidade no último ponto controlável

PYTHON / PROPOSTA AUTORAL

```python
def deliver(self, event_id):
    if event_id in self.sent:
        return self.sent[event_id]
    event = json.loads(self.events[event_id])
    pref = self.preferences.get(event["user"],
                                dict(show_values=False, enabled=True, version=0))
    if not pref["enabled"]:
        return {"status": "suppressed"}
    # A política é aplicada agora, não no instante de enfileirar.
    message = {"title": "Atualização financeira",
               "body": "Consulte os detalhes no aplicativo.",
               "preference_version": pref["version"]}
    if pref["show_values"]:
        whole, cents = divmod(event["amount_cents"], 100)
        message["body"] = f"Crédito disponível: R$ {whole},{cents:02d}."
    # Aceitação local pelo provedor simulado, sem garantia de leitura.
    self.sent[event_id] = {"status": "accepted", "message": message}
    return self.sent[event_id]
```

Trecho do laboratório sintético; execução verificada no arquivo completo.

O método primeiro reconhece uma entrega já aceita no modelo e evita repeti-la. Para uma mensagem ainda pendente, consulta a preferência vigente. O padrão oculta o valor; a exposição exige habilitação explícita. A representação monetária usa divmod sobre centavos inteiros, sem arredondamento binário de float.

O retorno “accepted” é apenas a resposta do provedor simulado. Depois desse ponto, mudar a preferência não recolhe uma mensagem já entregue, e o laboratório não tenta fazer essa promessa. Na implementação real, também seria necessário limitar o tempo entre ler a preferência e enviar, tratar indisponibilidade dessa leitura e documentar a fronteira de revogação.

ENGENHARIA EXPLICADA

Clojure: catálogo de mensagens como dados

CLOJURE / PROPOSTA AUTORAL

```clojure
(def templates
  {:income {:title "Atualização financeira"
            :private-body "Consulte os detalhes no aplicativo."}
   :cashback {:title "Cashback disponível"
              :private-body "Veja a atualização no aplicativo."}})

(defn cents-label [cents]
  (str "R$ " (quot cents 100) ","
       (format "%02d" (mod cents 100))))

(defn render-message [event preference]
  ;; Catálogo e contrato propostos; sem envio neste exemplo.
  (when-not (= 1 (:schema-version event))
    (throw (ex-info "unsupported-schema" {})))
  (when-not (true? (:enabled? preference))
    (throw (ex-info "communication-suppressed" {})))
  (let [template (get templates (:kind event))
        amount (:amount-cents event)]
    (when-not (and template (integer? amount) (<= 0 amount))
      (throw (ex-info "invalid-event" {})))
    {:title (:title template)
     :body (if (true? (:show-values? preference))
             (str "Crédito disponível: " (cents-label amount))
             (:private-body template))
     :policy-version (:version preference)}))
```

Catálogo e função propostos; Clojure não executado.

O exemplo torna o catálogo inspecionável e mantém a decisão de privacidade em uma função separada do transporte. Isso facilita testes de linguagem, valores e combinações de preferência. Os nomes de campos e as frases são uma proposta deste livro, não contratos reais do Nubank.

Um catálogo de produção teria governança de tradução, acessibilidade e finalidade, além de revisão de exposição. A função não resolve concorrência entre revogação e envio, nem substitui autorização para consultar o titular. Ela também não deve receber valores monetários não validados de fontes arbitrárias. O trecho é didático em Clojure e não foi compilado ou executado.

ANÁLISE E PROPOSTA EXTERNA

Versionar conteúdo sem quebrar consumidores antigos



O schema do evento e a versão da mensagem resolvem problemas diferentes. O primeiro define campos e significado; a segunda identifica como esses campos foram apresentados. Uma mudança de título pode ser compatível com o mesmo evento, enquanto a introdução de um novo estado financeiro pode exigir revisão do consumidor. Eu registraria essas versões separadamente para permitir investigação sem reconstruir a comunicação a partir do código que está em produção hoje.

No transporte, event_id precisa continuar estável quando o produtor repete a publicação. A deduplicação deve verificar se o mesmo identificador reaparece com outro conteúdo, pois isso sinaliza erro de contrato. O laboratório rejeita essa reutilização. Em uma fila persistente real, eu definiria janela de retenção, particionamento e tratamento de eventos não reconhecidos, garantindo que uma atualização de produtor não silencie mensagens críticas sem alertar o responsável.

A preferência também requer versão para explicar qual política foi aplicada. Isso não autoriza guardar o histórico completo de informações pessoais dentro do evento. O registro operacional mínimo pode apontar para uma política e uma decisão, com acesso restrito aos detalhes quando necessário. Na consulta do cliente, a configuração precisa ter significado estável: “ocultar valores nas notificações” não deveria ocultar apenas algumas categorias por acidente. As exceções, se houver, devem ser explícitas e justificadas.

ENGENHARIA EXPLICADA

Rejeitar ambiguidade antes de produzir texto

PYTHON / PROPOSTA AUTORAL

```python
def configure(self, user, show_values=False, enabled=True):
    if type(show_values) is not bool or type(enabled) is not bool:
        raise ValueError("boolean_preferences_required")
    old = self.preferences.get(user, {"version": 0})
    self.preferences[user] = dict(show_values=show_values, enabled=enabled,
                                  version=old["version"] + 1)

def enqueue(self, event):
    if set(event) != {"id", "user", "version", "kind", "amount_cents"}:
        raise ValueError("schema")
    if (type(event["version"]) is not int or event["version"] != 1
            or event["kind"] not in ("cashback", "income")):
        raise ValueError("unsupported")
    cents = event["amount_cents"]
    if type(cents) is not int or cents < 0:
        raise ValueError("money")
    payload = json.dumps(event, sort_keys=True)
    old = self.events.get(event["id"])
    if old is not None and old != payload:
        raise ValueError("id_conflict")
    self.events[event["id"]] = payload
```

Trecho do laboratório sintético; execução verificada no arquivo completo.

A entrada valida o conjunto de campos, a versão inteira, a categoria e o tipo monetário. Reutilizar event_id com outro conteúdo gera conflito. As preferências exigem booleanos reais: a string false não pode ser interpretada como autorização para expor valores. Uma configuração válida incrementa a versão da preferência, permitindo reconhecer que a decisão de comunicação ocorreu sob outra política.

O modelo é deliberadamente estrito: um produtor que acrescenta campos precisa negociar o contrato, em vez de depender de aceitação silenciosa. Essa escolha favorece o laboratório; uma plataforma real poderia adotar compatibilidade extensível com regras bem definidas. A fila em memória não resiste a reinício, portanto os testes desta página verificam semântica, não confiabilidade do transporte.

ENGENHARIA EXPLICADA

A preferência muda entre produzir e entregar

PYTHON / PROPOSTA AUTORAL

```python
def test_privacy_changed_while_queued_is_respected(self):
    self.n.configure("u", show_values=True)
    self.n.enqueue(self.event)
    self.n.configure("u", show_values=False)
    self.assertNotIn("123", str(self.n.deliver("e1")))

def test_opt_out_suppresses_delivery(self):
    self.n.enqueue(self.event)
    self.n.configure("u", enabled=False)
    self.assertEqual(self.n.deliver("e1"), {"status": "suppressed"})
    self.assertEqual(len(self.n.sent), 0)

def test_replay_is_identical(self):
    self.n.enqueue(self.event)
    first = self.n.deliver("e1")
    self.n.enqueue(self.event)
    self.assertEqual(self.n.deliver("e1"), first)
    self.assertEqual(len(self.n.sent), 1)

def test_payload_conflict_is_rejected(self):
    self.n.enqueue(self.event)
    with self.assertRaises(ValueError):
        self.n.enqueue(dict(self.event, amount_cents=999))
```

Trecho do laboratório sintético; execução verificada no arquivo completo.

O teste de privacidade altera a preferência depois de enfileirar o evento. Essa ordem reproduz a janela que um teste de tela isolado não encontra. Outro cenário desativa a comunicação e verifica que a mensagem não entra no conjunto de entregas aceitas. Os testes de repetição e conflito completam a proteção sem depender de um provedor externo.

O arquivo passou em oito testes, incluindo a rejeição de preferências e versões com tipos incorretos. A evidência não inclui apresentação na tela bloqueada, leitores de tela ou comportamento de sistemas operacionais. Essas camadas precisariam de testes de dispositivo e pesquisa. O laboratório demonstra apenas que o renderizador local respeita a preferência consultada antes de sua entrega simulada.

ANÁLISE E PROPOSTA EXTERNA

Medir utilidade sem otimizar curiosidade



A taxa de abertura é um sinal ambíguo para notificações financeiras. Uma mensagem clara pode reduzir aberturas porque já respondeu à pergunta; uma mensagem incompleta pode aumentá-las porque obriga o cliente a investigar. Eu não adotaria abertura como objetivo isolado. A avaliação começaria por compreensão: a pessoa consegue identificar o acontecimento, saber se precisa agir e reconhecer onde consultar detalhes sem interpretar incorretamente a situação?

As métricas operacionais propostas separariam eventos elegíveis, mensagens suprimidas por preferência, aceitação do provedor e falhas de renderização. Um aumento de supressão pode ser o comportamento correto após uma mudança de preferência, não uma regressão. Alertas procurariam desvios inesperados por categoria e versão, com vocabulário de baixa cardinalidade. Valores financeiros, destinatários e textos integrais não seriam rótulos de métricas.

No experimento de produto, eu mediria consultas motivadas apenas pela ausência de informação, contatos associados à mensagem e respeito à preferência. Participantes poderiam explicar a mensagem sem executar uma operação, reduzindo risco do teste. A experiência também precisa ser avaliada em tela bloqueada, fonte ampliada e leitores de tela. O sucesso seria entregar informação suficiente com exposição deliberada, e não maximizar quantidade de mensagens ou tempo gasto no aplicativo.

ANÁLISE E PROPOSTA EXTERNA

Migrar um catálogo sem inundar o cliente



O rollout começaria com um inventário de tipos de notificação, seus produtores, campos disponíveis e finalidades. Antes de mudar frases, eu identificaria quais mensagens são transacionais, quais são lembretes e quais são comunicação opcional. Essa classificação orienta preferência, fallback e urgência. O registro público serve de ponto de partida para algumas categorias, sem justificar uma alteração indiscriminada em todas as mensagens do app.

Eu validaria o novo renderizador em modo comparativo: produziria a versão candidata sem enviá-la e verificaria divergências de campos e política com dados sintéticos ou dados devidamente autorizados e minimizados. Em seguida, uma coorte pequena receberia apenas uma categoria revisada. Limites por destinatário e por evento impediriam que uma falha de integração gerasse repetição em massa. O atendimento teria exemplos das mensagens antigas e novas para reconhecer dúvidas de transição.

O rollback precisa preservar event_id e registros de entrega. Reprocessar o backlog com o catálogo anterior não pode reenviar tudo que já foi aceito. Mudanças de privacidade também não podem ser revertidas silenciosamente por voltar o binário. Eu trataria preferência como contrato persistente do cliente, independente da versão do renderizador. A expansão dependeria de evidências de compreensão e ausência de exposição inesperada, não somente de estabilidade técnica.

ANÁLISE E PROPOSTA EXTERNA

O corpo da mensagem atravessa uma nova fronteira



Ao sair do aplicativo, o texto pode aparecer em tela bloqueada, relógio, histórico do sistema e infraestrutura de um provedor. Por isso, a minimização deve ocorrer antes do envio, e não somente com uma camada visual no app. Ocultar um valor na tela interna não retira esse valor de um payload já transmitido. A proposta manteria o conteúdo sensível fora do corpo quando a preferência não autorizasse sua exposição.

Links de ação também precisam de desenho cuidadoso. Um identificador embutido não pode funcionar como credencial para consultar uma transação. Ao abrir a mensagem, o app deve autenticar o contexto necessário e resolver o destino com autorização. Eu evitaria colocar informação pessoal em URLs e verificaria como o link se comporta depois de logout, troca de aparelho ou recuperação de sessão. Um destino expirado precisa orientar sem revelar dados antigos.

Quanto ao custo, a consulta de preferência em cada entrega pode adicionar latência e carga. A decisão entre leitura direta e cache exige um limite explícito de desatualização e uma política conservadora quando esse limite não puder ser garantido. Otimizar esse caminho sem modelar revogação poderia tornar o sistema rápido e incorreto. O desempenho seria avaliado junto com o tempo de propagação da preferência, mantendo esses dois objetivos visíveis.

ANÁLISE E PROPOSTA EXTERNA

ADR: renderização central com responsabilidade distribuída



A decisão proposta centraliza as regras transversais de conteúdo e privacidade, mas mantém a propriedade do significado do evento no domínio produtor. A alternativa de cada equipe escrever seu próprio push facilita entregas locais e tende a criar variações de linguagem, campos e política. A alternativa de um serviço central decidir também todos os fatos financeiros concentraria responsabilidade demais e enfraqueceria a clareza do domínio.

Eu escolheria um catálogo governado e contratos por evento. O time produtor garantiria a semântica e a identidade; a plataforma de comunicação garantiria renderização, política e integração com provedores; produto e design garantiriam compreensão e acessibilidade. Essa divisão teria testes contratuais em ambos os lados. Uma mensagem desconhecida não deveria ser convertida automaticamente em texto genérico se isso escondesse uma ação importante do cliente.

O custo aceito é uma coordenação adicional para introduzir novos tipos de evento. O benefício pretendido é tornar mudanças de privacidade e conteúdo coerentes sem reimplementar a regra em cada domínio. Reabriria o ADR se o catálogo virasse gargalo de entrega ou se categorias com garantias diferentes exigissem isolamento operacional. A solução não é um único template para tudo: é um mecanismo comum para aplicar escolhas explícitas, com propriedade clara sobre cada mensagem e sua finalidade.

ANÁLISE E PROPOSTA EXTERNA

EM e Staff: governar comunicação como produto



Como EM, eu formaria um grupo pequeno com comunicação, mobile, domínio financeiro e privacidade para revisar as categorias de maior fricção. A entrega inicial seria um catálogo com dono, finalidade, campos obrigatórios e comportamento quando dados faltam. Cada alteração teria exemplos reais autorizados ou sintéticos e um teste de compreensão. Isso evita transformar o projeto em uma sequência de ajustes de texto sem resolver a origem das inconsistências.

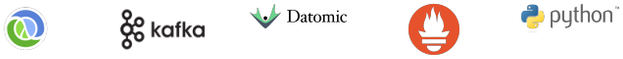
Como Staff, eu revisaria a identidade dos eventos, a fronteira de aplicação da preferência e a diferença entre aceitação do provedor e efeito percebido. Também exigiria uma estratégia para reprocessamento que preserve deduplicação. A contribuição técnica se torna especialmente valiosa quando várias equipes publicam eventos e uma única falha de contrato pode atingir comunicações de natureza diferente. O laboratório oferece uma base pequena para discutir essas invariantes sem alegar infraestrutura interna.

O valor seria expresso em tempo de consulta evitado, redução de contatos e compreensão, com o custo de operação e revisão incluído. Um ganho de abertura não seria apresentado como benefício se decorresse de mensagem incompleta. A pergunta que eu levaria à entrevista é: “Como provamos que a preferência aplicada a uma mensagem era a correta no instante em que ainda podíamos controlar sua exposição?” Essa resposta conecta produto, engenharia e responsabilidade operacional.

ANÁLISE E PROPOSTA EXTERNA

O subtotal precisa admitir o que não conhece

04



Open Finance: Cofrinhos do Mercado Pago não aparecem no Nubank

Captura da fonte pública consultada em 23/10/2026. Relato não equivale a incidente reproduzido.

O relato de 3 de outubro de 2026 distingue duas observações: saldo e movimentação de outra instituição apareciam no Nubank, mas posições dos Cofrinhos não apareciam no agregador de investimentos. O próprio participante não atribui a causa com certeza. Isso impede concluir, a partir da postagem, que houve erro no Nubank ou obrigação de disponibilizar aquele recurso. A evidência sustenta investigar cobertura, consentimento, disponibilidade e interpretação dos dados.

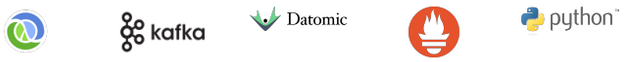
Uma agregação financeira tem dois produtos: os valores apresentados e a explicação de quão completa é aquela visão. Somar apenas o que chegou pode ser matematicamente correto e semanticamente enganoso se o resultado parecer representar todo o patrimônio. Minha proposta seria manter estado por instituição e recurso, com proveniência, atualização e motivo de ausência. Não receber uma posição não deve ser automaticamente interpretado como posição de valor zero.

O laboratório deste capítulo modela essa distinção com estados complete, unsupported e unavailable, além de recursos ausentes ou desatualizados. A soma é explicitamente um subtotal conhecido quando há lacunas. Não há conexão com APIs bancárias reais. A demonstração torna verificável uma decisão de produto importante: informação parcial pode continuar útil, desde que o aplicativo não apresente certeza maior do que a evidência disponível.

NuCommunity - Open Finance: Cofrinhos do Mercado Pago não aparecem no Nubank

ANÁLISE E PROPOSTA EXTERNA

Completude é uma propriedade do conjunto solicitado



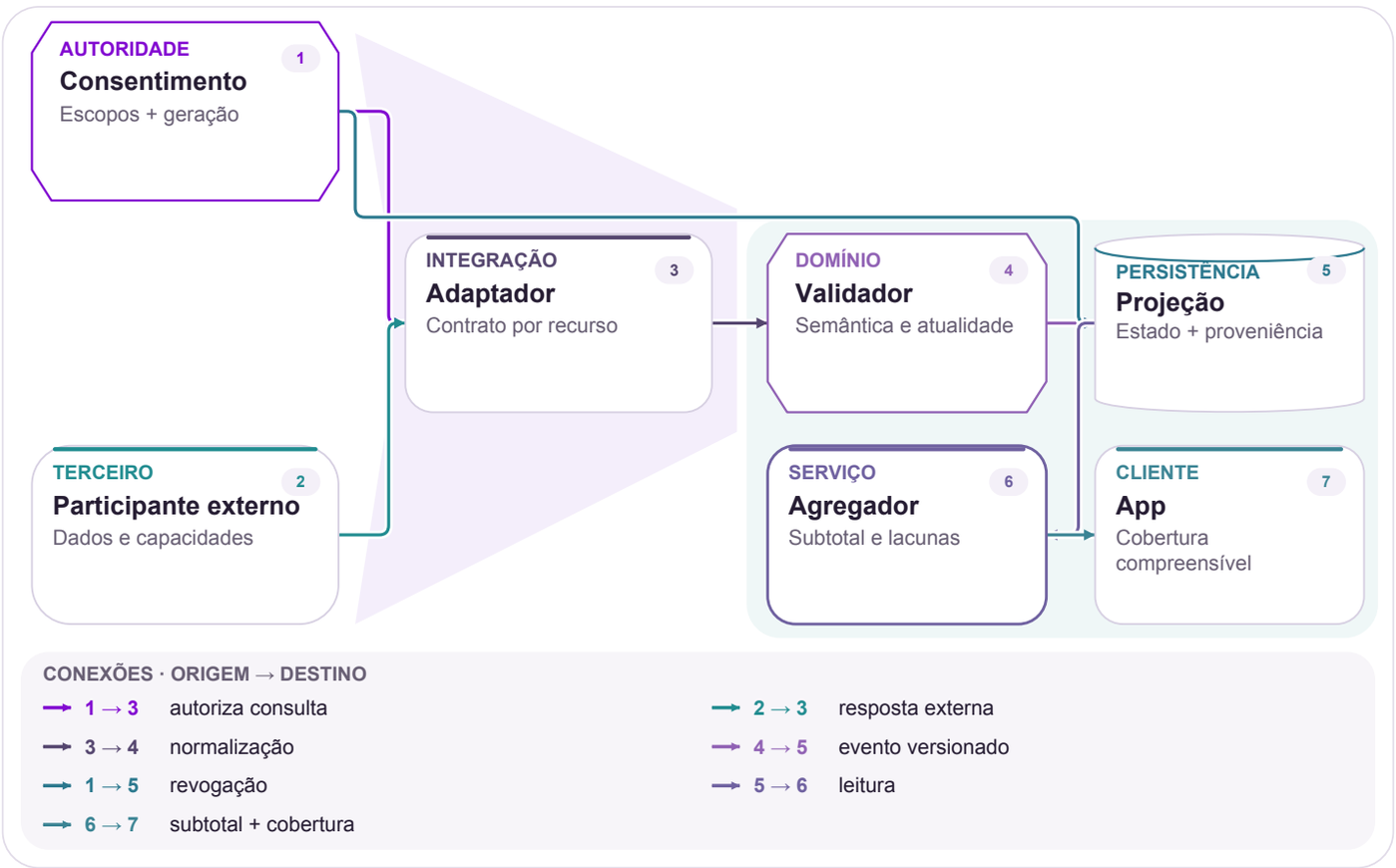
Um recurso completo não torna o portfólio completo. A resposta depende do conjunto de recursos esperado naquela conexão e do escopo autorizado pelo cliente. Se a conta corrente foi sincronizada e investimentos não foram consultados, o sistema precisa manter essa diferença. Eu representaria uma matriz de cobertura que combina participante, tipo de produto, consentimento e capacidade de integração, evitando um indicador único de “conectado” que esconda situações distintas.

O estado zero significa que uma fonte válida informou ausência de valor naquele recurso. Unsupported significa que o recurso não faz parte da capacidade considerada. Unavailable significa que a capacidade existe, mas a consulta não trouxe uma resposta utilizável naquele momento. Missing indica que ainda não há observação suficiente. Esses estados podem gerar ações diferentes: explicar cobertura, aguardar, repetir consulta ou revisar o consentimento, sem obrigar o cliente a desconectar tudo.

Também é necessário definir atualidade. Uma posição válida ontem pode ser útil como histórico e inadequada para um total apresentado como atual. A proposta guarda observed_at e uma versão ordenável por recurso, com limite de idade decidido por finalidade. O laboratório exclui do subtotal valores fora desse limite e registra a lacuna. Uma experiência real poderia mostrar o último valor conhecido separadamente, desde que a idade e o significado fossem inequívocos.

ARQUITETURA PROPOSTA • FRONTEIRAS EXPLÍCITAS

Proveniência por recurso, não um selo genérico de conexão



Desenho autoral proposto; não representa a arquitetura interna do Nubank.

A normalização ocorre por recurso para preservar diferenças entre respostas externas. O adaptador não decide que ausência significa zero; ele entrega dados e contexto para validação. A projeção mantém o resultado junto da fonte, versão e instante observado. O agregador combina apenas valores que atendem à política de atualidade e devolve as lacunas como parte do contrato, não como um detalhe escondido em logs.

O consentimento atravessa todo o fluxo. Uma resposta que estava em trânsito antes da revogação precisa ser reconhecida pela geração antiga e não repovoar silenciosamente a visão. O laboratório usa consent_epoch para demonstrar essa barreira, mas não implementa protocolos reais de consentimento.

O desenho permite investigar onde a informação desapareceu: capacidade, autorização, resposta, transformação ou apresentação. Essa separação ajuda a equipe a conversar com parceiros sem atribuir culpa prematuramente. A interface recebe informação suficiente para explicar a fronteira e oferecer uma ação proporcional à lacuna observada.

ENGENHARIA EXPLICADA

Somar valores conhecidos sem esconder lacunas

PYTHON / PROPOSTA AUTORAL

```python
# max_age mede a idade em segundos simulados.
# Uma lacuna não recebe um valor zero substituto.
# A interface deve preservar complete e gaps.
def summarize(self, now, max_age=60):
    total, gaps = 0, {}
    if not self.active:
        return dict(cents=None, complete=False, gaps={"consent": "revoked"})
    for key in sorted(self.required):
        item = self.resources.get(key)
        if item is None:
            gaps[key] = "missing"
        elif item.status != "complete":
            gaps[key] = item.status
        elif not 0 <= now - item.observed_at <= max_age:
            gaps[key] = "stale_or_future"
        else:
            total += item.cents
    return dict(cents=total, complete=not gaps, gaps=gaps)
```

Trecho do laboratório sintético; execução verificada no arquivo completo.

A função percorre o conjunto esperado e atribui um motivo a cada recurso que não pode contribuir. Um valor zero válido participa normalmente da soma; uma posição ausente não recebe zero como substituto. O campo complete depende da ausência de lacunas, e a revogação devolve valor indisponível em vez de um total aparentemente vazio.

O subtotal zero com complete=false precisa de apresentação explícita: “subtotal conhecido”, acompanhado das lacunas. O laboratório não implementa a interface. Também adota um limite de idade uniforme para simplificar o teste; em um produto real, a política de atualidade seria definida por tipo de recurso e finalidade.

ENGENHARIA EXPLICADA

Clojure: resultado parcial como dado de primeira classe

CLOJURE / PROPOSTA AUTORAL

```clojure
(defn usable? [now max-age resource]
  (and (= :complete (:status resource))
       (integer? (:cents resource))
       (<= 0 (- now (:observed-at resource)) max-age)))

(defn aggregate [expected resources now max-age]
  ;; Contrato proposto: total conhecido sempre vem com cobertura.
  (reduce
    (fn [acc key]
      (let [resource (get resources key)]
        (if (usable? now max-age resource)
          (update acc :known-cents + (:cents resource))
          (assoc-in acc [:gaps key]
            (cond
              (nil? resource) :missing
              (not= :complete (:status resource))
              (:status resource)
              :else :stale-or-future)))))
    {:known-cents 0 :gaps {}}
    expected))

(defn presentation [result]
  (assoc result :label
    (if (empty? (:gaps result))
      :covered-total :known-subtotal)))
```

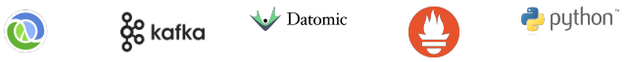
Contrato e funções didáticas propostas; Clojure não executado.

A estrutura proposta transporta lacunas junto do subtotal. Isso reduz a chance de uma camada intermediária descartar o contexto e apresentar apenas um número. A função presentation escolhe um rótulo sem alterar a soma, tornando visível que completude é uma decisão semântica distinta do cálculo aritmético.

O trecho pressupõe recursos previamente validados e não implementa consentimento, persistência ou ordenação de eventos. Não foi executado em Clojure. Sua função no capítulo é discutir o formato do contrato entre agregação e apresentação. Uma implementação real precisaria testar consumidores antigos para garantir que eles não ignorem gaps e mostrem known-cents como patrimônio completo.

ANÁLISE E PROPOSTA EXTERNA

Um contrato de cobertura exige vocabulário compartilhado



A proposta de resposta teria known_cents, coverage_complete, gaps e as referências de atualização por recurso. A unidade monetária também seria obrigatória: somar moedas diferentes sem conversão declarada produziria um número sem significado. O laboratório assume uma única unidade em centavos e não trata câmbio. A restrição está no escopo para impedir que a simplicidade do exemplo seja confundida com um agregador patrimonial completo.

Cada gap precisaria de um motivo estável que o app consiga apresentar e o suporte consiga investigar. Códigos técnicos de um participante não deveriam atravessar diretamente para o texto do cliente. O adaptador mapearia esses códigos para categorias de domínio, preservando detalhes de diagnóstico em um canal controlado. Quando não houver certeza sobre a causa, a categoria deve admitir desconhecimento em vez de culpar automaticamente o usuário ou a instituição externa.

A evolução do contrato também inclui recursos novos. O conjunto expected precisa ser versionado ou derivado de uma capacidade conhecida, pois adicionar uma categoria pode transformar uma visão antes completa em parcial. Isso pode ser correto, mas deve ser explicado. Eu trataria a matriz de cobertura como produto operacional com responsável, testes e revisão, não como uma lista informal de exceções mantida apenas no atendimento.

ENGENHARIA EXPLICADA

Recusar eventos antigos e preservar significado

PYTHON / PROPOSTA AUTORAL

```python
# A geração acompanha o recurso desde a solicitação.
def apply(self, key, resource):
    if not self.active or resource.consent_epoch != self.consent_epoch:
        return False
    if type(resource.sequence) is not int or resource.sequence < 0:
        raise ValueError("integer_sequence_required")
    if key not in self.required:
        raise ValueError("unknown_resource")
    if resource.status not in ("complete", "unsupported", "unavailable"):
        raise ValueError("status")
    if resource.status == "complete":
        if type(resource.cents) is not int or resource.cents < 0:
            raise ValueError("amount")
    elif resource.cents is not None:
        raise ValueError("unknown_is_not_zero")
    old = self.resources.get(key)
    if old and resource.sequence <= old.sequence:
        return False
    self.resources[key] = resource
    return True
```

Trecho do laboratório sintético; execução verificada no arquivo completo.

O método aplica a barreira de consentimento antes de considerar o conteúdo. Depois valida o recurso e exige que estados desconhecidos tragam cents=None. A versão por recurso impede que uma resposta atrasada substitua uma observação mais recente. O retorno booleano informa se a projeção mudou.

O laboratório ignora versões iguais ou antigas. Uma implementação real deveria investigar a mesma versão com conteúdo divergente, por exemplo comparando um fingerprint, em vez de tratar essa anomalia como duplicata normal. A sequência também precisa vir de um contrato confiável ou ser atribuída no ponto correto de ingestão; um horário de chegada isolado não estabelece necessariamente ordem de negócio.

ENGENHARIA EXPLICADA

Quatro testes contra a falsa precisão

PYTHON / PROPOSTA AUTORAL

```python
def test_partial_is_not_complete(self):
    self.p.apply("account", self.resource())
    self.assertEqual(self.p.summarize(110),
                     dict(cents=100, complete=False,
                          gaps={"investment": "missing"}))

def test_zero_is_a_real_complete_position(self):
    self.p.apply("account", self.resource())
    self.p.apply("investment", self.resource(0))
    self.assertTrue(self.p.summarize(110)["complete"])

def test_unsupported_requires_none(self):
    with self.assertRaises(ValueError):
        self.p.apply("investment", self.resource(0, status="unsupported"))
    self.p.apply("investment", self.resource(None, status="unsupported"))
    self.assertEqual(self.p.summarize(110)["gaps"]["investment"], "unsupported")

def test_old_or_duplicate_events_do_not_overwrite(self):
    self.p.apply("account", self.resource(200, sequence=2))
    self.assertFalse(self.p.apply("account", self.resource(100)))
    self.assertFalse(self.p.apply("account", self.resource(300, sequence=2)))
    self.assertEqual(self.p.summarize(110)["cents"], 200)
```

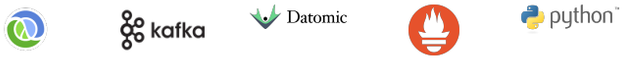
Trecho do laboratório sintético; execução verificada no arquivo completo.

Os cenários separam quatro situações visualmente parecidas: apenas parte dos recursos chegou, um recurso informou zero, um recurso não é suportado e uma versão antiga chegou depois da nova. A soma sozinha não distingue essas condições; por isso os testes também verificam cobertura e motivo da lacuna.

O arquivo completo passou em oito testes, cobrindo sequência inválida, expiração da observação, data futura e revogação com evento em trânsito. Não houve consulta a Open Finance real. A evidência demonstra o contrato local e fornece fixtures para uma futura integração, que precisaria validar escopos, schemas e comportamento de participantes autorizados.

ANÁLISE E PROPOSTA EXTERNA

Observabilidade que localiza a fronteira incompleta



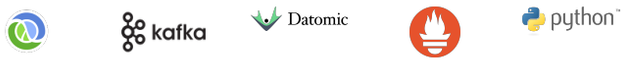
O painel proposto começaria pela proporção de conexões com cobertura completa e pela distribuição dos motivos de lacuna. Em seguida, mostraria idade das observações, falhas de validação e respostas descartadas por geração de consentimento. Uma conexão tecnicamente ativa com apenas parte dos recursos sincronizados não deveria ser contabilizada da mesma forma que uma visão completa e atual. A métrica precisa refletir o contrato que o cliente recebe.

Eu segmentaria por participante e classe de recurso quando a cardinalidade fosse controlada, evitando identificar contas individuais. Uma queda concentrada em um recurso pode indicar mudança de contrato; falhas espalhadas podem apontar para infraestrutura comum ou política interna. Esses sinais orientam investigação, mas não determinam causa sem examinar evidências. O runbook teria passos para conferir autorização, disponibilidade, resposta recebida, transformação e apresentação nessa ordem.

Também mediria reconexões repetidas, porque pedir ao usuário para refazer o consentimento pode ser uma solução inadequada para um recurso não suportado. O objetivo é reduzir ações inúteis e tempo de diagnóstico. Alertas precisam distinguir ausência esperada de dados de uma regressão de cobertura. Um produto novo ainda não suportado não deve disparar o mesmo incidente que a interrupção de uma integração anteriormente válida.

ANÁLISE E PROPOSTA EXTERNA

Lançar transparência antes de prometer mais cobertura



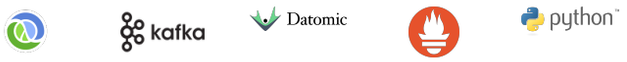
A primeira entrega poderia melhorar o rótulo do subtotal e informar a última atualização, sem adicionar novos participantes ou produtos. Para isso, seria necessário confirmar que os dados de proveniência já existem ou instrumentar sua coleta de forma segura. Mostrar uma data arbitrária apenas para preencher a tela criaria outra falsa certeza. A informação exibida precisa corresponder ao instante observado, ao processamento ou à consulta, com significado definido.

Depois, eu validaria a matriz de cobertura com contas de teste autorizadas e cenários conhecidos. Cada combinação de participante e recurso teria casos de valor positivo, zero, ausência de permissão, indisponibilidade e resposta atrasada. O lançamento gradual permitiria comparar a classificação antiga e a nova, verificando se os consumidores mantêm o significado de parcial. Uma mudança de total aparente exigiria explicação cuidadosa para não parecer perda de dinheiro.

O recuo não deveria restaurar um número incompleto como se fosse completo. A flag de apresentação pode mudar a forma da explicação, mas não deve apagar a semântica de cobertura. Eu manteria essa invariante no contrato. A expansão de suporte a novos recursos seria um projeto separado, com dependências de integração e critérios de aceitação próprios, evitando misturar transparência com uma promessa de capacidade ainda não demonstrada.

ANÁLISE E PROPOSTA EXTERNA

Consentimento revogado e resposta atrasada



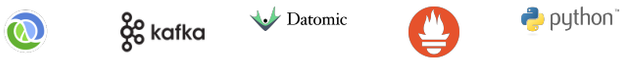
Uma integração pode estar consultando dados quando o cliente revoga o consentimento. Se o sistema olhar apenas o estado no início da requisição, uma resposta tardia pode repovoar uma projeção que acabou de ser limpa. A geração proposta resolve a parte lógica desse problema: a resposta carrega o contexto sob o qual foi solicitada, e o aplicador confere se esse contexto ainda é válido antes de publicar a atualização.

O laboratório limpa os recursos ao revogar e recusa eventos com geração antiga. Uma implementação real precisaria definir tratamento de cópias, caches, históricos e obrigações de retenção aplicáveis, com responsáveis de privacidade e segurança. O exemplo não estabelece regras jurídicas. Ele torna explícita uma condição técnica: a decisão de apresentar dados precisa respeitar a autorização vigente no ponto de uso, não apenas no começo de um trabalho assíncrono.

Para desempenho, eu evitaria consultar todas as instituições a cada abertura do app. Projeções podem fornecer leituras rápidas, desde que carreguem proveniência e limite de atualidade. O custo é lidar com estados parciais e atrasos. O benefício não autoriza ocultar esses estados. A pergunta de arquitetura seria quais recursos toleram qual idade para cada finalidade, preservando uma experiência útil sem transformar uma leitura antiga em representação do presente.

ANÁLISE E PROPOSTA EXTERNA

ADR: completude explícita em vez de preenchimento com zero



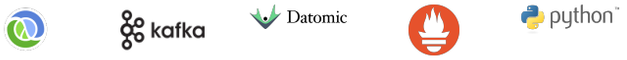
A decisão proposta é manter ausência e zero como valores semanticamente diferentes. A alternativa de preencher tudo com zero simplifica a soma e a interface, mas destrói informação sobre cobertura. A alternativa de esconder o agregador inteiro sempre que faltar um recurso evita falsa precisão, porém elimina utilidade de dados válidos e pode tornar a experiência instável. Eu escolheria um subtotal conhecido acompanhado de lacunas e atualização, com apresentação testada.

Essa decisão exige que consumidores tratem estado parcial como condição normal. Não basta adicionar um campo opcional e esperar que todas as telas o usem corretamente. Os contratos e testes de integração devem demonstrar que um valor parcial não aparece com rótulo de total completo. Também é necessário definir quando a experiência deixa de ser útil, por exemplo se nenhum recurso relevante está atual, para mostrar indisponibilidade em vez de um subtotal vazio sem contexto.

O custo aceito é maior complexidade de domínio e comunicação. A recompensa pretendida é confiança e diagnóstico mais rápido. Reabriria o ADR se surgisse uma garantia de cobertura mais forte ou se pesquisa mostrasse que o vocabulário escolhido confunde clientes. Eu mudaria a apresentação e o contrato de forma coordenada, preservando a distinção fundamental entre o que foi observado, o que não foi observado e o que está fora do escopo.

ANÁLISE E PROPOSTA EXTERNA

Liderança entre instituições, produto e suporte



Como EM, eu estruturaria o trabalho em duas trilhas: qualidade da explicação e ampliação de cobertura. A primeira pode avançar com estados e proveniência; a segunda depende de contratos e capacidades de participantes. Misturar as duas cria promessas que a equipe não controla. Um responsável por integração manteria a matriz de recursos, enquanto produto e atendimento validariam se os motivos de lacuna permitem orientar a pessoa sem reconexões desnecessárias.

Como Staff, eu revisaria a fronteira de consentimento, a ordenação de atualizações e a semântica de subtotal. A investigação de um caso começaria pela evidência recebida, não por uma suposição de defeito no parceiro ou no Nubank. Esse método melhora a qualidade da conversa técnica e torna mais fácil demonstrar qual camada precisa mudar. O laboratório fornece exemplos pequenos que todos os times conseguem reproduzir sem acesso a dados financeiros reais.

O valor seria estimado com redução de chamados, reconexões evitadas e tempo de diagnóstico. Retenção ou ativação incremental só entrariam após um experimento adequado. A pergunta que eu levaria à entrevista seria: “Quando mostramos um número agregado, qual é o contrato de completude desse número e como o cliente percebe suas fronteiras?” Responder bem exige combinar modelagem de dados, experiência, integração e responsabilidade operacional.

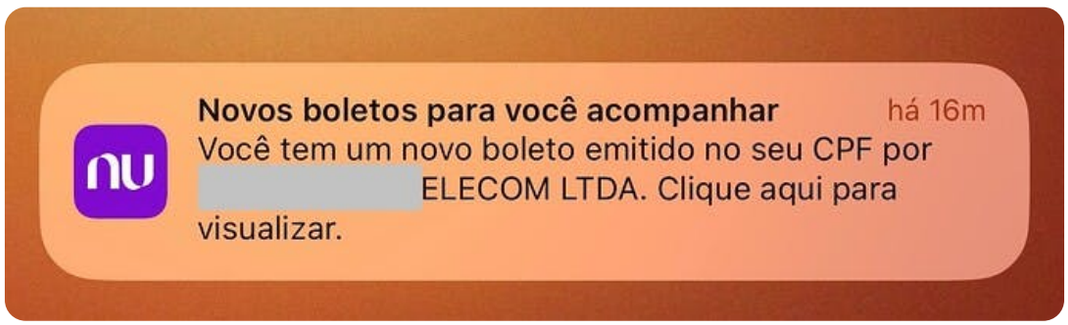
ANÁLISE E PROPOSTA EXTERNA

Organizar a próxima decisão de pagamento

05

Assistente de pagamentos do Nubank precisa de melhorias

Captura da fonte pública consultada em 23/10/2026. Relato não equivale a incidente reproduzido.



Print publicado por cliente na NuCommunity; contexto e limitações na fonte.

A publicação da NuCommunity reúne críticas ao horário de lembretes, à leitura do próximo vencimento e à navegação entre meses. Ela descreve a experiência de um participante e contém capturas anexadas; não estabelece a situação atual do produto nem demonstra que um aviso causou atraso. A pergunta de produto é mais específica: o assistente permite reconhecer o próximo compromisso sem reconstruir uma agenda mental a partir de várias telas?

Eu trataria consulta de vencimentos e execução de pagamento como jornadas diferentes. A primeira pode reduzir esforço com ordenação, linguagem e atualização de estado. A segunda envolve autorização, saldo, confirmação e consequência financeira. Adicionar uma lista melhor não autoriza automatizar pagamentos, e aumentar a quantidade de lembretes não garante organização. O projeto precisa preservar essa fronteira para melhorar a experiência sem introduzir ações inesperadas.

A proposta organiza obrigações por data civil, mostra o próximo vencimento de forma explícita e aplica preferências de horário aos lembretes. O laboratório testa mudança de mês, pagamento antes do envio, duplicatas e regras de fuso, incluindo horários ambíguos ou inexistentes em outro país. Esses cenários não descrevem problemas confirmados do Nubank; são condições de engenharia que uma solução de agenda precisa decidir de maneira previsível antes de ser expandida.

NuCommunity - Assistente de pagamentos do Nubank precisa de melhorias

ANÁLISE E PROPOSTA EXTERNA

Vencimento é data civil; envio é um instante



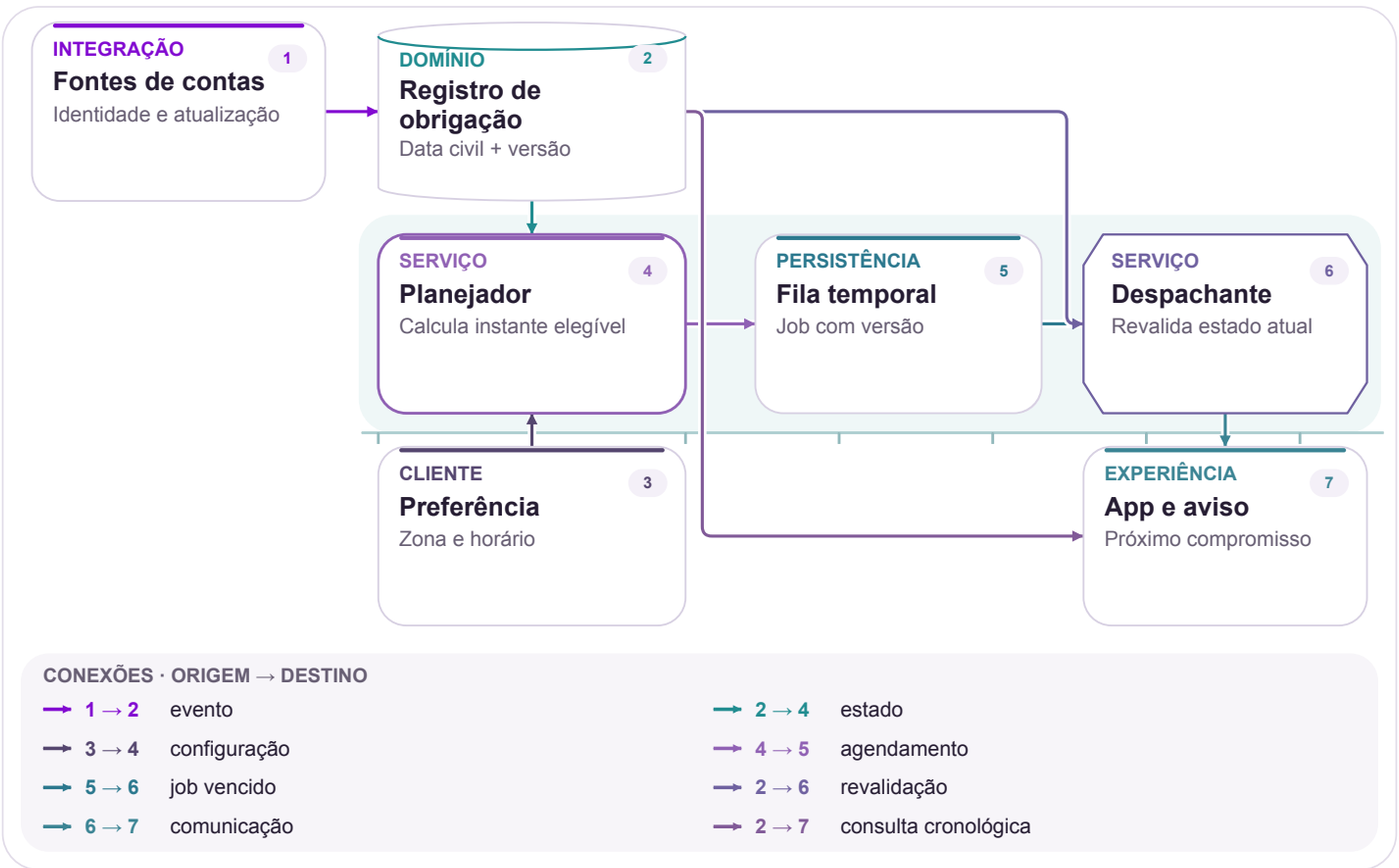
Uma conta que vence em primeiro de novembro possui uma data de negócio. Um lembrete enviado às nove da manhã possui um instante dependente do fuso escolhido. Guardar ambos como se fossem o mesmo timestamp cria ambiguidades: converter meia-noite UTC para outro fuso pode deslocar a data que o cliente entende como vencimento. Eu manteria due_date como data civil e reminder_at como instante calculado a partir de preferência e zona explícitas.

A obrigação também possui identidade e versão. Se o valor, vencimento ou estado mudar, um job gerado anteriormente pode ficar obsoleto. O despachante precisa comparar a versão atual antes de enviar. O estado pago não deve depender apenas de um clique na interface; precisa indicar a origem e a certeza da confirmação disponível. Quando uma conta é paga em outro canal e isso ainda não foi observado, a mensagem deve evitar afirmar uma dívida como fato definitivo.

O laboratório usa versões crescentes e considera o lembrete inválido quando a obrigação foi atualizada ou paga. Ele não integra DDA, calendário bancário, feriados ou liquidação. Essa redução permite testar a disciplina temporal sem misturar regras externas. A interface proposta poderia mostrar “próximo vencimento conhecido” e permitir conferir detalhes, mantendo clareza sobre a fonte da informação e a ação que ainda precisa ser tomada.

ARQUITETURA PROPOSTA • FRONTEIRAS EXPLÍCITAS

Uma agenda com relógios e versões explícitos



Desenho autoral proposto; não representa a arquitetura interna do Nubank.

O planejador transforma uma data de negócio e uma preferência em um instante de envio. A fila mantém a versão da obrigação usada nesse cálculo. Quando o job vence, o despachante consulta o estado atual: uma conta paga, alterada ou cancelada torna o trabalho antigo obsoleto. O fluxo de consulta cronológica não depende de executar um pagamento nem de disparar uma mensagem.

Essa arquitetura proposta aceita que uma fila entregue atrasado ou mais de uma vez. A identidade do lembrete evita duplicação local, enquanto uma janela de validade impede enviar no dia seguinte uma mensagem que perdeu seu contexto. A política de atraso deve variar por finalidade; o laboratório escolhe expirar ao mudar o dia civil.

O relógio do teste é passado como parâmetro, permitindo reproduzir fronteiras de data sem esperar o tempo real. Uma operação de produção precisaria considerar persistência, sincronização de relógios, mudanças de preferência e o comportamento do provedor de notificação.

ENGENHARIA EXPLICADA

Resolver horários que não existem no relógio local

PYTHON / PROPOSTA AUTORAL

```python
# day é uma data civil já validada.
# zone usa regras IANA, não um offset fixo.
# A ida e volta detecta horários inexistentes.
# fold considera as duas ocorrências de uma dobra.
def local_instant(day, hour, minute, zone):
    # Política proposta: primeiro instante válido; na dobra, primeiro UTC.
    naive = datetime.combine(day, time(hour, minute))
    for shift in range(181):
        candidate = naive + timedelta(minutes=shift)
        valid = []
        for fold in (0, 1):
            instant = candidate.replace(tzinfo=zone, fold=fold).astimezone(UTC)
            roundtrip = instant.astimezone(zone).replace(tzinfo=None)
            if roundtrip == candidate:
                valid.append(instant)
        if valid:
            return min(valid)
    raise ValueError("no_valid_instant")
```

Trecho do laboratório sintético; execução verificada no arquivo completo.

A função constrói candidatos para o horário preferido e verifica a ida e volta entre a zona local e UTC. Um horário inexistente na transição de fuso não retorna ao mesmo valor civil, então a busca avança até o primeiro minuto válido. Quando existem duas ocorrências, a política sintética escolhe o primeiro instante UTC.

Essa política deve ser uma decisão explícita do produto, não um efeito acidental da biblioteca. O código usa zoneinfo da biblioteca padrão e a base de fusos disponível no sistema. Os testes incluem Berlim apenas para exercitar transições; isso não afirma que o relato brasileiro envolveu horário de verão.

Python - zoneinfo

ENGENHARIA EXPLICADA

Dart: uma lista por vencimento não precisa executar nada

DART / PROPOSTA AUTORAL

```dart
class CivilDate implements Comparable<CivilDate> {
  final int year, month, day;
  const CivilDate(this.year, this.month, this.day);
  int get orderKey => year * 10000 + month * 100 + day;
  @override
  int compareTo(CivilDate other) =>
      orderKey.compareTo(other.orderKey);
}

class BillView {
  final String id;
  final CivilDate due;
  final int cents;
  final bool paid;
  const BillView(this.id, this.due, this.cents, this.paid);
}

List<BillView> upcoming(List<BillView> source) {
  // Datas já validadas na fronteira do contrato proposto.
  final visible = source.where((bill) => !bill.paid).toList();
  visible.sort((a, b) {
    final byDate = a.due.compareTo(b.due);
    return byDate != 0 ? byDate : a.id.compareTo(b.id);
  });
  return List.unmodifiable(visible);
}
```

Modelo de apresentação proposto; Dart/Flutter não executados.

O modelo de apresentação usa uma data civil, preserva dinheiro em centavos e ordena com um critério estável de desempate. Ele não transforma vencimento em horário UTC e não contém autorização de pagamento. A lista resultante é imutável para reduzir efeitos inesperados entre componentes de interface.

O exemplo pressupõe datas válidas recebidas do contrato; o construtor simplificado não valida calendário. Em Flutter, a tela ainda precisaria de semântica acessível, agrupamentos compreensíveis e estado de carregamento ou atualização. O trecho é uma proposta didática em Dart, não executada. O laboratório Python testa a lógica de agenda e envio, não esse widget ou modelo móvel.

ANÁLISE E PROPOSTA EXTERNA

Persistir a intenção do lembrete, não apenas um horário



Um job de lembrete precisa carregar referência à obrigação, versão, finalidade e política que produziu o instante. Guardar somente “enviar às nove” dificulta saber se o trabalho continua válido depois de uma alteração de vencimento. Eu proporia uma identidade formada por obrigação, versão e tipo de lembrete, com um registro separado do resultado de entrega. A confirmação de envio não muda o estado financeiro da conta.

Alterar uma preferência de horário exige uma política para jobs já criados. Recalculá-los pode ser correto, desde que não duplique um aviso aceito anteriormente. Ignorá-los até o próximo ciclo pode ser surpreendente. A escolha deve ser apresentada ao cliente quando relevante e aplicada com uma geração de configuração. O laboratório simplifica essa dimensão ao receber o horário no despacho; uma implementação persistente precisaria versionar a preferência e tratar replanejamento.

A data de vencimento também pode carregar uma origem e um nível de confiança. Informação cadastrada manualmente, detectada automaticamente ou atualizada por um parceiro não deveria ser indistinguível na investigação. A interface não precisa mostrar todos os detalhes técnicos, mas o sistema deve preservá-los. Isso permite explicar divergências sem responsabilizar o cliente por uma informação que mudou depois do cadastro e sem prometer que uma agenda substitui a conferência do documento de cobrança.

ENGENHARIA EXPLICADA

Antes de enviar, conferir se o job ainda faz sentido

PYTHON / PROPOSTA AUTORAL

```python
# Validar a versão antes de registrar uma entrega.
def dispatch(self, bill_id, expected_version, now, hour=9, minute=0):
    if now.tzinfo is None:
        raise ValueError("timezone_required")
    bill = self.bills[bill_id]
    if bill["version"] != expected_version or bill["paid"]:
        return "obsolete"
    target = local_instant(bill["due"], hour, minute, self.zone)
    key = (bill_id, expected_version, bill["due"].isoformat())
    if key in self.sent:
        return "duplicate"
    if now < target:
        return "not_due"
    # Não entregar lembrete que ficou preso até outro dia civil.
    if now.astimezone(self.zone).date() != bill["due"]:
        return "expired"
    self.sent.add(key)
    return "sent_locally"
```

Trecho do laboratório sintético; execução verificada no arquivo completo.

O despachante rejeita relógios sem fuso, compara a versão e verifica se a conta já foi paga. Em seguida, calcula o instante elegível e reconhece um lembrete repetido. Um job que só chega em outro dia civil expira no modelo, evitando comunicar urgência que perdeu contexto.

O conjunto sent fica em memória; portanto, a deduplicação demonstrada não sobrevive a reinício e não garante efeito único no provedor externo. Para produção, eu persistiria a decisão e trataria a janela entre aceitação externa e confirmação local. O teste isola a semântica temporal para que essa limitação seja visível, em vez de escondida por infraestrutura.

ENGENHARIA EXPLICADA

O calendário deve aparecer na suíte de testes

PYTHON / PROPOSTA AUTORAL

```python
def test_orders_across_month_boundaries(self):
    self.p.upsert("a", date(2026, 10, 31), 999, 1)
    self.assertEqual([key for key, _ in self.p.upcoming()], ["a", "b"])

def test_paid_before_dispatch_suppresses_reminder(self):
    self.p.upsert("b", date(2026, 11, 1), 1234, 2, paid=True)
    self.assertEqual(self.p.dispatch(
        "b", 1, datetime(2026, 11, 1, 12, tzinfo=UTC)),
                     "obsolete")
    self.assertEqual(self.p.upcoming(), [])

def test_gap_advances_to_first_valid_minute(self):
    zone = ZoneInfo("Europe/Berlin")
    value = local_instant(date(2026, 3, 29), 2, 30, zone)
    self.assertEqual(value.astimezone(zone).strftime("%H:%M"), "03:00")

def test_fold_selects_first_occurrence(self):
    zone = ZoneInfo("Europe/Berlin")
    value = local_instant(date(2026, 10, 25), 2, 30, zone)
    self.assertEqual(value, datetime(2026, 10, 25, 0, 30, tzinfo=UTC))
```

Trecho do laboratório sintético; execução verificada no arquivo completo.

Os testes verificam uma lista que atravessa meses, um pagamento observado antes do envio e as duas formas de ambiguidade temporal: horário inexistente e horário repetido. As datas foram escolhidas como fixtures, sem relação com contas de clientes. O resultado esperado torna a política de calendário revisável.

O arquivo passou em nove testes, cobrindo versão inválida, deduplicação, expiração de lembrete atrasado e rejeição de relógio sem zona. Não houve integração com calendário bancário ou DDA. Em uma entrega real, feriados, regras do documento e diferenças entre vencimento e compensação exigiriam contratos e testes adicionais, sem alterar silenciosamente a data que o cliente vê.

ANÁLISE E PROPOSTA EXTERNA

Medir a tarefa de encontrar o próximo compromisso



A principal métrica de experiência seria o sucesso ao identificar a próxima conta relevante, com valor e vencimento corretos. Eu mediria tempo e erros de interpretação em tarefas concretas, incluindo mudança de mês e ausência de contas no período atual. Cliques a menos são úteis apenas se a pessoa continuar entendendo o que está vendo. Um atalho que reduz navegação e aumenta confusão sobre vencimento não cumpre o objetivo.

Na operação, separaria jobs planejados, enviados, suprimidos por atualização, repetidos e expirados por atraso. Um aumento de supressões por pagamento pode ser positivo; um aumento de expiração pode indicar atraso da fila. A idade do job e a diferença entre instante planejado e despacho ajudam a localizar esse problema. Eu também observaria mensagens enviadas fora da preferência, com classificação de motivo, sem usar identificadores pessoais como rótulos.

O efeito sobre atrasos de pagamento exige cuidado causal. Pessoas podem atrasar por falta de saldo, decisão própria ou informação incorreta, e não apenas por esquecer o vencimento. O experimento deveria distinguir essas condições antes de atribuir valor econômico ao assistente. A proposta inicial pode ser validada por organização e compreensão, deixando redução de encargos ou inadimplência como hipóteses a medir, não como promessa derivada de uma melhoria visual.

ANÁLISE E PROPOSTA EXTERNA

Separar a migração visual da mudança de agendamento



Eu lançaria primeiro a consulta cronológica com o mesmo conjunto de obrigações já conhecido, preservando a possibilidade de conferir mês e detalhes. A pesquisa compararia a tarefa de encontrar o próximo compromisso, não apenas preferência estética. Pessoas com muitas contas, contas recorrentes e contas distribuídas entre meses precisariam participar da validação. A nova apresentação não deve alterar vencimentos, valores ou autorizações existentes.

A mudança de lembretes teria rollout próprio. Antes de enviar, o planejador candidato produziria decisões em modo comparativo para verificar horário, zona e supressão por pagamento. A equipe revisaria casos em que o candidato diverge do comportamento vigente, sem inundar clientes com duas versões da mensagem. Uma coorte controlada permitiria medir horários inadequados e recontato, com limite de mensagens e um mecanismo claro de interrupção.

O rollback preservaria registros de entrega e versões de obrigações. Voltar o agendador não pode fazer o backlog reaparecer como trabalho novo. Também seria necessário manter a preferência escolhida pelo cliente, mesmo que a apresentação antiga não ofereça todos os controles. O plano de migração incluiria compatibilidade de leitura e uma resposta do atendimento para explicar por que um lembrete foi enviado, suprimido ou expirado em cada etapa.

ANÁLISE E PROPOSTA EXTERNA

Uma agenda útil precisa ser legível sob pressão



A pessoa pode consultar vencimentos no transporte, durante trabalho ou com baixa conectividade. Eu priorizaria valor, data e estado, deixando gráficos e explicações secundárias sem competir com a decisão imediata. Uma lista ordenada deve manter contexto quando atualiza: se uma conta desaparece porque foi marcada como paga, a interface precisa evitar deslocar o foco de forma que uma ação seguinte recaia sobre outro item sem percepção do usuário.

Em acessibilidade, data relativa e absoluta podem se complementar. “Amanhã” é rápido de ler, enquanto a data explícita evita ambiguidade ao revisitar uma captura ou mensagem atrasada. Cores de urgência precisam de texto equivalente, e valores devem ser anunciados de forma compreensível. A proposta visual de vidro e transparência só seria aceitável se contraste, foco e leitura permanecessem claros em diferentes fundos e configurações de movimento.

Para desempenho, eu usaria paginação ou carregamento incremental quando o volume exigisse, preservando uma consulta rápida ao próximo vencimento. O servidor pode oferecer uma visão ordenada, mas o contrato deve esclarecer filtros e cobertura. Uma lista parcial não deveria ser apresentada como ausência de outras contas. A qualidade do assistente depende de organização e honestidade sobre o que foi carregado, não somente de uma animação fluida.

ANÁLISE E PROPOSTA EXTERNA

ADR: data civil no domínio e instante na operação



A decisão proposta mantém vencimento como data civil e calcula o instante de notificação com zona e preferência explícitas. A alternativa de armazenar tudo como timestamp UTC simplifica alguns mecanismos de fila, mas desloca uma interpretação de negócio para conversões implícitas. A alternativa de usar apenas a hora local do dispositivo dificulta consistência quando o usuário muda de aparelho, viaja ou está desconectado. Eu escolheria um contrato que preserve as duas representações e sua finalidade.

O sistema também precisa decidir como tratar transições de horário. O laboratório avança para o primeiro minuto válido e escolhe a primeira ocorrência quando há duplicidade. Essa política é demonstrativa e deve ser revisada conforme o produto. Para lembretes financeiros, previsibilidade e ausência de duplicação importam mais do que uma conversão aparentemente elegante que ninguém consegue explicar ao cliente ou ao suporte.

O custo aceito é armazenar mais contexto e manter testes de calendário. O benefício é evitar que mudanças de fuso alterem o significado do vencimento. Reabriria o ADR se a aplicação passasse a operar em mais países, se obrigações tivessem horários contratuais específicos ou se a preferência precisasse acompanhar viagem automaticamente. Essas mudanças exigem uma decisão de produto e dados, não apenas trocar uma biblioteca de datas.

ANÁLISE E PROPOSTA EXTERNA

EM: uma entrega de organização, não uma coleção de funções



Como EM, eu definiria a tarefa principal com produto e pesquisa: reconhecer o próximo compromisso e saber qual ação está disponível. A partir daí, separaria a entrega de consulta, a revisão de lembretes e qualquer evolução de pagamento. Essa divisão reduz dependências e evita que uma melhoria de organização se torne um projeto amplo de automação sem hipótese clara. O atendimento participaria com motivos de contato e exemplos de interpretação equivocada.

Como Staff, eu revisaria o modelo temporal, a identidade das obrigações e a revalidação de jobs. A conversa técnica incluiria falhas de fila, pagamento observado após agendamento e preferência alterada durante a espera. Os nove testes do laboratório funcionam como exemplos executáveis dessas decisões. Para uma implementação real, eu adicionaria persistência e testes de integração somente depois de fechar a semântica, evitando usar infraestrutura para esconder perguntas de domínio não respondidas.

O valor inicial seria tempo poupado por sessão de planejamento e redução de contatos sobre localização de contas ou horário de aviso. Benefícios financeiros ligados a atraso precisariam de medição causal. A pergunta que eu levaria à entrevista seria: “Quando uma obrigação muda entre planejar e notificar, qual versão tem autoridade e como o cliente percebe essa mudança?” A resposta conecta calendário, consistência, experiência e execução responsável.

ANÁLISE E PROPOSTA EXTERNA

Explicar por que a proteção permanece ativa

06

Modo Rua precisa de um pequeno ajuste né

Captura da fonte pública consultada em 23/10/2026. Relato não equivale a incidente reproduzido.



Print publicado por cliente na NuCommunity; contexto e limitações na fonte.

O participante relata que o Modo Rua continuava pedindo biometria mesmo conectado a um Wi-Fi cadastrado quando não concedia permissão de localização. A publicação não informa um diagnóstico de plataforma e não prova que a interpretação sobre GPS ou bateria esteja correta. O caso é útil para examinar a diferença entre a promessa percebida pelo cliente e os sinais técnicos necessários para confirmar o contexto.

Eu não trataria a fricção como justificativa para remover uma proteção. A proposta seria apresentar estados compreensíveis: contexto confirmado, contexto não confirmado, permissão necessária ou informação expirada. Isso permite explicar por que a regra conservadora continua valendo, sem afirmar que uma rede é segura apenas porque seu nome parece familiar. O cliente precisa saber qual ação pode tomar e qual proteção permanece, mesmo quando decide não conceder a permissão.

O laboratório usa uma prova contextual fictícia assinada por HMAC para testar validade, vínculo com dispositivo e geração. Ele não detecta Wi-Fi, não atesta segurança de rede e não autentica pessoas. A simulação serve para demonstrar uma invariante: um sinal ausente, adulterado ou vencido não pode ser promovido a confiança. A decisão sobre um desafio específico também não equivale a autorização para movimentar dinheiro.

NuCommunity - Modo Rua precisa de um pequeno ajuste né

ANÁLISE E PROPOSTA EXTERNA

Contexto observado, confiança concedida e ação autorizada



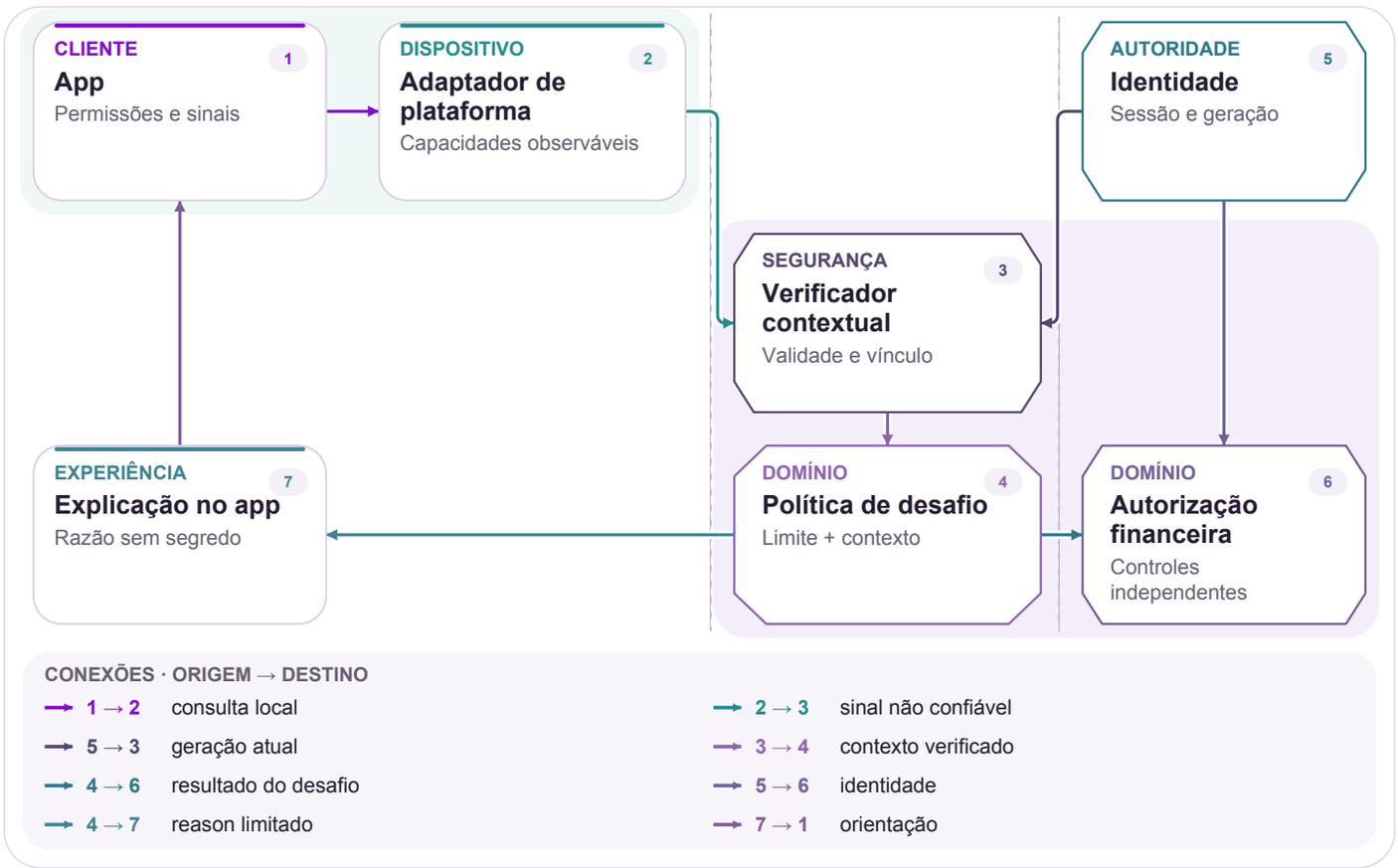
Três conceitos precisam permanecer separados. Contexto observado é o conjunto de sinais disponíveis no aparelho. Confiança concedida é a interpretação autorizada desses sinais dentro de uma política. Ação autorizada é a decisão final sobre uma operação financeira. Um Wi-Fi reconhecido pode participar de um desafio de segurança, mas não deve substituir saldo, identidade, limites ou demais controles. Misturar esses conceitos torna um indicador visual mais poderoso do que deveria.

O contrato proposto da decisão devolve context_trusted, challenge e reason. A interface usa reason para explicar a experiência, enquanto a autorização financeira continua em outra fronteira. A prova contextual do laboratório contém dispositivo, rede fictícia, geração e intervalo de validade. Mesmo com assinatura válida, ela só é aceita para o dispositivo correspondente e para um identificador aprovado. O nome público da rede não participa como prova suficiente.

Quando falta permissão, o sistema conserva o estado restritivo e apresenta a razão específica. Quando a prova expira, a solução precisa obter uma nova observação ou manter o desafio. Não é aceitável reutilizar indefinidamente um estado de confiança apenas para evitar latência. Essas regras são propostas de modelagem; não descrevem a implementação do Modo Rua. A utilidade do caso está em tornar explícito como segurança e explicação devem concordar.

ARQUITETURA PROPOSTA • FRONTEIRAS EXPLÍCITAS

A interface explica; a política decide; o pagamento autoriza



Desenho autoral proposto; não representa a arquitetura interna do Nubank.

O diagrama deixa o dado do dispositivo fora da fronteira de confiança até ser avaliado. O adaptador precisa distinguir indisponibilidade de permissão, falta de capacidade e observação válida. O verificador confere vínculo, geração e atualidade antes de entregar contexto à política. A autorização financeira recebe o resultado do desafio como uma entrada, sem delegar toda a decisão a ele.

A explicação utiliza categorias limitadas para orientar o cliente. Ela não precisa revelar limiares antifraude, sinais detalhados ou material de verificação. Essa separação permite melhorar a compreensão sem transformar a interface em uma descrição completa dos mecanismos de defesa.

No laboratório, a prova HMAC é emitida pela própria fixture e representa uma autoridade fictícia. A assinatura demonstra integridade do exemplo, não a verdade física da rede. Uma implementação real teria de justificar como obtém sinais confiáveis e qual ameaça cada sinal consegue enfrentar.

ENGENHARIA EXPLICADA

Validade não é apenas conferir uma assinatura

PYTHON / PROPOSTA AUTORAL

```python
def decide(self, device, amount_cents, street_limit_cents, proof, now,
           permission=True):
    if type(permission) is not bool:
        raise ValueError("boolean_permission_required")
    if (type(amount_cents) is not int or amount_cents <= 0
            or type(street_limit_cents) is not int or street_limit_cents < 0):
        raise ValueError("invalid_money")
    reason = "context_missing"
    trusted = False
    if not permission:
        reason = "permission_missing"
    elif proof:
        body = proof["body"]
        expected = hmac.new(self.key, canonical(body), hashlib.sha256).hexdigest()
        valid = hmac.compare_digest(expected, proof["signature"])
        valid = valid and body["device"] == device
        valid = valid and body["epoch"] == self.epoch
        valid = valid and body["issued"] <= now < body["expires"]
        valid = valid and body["network"] in self.approved.get(device, set())
        trusted = valid
        reason = "verified_context" if valid else "context_invalid"
    # Contexto apenas decide este desafio; não autoriza pagamento.
    challenge = amount_cents > street_limit_cents and not trusted
    return dict(challenge=challenge, context_trusted=trusted, reason=reason,
                payment_authorized=False)
```

Trecho do laboratório sintético; execução verificada no arquivo completo.

A decisão combina assinatura, dispositivo, geração, tempo e associação aprovada. Falhar em qualquer condição mantém o contexto não confiável. Mesmo quando o desafio deixa de ser exigido no exemplo, payment_authorized continua false, deixando explícito que a função não movimenta dinheiro nem substitui controles financeiros.

A permissão ausente produz uma razão própria e conserva a política restritiva. O laboratório recebe objetos de teste já estruturados; não implementa validação completa de entrada hostil ou autenticação do emissor. A chave fixa aparece somente no teste sintético. Em produção, emissão, proteção de chaves e validação de esquema exigiriam outra arquitetura e revisão especializada.

ENGENHARIA EXPLICADA

Dart: explicar o estado sem expor a regra secreta

DART / PROPOSTA AUTORAL

```dart
enum ContextReason {
  verified, permissionMissing, unavailable, expired,
}

class ContextMessage {
  final String title, body;
  final bool offerSettings;
  const ContextMessage(this.title, this.body, this.offerSettings);
}

ContextMessage explain(ContextReason reason) {
  // Texto proposto: não informa critérios internos de risco.
  switch (reason) {
    case ContextReason.verified:
      return const ContextMessage('Contexto confirmado',
          'Outras verificações de segurança continuam valendo.', false);
    case ContextReason.permissionMissing:
      return const ContextMessage('Proteção mantida',
          'Não foi possível confirmar o contexto sem a permissão.', true);
    case ContextReason.unavailable:
      return const ContextMessage('Proteção mantida',
          'O contexto não está disponível neste momento.', false);
    case ContextReason.expired:
      return const ContextMessage('Precisamos verificar novamente',
          'A confirmação anterior perdeu a validade.', false);
  }
}
```

Mapeamento de mensagens proposto; Dart/Flutter não executados.

O exemplo traduz categorias de domínio para uma mensagem estável. Ele evita chamar ausência de permissão de “erro de segurança” e não promete liberar a conta ao abrir configurações. O texto informa que outros controles continuam existindo mesmo quando o contexto foi confirmado.

Uma interface Flutter precisaria conectar essas mensagens à navegação autorizada, ao foco e à semântica acessível. A opção de configurações deve orientar, sem pressionar o cliente a conceder acesso além do necessário. O trecho é uma proposta didática em Dart e não foi executado. A versão final exigiria pesquisa de compreensão e revisão conjunta de produto, segurança e plataformas móveis.

ANÁLISE E PROPOSTA EXTERNA

A permissão depende da API e da versão da plataforma



A documentação Android distingue permissões de dispositivos Wi-Fi próximos e permissões de localização, com diferenças por versão e por API utilizada. Isso mostra por que uma afirmação genérica como “Wi-Fi nunca precisa de localização” pode estar errada. Não permite, porém, inferir qual API o Nubank usa ou diagnosticar o relato. A investigação começaria por versão do sistema, target da aplicação, operação solicitada e resultado real da chamada.

Eu criaria uma matriz de capacidades por plataforma e versão, com testes para permissão concedida, negada, revogada e alterada durante a sessão. O adaptador devolveria uma categoria de domínio, preservando o detalhe técnico para diagnóstico controlado. A política de segurança não deveria interpretar uma exceção de plataforma como confirmação positiva. A interface, por sua vez, não deveria pedir uma permissão sem explicar a finalidade específica que o produto realmente utiliza.

O contrato também precisa admitir que capacidades mudam. Uma atualização de sistema pode alterar o acesso a sinais sem que o cliente tenha mudado sua rede. O comportamento conservador deve permanecer compreensível, e o time precisa conseguir identificar regressões por versão sem coletar nomes de redes ou localização desnecessariamente. Essa é uma responsabilidade de integração móvel e segurança, não apenas uma revisão de texto.

Android - Wi-Fi permissions

ENGENHARIA EXPLICADA

Uma prova íntegra pode estar vencida ou fora de contexto

PYTHON / PROPOSTA AUTORAL

```python
def issue_demo_proof(self, device, network, now, ttl=30):
    body = dict(device=device, network=network, issued=now,
                expires=now + ttl, epoch=self.epoch)
    signature = hmac.new(self.key, canonical(body), hashlib.sha256).hexdigest()
    return dict(body=body, signature=signature)

def test_missing_permission_keeps_conservative_state(self):
    result = self.decide(permission=False)
    self.assertTrue(result["challenge"])
    self.assertEqual(result["reason"], "permission_missing")

def test_expired_and_future_proofs_are_rejected(self):
    for now in (99, 130, 1000):
        self.assertTrue(self.decide(now=now)["challenge"])

def test_tampered_signature_is_rejected(self):
    proof = json.loads(json.dumps(self.proof))
    proof["body"]["expires"] = 9999
    self.assertTrue(self.decide(proof=proof)["challenge"])
```

Trecho do laboratório sintético; execução verificada no arquivo completo.

A fixture emite uma prova com intervalo de validade e geração. Os testes alteram o tempo, a permissão e o conteúdo assinado. A adulteração do vencimento precisa invalidar a assinatura, enquanto uma prova intacta também deve falhar se estiver fora do intervalo permitido.

Esses cenários evitam reduzir a revisão a “HMAC passou, então confie”. Eles não validam a origem física do sinal nem a proteção da chave em um aparelho real. O emissor é fictício e compartilha a chave com o verificador apenas para tornar a regra executável. O capítulo usa essa limitação para separar integridade de mensagem de verdade do contexto.

ENGENHARIA EXPLICADA

Recuperação e troca de dispositivo invalidam o contexto antigo

PYTHON / PROPOSTA AUTORAL

```python
def test_valid_context_removes_only_this_challenge(self):
    result = self.decide()
    self.assertFalse(result["challenge"])
    self.assertFalse(result["payment_authorized"])

def test_proof_is_bound_to_device_and_approved_network(self):
    self.assertTrue(self.decide(device="other")["challenge"])
    other = self.p.issue_demo_proof("device-demo", "same-name-fake", 100)
    self.assertTrue(self.decide(proof=other)["challenge"])

def test_recovery_invalidates_old_context(self):
    self.p.epoch += 1
    self.assertTrue(self.decide()["challenge"])

def test_below_limit_is_not_automatic_payment_authorization(self):
    result = self.decide(amount_cents=500, proof=None)
    self.assertFalse(result["challenge"])
    self.assertFalse(result["payment_authorized"])
```

Trecho do laboratório sintético; execução verificada no arquivo completo.

O teste de vínculo impede usar a prova em outro dispositivo e rejeita uma rede fictícia não aprovada, mesmo quando o emissor de teste a assinou. A alteração da geração representa uma recuperação de acesso que invalida contextos anteriores. O último cenário demonstra que ficar abaixo do limite não significa autorizar o pagamento.

O arquivo passou em oito testes, incluindo a rejeição de permissões que não sejam booleanas. Não houve comunicação com redes, sensores, biometria ou serviços de risco. A evidência é a consistência da política local diante dos inputs sintéticos. Uma revisão real precisaria avaliar a origem dos sinais, o adversário considerado e o comportamento do sistema quando essas fontes deixam de ser confiáveis.

ANÁLISE E PROPOSTA EXTERNA

Observar fricção sem recompensar a remoção de proteção



Eu acompanharia quantas vezes o contexto não pôde ser confirmado e por qual categoria: permissão ausente, sinal indisponível, prova expirada ou geração incompatível. Isso ajuda a distinguir dificuldade de configuração de regressão técnica. A métrica não deveria ser usada para exigir que todo desafio desapareça. Uma queda artificial de desafios pode indicar que a política ficou permissiva, enquanto um aumento pode refletir uma proteção funcionando diante de sinais insuficientes.

O resultado de produto seria compreensão do estado e conclusão legítima da tarefa com a proteção adequada. Pesquisa e contatos de suporte poderiam revelar se o cliente entende por que a biometria continua sendo solicitada. A taxa de desativação do recurso após uma dificuldade também é útil, mas precisa de interpretação: ela pode apontar frustração, mudança de necessidade ou preferência consciente. Não se deve deduzir causa apenas pelo evento.

Nos dados operacionais, eu evitaria nomes de redes, coordenadas e identificadores persistentes de aparelho em métricas agregadas. Classes de versão e motivo podem orientar a investigação com menos exposição. Alertas seriam acionados por mudanças inesperadas, por exemplo após uma atualização do sistema operacional, e teriam um runbook de reprodução. O painel mostraria segurança e usabilidade lado a lado, impedindo que uma otimização de conversão esconda uma perda de controle.

ANÁLISE E PROPOSTA EXTERNA

Lançar uma explicação nova sem mudar silenciosamente a política



A primeira etapa seria revisar mensagens e diagnóstico mantendo a política de segurança vigente. Isso permite testar se a pessoa entende a diferença entre permissão ausente e falha temporária sem introduzir uma mudança de risco. A equipe compararia as categorias técnicas observadas com o texto exibido e verificaria se a ação sugerida realmente pode resolver aquela condição. Um botão para configurações não ajuda quando a API está indisponível por outro motivo.

Qualquer alteração da política de confiança teria uma trilha separada, com avaliação de ameaça, validação em dispositivos e critérios de interrupção. O rollout poderia observar decisões candidatas em paralelo, sem usá-las para autorizar operações, para entender divergências. Essa comparação precisaria de minimização e finalidade clara dos dados. A expansão dependeria de evidência técnica e de revisão de segurança, não apenas de redução de desafios.

O rollback manteria o estado conservador e preservaria a capacidade de explicar a razão ao cliente. Não seria aceitável reutilizar provas antigas para suavizar a transição ou apagar a geração de recuperação. Eu também incluiria atendimento e documentação de configuração no lançamento. Uma política consistente que ninguém consegue explicar continua produzindo fricção e incentiva improvisações, justamente na jornada em que previsibilidade é importante.

ANÁLISE E PROPOSTA EXTERNA

O modelo de ameaça define o que um sinal pode afirmar



Uma rede com nome conhecido pode ser imitada; um dispositivo pode relatar dados incompletos; uma observação válida pode envelhecer. Por isso, o desenho precisa declarar quais ameaças considera e quais propriedades cada sinal realmente oferece. O laboratório não tenta resolver essas ameaças físicas. Ele supõe um emissor fictício e demonstra que, mesmo nessa hipótese favorável, ainda é necessário verificar vínculo, validade e geração antes de usar o contexto.

Eu evitaria uma experiência que diga “você está seguro” com base em um único indicador. A mensagem proposta fala sobre confirmação de contexto e mantém outros controles explícitos. Isso reduz a chance de o cliente interpretar uma conveniência como garantia ampla. O mesmo cuidado vale para performance: cachear um resultado contextual pode reduzir latência, mas aumenta a janela em que uma mudança de ambiente não é refletida. Essa janela deve ser uma decisão de risco, não apenas uma configuração de cache.

O custo de coletar sinais também importa. A equipe deveria medir consumo, frequência de consulta e necessidade real de cada permissão, sem presumir que uma dependência técnica justifica qualquer coleta. O melhor desenho pode reduzir chamadas, explicar a finalidade e manter uma alternativa conservadora. Segurança compreensível exige precisão sobre o que se sabe, o que falta e qual ação continua protegida.

ANÁLISE E PROPOSTA EXTERNA

ADR: desconhecido permanece desconhecido



A decisão proposta é manter o contexto não confirmado quando faltam sinais, permissões ou validade. A alternativa permissiva reduziria desafios, mas trataria ausência de evidência como evidência de segurança. A alternativa de bloquear qualquer uso do aplicativo diante de contexto ausente poderia impor fricção além da finalidade do recurso. Eu escolheria uma política limitada ao desafio relevante, preservando a autoridade independente de identidade e pagamento.

O contrato precisa devolver uma razão que a interface consiga explicar sem expor regras sensíveis. Isso aumenta a qualidade do suporte e evita mensagens genéricas que culpam o cliente. A implementação também deve distinguir uma condição esperada, como permissão negada, de uma falha do adaptador. Ambas podem manter a proteção, mas demandam ações operacionais diferentes. O estado conservador não deve servir como desculpa para ignorar regressões técnicas.

O custo aceito é manter uma matriz de plataforma e regras de validade. O benefício é previsibilidade: a política não muda de sentido porque uma API retornou vazio. Reabriria o ADR diante de novos sinais com garantias demonstráveis, mudanças de ameaça ou evidência de que o escopo do desafio está excessivo. A revisão teria de apresentar quais propriedades melhoraram e quais testes continuam garantindo que um sinal antigo ou fora de contexto não ganha autoridade indevida.

ANÁLISE E PROPOSTA EXTERNA

EM e Staff: segurança e compreensão na mesma revisão



Como EM, eu reuniria responsáveis por plataforma móvel, identidade, segurança, produto e atendimento para revisar uma jornada reproduzível. O objetivo seria mapear a condição observada, a política aplicada e a mensagem exibida. A evidência pública orienta essa pergunta, mas não substitui reprodução com autorização e dados controlados. A primeira entrega deveria corrigir desencontros entre estado e explicação antes de discutir mudanças mais profundas de confiança.

Como Staff, eu examinaria a origem dos sinais, o vínculo com dispositivo e a geração, além da separação entre desafio contextual e autorização financeira. Os oito testes sintéticos oferecem um ponto de partida para tornar essas decisões concretas. Eu também exigiria uma matriz de dispositivos e versões para o comportamento que Python não simula. A contribuição é conectar um detalhe de permissão à arquitetura de confiança e à experiência, sem prometer que uma assinatura resolve a verdade do mundo físico.

O valor seria medido por configuração concluída, compreensão e redução de chamados, com indicadores de proteção preservados. Não converteria adesão ao recurso diretamente em fraude evitada. A pergunta de entrevista seria: “Quando um sinal contextual desaparece, qual é a decisão segura e qual explicação permite ao cliente continuar sem improvisar?” Essa resposta precisa ser construída em conjunto por engenharia, risco e design.

ANÁLISE E PROPOSTA EXTERNA

Pequeno valor, grande contrato de interoperabilidade

07



Captura da fonte pública consultada em 23/10/2026. Relato não equivale a incidente reproduzido.

O relato público descreve um valor negativo de um centavo representado de forma inválida em um extrato OFX e dificuldades de importação após mudanças na formatação numérica. O arquivo original não foi obtido, e o comportamento não foi reproduzido na versão real do aplicativo. Portanto, a fonte não permite atribuir a causa ao exportador ou ao importador. Ela oferece um exemplo concreto para discutir correção monetária e compatibilidade de contratos externos.

O impacto potencial não depende do valor de um centavo. Um consumidor pode rejeitar o arquivo inteiro, importar parcialmente ou exigir conciliação manual quando um campo viola o formato esperado. A investigação precisaria separar formato inválido, mudança válida não suportada pelo consumidor e divergência de conteúdo. Essas situações demandam respostas diferentes; “o arquivo abre” não é um critério suficiente para concluir que a integração está correta.

O laboratório demonstra serialização de centavos inteiros, leitura de uma representação decimal estrita e importação idempotente de um fragmento XML inspirado em campos OFX. Ele não produz um arquivo OFX completo nem certifica interoperabilidade com aplicativos financeiros. Essa escolha permite testar invariantes pequenas e relevantes, deixando claro que uma implementação real ainda precisaria da especificação de versão, metadados obrigatórios, contratos de consumidores e validação de segurança de arquivos.

NuCommunity - Extrato OFX com erro

ANÁLISE E PROPOSTA EXTERNA

Dinheiro, representação e identidade são três contratos





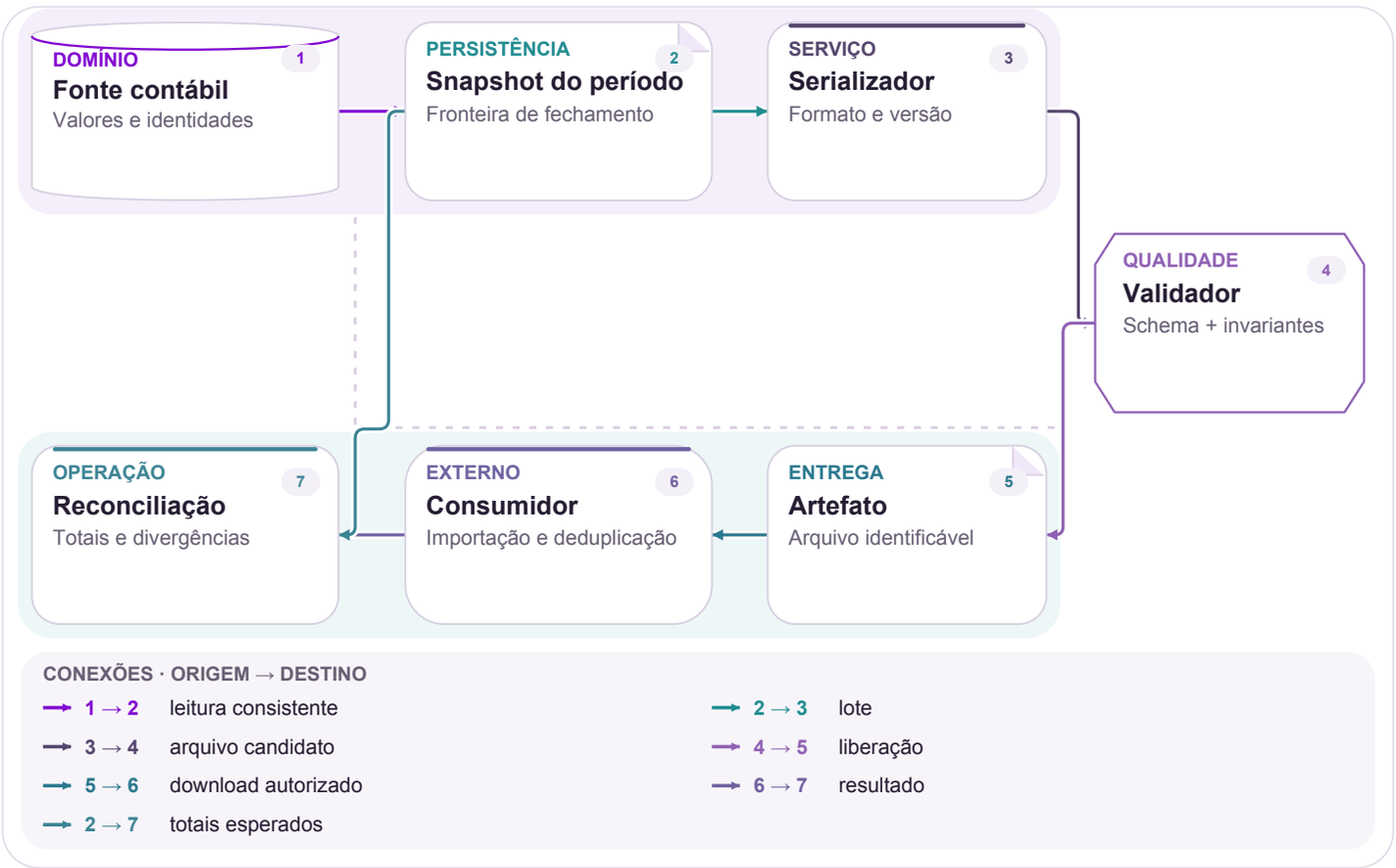
O valor monetário existe independentemente da string usada para transportá-lo. No modelo local, um centavo negativo é o inteiro -1. A representação externa correspondente é uma string com sinal, parte inteira e duas casas decimais. Separar essas camadas evita depender de arredondamentos de ponto flutuante e torna os casos de fronteira explícitos. O exportador deve receber uma unidade conhecida, não adivinhar se um número representa reais, centavos ou outra moeda.

O terceiro contrato é a identidade da transação. Uma reemissão do mesmo período não deveria transformar lançamentos já importados em novas movimentações. Por isso, a identidade precisa permanecer estável quando o conteúdo não mudou. Se uma correção altera o conteúdo associado à mesma identidade, o consumidor precisa de uma política clara: rejeitar conflito, aplicar uma revisão reconhecida ou seguir um mecanismo de correção do formato. O laboratório escolhe rejeitar conflito e preservar o lote anterior.

Essas regras são independentes da aparência do documento. Um PDF visualmente correto pode coexistir com um arquivo de integração inválido. A estratégia de qualidade deve tratar cada formato como contrato próprio, com testes de valor, datas, identidade e completude. A pergunta inicial da investigação seria qual representação foi gerada e qual consumidor a interpretou, preservando o arquivo e sua versão como evidência técnica.

ARQUITETURA PROPOSTA • FRONTEIRAS EXPLÍCITAS

Exportar com identidade e importar com reconciliação



Desenho autoral proposto; não representa a arquitetura interna do Nubank.

A proposta fixa primeiro a fronteira do período e a origem dos lançamentos. O serializador transforma valores e identidades sem redefinir sua semântica. Um validador separado confere formato e invariantes antes de liberar o artefato. A reconciliação compara o resultado consumido com o snapshot esperado, permitindo encontrar problemas que uma simples validação sintática não detecta.

A reemissão deve indicar sua relação com o arquivo anterior e preservar identidades quando os lançamentos são os mesmos. Isso reduz risco de duplicação no consumidor, mas depende do contrato externo adotado. O laboratório demonstra esse princípio com FITID em um fragmento simplificado, sem afirmar conformidade completa com uma versão OFX.

O download também é uma fronteira de autorização: um identificador de artefato não deve permitir acesso irrestrito ao extrato. Persistência, retenção e distribuição seriam decisões adicionais. O desenho concentra a discussão na correção do conteúdo e na possibilidade de reproduzir sua geração.

ENGENHARIA EXPLICADA

Serializar o sinal sem converter dinheiro para float

PYTHON / PROPOSTA AUTORAL

```python
# Entrada monetária: inteiro em centavos.
# Saída externa: decimal com ponto e duas casas.
# O parser rejeita formatos fora deste contrato sintético.
# Não existe etapa de arredondamento neste percurso.
def format_cents(cents):
    if type(cents) is not int:
        raise ValueError("integer_cents_required")
    sign = "-" if cents < 0 else ""
    units, remainder = divmod(abs(cents), 100)
    return f"{sign}{units}.{remainder:02d}"

def parse_cents(text):
    if not re.fullmatch(r"-?(?:0|[1-9][0-9]*)\.[0-9]{2}", text):
        raise ValueError("invalid_decimal_format")
    negative = text.startswith("-")
    units, fraction = text.lstrip("-").split(".")
    result = int(units) * 100 + int(fraction)
    return -result if negative else result
```

Trecho do laboratório sintético; execução verificada no arquivo completo.

A formatação usa o valor absoluto para separar unidades e centavos, aplicando o sinal apenas no início. Isso mantém -1 como -0.01, sem combinações inválidas entre ponto e sinal. O parser aceita um formato restrito e reconstrói o inteiro com multiplicação e soma, sem passar por float.

O contrato do laboratório exige duas casas decimais e não representa todas as variantes aceitas por versões ou consumidores OFX reais. Essa restrição é deliberada para testar roundtrip e casos inválidos. Uma implementação de exportação precisaria alinhar o formato à especificação escolhida e manter testes de compatibilidade com consumidores relevantes.

ENGENHARIA EXPLICADA

Clojure: manter exatidão até a fronteira textual

CLOJURE / PROPOSTA AUTORAL

```clojure
(require '[clojure.string :as str])

(defn money-text [cents]
  ;; Valor inteiro e unidade definidos pelo contrato proposto.
  (when-not (integer? cents)
    (throw (ex-info "integer-cents-required" {})))
  (let [value (bigint cents)
        negative? (neg? value)
        magnitude (if negative? (- value) value)
        units (quot magnitude 100)
        fraction (long (rem magnitude 100))]
    (str (when negative? "-")
         units "." (format "%02d" fraction))))

(defn parse-money [text]
  (when-not (re-matches #"-?(0|[1-9][0-9]*)\.[0-9]{2}" text)
    (throw (ex-info "invalid-decimal-format" {})))
  (let [negative? (.startsWith text "-")
        unsigned (if negative? (subs text 1) text)
        [units fraction] (str/split unsigned #"\.")
        cents (+ (* (bigint units) 100) (bigint fraction))]
    (if negative? (- cents) cents)))

;; Propriedade pretendida, a verificar no runtime de destino:
;; (= x (parse-money (money-text x)))
;; Nenhuma etapa exige conversão para Double.
```

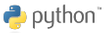
Serializador proposto; Clojure não executado.

O exemplo usa inteiros arbitrários para evitar que a obtenção do valor absoluto dependa do limite de um inteiro de tamanho fixo. Apenas o resto entre zero e noventa e nove é convertido para long ao formatar duas casas. A representação textual é construída no limite de saída, preservando a unidade em centavos no domínio.

O trecho é didático e não foi executado em Clojure. Ele também não define moeda, escala variável ou limites aceitos pelo sistema de destino. Essas condições pertencem ao contrato de exportação. Os testes executados em Python verificam a propriedade correspondente no modelo local, não a compilação desta variante.

ANÁLISE E PROPOSTA EXTERNA

O snapshot explica por que dois arquivos diferem


Um extrato precisa declarar o período e a fronteira de informação que representa. Transações pendentes, lançamentos posteriores e estornos podem alterar a visão de um intervalo quando o arquivo é reemitido. Eu registraria a versão de geração e o snapshot utilizado, permitindo distinguir uma mudança legítima do conjunto de dados de uma regressão no serializador. Sem essa referência, dois arquivos diferentes podem parecer contraditórios mesmo quando foram gerados em instantes distintos.

O contrato de identidade também precisa sobreviver à reemissão. A proposta não usaria a posição da linha ou o instante do download como identificador de transação. Uma ordenação diferente não deveria produzir lançamentos novos no consumidor. No laboratório, o campo id permanece estável e é transportado como FITID. Se o mesmo id chegar com outro valor ou descrição, o importador sintético rejeita o lote, preservando o estado anterior em vez de escolher silenciosamente uma versão.

Essa política é conservadora e não substitui mecanismos de correção previstos por um formato real. Ela evidencia uma pergunta que precisa ser respondida antes da integração: como o consumidor reconhece uma correção legítima sem duplicar ou apagar informação? A resposta deve aparecer em documentação, testes de compatibilidade e orientação de suporte, especialmente quando uma reemissão é oferecida após um incidente confirmado.

ENGENHARIA EXPLICADA

XML deve ser construído como estrutura, não concatenado

PYTHON / PROPOSTA AUTORAL

```python
# Fragmento didático: BANKTRANLIST não é um documento OFX completo.
# FITID preserva a identidade entre reemissões.
# O gerador XML cuida de escapar o texto da descrição.
# A ordem recebida é preservada neste fragmento.
# Identidades repetidas são recusadas antes da publicação.
# O consumidor de teste reconcilia o resultado do lote.
def export_transactions(transactions):
    root = ET.Element("BANKTRANLIST")
    seen = set()
    for tx in transactions:
        if tx["id"] in seen:
            raise ValueError("duplicate_transaction")
        seen.add(tx["id"])
        row = ET.SubElement(root, "STMTTRN")
        ET.SubElement(row, "FITID").text = tx["id"]
        ET.SubElement(row, "TRNAMT").text = format_cents(tx["cents"])
        ET.SubElement(row, "MEMO").text = tx["memo"]
    return ET.tostring(root, encoding="unicode")
```

Trecho do laboratório sintético; execução verificada no arquivo completo.

O gerador cria elementos e atribui texto, permitindo que a biblioteca escape caracteres como ampersand e sinais de comparação. Concatenar descrições diretamente em uma string XML poderia quebrar a estrutura. O conjunto seen recusa identidades duplicadas na fonte, evitando produzir um lote ambíguo.

O resultado é apenas um fragmento usado pelos testes. Ele omite cabeçalhos, datas e outros elementos exigidos por um arquivo OFX real. Também não transforma o parser do laboratório em um serviço seguro para entrada hostil: os testes trabalham com fixtures controladas. Uma integração real precisaria de limites, validação e revisão apropriada do processamento de arquivos.

ENGENHARIA EXPLICADA

Roundtrip, reemissão e conflito precisam ser demonstrados

PYTHON / PROPOSTA AUTORAL

```python
def test_roundtrip_range_of_integer_cents(self):
    for cents in range(-10000, 10001):
        self.assertEqual(parse_cents(format_cents(cents)), cents)

def test_invalid_formats_and_float_are_rejected(self):
    for text in (".-1", "1,00", "1.0", "1e2", " 1.00", "01.00"):
        with self.subTest(text=text), self.assertRaises(ValueError):
            parse_cents(text)
    for cents in (0.1, True):
        with self.assertRaises(ValueError):
            format_cents(cents)

def test_reissue_does_not_duplicate_import(self):
    document = export_transactions([dict(id="tx1", cents=-123, memo="x")])
    consumer = Importer()
    self.assertEqual(consumer.load(document), -123)
    self.assertEqual(consumer.load(document), -123)
    self.assertEqual(len(consumer.entries), 1)

def test_conflicting_reissue_is_atomic(self):
    consumer = Importer()
    consumer.load(export_transactions([dict(id="tx1", cents=-123, memo="x")]))
    changed = [dict(id="tx2", cents=100, memo="new"),
               dict(id="tx1", cents=-124, memo="x")]
    with self.assertRaises(ValueError):
        consumer.load(export_transactions(changed))
    self.assertEqual(consumer.entries, {"tx1": (-123, "x")})
```

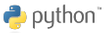
Trecho do laboratório sintético; execução verificada no arquivo completo.

O roundtrip percorre 20.001 valores inteiros de teste, incluindo negativos, zero e positivos. Outros cenários rejeitam formatos inválidos e floats, repetem a importação do mesmo documento e tentam aplicar uma reemissão conflitante depois de um lançamento novo no mesmo lote. O estado anterior deve permanecer intacto nesse conflito.

O arquivo passou em sete testes. Isso demonstra as propriedades do fragmento e do importador sintético, não compatibilidade com aplicativos financeiros ou todas as versões OFX. Para produção, eu acrescentaria fixtures autorizadas de consumidores reais e reconciliação de saldo por período, preservando a distinção entre teste local e certificação externa.

ANÁLISE E PROPOSTA EXTERNA

Observar validade, completude e consumo separadamente

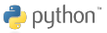

Eu separaria a operação de exportação em três resultados: arquivo gerado, arquivo validado e arquivo consumido corretamente quando houver evidência disponível. O primeiro mede disponibilidade do gerador; o segundo mede conformidade com o contrato; o terceiro mede interoperabilidade. Confundir esses estados permite declarar sucesso porque houve download, mesmo quando o cliente precisou editar o arquivo manualmente para importá-lo.

As métricas propostas incluiriam falhas por categoria de validação, divergência de contagem de lançamentos, diferença entre totais esperados e exportados e reemissões associadas a incidentes. Categorias como sinal inválido, identidade duplicada ou campo obrigatório ausente são úteis para diagnóstico. Valores individuais, descrições de compras e números de conta não deveriam ser expostos em métricas agregadas. Uma amostra de arquivo para investigação exigiria controle e minimização próprios.

O alerta mais importante seria uma regressão de contrato após mudança de versão, mesmo com baixa incidência. Um único caso de centavo negativo pode revelar uma classe inteira de valores afetados. A investigação deve produzir uma consulta de alcance sobre dados autorizados, sem presumir que todos os arquivos estão errados. A comunicação ao cliente dependeria dessa confirmação e ofereceria reemissão ou orientação proporcional, distinguindo formato corrigido de alteração de conteúdo financeiro.

ANÁLISE E PROPOSTA EXTERNA

Migrar o serializador com comparação semântica


Eu começaria gerando arquivos candidatos a partir de fixtures e snapshots autorizados, sem distribuí-los aos clientes. A comparação entre versões precisaria ser semântica: ordem de atributos ou espaçamento podem mudar sem alterar o conteúdo, enquanto um sinal ou uma identidade diferente pode alterar a interpretação financeira. O diff deveria destacar valores, datas, moedas, identidades e contagens, evitando tratar toda diferença textual como equivalente.

Depois, um conjunto de consumidores representativos importaria os arquivos em ambiente controlado. A aceitação exigiria conferir o resultado, não apenas ausência de erro. Se um consumidor importar parcialmente sem avisar, a reconciliação deve detectar a diferença. A migração também testaria reemissão do mesmo período e convivência entre versões, garantindo que o cliente não duplique lançamentos ao trocar o arquivo corrigido.

O rollout teria capacidade de interromper a nova geração e manter acesso a artefatos já válidos. Um rollback do serializador não deveria alterar retroativamente a identidade dos documentos ou esconder quais versões foram afetadas. Eu prepararia um procedimento de alcance e reemissão antes de ampliar o tráfego. Essa disciplina trata o extrato como uma interface externa de engenharia, com compatibilidade e responsabilidade, e não como um botão de download de baixo risco.

ANÁLISE E PROPOSTA EXTERNA

Precisão exige limites de entrada e de recurso





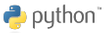
Usar inteiros evita uma classe de arredondamento, mas não resolve sozinho o processamento de arquivos. O sistema ainda precisa validar escala, moeda, faixa de valores e tamanho de lote. Um inteiro arbitrariamente grande pode ser matematicamente exato e incompatível com o armazenamento ou consumidor de destino. O contrato deve declarar esses limites e rejeitar entradas fora deles antes de produzir um artefato que pareça utilizável.

O laboratório lê XML gerado pelas próprias fixtures. Ele não foi desenhado para analisar documentos hostis enviados por terceiros. Uma aplicação real precisaria considerar tamanho máximo, profundidade, tempo de processamento, recursos do parser e campos inesperados. Também deve controlar a autorização para baixar extratos e evitar que logs de erro incluam conteúdo financeiro inteiro. Exatidão monetária e segurança de processamento são propriedades complementares, com testes e responsáveis diferentes.

Para desempenho, eu mediria memória e tempo de geração por quantidade de lançamentos, preservando uma visão consistente do período. Processar em lotes pode reduzir memória, mas exige cuidado com ordenação, fechamento do arquivo e recuperação de geração interrompida. Um arquivo parcialmente escrito não deve ser liberado como concluído. A publicação do artefato precisaria ocorrer somente depois de validação e confirmação de completude.

ANÁLISE E PROPOSTA EXTERNA

ADR: serializador central e identidade estável

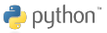

A decisão proposta centraliza a conversão monetária e a validação de identidade em uma fronteira de exportação, com contratos por formato. A alternativa de cada produto formatar seus próprios valores pode parecer rápida, mas multiplica variações de sinal, escala e arredondamento. A alternativa de converter tudo para uma string genérica cedo demais perde informação de unidade e dificulta validação. Eu manteria valores tipados no domínio e formatação apenas no limite externo.

Para reemissão, a identidade estável é uma invariante. O conteúdo corrigido precisa seguir uma política explícita de revisão, em vez de receber identificadores novos sem explicação. O laboratório escolhe conflito atômico porque torna a ambiguidade visível. Uma implementação completa poderia adotar outra semântica prevista pelo formato e pelo consumidor, mas teria de demonstrar que ela não duplica nem apaga lançamentos indevidamente.

O custo aceito é manter uma suíte de compatibilidade e revisar mudanças aparentemente pequenas de serialização. O benefício é reduzir falhas silenciosas e conciliação manual. Reabriria o ADR ao adicionar moedas com outras escalas, novos formatos ou consumidores com requisitos incompatíveis. Essas mudanças deveriam criar contratos versionados, não condicionais ocultas espalhadas pelo código. A clareza da fronteira permite evoluir sem transformar cada exportação em uma exceção difícil de testar.

ANÁLISE E PROPOSTA EXTERNA

Como eu transformaria um relato pequeno em qualidade sistêmica


Como EM, eu trataria a investigação com alcance definido: obter evidência autorizada, reproduzir a representação, identificar consumidores afetados e separar conteúdo de formato. O time de extratos, a plataforma financeira e o suporte teriam responsáveis claros. A primeira entrega seria uma reprodução automatizada do defeito confirmado ou, se ele não se reproduzisse, uma classificação honesta do que ainda falta investigar. A pressão para responder rápido não justificaria anunciar causa sem evidência.

Como Staff, eu buscaria a classe de falha: sinal, escala, identidade, fronteira temporal ou compatibilidade. O caso do centavo orienta uma suíte mais ampla, como o roundtrip de 20.001 valores do laboratório, mas não prova que a implementação real possui o mesmo problema. A contribuição é converter um exemplo pequeno em invariantes reutilizáveis, com decisões de serialização e reemissão que outras equipes consigam revisar.

O valor seria calculado por horas de conciliação manual evitadas, redução de chamados e menor custo de reemissão, com volumes internos medidos. Eu não presumiria perda financeira a partir de um arquivo rejeitado. A pergunta de entrevista seria: “Se reemitimos o extrato corrigido, como demonstramos que o consumidor recupera o conteúdo certo sem duplicar o que já importou?” Essa pergunta liga precisão de código, contrato externo e confiança do cliente.

ANÁLISE E DECISÃO

08 · A entrega é parte do produto

08








Nubank: conta, cartão e mais

Captura da fonte pública consultada em 23/10/2026. Relato não equivale a incidente reproduzido.

Uma resenha pública da loja de aplicativos descreve dificuldade com a reposição de um cartão e pouca clareza no acompanhamento. Isso é um sinal para investigação, sem diagnóstico confirmado de integração, transportadora ou atendimento. O desfecho não está disponível. A pergunta útil é quais informações permitiriam ao cliente entender a situação e ao operador agir sem reiniciar a conversa a cada contato.

O objeto da proposta é a continuidade da jornada física. Pedido aceito, produção, expedição, tentativa, exceção e recebimento têm responsáveis distintos. Mesmo que cada participante cumpra seu contrato local, o cliente pode permanecer sem uma orientação concreta. Um painel que apenas repete o último texto recebido não resolve esse problema. Precisamos distinguir evento, estado interpretado, prazo prometido e próxima ação autorizada.

Antes de construir, eu escolheria uma amostra de entregas com recontato, confrontaria os eventos disponíveis com as respostas dadas e procuraria lacunas reproduzíveis. Entrevistaria atendimento e operações para entender decisões que hoje dependem de contexto fora das ferramentas. A intervenção inicial poderia ser informação melhor, reconciliação ou uma política de exceção; não pressuporia um serviço novo. O critério de sucesso seria reduzir tempo sem responsável e contatos repetidos, preservando a segurança da entrega. Este capítulo testa apenas uma parte: ingestão durável de eventos sintéticos.

Google Play - resenha de Murilo Meyrelles sobre entrega de cartão

ANÁLISE E DECISÃO

Uma remessa não é uma sequência de notificações



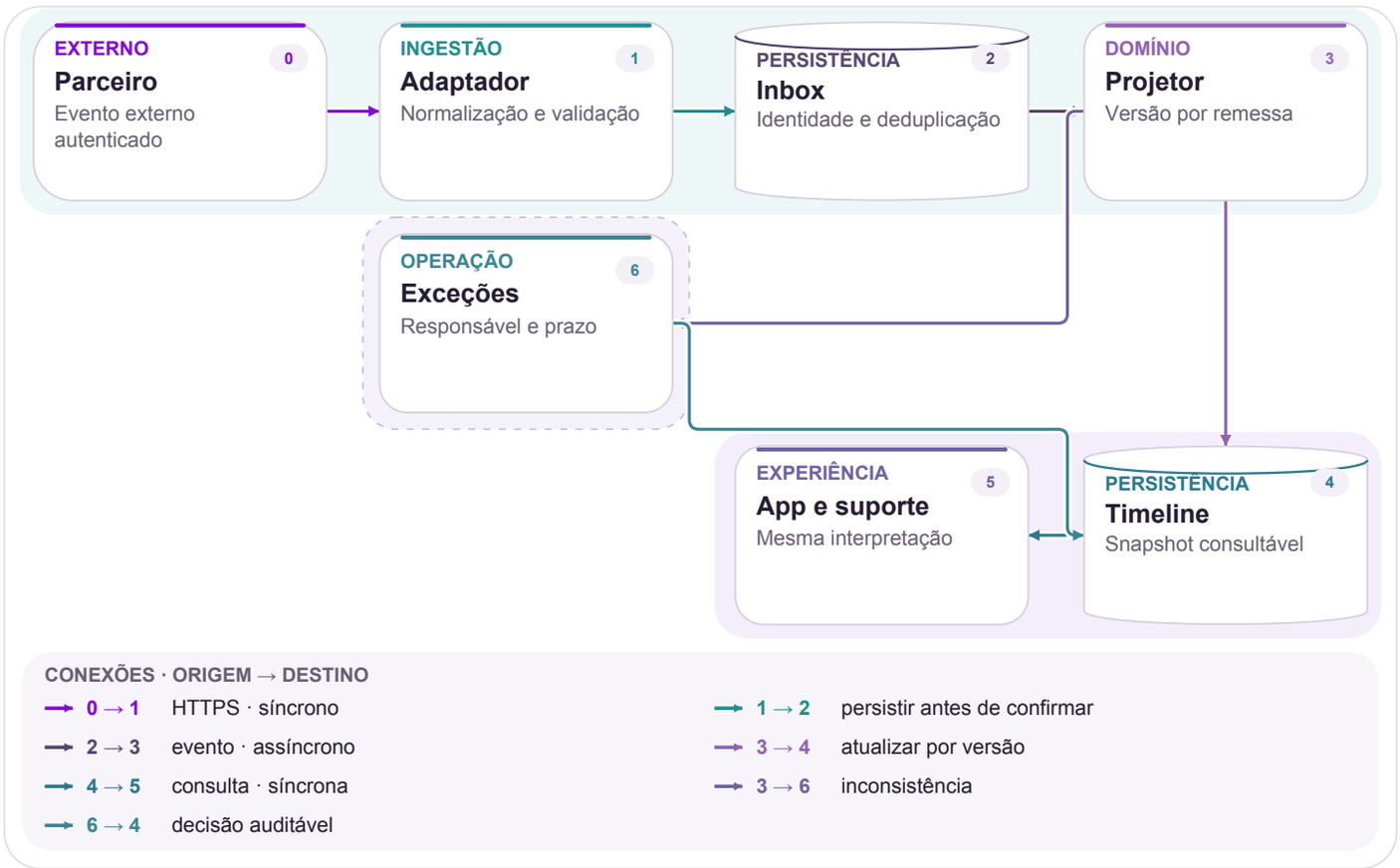
O identificador de pedido liga a jornada ao cliente; o identificador de remessa representa uma tentativa física específica. Reemissão, devolução e troca de endereço não deveriam ser disfarçadas como simples alterações de texto na mesma linha do tempo. Sem essa distinção, um evento tardio da entrega anterior pode contaminar a visão do cartão novo e produzir uma ação operacional errada.

Na proposta, cada atualização normalizada possui identidade, versão monotônica da remessa, estado, origem e próximo passo. O laboratório reduz o vocabulário a trânsito, exceção e entregue. A simplificação é deliberada: não modela todos os estados de uma transportadora. Uma remessa entregue não volta para trânsito nesse modelo; devolução posterior exige outro processo. Exceção, por sua vez, pode voltar a trânsito após uma solução legítima.

O domínio precisa definir quem pode afirmar cada estado. Receber um webhook não comprova entrega ao destinatário correto. A origem pode ser confiável para registrar uma tentativa, mas insuficiente para encerrar uma contestação. Por isso, o estado exibido e a decisão de encerrar atendimento são contratos separados. O mapa inclui também situações sem atualização: silêncio não é sucesso nem prova de extravio. O próximo passo deve dizer se há espera, investigação ou ação do cliente, indicando um prazo operacional cujo responsável possa ser identificado.

ARQUITETURA PROPOSTA · NÃO INTERNA

Do parceiro à próxima ação



Arquitetura externa proposta. Caixas representam responsabilidades; setas explicitam contratos.

O adaptador é a fronteira de confiança: verifica origem, formato e identidade antes de aceitar o evento. A confirmação ao parceiro depende de persistência, não da velocidade da atualização visual. Um consumidor pode reiniciar e repetir trabalho; o projetor precisa reconhecer essa repetição sem avançar duas vezes a mesma remessa. Kafka seria uma opção de transporte, não uma garantia de correção do domínio.

A timeline é uma leitura derivada. O registro bruto normalizado permanece disponível para explicar como o estado surgiu, respeitando retenção e minimização. Divergência não deve ser resolvida sobrescrevendo silenciosamente o histórico. O caminho de exceção possui dono e prazo, permitindo reconciliação com o parceiro. App e suporte consultam a mesma interpretação, com permissões diferentes: o cliente recebe orientação suficiente; o operador recebe contexto necessário para agir. A separação evita transformar detalhes internos em uma experiência confusa e permite testar cada fronteira com eventos falsos.

CÓDIGO COM ESCOPO EXPLÍCITO

Ingestão: a decisão cabe em uma transação

PYTHON / PROPOSTA AUTORAL

```python
def ingest(self, event, crash=False):
    payload = json.dumps(event, sort_keys=True)
    digest = hashlib.sha256(payload.encode()).hexdigest()
    if event['status'] not in {'transit', 'exception', 'delivered'}:
        raise ValueError('status desconhecido')
    if type(event['seq']) is not int or event['seq'] < 1:
        raise ValueError('sequência inválida')
    with self.db:
        # A decisão de versão deve usar a mesma transação da gravação.
        self.db.execute('BEGIN IMMEDIATE')
        old = self.db.execute('SELECT digest FROM events WHERE event_id=?',
                              (event['event_id'],)).fetchone()
        if old:
            if old[0] != digest:
                raise ValueError('evento conflitante')
            return 'duplicate'
        row = self.db.execute('SELECT seq,status FROM shipments '
            'WHERE shipment_id=?', (event['shipment_id'],)).fetchone()
        self.db.execute('INSERT INTO events VALUES (?,?)',
                        (event['event_id'], digest))
        if row and event['seq'] <= row[0]:
            return 'stale'
        if row and row[1] == 'delivered' and event['status'] != 'delivered':
            raise ValueError('regressão terminal')
        self.db.execute('INSERT OR REPLACE INTO shipments VALUES (?,?,?,?)',
            (event['shipment_id'], event['seq'], event['status'],
             event['next_step']))
        if crash:
            raise RuntimeError('falha antes do commit')
    return 'applied'
```

Fragmento literal de labs/case_08.py. Python executado localmente; dados e contratos sintéticos.

Este fragmento vem do laboratório executado. BEGIN IMMEDIATE adquire a transação de escrita antes de ler identidade ou versão; isso impede que conexões concorrentes decidam sobre um estado obsoleto e regridam a projeção. Identidade e projeção são gravadas juntas. Uma exceção antes do commit desfaz ambas. Repetição com conteúdo igual é reconhecida; identidade reutilizada com outro conteúdo provoca conflito.

A sequência é uma premissa do contrato sintético. Uma integração real precisaria verificar se o parceiro fornece ordenação confiável ou construir outra regra de reconciliação. O código não autentica webhooks, não opera Kafka e não decide reemissão. Ele isola a pergunta sobre consistência local para que o comportamento possa ser inspecionado.

CÓDIGO COM ESCOPO EXPLÍCITO

Clojure: normalizar sem adivinhar o parceiro

CLOJURE / PROPOSTA AUTORAL

```clojure
(ns portfolio.delivery)

(def allowed-status
  #{:transit :exception :delivered})

(defn normalize-event [raw]
  (let [{:keys [event-id shipment-id sequence
                status next-step]} raw]
    (when-not (and (string? event-id)
                   (string? shipment-id)
                   (integer? sequence)
                   (pos? sequence)
                   (contains? allowed-status status)
                   (string? next-step))
      (throw (ex-info "Invalid partner contract"
                      {:kind :invalid-event})))
    {:event-id event-id
     :shipment-id shipment-id
     :sequence sequence
     :status status
     :next-step next-step}))

;; Autenticação ocorre antes desta função pura.
;; Persistência e confirmação ocorrem depois dela.
```

Trecho Clojure didático não executado; função e contrato autorais propostos.

A função proposta devolve um valor normalizado ou um erro de domínio. Ela não faz chamadas, não confirma recebimento e não interpreta ausência de campo como sucesso. Isso permite exercitar o contrato com conjuntos pequenos de exemplos antes de integrar qualquer fornecedor. O nome de cada campo pertence a esta proposta; não foi extraído de uma API do Nubank.

Na adaptação de produção, campos específicos do parceiro ficariam fora do núcleo. A equipe teria de decidir compatibilidade, autenticação e tratamento de eventos inválidos. Encaminhar tudo para uma fila de erro sem responsável apenas troca uma falha visível por uma pendência invisível.

CÓDIGO COM ESCOPO EXPLÍCITO

Envelope de evento com responsabilidade explícita

JSON / PROPOSTA AUTORAL

```json
{
  "contract": "portfolio.delivery.v1",
  "event_id": "demo-event-42",
  "shipment_id": "demo-shipment-7",
  "sequence": 12,
  "status": "exception",
  "occurred_at": "2026-10-20T14:00:00Z",
  "received_at": "2026-10-20T14:01:00Z",
  "origin": {
    "partner": "synthetic-carrier",
    "schema_revision": 1
  },
  "next_action": {
    "owner": "delivery-operations",
    "kind": "review-attempt",
    "due_at": "2026-10-21T14:00:00Z"
  },
  "privacy": {
    "contains_full_address": false,
    "customer_visible": true
  }
}
```

Contrato JSON autoral, sintético e não integrado; campos adicionais não são implementados pelo laboratório.

O envelope estende o laboratório para discutir tempo e operação. Horário de ocorrência e horário de recebimento respondem perguntas diferentes: o primeiro ajuda a reconstruir o percurso; o segundo mede atraso de integração. Nenhum deles substitui automaticamente uma sequência contratual. Relógios podem divergir e eventos podem ser corrigidos pelo parceiro.

O próximo passo tem responsável e vencimento, mas não inclui endereço completo nem dados pessoais de exemplo. Em um produto real, a visibilidade seria derivada de uma política, não confiada a um booleano recebido do parceiro. O JSON é documentação de contrato proposto e não um payload de uma integração existente.

ANÁLISE E DECISÃO

Falhas que não podem virar reemissão automática



Uma entrega atrasada e uma entrega perdida podem parecer iguais para quem olha apenas o último horário registrado. A primeira falha relevante é ausência de evento: o consumidor pode estar saudável enquanto o parceiro parou de enviar atualizações. O sistema deve acompanhar tanto atraso de consumo quanto silêncio por remessa elegível. A correção pode exigir consulta ao parceiro, investigação humana ou mudança do prazo informado.

Outra falha é a ordem invertida. Uma confirmação de recebimento pode chegar antes de uma tentativa anterior. O laboratório preserva a versão maior, mas a regra só funciona porque a sequência é assumida como confiável. Quando isso não existe, ordenar pelo relógio de chegada produz uma história falsa. Uma integração robusta mantém evidências e encaminha contradições, especialmente quando elas mudariam uma ação irreversível.

Reemissão merece uma barreira própria. Antes de emitir outro cartão, é necessário relacionar remessas, estado do cartão e decisão operacional. Um evento duplicado não pode gerar duas solicitações e uma confirmação tardia precisa chegar ao mesmo caso. Eu trataria a reemissão como comando idempotente separado da ingestão. O consumidor da timeline nunca deveria possuir permissão implícita para executar essa ação apenas porque reconheceu uma palavra como extravio. Essa separação limita impacto e facilita explicar o que foi decidido.

CÓDIGO COM ESCOPO EXPLÍCITO

Testar replay e ordem com resultado observável

PYTHON / PROPOSTA AUTORAL

```python
def test_duplicate(self):
    self.model.ingest(self.event())
    self.assertEqual(self.model.ingest(self.event()), 'duplicate')
    self.assertEqual(len(self.model.state()), 1)

def test_conflicting_identity(self):
    self.model.ingest(self.event())
    with self.assertRaises(ValueError):
        self.model.ingest(self.event(status='delivered'))
    self.assertEqual(self.model.state()[0][2], 'transit')

def test_out_of_order(self):
    self.model.ingest(self.event(seq=5, status='delivered'))
    result = self.model.ingest(self.event(event_id='e2', seq=2))
    self.assertEqual(result, 'stale')
    self.assertEqual(self.model.state()[0][2], 'delivered')

def test_terminal_guard(self):
    self.model.ingest(self.event(status='delivered'))
    with self.assertRaises(ValueError):
        self.model.ingest(self.event(event_id='e2', seq=2))
```

Fragmento de testes de labs/case_08.py. Resultado: 8 testes passaram; ver results_08.txt.

Os oito testes do arquivo completo passaram. O fragmento cobre repetição, conflito, ordem e barreira terminal. Os demais verificam rollback, reabertura, aquisição da transação antes das leituras e oito conexões concorrentes. A última verificação exige que a projeção preserve a maior sequência, independentemente da ordem de execução. As asserções examinam estado persistido.

Esse resultado não testa um parceiro real nem prova recuperação de queda de energia. A falha injetada é uma exceção antes do commit. O próximo nível de validação precisaria incluir o adaptador, confirmações perdidas, reprocessamento com versão nova e casos operacionais rotulados. Um teste pequeno é útil quando sua fronteira permanece clara.

ANÁLISE E DECISÃO

O painel começa pelo tempo em incerteza



Eu organizaria observabilidade por pergunta operacional. Quantas remessas estão sem próximo passo? Há quanto tempo? Qual responsável pode agir? Um gráfico de eventos por segundo ajuda a detectar interrupções, mas não mostra se o cliente está preso. Uma fila de exceções precisa permitir recorte por parceiro, etapa e idade, sem colocar identificadores de clientes em rótulos de métricas de alta cardinalidade.

As métricas técnicas incluem taxa de duplicação, conflitos de identidade, eventos descartados por versão e atraso entre ocorrência e processamento. Cada uma exige interpretação. Duplicação pode ser efeito normal de retry; conflito de conteúdo é qualitativamente diferente. Uma queda de eventos antigos pode indicar melhora de integração ou perda silenciosa de dados. Comparar totais com uma fonte de reconciliação evita comemorar um painel aparentemente limpo.

No nível de experiência, mediria contatos por entrega, tempo até ação resolutiva e encerramentos reabertos. Não misturaria uma solicitação de informação com uma contestação de recebimento. O alerta teria janela, condição, parceiro afetado e link para casos autorizados, além de uma ação possível. Se ninguém consegue agir quando ele dispara, falta contrato operacional. O objetivo é orientar trabalho e proteger a jornada; volume de alertas não é evidência de maturidade.

ANÁLISE E DECISÃO

Migração paralela antes da troca de confiança



A primeira implantação seria uma leitura paralela, alimentada pelos mesmos eventos que a experiência existente. Ela não enviaria mensagens nem abriria reemissões. Compararíamos estados em uma amostra estratificada de parceiros e tipos de remessa, anotando divergências com uma pessoa de operações. A amostra precisa incluir atrasos e exceções: avaliar apenas entregas simples cria uma confiança artificial no modelo.

Depois da reconciliação inicial, o suporte poderia consultar a nova timeline como informação complementar. Essa etapa testa vocabulário, utilidade do próximo passo e capacidade de encontrar o responsável. Só avançaria para exposição ao cliente quando as diferenças relevantes estivessem explicadas e o caminho de suporte estivesse preparado. A comunicação de prazos precisa seguir a política operacional, não um valor estático escolhido pelo time de interface.

A reversão separa leitura de comandos. É simples retirar uma nova timeline da tela, mas não desfazer uma reemissão já aceita por outro sistema. Por isso, a ativação de comandos viria depois, com chave de idempotência, auditoria e limite de exposição. O plano de rollout documentaria quais mudanças são reversíveis, quais exigem compensação e quem autoriza cada avanço. Esse desenho evita confundir feature flag com uma estratégia completa de recuperação.

ANÁLISE E DECISÃO

Endereço é dado sensível, não contexto para todo evento



Uma integração logística pode transportar nome, endereço, telefone e instruções de acesso. Esses dados não precisam circular em todos os eventos usados para atualizar uma timeline. Eu separaria referência operacional e dados necessários à execução física, limitando acesso por função. O identificador de remessa ainda pode ser vinculável a uma pessoa; chamá-lo de anônimo não elimina essa responsabilidade.

Na fronteira externa, validaria origem, integridade, janela de replay e limites de tamanho antes de processar conteúdo. Assinatura de webhook e chave de evento resolvem problemas diferentes: uma confirma a origem esperada, a outra evita repetir o mesmo fato. O laboratório não implementa autenticação. A escolha de mecanismo deve seguir o contrato real do parceiro e incluir rotação de credenciais sem interromper toda a operação.

Também evitaria respostas que revelam detalhes indevidos. O cliente pode receber informação sobre uma tentativa sem ver notas internas ou dados de terceiros. O suporte precisa de contexto suficiente, com trilha de acesso e revisão de permissões. Testes devem incluir conteúdo inesperado em campos livres, anexos enormes e eventos de outra remessa. Um payload bem formado não autoriza acesso a qualquer objeto. A segurança da cadeia depende dessas fronteiras tanto quanto da consistência da base.

ANÁLISE E DECISÃO

ADR 08 · Projeção versionada, comando separado



A decisão proposta é manter uma projeção de entrega versionada por remessa e reservar ações como reemissão a comandos explícitos. O contexto é uma cadeia com participantes independentes, eventos atrasados e consequências que extrapolam a tela. O objetivo é tornar o estado explicável e impedir que uma atualização de acompanhamento execute uma ação financeira ou física sem avaliação apropriada.

Uma alternativa é consultar o parceiro a cada abertura do app. Ela reduz necessidade de armazenamento local, mas transfere latência e disponibilidade externas para a experiência, dificulta histórico e pode aumentar custo. Outra é aceitar sempre a última mensagem recebida. É simples, porém transforma ordem de transporte em verdade de negócio. A projeção versionada oferece leitura previsível e permite reprocessamento, ao custo de operar consumidor, retenção e reconciliação.

Eu registraria duas condições para reconsiderar a escolha. A primeira é descobrir que nenhum parceiro fornece sequência ou snapshot comparável: seria necessário um modelo de precedência e investigação mais rico. A segunda é a carga operacional da reconciliação superar o benefício da projeção. Nesse cenário, reduziríamos escopo ou reveríamos contratos em vez de adicionar exceções silenciosas. A arquitetura deve permanecer subordinada à qualidade da informação e à capacidade real de operar os casos difíceis.

ANÁLISE E DECISÃO

Execução EM: um responsável até o cartão chegar



Como Engineering Manager, começaria com operações e atendimento, não com uma lista de tecnologias. Nomearia uma pessoa responsável pelo piloto e construiria um mapa dos pontos em que o caso muda de equipe. O acordo incluiria definição de conclusão, prazo para investigar divergência e disponibilidade de quem conhece os parceiros. Engenharia não pode assumir sozinha uma promessa que depende de produção, transporte e decisões de cartão.

O valor seria calculado a partir de uma população elegível, sem extrapolar a resenha individual para a base. Contatos evitados multiplicados pelo custo incremental de atendimento e reemissões evitadas multiplicadas pelo custo unitário são componentes possíveis. Deles seriam descontados integração, observabilidade, operação de exceções e manutenção. Capacidade recuperada no suporte não vira automaticamente economia de caixa. Ativação adicional só entraria com evidência de causalidade.

Eu revisaria semanalmente a qualidade das decisões e a carga de quem opera o processo. A equipe precisaria de espaço para corrigir contratos e documentação, além de desenvolver telas. A pergunta Staff seria: qual fato permite encerrar uma remessa quando parceiro e cliente discordam? Uma resposta madura explicita autoridade, evidência e escalonamento, reconhecendo que nenhum algoritmo elimina toda controvérsia. O resultado esperado é menos incerteza sustentada por um sistema que sabe quem deve agir.

ANÁLISE E DECISÃO

09 · Preservar a tarefa durante a evolução visual

09






Nubank: conta, cartão e mais

Captura da fonte pública consultada em 23/10/2026. Relato não equivale a incidente reproduzido.

Uma resenha pública questiona mudanças na visualização da fatura, incluindo relação entre ocultar gráfico, enxergar total e navegar entre meses. Não houve reprodução independente da versão citada. O sinal orienta uma pergunta de produto: quais informações devem permanecer disponíveis independentemente de uma preferência de apresentação? O objetivo deste capítulo é transformar essa pergunta em contratos verificáveis, sem declarar um defeito atual do aplicativo.

A fatura é uma visão financeira com expectativas estabelecidas. Pessoas voltam para localizar valor, vencimento e lançamentos, comparar períodos e entender situações como pagamento parcial ou estorno. Uma interface pode ganhar consistência visual e, ainda assim, aumentar esforço para essas tarefas. O risco aparece quando a implementação usa a presença de um componente decorativo como condição para renderizar dados essenciais.

Eu investigaria tarefas antes de escolher um novo layout. Observaria pessoas que usam a experiência regularmente, incluindo leitor de tela, ampliação de texto e conexão instável. Separaria preferência estética de erro de interpretação: ambos importam, mas exigem respostas diferentes. A proposta técnica mantém snapshot financeiro, seleção de período e preferências visuais em estados separados. O laboratório demonstra essa separação com dados sintéticos; a validação real precisaria incluir regras da fatura, pesquisa e os componentes efetivamente utilizados.

Google Play - resenha de Fernando Cordeiro sobre visualização da fatura

ANÁLISE E DECISÃO

Três estados, três responsabilidades



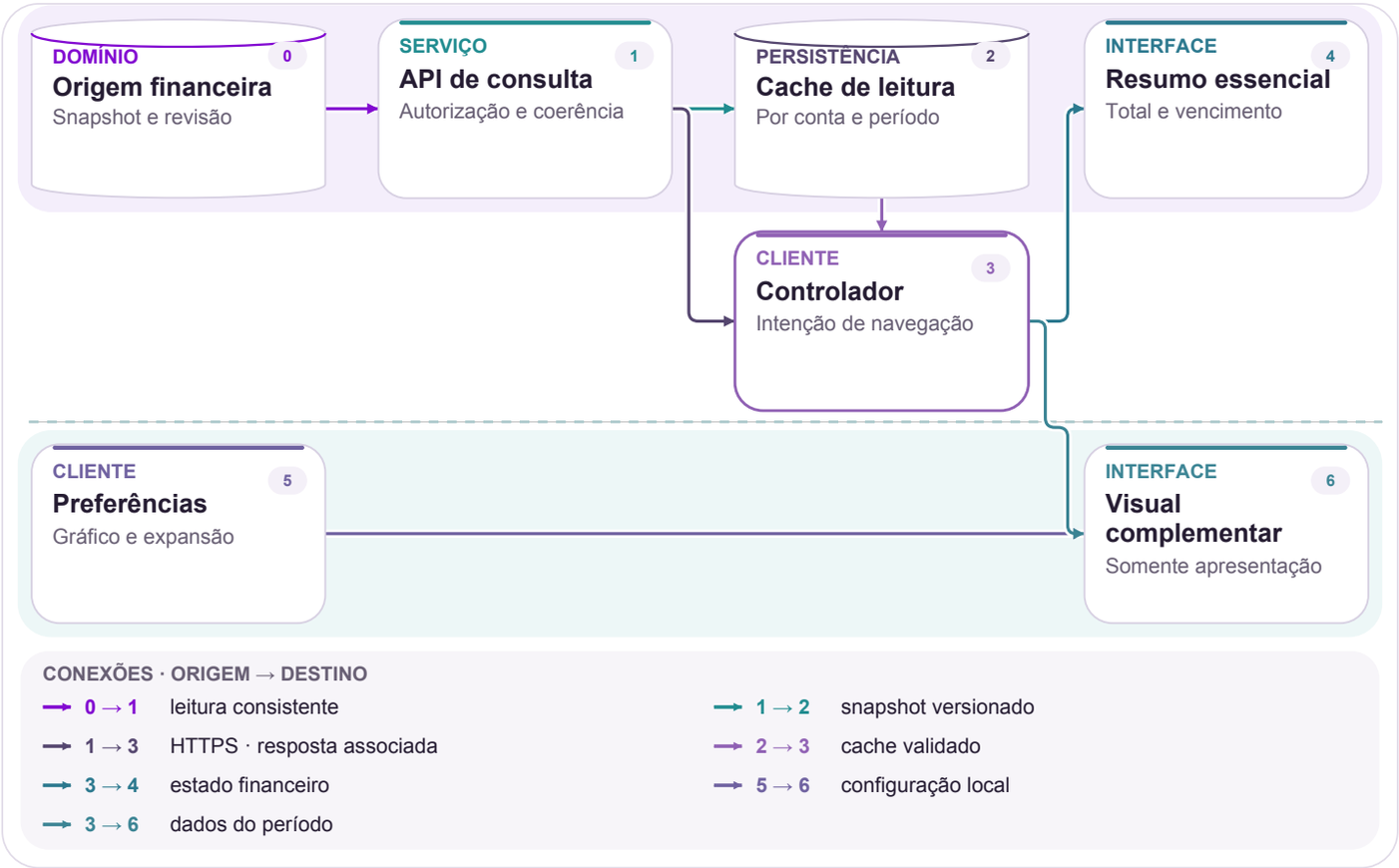
O primeiro estado é financeiro: período, revisão, total, vencimento e lançamentos de um snapshot coerente. O segundo é de navegação: qual período a pessoa escolheu e qual consulta está em andamento. O terceiro é de apresentação: gráfico visível, ordenação ou expansão de uma seção. Misturá-los cria dependências acidentais, como esconder o total ao desativar uma preferência visual ou substituir o mês atual quando chega uma resposta antiga.

No laboratório, Invoice exige uma tupla de lançamentos, valida centavos inteiros e confere a soma. O bloqueio de listas mutáveis preserva o snapshot após a validação. Essa equivalência é uma simplificação local, não a fórmula completa de uma fatura real. Encargos, pagamentos e ajustes exigiriam entradas explícitas ou outro contrato reconciliável. A revisão identifica conteúdo; duas respostas com a mesma revisão e vencimentos diferentes são um conflito, não uma atualização inocente.

A seleção de mês é explícita. Uma resposta de outro período não pode ser aplicada silenciosamente sobre o que está na tela. Quando o usuário troca de mês, o controlador mantém a intenção de navegação e associa a resposta à requisição pertinente. Preferências podem sobreviver à troca, mas não alteram o snapshot. Esse desenho ajuda a testar estados vazios, atrasados e conflitantes sem depender de um teste visual completo para cada regra de domínio.

ARQUITETURA PROPOSTA · NÃO INTERNA

Dados financeiros e preferência não se confundem



Arquitetura externa proposta. Caixas representam responsabilidades; setas explicitam contratos.

A proposta mantém o resumo essencial fora da condição que decide renderizar o gráfico. O controlador associa cada resposta ao período solicitado e impede que uma resposta atrasada troque o contexto escolhido. Um cache pode melhorar abertura, desde que indique atualização e nunca apresente uma revisão antiga como confirmação de um pagamento recente.

O desenho não atribui ao cliente a autoridade de calcular a dívida. A validação local compara consistência de um contrato sintético; em produção, a origem financeira define o significado dos campos. Separar preferências permite revisar visualização sem migrar dados financeiros. Também reduz o conjunto de estados impossíveis: um gráfico pode estar oculto enquanto total e vencimento continuam expostos. O teste arquitetural central é seguir uma interação até a dependência que ela modifica. Se esconder um componente muda o snapshot, a fronteira está errada ou precisa de uma justificativa explícita de produto.

CÓDIGO COM ESCOPO EXPLÍCITO

Um controlador que recusa respostas incoerentes

PYTHON / PROPOSTA AUTORAL

```python
def apply(self, invoice):
    invoice.validate()
    if invoice.period != self.invoice.period:
        raise ValueError('seleção explícita de período necessária')
    if invoice.revision < self.invoice.revision:
        return False
    if invoice.revision == self.invoice.revision:
        if invoice != self.invoice:
            raise ValueError('mesma revisão com conteúdo diferente')
        return False
    self.invoice = invoice
    return True

def select_period(self, invoice):
    self.invoice = invoice.validate()

def view(self):
    return dict(period=self.invoice.period,
                total_cents=self.invoice.total_cents,
                due_date=self.invoice.due_date,
                chart=self.invoice.entries if self.show_chart else None)
```

Fragmento literal de labs/case_09.py. Modelo Python local executado; não é implementação do app.

O fragmento do laboratório aplica revisões apenas ao período atualmente selecionado. Uma revisão menor é ignorada; uma revisão igual com conteúdo distinto é rejeitada. Preferência de gráfico influencia somente o campo chart da projeção. Total e vencimento são derivados do snapshot, preservando a tarefa principal quando o gráfico está oculto.

O modelo não inclui assincronia de rede nem persistência entre sessões. Em uma aplicação real, o controlador precisaria de identidade de requisição, cancelamento lógico e estados de carregamento. O valor do exercício é tornar explícitas as condições que qualquer mecanismo de gerenciamento de estado deveria preservar, independentemente da biblioteca escolhida.

CÓDIGO COM ESCOPO EXPLÍCITO

Flutter: o resumo existe sem o gráfico

DART / PROPOSTA AUTORAL

```dart
import 'package:flutter/material.dart';

class InvoiceSummary extends StatelessWidget {
  const InvoiceSummary({super.key, required this.total,
    required this.dueDate, required this.showChart,
    required this.chart});
  final String total;
  final String dueDate;
  final bool showChart;
  final Widget chart;

  @override
  Widget build(BuildContext context) {
    return Column(
      crossAxisAlignment: CrossAxisAlignment.start,
      children: [
        Semantics(
          header: true,
          child: Text('Total da fatura: $total'),
        ),
        Text('Vencimento: $dueDate'),
        if (showChart) chart,
      ],
    );
  }
}
```

Trecho Flutter/Dart didático, não executado. Componente autoral, sem dependência de NuDS interno.

O exemplo Flutter destaca a fronteira de apresentação: o controle showChart envolve apenas o visual complementar. Os valores formatados são entradas do componente; sua origem e autorização ficam fora desta árvore. O widget Semantics fornece informação estrutural, mas não substitui avaliação real com tecnologias assistivas nem resolve sozinho contraste, foco e ordem de navegação.

A integração completa precisaria dos componentes e tokens aprovados, localização monetária e tratamento de valores indisponíveis. Evitaria substituir ausência por zero, pois isso comunica uma situação financeira diferente. O trecho é ilustrativo e não foi compilado neste ambiente; somente o laboratório Python associado foi executado.

Flutter · Semantics

CÓDIGO COM ESCOPO EXPLÍCITO

Contrato de snapshot, não soma feita na tela

JSON / PROPOSTA AUTORAL

```json
{
  "contract": "portfolio.invoice.v1",
  "period": "2026-10",
  "revision": 4,
  "currency": "BRL",
  "total_cents": 900,
  "due_date": "2026-11-10",
  "status": "open",
  "entries": [
    {
      "id": "demo-purchase",
      "kind": "purchase",
      "amount_cents": 1000
    },
    {
      "id": "demo-adjustment",
      "kind": "adjustment",
      "amount_cents": -100
    }
  ],
  "complete": true,
  "updated_at": "2026-10-23T12:00:00Z"
}
```

Contrato JSON proposto com dados sintéticos; apenas o subconjunto Invoice é executado no laboratório.

O contrato proposto explicita moeda, período, revisão e completude. Valores negativos representam ajustes neste exemplo; não são erros de validação por definição. Uma API real precisaria descrever exatamente quais componentes entram no total e como parcelas, pagamentos, encargos e estornos são refletidos. O cliente não deveria reconstruir uma regra financeira incompleta a partir de itens carregados parcialmente.

O campo complete diferencia uma lista completa de uma página de resultados, mas confiar nele exige um produtor autorizado. O laboratório usa uma tupla completa para verificar a consistência. Esse JSON acrescenta documentação de intenção e não afirma o formato utilizado pelo Nubank.

ANÁLISE E DECISÃO

A corrida entre meses precisa de uma resposta definida






Imagine duas consultas: a pessoa abre outubro e, antes da resposta, seleciona setembro. A resposta de setembro chega primeiro; depois chega outubro. Um controlador que aplica qualquer resposta ao estado atual muda o mês sem consentimento. Cancelar a requisição anterior pode economizar recursos, mas não é garantia suficiente: ela pode ter terminado ou atravessado um cache. A regra de aplicação deve conferir a intenção vigente.

Outra corrida envolve revisão. Um pagamento ou ajuste altera o snapshot enquanto o app lê o período. Misturar total de uma revisão com lançamentos de outra produz uma inconsistência percebida como erro financeiro. A solução pode ser snapshot completo, token de leitura ou contrato de paginação ligado a uma revisão. O mecanismo depende da origem; o princípio é não combinar dados sem saber se pertencem ao mesmo estado.

Falhas de apresentação merecem estados próprios. Texto cortado, moeda ausente e carregamento que apaga o total são experiências diferentes de indisponibilidade da fatura. Eu preservaria contexto útil durante atualização, identificando quando ele é anterior. Não mostraria zero para uma consulta falhada nem esconderia o período para ganhar espaço. O cliente precisa saber o que está vendo e se a informação está pronta para sustentar uma decisão.

CÓDIGO COM ESCOPO EXPLÍCITO

A asserção importante não é a presença do gráfico

PYTHON / PROPOSTA AUTORAL

```python
def test_hide_chart_keeps_financial_information(self):
    before = self.screen.view()
    self.screen.show_chart = False
    after = self.screen.view()
    for key in ('period', 'total_cents', 'due_date'):
        self.assertEqual(before[key], after[key])
    self.assertIsNone(after['chart'])

def test_empty_period(self):
    self.screen.select_period(Invoice('2026-11', 1, 0, '2026-12-10', ()))
    self.assertEqual(self.screen.view()['total_cents'], 0)

def test_refund_is_signed(self):
    self.assertEqual(self.screen.view()['total_cents'], 900)

def test_stale_revision_ignored(self):
    self.screen.apply(replace(self.invoice, revision=2))
    self.assertFalse(self.screen.apply(self.invoice))
    self.assertEqual(self.screen.invoice.revision, 2)
```

Fragmento de labs/case_09.py. Resultado local: 8 testes passaram; ver results_09.txt.

Oito testes passaram no laboratório completo. O fragmento verifica independência da preferência visual, mês vazio, ajuste com sinal e revisão atrasada. Os demais rejeitam snapshot inconsistente, conflito de revisão, período incorreto e lançamentos em lista mutável. A última barreira impede mudar o conteúdo depois da validação. As propriedades examinadas são semânticas.

Isso não substitui testes de widget. Eles precisariam confirmar que a árvore realmente expõe o total, que texto ampliado continua utilizável e que a navegação por mês é operável. A combinação é valiosa: regras rápidas isolam o domínio, enquanto avaliações de interface verificam sua tradução para a experiência.

ANÁLISE E DECISÃO

Medir entendimento, não apenas abertura da tela






Aberturas de fatura podem aumentar quando a experiência piora: a pessoa retorna várias vezes para confirmar o mesmo valor. Por isso, eu não usaria visualizações como indicador principal de sucesso. A medição começaria por tarefas: localizar total e vencimento, identificar o período, encontrar um lançamento e explicar o estado da fatura. Pesquisa moderada ajuda a observar erros que eventos de clique não revelam.

Na telemetria, coletaria conclusão de navegação e falhas de carregamento com revisão do app, tamanho de texto e classe de dispositivo quando permitido. Não registraria valores financeiros em eventos de analytics para responder perguntas que podem ser medidas de forma agregada. Mudanças de período repetidas podem sinalizar exploração legítima; precisam de contexto antes de serem chamadas de fricção.

O painel de saúde combinaria inconsistências de snapshot, respostas descartadas por contexto, latência de leitura e contatos categorizados sobre compreensão. Segmentaria por uso de acessibilidade e conexão sem presumir que uma média representa todos. Uma melhora de velocidade acompanhada de mais erros ao identificar o vencimento seria regressão. A equipe precisa de uma métrica de entendimento e de uma proteção contra dano, além da capacidade de inspecionar exemplos consentidos quando os números entram em conflito.

ANÁLISE E DECISÃO

Uma migração visual também tem compatibilidade



O rollout começa com um inventário de tarefas e estados da experiência anterior. Não basta listar componentes substituídos. Precisamos saber como uma pessoa habituada encontra meses anteriores, reconhece um estorno e retoma uma consulta interrompida. A migração preserva esses resultados ou oferece um caminho claramente melhor, validado com pessoas reais. Um novo padrão de design não é evidência suficiente de menor esforço.

Eu separaria mudanças de conteúdo, comportamento e estilo para que o experimento possa ser interpretado. Alterar navegação mensal, gráfico e hierarquia do total simultaneamente dificulta atribuir qualquer queda de conclusão. A primeira coorte incluiria diversidade de dispositivos e necessidades de acesso, com caminho de reversão definido. Configurações locais precisam continuar válidas; uma preferência antiga não deveria esconder um novo componente essencial por acidente.

Antes de ampliar, revisaria métricas quantitativas junto com observações de pesquisa e contatos de suporte. Um resultado inconclusivo não autoriza expandir só porque não houve crash. A reversão deve restaurar uma experiência compatível com os dados atuais, sem trocar a fonte financeira ou apagar preferências. Se o custo de manter variantes se tornar alto, definiria uma janela de decisão e critérios explícitos de encerramento, evitando um experimento permanente que ninguém consegue operar.

ANÁLISE E DECISÃO

Acessibilidade é semântica financeira



Uma árvore acessível deve comunicar mais do que o texto visível. Total, vencimento, período e situação precisam de estrutura e ordem compreensíveis. Um botão que anuncia apenas seta não explica se muda de mês ou expande uma seção. O foco não deveria saltar para o início quando uma atualização chega, nem desaparecer quando o gráfico é ocultado. Essas propriedades exigem teste em dispositivos e tecnologias assistivas reais.

Texto ampliado muda a composição. A solução não é reduzir fonte para caber: é permitir reflow, priorizar informação e evitar altura fixa onde o conteúdo varia. Também verificaria a distinção entre valores positivos, ajustes negativos e estados indisponíveis. Cor pode reforçar significado, mas não deve ser a única forma de informar dívida, crédito ou erro. A formatação monetária precisa manter sinal e unidade identificáveis.

Segurança inclui privacidade na apresentação. Capturas, prévias e registros de erro podem expor dados da fatura se a equipe tratar a tela como conteúdo comum. O desenho de telemetria deve minimizar esses dados e evitar que exceções imprimam o snapshot inteiro. O objetivo é preservar autonomia sem transformar observabilidade em um repositório paralelo de informações financeiras. Isso conecta qualidade visual, semântica e disciplina de engenharia em uma única revisão de produto.

ANÁLISE E DECISÃO

ADR 09 · Snapshot financeiro independente do visual



A decisão é modelar informação financeira como snapshot imutável, mantendo navegação e preferência em estados separados. O contexto é uma interface evolutiva que recebe respostas assíncronas e precisa preservar tarefas habituais. O benefício principal não é uma biblioteca específica: é reduzir estados inválidos e tornar previsível qual interação pode alterar qual parte da experiência.

Uma alternativa é um único objeto mutável com campos de dados, carregamento e apresentação. Ele simplifica o início, mas permite combinações sem significado: mês novo com total antigo, gráfico oculto com resumo ausente, revisão atual com lista anterior. Outra alternativa é derivar toda a fatura dos lançamentos visíveis. Isso só seria correto se o contrato garantisse completude e a regra financeira estivesse integralmente representada, condições que uma tela não deveria presumir.

A separação exige um controlador explícito e testes de compatibilidade entre snapshots e componentes. Também exige decidir como apresentar informação anterior durante atualização. Eu registraria essa política como parte do contrato de experiência, não como detalhe deixado a cada tela. Reavaliaria o desenho se a origem não conseguir fornecer revisão coerente ou se a paginação impedir validação do conjunto. Nesse caso, a mudança correta é fortalecer o contrato, e não esconder divergências com estados visuais mais complexos.

ANÁLISE E DECISÃO

Execução EM: proteger a compreensão durante a mudança



Como Engineering Manager, formaria uma frente curta com Design, Pesquisa, Mobile e responsáveis pela fatura. A primeira entrega seria um acordo sobre tarefas essenciais e estados financeiros, acompanhado de exemplos verificáveis. Distribuiria ownership entre contrato de dados, comportamento e acessibilidade, evitando que uma pessoa concentre toda a revisão porque conhece a experiência antiga. O planejamento incluiria participação de usuários habituais e tempo para corrigir descobertas.

O valor começaria em compreensão e esforço: consultas concluídas, tempo para encontrar informação e redução de recontatos demonstrada. Para converter suporte em valor operacional, usaria volume elegível, mudança causal e custo unitário adequado. Não apresentaria uma redução de cliques como receita. Também acompanharia custo da migração e manutenção temporária de variantes. Uma mudança simples pode ser melhor investimento do que reconstruir toda a experiência.

A pergunta Staff seria: qual invariante continua verdadeiro se o cliente alterna gráfico, mês e tamanho de texto enquanto as respostas chegam fora de ordem? A resposta precisa conectar domínio, estado e renderização. A pergunta de liderança seria quem decide interromper a expansão quando a média melhora, mas um grupo relevante perde capacidade de entender a fatura. Essa decisão deve estar combinada antes do experimento, com evidências e responsabilidades claras.

ANÁLISE E DECISÃO

10 · Primeiro definir o documento certo

10






Nubank: conta, cartão e mais

Captura da fonte pública consultada em 23/10/2026. Relato não equivale a incidente reproduzido.

Uma resenha pública pede compartilhar um comprovante de compra no crédito sem depender de atendimento por e-mail. A necessidade é compreensível, mas a fonte não esclarece qual documento resolveria a situação. Registro de compra, confirmação de pagamento da fatura e documento do estabelecimento não são equivalentes. Criar um botão antes de resolver essa ambiguidade poderia acelerar o envio de uma informação inadequada.

A investigação começaria pelos motivos de solicitação. A pessoa quer comprovar que reconhece uma compra, contestar uma cobrança, solicitar reembolso ou demonstrar pagamento? Cada finalidade exige campos, linguagem e responsabilidade diferentes. Eu observaria exemplos anonimizados com suporte e responsáveis pelo produto, classificando solicitações elegíveis para autosserviço e situações que precisam continuar em atendimento. A amostra não deveria incluir apenas documentos fáceis de gerar.

A proposta deste capítulo é um registro compartilhável versionado, associado ao estado conhecido da transação. Ele informa seu alcance e minimiza dados desnecessários. O laboratório produz esse registro a partir de um objeto imutável, verifica autorização sintética e detecta alteração do conteúdo com um segredo local. Não cria um recibo fiscal, não confirma liquidação externa e não substitui assinatura legal. Sua contribuição é mostrar que semântica, acesso e integridade precisam nascer juntos, antes de discutir aparência ou formato do arquivo.

Google Play - resenha de Neidslem Fernandes da Costa sobre comprovantes

ANÁLISE E DECISÃO

Autorizado, confirmado e estornado não são sinônimos





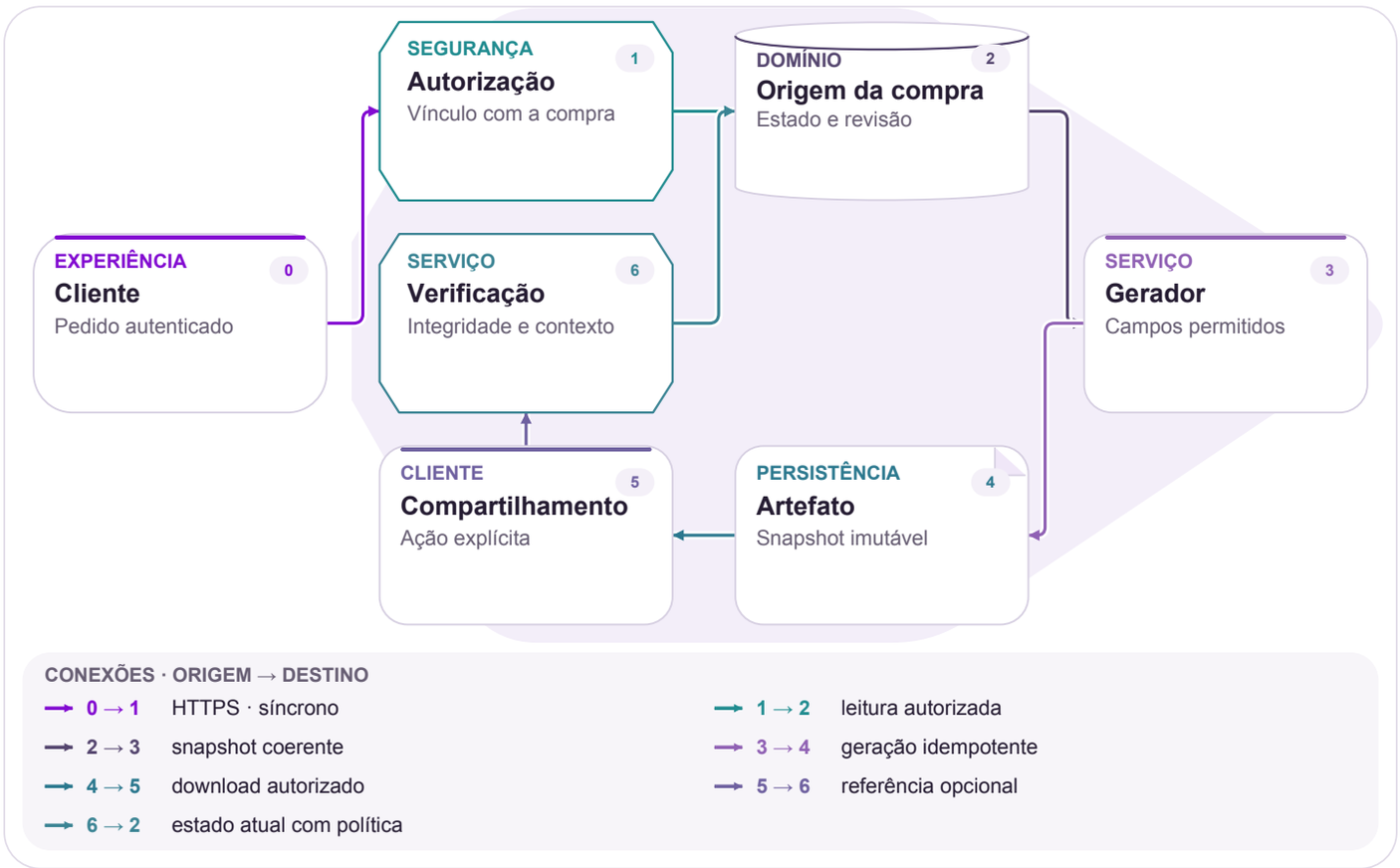
O domínio do documento começa pelo ciclo da compra. Uma autorização pendente pode reservar capacidade sem representar a mesma situação de uma compra confirmada. Um estorno posterior muda o significado do registro, mas não deveria apagar que uma versão anterior foi emitida. A experiência precisa dizer o estado e a data de referência, evitando o rótulo genérico pago quando a informação disponível não sustenta essa afirmação.

No laboratório, Purchase contém identificador, proprietário, revisão, valor, estabelecimento e estado. Um conjunto fechado de rótulos traduz os três estados sintéticos. A revisão muda quando o conteúdo relevante muda. O documento recebe uma cópia do snapshot e não consulta campos individualmente durante a montagem. Isso evita misturar valor de uma revisão com situação de outra. A regra completa de um cartão real exigiria um vocabulário mais detalhado.

Documento emitido e documento atual são objetos diferentes. Um arquivo compartilhado pode continuar existindo depois de um estorno, portanto não se pode prometer atualização automática de cópias já distribuídas. Uma referência verificável pode informar a situação mais recente, desde que não exponha dados a terceiros sem autorização. A decisão de produto deve explicar essa limitação. O cliente precisa saber se está enviando um retrato histórico ou uma consulta viva, e o destinatário deve conseguir interpretar o alcance.

ARQUITETURA PROPOSTA · NÃO INTERNA

Exportar um snapshot autorizado



Arquitetura externa proposta. Caixas representam responsabilidades; setas explicitam contratos.

A autorização pertence ao acesso ao objeto, não apenas ao fato de existir uma sessão. O gerador recebe um snapshot coerente e aplica uma lista de campos permitidos. Ele não copia automaticamente todos os dados da origem para o documento. O artefato é imutável; outra revisão produz outro registro e preserva a relação entre eles.

A seta de verificação é opcional e exige uma decisão de privacidade. Tornar um identificador público consultável pode transformar conveniência em exposição de atividade financeira. Uma verificação externa poderia confirmar apenas integridade e versão, sem devolver o histórico completo. O laboratório usa HMAC local para detectar adulteração, mas não implementa uma infraestrutura pública de verificação. A arquitetura separa essas responsabilidades para que uma escolha de formato, como PDF ou imagem, não esconda decisões importantes sobre acesso, validade temporal e significado do que está sendo compartilhado.

CÓDIGO COM ESCOPO EXPLÍCITO

Uma lista explícita de campos compartilháveis

PYTHON / PROPOSTA AUTORAL

```python
def export_record(purchase, subject, secret):
    if purchase.owner != subject:
        raise PermissionError('objeto não autorizado')
    if purchase.status not in LABELS:
        raise ValueError('estado desconhecido')
    if type(purchase.amount_cents) is not int or purchase.amount_cents <= 0:
        raise ValueError('centavos inteiros positivos')
    if purchase.revision < 1:
        raise ValueError('revisão inválida')
    body = {
        'kind': 'synthetic-purchase-record',
        'purchase_id': purchase.purchase_id,
        'revision': purchase.revision,
        'amount_cents': purchase.amount_cents,
        'currency': 'BRL',
        'merchant': purchase.merchant,
        'status': purchase.status,
        'label': LABELS[purchase.status],
        'notice': 'Não é recibo fiscal nem prova de quitação da fatura.'
    }
    signature = hmac.new(secret, canonical(body), hashlib.sha256).hexdigest()
    return {'body': body, 'integrity_tag': signature}
```

Fragmento literal de labs/case_10.py. HMAC local com chave de teste; não é assinatura legal ou infraestrutura real.

O gerador executado compara o proprietário com o sujeito recebido e recusa estados ou valores inválidos. O documento é construído campo a campo; owner não é exportado. O aviso acompanha o conteúdo para impedir que o registro seja confundido com quitação da fatura ou recibo fiscal. A etiqueta de integridade cobre o corpo canônico.

O sujeito e o segredo são entradas controladas do teste, não mecanismos de autenticação de produção. Um serviço real precisaria obtê-los de fontes confiáveis, gerenciar chaves e aplicar autorização consistente. O exemplo demonstra a decisão de minimizar conteúdo e preservar estado, sem alegar segurança completa do compartilhamento.

CÓDIGO COM ESCOPO EXPLÍCITO

Clojure: contrato antes de renderização

CLOJURE / PROPOSTA AUTORAL

```clojure
(ns portfolio.purchase-record)

(def labels
  {:pending "Autorização pendente"
   :confirmed "Registro de compra confirmada"
   :reversed "Compra estornada"})

(defn public-record [purchase subject]
  (when-not (= subject (:owner purchase))
    (throw (ex-info "Forbidden" {:kind :forbidden})))
  (let [status (:status purchase)
        label (get labels status)]
    (when-not label
      (throw (ex-info "Unsupported state"
                      {:kind :invalid-state})))
    (assoc
      (select-keys purchase
        [:purchase-id :revision :amount-cents
         :merchant :status])
      :label label
      :kind :purchase-record
      :notice "Não comprova quitação da fatura")))

;; Proposta pura: autenticação e assinatura são externas.
```

Trecho Clojure ilustrativo não executado; nomes e contratos propostos pelo autor.

Esta função didática expressa a mesma fronteira em Clojure: verificar acesso ao objeto e selecionar campos antes de qualquer renderização. Um template visual recebe somente o modelo público, reduzindo a chance de incluir por acidente informação interna. O tratamento completo de tipos e valores continuaria necessário; o fragmento enfatiza apenas acesso e semântica.

O contrato é autoral e não corresponde a uma API conhecida do Nubank. A escolha de função pura permite revisar a transformação separadamente de PDF, armazenamento e transporte. Se o modelo público estiver errado, trocar o renderizador não corrige o problema; apenas distribui a interpretação errada de outra forma.

CÓDIGO COM ESCOPO EXPLÍCITO

Persistir revisão e propósito do artefato

SQL / PROPOSTA AUTORAL

```sql
-- Esquema proposto; não executado pelo laboratório.
CREATE TABLE purchase_records (
  document_id TEXT PRIMARY KEY,
  purchase_id TEXT NOT NULL,
  purchase_revision BIGINT NOT NULL,
  template_revision INTEGER NOT NULL,
  document_kind TEXT NOT NULL,
  state_at_issue TEXT NOT NULL,
  payload_digest TEXT NOT NULL,
  storage_reference TEXT NOT NULL,
  issued_at TIMESTAMP NOT NULL,
  owner_reference TEXT NOT NULL,
  UNIQUE (purchase_id, purchase_revision,
          template_revision, document_kind),
  CHECK (purchase_revision > 0),
  CHECK (template_revision > 0)
);

CREATE INDEX records_by_purchase
  ON purchase_records (purchase_id, purchase_revision);

-- Acesso ao blob ainda exige autorização.
-- Digest não torna o arquivo anônimo ou público.
```

SQL de referência proposto. Não executado; laboratório usa objetos Python imutáveis.

O esquema é uma opção proposta para armazenar metadados de artefatos, não uma descrição da persistência do Nubank. A identidade combina revisão de compra, revisão de template e tipo de documento. Assim, uma mudança visual pode gerar um arquivo diferente sem fingir que a transação mudou. A referência de armazenamento não deve ser tratada como URL pública permanente.

Uma implementação precisaria decidir retenção, criptografia, política de acesso e reconciliação entre registro e blob. O laboratório mantém apenas objetos em memória e não executa este SQL. A distinção evita usar um exercício pequeno como prova de persistência ou distribuição segura de documentos.

ANÁLISE E DECISÃO

O estorno pode acontecer enquanto o arquivo é gerado





A principal corrida está entre leitura da compra e emissão do documento. Se o estado muda durante a geração, não é necessariamente correto cancelar o arquivo: ele pode representar um snapshot histórico válido. O que não pode acontecer é combinar campos de revisões diferentes ou omitir a referência temporal. A decisão depende do propósito do documento e precisa ser comunicada antes de compartilhá-lo.

Outra falha ocorre depois da geração. O arquivo existe, mas o cliente perde a resposta e tenta novamente. Uma chave baseada em compra, revisão, template e finalidade pode retornar o mesmo artefato, evitando custo duplicado e versões indistinguíveis. A operação de compartilhar permanece explícita no dispositivo; gerar o arquivo não autoriza enviá-lo a um destinatário previamente utilizado. São consentimentos e efeitos diferentes.

Também há falhas de integridade e acesso. Uma etiqueta válida não significa que o solicitante pode ler o objeto. Um link expirado não deveria vazar metadados para explicar o erro. Um template defeituoso pode produzir um documento incorreto mesmo com hash correto, porque integridade só demonstra que os bytes não mudaram. Por isso, testes precisam examinar significado, autorização e renderização, além da verificação criptográfica. Nenhuma dessas camadas substitui as demais.

CÓDIGO COM ESCOPO EXPLÍCITO

Testes distinguem integridade de significado

PYTHON / PROPOSTA AUTORAL

```python
def test_pending_has_no_paid_claim(self):
    self.assertEqual(self.make()['body']['label'], 'Autorização pendente')

def test_reversed_has_explicit_status(self):
    doc = self.make(status='reversed', revision=2)
    self.assertEqual(doc['body']['label'], 'Compra estornada')
    self.assertEqual(doc['body']['revision'], 2)

def test_owner_not_in_shareable_body(self):
    self.assertNotIn('owner', self.make()['body'])
    self.assertNotIn('alice', canonical(self.make()).decode())

def test_wrong_subject_is_denied(self):
    with self.assertRaises(PermissionError):
        export_record(self.purchase, 'bob', self.secret)

def test_same_snapshot_same_document(self):
    self.assertEqual(self.make(), self.make())
```

Fragmento de labs/case_10.py. Resultado: 7 testes passaram; ver results_10.txt.

Sete testes passaram no laboratório completo. O trecho verifica rótulos, revisão, minimização, negação de acesso e determinismo do mesmo snapshot. Os testes restantes alteram o valor depois da emissão e validam rejeição de valores inadequados. O documento pode ser íntegro e ainda assim ser inadequado à finalidade; por isso, os rótulos também são objeto de asserção.

Os testes não validam autenticação, gestão de segredo ou interpretação por terceiros. Para um piloto, acrescentaria revisão de conteúdo com Produto e Jurídico, amostras de compras parceladas e estornadas e avaliação de leitura do documento fora do app. O contexto externo muda o que precisa estar explícito.

ANÁLISE E DECISÃO

Autosserviço que resolve, não apenas gera arquivos





Contar downloads é insuficiente para avaliar o produto. Uma pessoa pode baixar três documentos porque nenhum responde à necessidade. Eu mediria o motivo declarado ou inferido com cuidado, a conclusão da geração e o recontato posterior sobre a mesma compra. O objetivo é descobrir se o registro substitui um pedido elegível de suporte sem empurrar a dúvida para outro canal.

Na saúde técnica, acompanharia falhas por etapa: autorização, leitura, renderização, armazenamento e entrega. Essa separação permite distinguir indisponibilidade de acesso negado e detectar templates que falham em determinados conteúdos. Um identificador de correlação deve ligar a solicitação às etapas sem colocar o documento inteiro em logs. Valores, estabelecimentos e nomes não são necessários para toda métrica operacional.

Qualidade exige amostragem de documentos emitidos sob controle de acesso e finalidade adequada. O painel deveria destacar estado incorreto, revisão ausente e divergência entre artefato e origem. Também observaria custo por emissão e taxa de reutilização de artefatos. Um aumento de eficiência que mantém documentos semanticamente confusos não representa sucesso. O suporte precisa de um caminho para registrar que o cliente recebeu o arquivo, mas continuou sem uma prova apropriada para a situação que motivou o pedido.

ANÁLISE E DECISÃO

Lançar por finalidade, depois ampliar cobertura



O primeiro piloto deveria escolher uma finalidade estreita e um conjunto bem compreendido de estados. Por exemplo, oferecer registro de uma compra confirmada para consulta e compartilhamento informativo, com linguagem revisada. Começar incluindo todas as situações cria um contrato implícito amplo demais e dificulta descobrir quais documentos realmente resolvem o pedido. A restrição inicial deve ser clara para o cliente e para o atendimento.

Antes da exposição, compararia exemplos gerados com a informação fornecida hoje pelo suporte, investigando diferenças de semântica e conteúdo. A equipe testaria a experiência completa de obter o arquivo, abrir em outro dispositivo e compreender seus limites. Pessoas que recebem o documento fora do app não contam com o contexto da tela anterior. O arquivo precisa carregar informação suficiente sem acumular dados pessoais desnecessários.

A expansão seria orientada por resolução e recontato, com amostragem de casos problemáticos. Reverter o botão impede novas emissões, mas não recolhe arquivos já compartilhados; essa consequência precisa entrar no plano de risco. Revisões de template devem ser rastreáveis e a comunicação de correções precisa de um processo apropriado. A feature flag controla exposição futura, enquanto governança documental cuida do que já foi emitido. Tratar ambas como a mesma coisa deixa uma parte importante da jornada sem responsável.

ANÁLISE E DECISÃO

Compartilhar é uma nova fronteira de privacidade





Dentro do aplicativo, acesso pode depender de sessão e autorização. Depois do compartilhamento, o arquivo pode circular além da intenção original. A proposta minimiza informação desde a geração: nenhum número completo de cartão, segredo ou identificador interno entra automaticamente no modelo público. A seleção de campos deve corresponder à finalidade do documento, e não à conveniência de reutilizar um objeto da API.

Links verificáveis exigem atenção adicional. Um token longo pode reduzir adivinhação, mas continua sendo uma credencial se dá acesso ao conteúdo. Expiração, revogação, escopo e registro de uso precisam ser desenhados conforme o caso. Uma página pública que confirma a existência de uma compra pode revelar informação mesmo sem mostrar valor. A decisão de verificação deve separar prova de integridade de consulta ao estado financeiro atual.

O HMAC do laboratório usa segredo fixo de teste para tornar adulteração observável. Isso não constitui gestão de chaves de produção, assinatura pública ou validade jurídica. Em um serviço real, haveria rotação, armazenamento seguro, separação de ambientes e revisão de permissões. Também seria necessário testar conteúdo malicioso em nomes de estabelecimentos para evitar problemas no renderizador. Segurança do documento envolve dados, representação e distribuição; não se encerra na escolha de um algoritmo criptográfico.

ANÁLISE E DECISÃO

ADR 10 · Emitir um retrato verificável da compra



A decisão proposta é emitir um snapshot imutável com estado, revisão e finalidade explícitos. O contexto é uma demanda de autosserviço cujo significado varia entre clientes. O objetivo é reduzir dependência de atendimento para pedidos elegíveis sem chamar qualquer registro de comprovante de pagamento. O modelo público é separado do objeto interno e precede a escolha do formato visual.

Uma alternativa é exportar diretamente a tela. Ela preserva aparência familiar, mas pode incluir informação desnecessária, omitir contexto temporal e depender do tamanho do dispositivo. Outra é gerar um documento sempre com os dados mais recentes, sem referência à revisão. Isso simplifica leitura atual, porém dificulta explicar o que uma pessoa compartilhou antes de um estorno. O snapshot explícito oferece rastreabilidade, ao custo de retenção e comunicação clara sobre atualização.

Eu reavaliaria a decisão se a finalidade dominante exigir um documento de outro participante, como o estabelecimento, ou se um registro informativo não reduzir recontato. Nesse caso, o melhor produto pode ser orientar o pedido correto, não criar mais formatos. Também registraria que integridade criptográfica não valida a verdade comercial do conteúdo. A arquitetura garante associação a uma fonte e uma revisão; o domínio continua responsável pelo significado e pela legitimidade do documento emitido.

ANÁLISE E DECISÃO

Execução EM: linguagem correta evita retrabalho





Como Engineering Manager, reuniria Cartões, Suporte, Design e responsáveis pela interpretação documental. A primeira decisão seria nomear a finalidade e delimitar estados cobertos, com exemplos aprovados. O trabalho técnico então teria critérios concretos de aceite: origem coerente, autorização por objeto, minimização, rastreabilidade e compreensão fora do app. Eu evitaria dividir a equipe entre gerar PDF e criar botão antes desse alinhamento.

O modelo de valor usaria solicitações elegíveis, adoção efetiva, resolução sem recontato e custo incremental por contato. Descontaria emissão, armazenamento, manutenção e investigação de documentos problemáticos. Um download não equivale a um atendimento evitado, e capacidade liberada não vira economia financeira automaticamente. O piloto precisa demonstrar que a pessoa resolve a finalidade, não apenas que conseguiu exportar um arquivo.

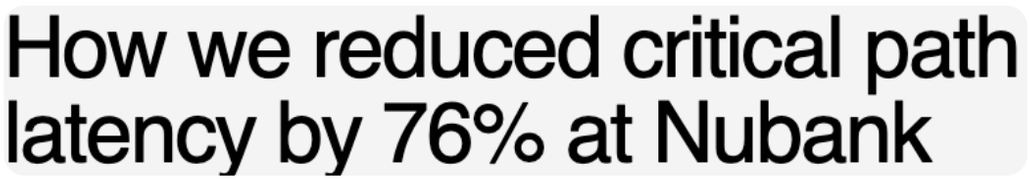
A pergunta Staff seria: o que continua verdadeiro quando a compra é estornada entre geração, download e leitura pelo destinatário? A resposta deve separar snapshot histórico, estado atual e autorização de consulta. A pergunta de liderança seria quem responde quando o documento é íntegro, mas não serve à finalidade esperada. Uma equipe madura não usa o sucesso técnico da geração para encerrar um problema de produto. Ela observa o desfecho e ajusta a promessa feita ao cliente.

ANÁLISE E DECISÃO

11 · O resultado publicado é ponto de partida

11





Captura da publicação oficial consultada em 23/10/2026.

O Nubank publicou um caso de redução do P90 de uma API de condições de pagamento de aproximadamente 1.200 para 280 milissegundos. O relato descreve melhorias locais e, depois, agregação por eventos com leitura persistida e fallback. Esse resultado pertence ao fluxo e período documentados; não representa a latência de todo o aplicativo nem um ganho produzido por este portfólio.

A contribuição autoral começa na pergunta seguinte: quais critérios permitem escolher outra leitura candidata à materialização? A resposta precisa combinar custo de composição, variabilidade das dependências e tolerância explícita a desatualização. Não basta observar que uma consulta é lenta. Uma condição exibida ao cliente pode aceitar um snapshot por alguns instantes, enquanto a autorização financeira exige validação em outra fronteira.

Eu começaria com tracing, identificação de chamadas redundantes e distribuição de latência por dependência. Só proporia nova persistência depois de esgotar mudanças simples e entender a semântica dos dados. O laboratório deste capítulo modela componentes versionados, orçamento de atualização e fallback. Ele não mede desempenho nem executa pagamento. A intenção é tornar verificável uma decisão de leitura: quando a projeção pode ser usada e quando deve ceder à origem, sem esconder o custo operacional transferido para o processamento de eventos.

How we reduced critical path latency by 76% at Nubank

ANÁLISE E DECISÃO

Atualização é um contrato por componente



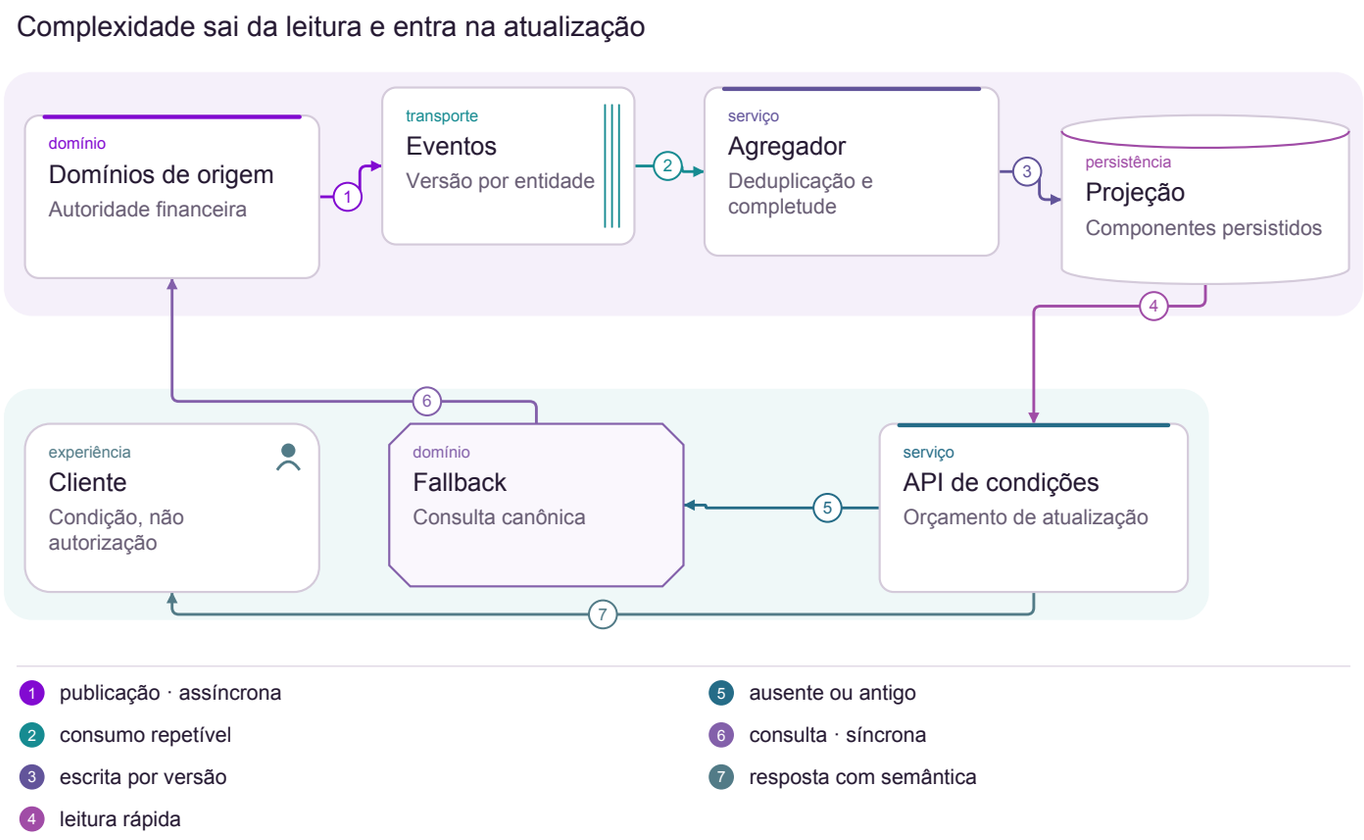
Uma projeção composta reúne dados que podem mudar em ritmos diferentes. Um carimbo único de atualização pode esconder que apenas um componente foi renovado. Por isso, o modelo sintético mantém versão e horário observado para saldo e limite separadamente. A leitura só usa a projeção quando ambos estão presentes e dentro do orçamento configurado. A escolha de orçamento pertence ao domínio, não ao cache isoladamente.

Versão e horário respondem perguntas diferentes. A versão impede que uma atualização antiga substitua uma nova; o horário ajuda a avaliar idade. Um evento recente no transporte pode carregar um estado antigo, e um relógio futuro pode fazer um dado parecer fresco indefinidamente. O laboratório rejeita horário futuro em relação ao relógio fornecido pelo teste. Uma implantação distribuída precisaria definir fontes temporais e tolerância a diferença de relógios.

Também é necessário distinguir completude de validade. Ter todos os campos não significa que eles possam autorizar uma operação. A projeção pode servir uma estimativa ou condição de apresentação, enquanto a confirmação consulta o serviço responsável. O contrato deve explicitar essa diferença para que outro time não reutilize uma leitura rápida como autoridade financeira. A velocidade de uma interface cria incentivos para expansão de uso; limites documentados e verificáveis evitam transformar conveniência em acoplamento perigoso.

ARQUITETURA PROPOSTA · NÃO INTERNA

Complexidade sai da leitura e entra na atualização



Arquitetura externa proposta. Caixas representam responsabilidades; setas explicitam contratos.

A arquitetura proposta favorece previsibilidade da leitura, mas acrescenta um sistema de atualização que precisa ser operado. O consumidor pode atrasar, receber eventos duplicados ou reconstruir uma projeção. A API não deve interpretar disponibilidade do banco de projeção como prova de atualização: o contrato de leitura verifica completude e idade de cada componente relevante.

O fallback preserva um caminho de correção, porém pode receber uma avalanche quando a projeção falha. Portanto, capacidade, limites de concorrência e prioridade do fallback fazem parte do desenho inicial. A seta para o cliente transporta condições com significado definido; não transfere autoridade de autorização financeira. O laboratório usa funções locais para representar essas fronteiras. A proposta real precisaria de observabilidade, políticas de degradação e contratos com os domínios. Materializar uma leitura só é uma boa decisão quando o custo total da nova operação compensa a variabilidade retirada do caminho crítico.

CÓDIGO COM ESCOPO EXPLÍCITO

Aplicar uma versão sem aceitar uma regressão

PYTHON / PROPOSTA AUTORAL

```python
def apply(self, component, version, cents, observed_at, now):
    if component not in {'balance', 'limit'}:
        raise ValueError('componente desconhecido')
    if type(cents) is not int or cents < 0:
        raise ValueError('centavos inválidos')
    if type(version) is not int or version < 1:
        raise ValueError('versão inválida')
    if observed_at > now:
        raise ValueError('timestamp futuro')
    candidate = (version, cents, observed_at)
    previous = self.parts.get(component)
    if previous and version < previous[0]:
        return False
    if previous and version == previous[0]:
        if previous != candidate:
            raise ValueError('versão conflitante')
        return False
    self.parts[component] = candidate
    return True
```

Fragmento literal de labs/case_11.py. Python executado; projeção sintética sem autorização financeira.

Este núcleo executado recebe um componente, versão, valor e horário. Atualizações antigas são ignoradas; uma versão repetida com conteúdo diferente gera conflito. A comparação não depende apenas do horário, pois transporte e relógio não estabelecem sozinhos a ordem de negócio. O conjunto de componentes é fechado para manter o contrato inspecionável.

O armazenamento é em memória e não demonstra atomicidade entre processos. Em produção, a comparação e a atualização precisariam ocorrer em uma fronteira consistente, com recuperação e reprocessamento definidos. O trecho serve para discutir semântica de versão antes de escolher Datomic, DynamoDB ou outro mecanismo de persistência.

CÓDIGO COM ESCOPO EXPLÍCITO

Clojure: leitura com caminho alternativo explícito

CLOJURE / PROPOSTA AUTORAL

```clojure
(ns portfolio.conditions)

(def required-components #{:balance :limit})

(defn fresh? [now max-age parts]
  (and
    (every? #(contains? parts %) required-components)
    (every?
      (fn [{:keys [observed-at]}]
        (<= 0 (- now observed-at) max-age))
      (vals parts))))

(defn read-conditions
  [projection now max-age canonical-reader]
  (if (fresh? now max-age projection)
    {:source :projection
     :purpose :display-conditions
     :data projection}
    {:source :canonical
     :purpose :display-conditions
     :data (canonical-reader)}))

;; canonical-reader é contrato proposto, não API do Nu.
;; Autorizar pagamento pertence a outra fronteira.
```

Trecho Clojure proposto e não executado; canonical-reader é uma dependência autoral abstrata.

A função recebe o leitor canônico como dependência explícita. Isso torna possível testar escolha de caminho sem abrir conexão real. A resposta carrega origem e finalidade para facilitar diagnóstico, mas metadados não substituem disciplina de acesso. Um consumidor não deveria tratar a finalidade declarada como uma autorização para ignorar validação financeira.

O fragmento é didático e não inclui timeout, bulkhead, limite de concorrência ou tratamento de falha do caminho canônico. Esses controles são necessários para que uma falha da projeção não derrube a origem. O desenho precisa especificar o comportamento quando ambos os caminhos estão indisponíveis, mantendo a resposta honesta ao cliente.

CÓDIGO COM ESCOPO EXPLÍCITO

Kafka transporta um fato versionado, não uma garantia global

JSON / PROPOSTA AUTORAL

```json
{
  "topic_proposed": "portfolio.conditions-state.v1",
  "partition_key": "synthetic-customer-8",
  "event_id": "demo-limit-12",
  "schema_version": 1,
  "entity": {
    "type": "limit",
    "id": "synthetic-limit-8",
    "version": 12
  },
  "state": {
    "amount_cents": 2000,
    "currency": "BRL"
  },
  "time": {
    "observed_at": "2026-10-23T12:00:00Z",
    "emitted_at": "2026-10-23T12:00:01Z"
  },
  "purpose": "conditions-projection",
  "authorization_authority": false,
  "synthetic": true
}
```

Envelope e nome de tópico autorais propostos. Não executado em Kafka; laboratório usa chamadas locais.

O envelope proposto mantém identidade e versão do domínio, mesmo quando o transporte oferece ordenação por partição. Partições diferentes, reprocessamento e consumidores que escrevem em armazenamento externo continuam exigindo regras próprias. Não se deve transformar uma configuração de produtor em promessa de efeito único de ponta a ponta.

A chave de partição é uma escolha de distribuição e afinidade, com impacto em volume e pontos quentes. O exemplo não contém dados reais e não cria um tópico. O campo de finalidade documenta o contrato; não é controle de segurança suficiente. A implantação precisaria de schema compatível, permissões, retenção e um plano verificável de reconstrução da projeção.

Apache Kafka · Design

ANÁLISE E DECISÃO

O fallback também pode ser a próxima falha



Uma projeção indisponível muda a distribuição de carga imediatamente. Se todas as leituras migram para a origem ao mesmo tempo, o mecanismo que deveria proteger correção pode produzir saturação e aumentar a indisponibilidade. O desenho precisa considerar capacidade de contingência, priorização e limites de concorrência. A resposta correta para excesso de carga pode ser uma espera explícita, e não retries ilimitados em várias camadas.

Eventos atrasados criam outra categoria de falha. O armazenamento responde rápido, mas o conteúdo está antigo. Nesse caso, taxa de sucesso HTTP pode permanecer excelente enquanto a experiência usa informação inadequada. A verificação de atualização deve estar no caminho de leitura e ter critérios por componente. A política também precisa decidir como agir quando o horário não é confiável ou quando a fonte não publica heartbeat de estado estável.

Reconstrução merece tratamento de primeira classe. Reprocessar histórico com lógica nova pode produzir resultados diferentes dos existentes. Eu compararia projeções paralelas e registraria divergências antes de trocar tráfego. Atualizações antigas não podem sobrescrever dados novos durante o backfill. A solução pode envolver namespaces de revisão e uma barreira de promoção. Esses mecanismos acrescentam complexidade, mas tornam explícito o custo real de retirar composição síncrona do caminho crítico.

CÓDIGO COM ESCOPO EXPLÍCITO

Testes de atualização, não benchmark de latência

PYTHON / PROPOSTA AUTORAL

```python
def test_fresh_projection_used(self):
    result = self.view.read(105, 10, self.origin)
    self.assertEqual(result['source'], 'projection')

def test_old_component_forces_fallback(self):
    self.view.apply('balance', 2, 900, 119, 119)
    result = self.view.read(120, 10, self.origin)
    self.assertEqual(result['source'], 'canonical')
    self.assertEqual(self.view.fallbacks, 1)

def test_old_version_does_not_replace(self):
    self.view.apply('balance', 2, 900, 101, 101)
    self.assertFalse(self.view.apply('balance', 1, 1000, 100, 101))
    self.assertEqual(self.view.parts['balance'][0], 2)

def test_conflicting_version_rejected(self):
    with self.assertRaises(ValueError):
        self.view.apply('limit', 1, 3000, 100, 100)
```

Fragmento de labs/case_11.py. Resultado: 7 testes passaram; ver results_11.txt. Sem benchmark.

Os sete testes locais passaram. O fragmento verifica seleção da projeção fresca, fallback por componente antigo e rejeição de regressão de versão. Outros testes cobrem conflito na mesma versão, timestamp futuro, componente ausente e indisponibilidade explícita. O relógio é uma entrada, permitindo examinar limites sem esperar tempo real.

Nenhum resultado mede milissegundos ou reproduz o ganho publicado pelo Nubank. Para avaliar desempenho, seriam necessários carga representativa, distribuição de dependências e observação de cauda. O laboratório comprova somente decisões locais de leitura. Essa separação protege a credibilidade do material: uma propriedade funcional pode estar correta mesmo quando a arquitetura ainda não demonstrou benefício econômico.

ANÁLISE E DECISÃO

Observar a escrita para confiar na leitura



O painel de leitura precisa mostrar latência e origem da resposta, mas isso é apenas metade da história. A saúde da projeção depende de atraso de consumo, completude, idade por componente e divergência em comparação com a origem. Eu evitaria um único indicador de cache hit: uma taxa alta pode significar eficiência ou uso persistente de conteúdo inadequado, dependendo do contrato de atualização.

As métricas de cauda devem ser segmentadas por caminho. Misturar leitura materializada e fallback pode esconder degradação do segundo até o momento em que ele recebe mais tráfego. Também acompanharia frequência e motivo do fallback, volume de conflitos de versão e tempo de reconstrução. Identificadores individuais ficam em rastros com controle de acesso, não em rótulos de métricas que multiplicam cardinalidade sem necessidade.

O objetivo de negócio é conclusão da jornada com condição correta e espera aceitável. Uma melhora de P90 acompanhada de mais divergências ou falhas de confirmação não é vitória. Definiria indicadores de proteção antes do rollout e manteria um mecanismo de comparação em amostra. Alertas precisam distinguir transporte atrasado, produtor silencioso e origem indisponível, porque cada situação exige ação diferente. Operar uma leitura rápida significa compreender o caminho que a alimenta, não apenas o banco que responde.

ANÁLISE E DECISÃO

Comparação silenciosa antes de servir condições



A implantação começaria em modo de comparação: produzir a projeção e consultar seu resultado em uma amostra, sem alterar a resposta do cliente. As diferenças seriam classificadas por atraso esperado, erro de transformação, fonte incompleta e semântica incompatível. Uma porcentagem agregada de igualdade não basta; os campos mais críticos precisam de análise própria e um conjunto de exemplos explicáveis.

Após validar atualização e correção, uma coorte pequena poderia receber a leitura materializada. O fallback seria exercitado deliberadamente em ambiente controlado para confirmar capacidade e comportamento. O plano incluiria limites de exposição, critérios de pausa e retorno ao caminho anterior. Também testaria a situação inversa: origem degradada enquanto a projeção parece saudável, pois servir dados antigos sem limite pode mascarar um problema relevante.

Expansão depende do benefício líquido, não de preferência arquitetural. Mediria latência percebida, conclusão da jornada, custo de processamento e esforço operacional. Se a maior parte das consultas precisar de fallback, talvez a tolerância a atraso seja pequena demais para o padrão escolhido. Nesse caso, reveria a composição ou o contrato em vez de afrouxar silenciosamente a garantia de atualização. O experimento deve poder concluir que a materialização não é adequada para aquele fluxo.

ANÁLISE E DECISÃO

Dados financeiros rápidos ainda precisam de fronteiras



A projeção aumenta o número de lugares em que dados relevantes existem. Isso exige propósito, acesso e retenção explícitos. Um armazenamento desenhado para leitura rápida não deveria se tornar um repositório genérico acessível a qualquer serviço. Cada consumidor precisa demonstrar que seu uso é compatível com a semântica e a atualização oferecidas. Autorizar pagamento com um campo criado para apresentação seria uma mudança de contrato, não uma simples otimização.

Na performance, a primeira pergunta é onde o tempo está sendo gasto. Reduzir chamadas duplicadas, propagar contexto validado e remover camadas sem função de domínio pode ser suficiente. A materialização só se justifica quando o custo estrutural permanece e a atualização assíncrona é aceitável. Também consideraria tamanho do objeto, frequência de invalidação e concentração de tráfego, pois uma projeção muito ampla pode transferir gargalos para escrita e armazenamento.

O laboratório mantém saldo e limite sintéticos, sem identidade autenticada ou isolamento entre clientes. Uma implementação real precisaria preservar autorização de leitura e evitar mistura de entidades no agregador. Testes incluiriam chaves equivocadas, permissões insuficientes e restauração de backups. Confiabilidade e segurança se encontram nessa fronteira: uma resposta rápida para a pessoa errada é uma falha grave, mesmo que versão, idade e latência estejam dentro dos limites.

ANÁLISE E DECISÃO

ADR 11 · Materializar somente com semântica de atualização



A decisão proposta é materializar uma leitura composta apenas quando existe tolerância de atualização documentada e um caminho canônico de contingência operável. O contexto é a variabilidade causada por dependências síncronas, não uma regra de que caches sempre melhoram arquitetura. O objetivo é reduzir cauda e tornar a leitura previsível, preservando o significado dos dados utilizados pela jornada.

Uma alternativa é manter composição síncrona e otimizar chamadas, paralelismo e propagação de contexto. Ela reduz estado operacional, mas conserva dependência da disponibilidade e cauda dos serviços. Outra é usar cache com expiração única, sem versões por componente. É mais simples, porém pode ocultar inconsistência e não explica qual dado ficou antigo. A projeção versionada oferece controle mais preciso ao custo de consumo, reconciliação, backfill e observabilidade.

Eu registraria condições de abandono: baixa taxa de leituras elegíveis, custo operacional desproporcional ou impossibilidade de definir atualização segura. Também definiria o que não está sendo decidido: mecanismo de autorização financeira e arquitetura interna do Nubank não fazem parte do exercício. A revisão arquitetural deve comparar alternativas com dados da jornada. O resultado publicado inspira uma forma de raciocinar; não dispensa a equipe de demonstrar que o mesmo padrão faz sentido em outro contexto.

ANÁLISE E DECISÃO

Execução EM: distribuir o custo da nova arquitetura



Como Engineering Manager, alinharia Produto, domínios de origem e Plataforma sobre o problema e as garantias. Um time pode ganhar latência enquanto outro recebe carga de eventos, suporte e manutenção de contratos. O planejamento precisa reconhecer essa transferência para evitar uma otimização local que cria dívida operacional em outra equipe. Definiria responsáveis por producer, projeção, API, fallback e reconciliação, com critérios de operação antes do lançamento.

O valor seria estimado por tentativas elegíveis, variação causal na conclusão e margem por conclusão, descontando infraestrutura e operação. Também mediria capacidade recuperada por redução de chamadas. Esses componentes não podem ser somados sem verificar sobreposição. O ganho de 76% pertence ao caso oficial; nenhum número do laboratório permite projetar receita ou disponibilidade. A decisão de investimento exige baseline interno e sensibilidade às hipóteses.

A pergunta Staff seria: como provar que uma leitura rápida permanece aceitável quando apenas um componente para de atualizar e o fallback recebe todo o tráfego? A resposta deve incluir versão, atualização, capacidade e experiência de falha. A pergunta de liderança seria quem pode interromper o rollout e qual custo de contingência foi financiado. Uma arquitetura só está pronta para ampliar quando a organização consegue operá-la com a mesma clareza com que consegue desenhá-la.

ANÁLISE E DECISÃO

12 · A nuvem tem limites e tempo de resposta

12



Managing Cloud Limits

Captura da publicação oficial consultada em 23/10/2026.

Uma publicação de engenharia do Nubank descreve múltiplas contas, sharding e acompanhamento de utilização, quotas e custos. O caso mostra capacidade como responsabilidade compartilhada entre plataforma e domínios. Os números de escala do artigo são históricos; a contribuição deste capítulo não depende de tratá-los como inventário atual. A pergunta é como transformar sinais de consumo em decisões antes de uma jornada encontrar um limite.

Uma quota não é apenas uma linha de configuração. Ela possui escopo, unidade, fonte, prazo para aumento e relação com dependências. Há limites por região, conta, recurso ou serviço. A aplicação pode ter CPU disponível e ainda não conseguir criar conexões, endereços ou componentes necessários. Planejar apenas a dimensão mais visível cria uma falsa sensação de margem. O modelo de capacidade precisa representar o gargalo relevante para a jornada.

A proposta combina inventário de limites, previsão explícita e reserva antes de admitir mudanças planejadas. O laboratório testa reserva multidimensional e concorrência local, sem provisionar recursos. Ele não simula AWS nem Kubernetes. Seu papel é demonstrar que duas equipes não podem consumir a mesma margem prometida simultaneamente. O desenho de produção exigiria fontes atualizadas, reconciliação de reservas com recursos reais e tratamento do tempo entre solicitar capacidade e tê-la efetivamente disponível.

Managing Cloud Limits

ANÁLISE E DECISÃO

Observado, reservado e disponível são quantidades diferentes



Consumo observado descreve o que uma fonte mediu. Reserva descreve intenção já aceita, mas ainda não refletida nessa medição. Margem de segurança representa capacidade que não queremos comprometer no funcionamento normal. A disponibilidade segura é limite menos esses três componentes, respeitando a unidade de cada recurso. Misturar reserva e consumo observado pode contar a mesma demanda duas vezes ou, pior, não contar nenhuma.

No laboratório, os baselines são constantes e a reserva é mantida sob um lock. Não há provisionamento, portanto não existe transição automática de reserva para consumo real. Essa limitação evita esconder o problema mais difícil: reconciliar identificador de pedido, recurso criado e medição posterior. Uma implementação real precisaria de estados como solicitado, reservado, provisionando, observado e encerrado, com timeout e recuperação definidos para cada um.

Capacidade é multidimensional. Uma demanda só é aceita quando todas as dimensões necessárias cabem; reservar CPU e falhar em endereço de rede deixaria um fragmento inútil se não houvesse atomicidade. O modelo rejeita a demanda inteira, permitindo que a equipe revise tamanho ou calendário. A política de prioridade também precisa ser explícita. Uma expansão crítica pode justificar outra decisão, mas essa exceção deve ter responsável e evidência, não um caminho informal que ignora a margem sem registro.

ARQUITETURA PROPOSTA · NÃO INTERNA

Da quota ao compromisso de lançamento



Arquitetura externa proposta. Caixas representam responsabilidades; setas explicitam contratos.

O coletor normaliza unidades e marca a atualização da fonte. Uma quota sem dado recente não deveria parecer ilimitada. O inventário separa escopos para evitar comparar consumo de uma região com limite de outra. A previsão produz cenários, não certezas: crescimento linear pode ser útil para antecipação, mas não descreve automaticamente campanhas ou eventos concentrados.

A admissão liga capacidade ao compromisso de lançamento. Ela reserva todas as dimensões relevantes e acompanha o resultado do provisionamento. O retorno assíncrono evita manter uma transação aberta enquanto um fornecedor cria recursos. A reconciliação precisa reconhecer operações concluídas cuja resposta se perdeu, além de reservas abandonadas. A jornada informa prioridade e calendário; a plataforma informa margem e tempo necessário. Esse contrato permite discutir adiamento, redução de escopo ou expansão com dados, sem transformar uma estimativa técnica em uma promessa de disponibilidade irrestrita.

CÓDIGO COM ESCOPO EXPLÍCITO

Reserva multidimensional sem overbooking local

PYTHON / PROPOSTA AUTORAL

```python
def allocate(self, token, demand):
    with self.lock:
        if token in self.requests:
            if self.requests[token] != demand:
                raise ValueError('token conflitante')
            return False
        if not demand or set(demand) - set(self.limits):
            raise ValueError('recurso desconhecido ou demanda vazia')
        if any(type(v) is not int or v <= 0 for v in demand.values()):
            raise ValueError('demanda inteira positiva obrigatória')
        available = {}
        for resource, limit in self.limits.items():
            pending = sum(x.get(resource, 0)
                          for x in self.allocations.values())
            available[resource] = (limit - self.used[resource]
                - self.reserve_floor[resource] - pending)
        if any(amount > available[key] for key, amount in demand.items()):
            raise RuntimeError('capacidade insuficiente')
        self.allocations[token] = dict(demand)
        self.requests[token] = dict(demand)
        return True
```

Fragmento literal de labs/case_12.py. Reserva sintética em memória; não provisiona AWS ou Kubernetes.

O código executado calcula e compromete disponibilidade sob o mesmo lock. Duas threads não podem prometer a mesma margem. O histórico de tokens permanece após release: replay reconhece a operação encerrada, sem recriar a reserva; outro conteúdo provoca conflito. Uma nova intenção exige novo token. Cópias independentes protegem os registros contra alteração do dicionário recebido.

O lock protege um processo e o histórico é volátil, sem política de retenção. Coordenação distribuída exigiria persistência, comparação atômica e expiração ligada à janela de replay. O laboratório especifica comportamento; memória local não governa recursos reais nem oferece idempotência após reinício.

CÓDIGO COM ESCOPO EXPLÍCITO

Prometheus: alertar sobre margem e dados antigos

YAML / PROPOSTA AUTORAL

```yaml
groups:
  - name: portfolio-capacity
    rules:
      - alert: SafeCapacityExhausted
        expr: portfolio_safe_headroom < 0
        for: 5m
        labels:
          severity: warning
          owner: platform-demo
        annotations:
          summary: "Margem segura negativa"
          description: "Revisar reservas e demanda planejada."
      - alert: CapacitySourceStale
        expr: time() - portfolio_capacity_collected_at > 900
        for: 10m
        labels:
          severity: warning
          owner: platform-demo
        annotations:
          summary: "Fonte de capacidade desatualizada"
          description: "Confirmar coleta antes de admitir expansão."

# Métricas, limiares e responsável são exemplos propostos.
# Validar regras com dados e ferramentas do ambiente real.
```

YAML de referência não executado. Métricas e nomes são propostos, não telemetria interna do Nubank.

As regras de referência distinguem falta de margem e falta de informação. Se a coleta parou, um painel estável pode parecer saudável enquanto o consumo continua crescendo. A idade da fonte deve entrar na decisão operacional. Os limites do exemplo são sintéticos e precisam ser calibrados com frequência de coleta e tempo de reação disponível.

Prometheus pode avaliar essas condições, mas um alerta não reserva capacidade nem solicita aumento de quota. O responsável precisa de um procedimento verificável e acesso às fontes. Rótulos devem usar dimensões controladas, como conta e tipo de quota, evitando identificadores de cada pedido que multiplicariam séries sem benefício proporcional.

Prometheus · Alerting rules

CÓDIGO COM ESCOPO EXPLÍCITO

Uma previsão precisa carregar suas premissas

JSON / PROPOSTA AUTORAL

```json
{
  "scope": "synthetic-account-region",
  "resource": "demo-network-addresses",
  "unit": "count",
  "observed_used": 70,
  "provider_limit": 100,
  "safety_reserve": 10,
  "pending_reservations": 0,
  "forecast": {
    "method": "linear-scenario",
    "daily_growth": 5,
    "days_to_safe_threshold": 4,
    "assumption": "constant growth, no releases"
  },
  "expansion": {
    "lead_time_days": 7,
    "status": "not_requested"
  },
  "collected_at": "2026-10-23T12:00:00Z",
  "source": "synthetic-fixture",
  "decision": "review-before-new-demand"
}
```

Contrato e números inteiramente sintéticos; aritmética linear coberta pelo teste de forecast.

Os valores ilustram uma margem de vinte unidades até o limite seguro e crescimento de cinco por dia, resultando em quatro dias. Se a expansão leva sete dias, esperar o alerta de saturação seria tarde. Isso é um cenário aritmético, não previsão sobre o Nubank nem garantia de prazo de um fornecedor.

O contrato registra método, unidade e hipótese para que a estimativa possa ser contestada. Crescimento sazonal, campanhas e mudanças de arquitetura exigem outros cenários. Também é necessário distinguir pedido de aumento aprovado de capacidade utilizável. A decisão de lançamento precisa considerar o momento em que o recurso estará disponível, não apenas a abertura de uma solicitação.

ANÁLISE E DECISÃO

Reservas abandonadas e medições atrasadas



Um controlador pode reservar capacidade e falhar antes de iniciar o provisionamento. Sem expiração ou reconciliação, a reserva fica ocupando margem indefinidamente. Por outro lado, liberar apenas porque um timeout ocorreu pode permitir overbooking se o fornecedor concluiu a operação. A solução exige identidade rastreável e consulta de resultado, não uma escolha universal entre reservar para sempre ou soltar imediatamente.

Medição atrasada cria uma segunda dificuldade. Depois que o recurso aparece na fonte, a reserva correspondente precisa deixar de ser somada como demanda pendente. Essa passagem deve ser baseada em vínculo verificável, não em aproximação de quantidade. Caso contrário, uma atualização parcial pode reduzir ou inflar disponibilidade. O laboratório não modela esse ciclo; a limitação é importante porque o problema de produção está na coordenação entre intenção e observação.

Falhas correlacionadas também mudam a margem necessária. Perder um shard pode deslocar tráfego para outros, e uma expansão pode depender de um recurso que já está no limite. Eu testaria cenários de contingência com gargalos múltiplos, incluindo limites de conexões e taxa de chamadas. A política de reserva normal não substitui capacidade de falha. São compromissos diferentes, que precisam ser discutidos com os objetivos de disponibilidade das jornadas afetadas.

CÓDIGO COM ESCOPO EXPLÍCITO

Concorrência transforma previsão em regra verificável

PYTHON / PROPOSTA AUTORAL

```python
def test_multidimensional_reservation_is_atomic(self):
    with self.assertRaises(RuntimeError):
        self.capacity.allocate('a', {'cpu': 2, 'ip': 6})
    self.assertEqual(self.capacity.allocations, {})

def test_same_token_not_reserved_twice(self):
    self.assertTrue(self.capacity.allocate('a', {'cpu': 3}))
    self.assertFalse(self.capacity.allocate('a', {'cpu': 3}))
    self.assertEqual(len(self.capacity.allocations), 1)

def test_conflicting_token(self):
    self.capacity.allocate('a', {'cpu': 3})
    with self.assertRaises(ValueError):
        self.capacity.allocate('a', {'cpu': 4})

def test_concurrent_demands_cannot_overbook(self):
    barrier = threading.Barrier(2)
    def attempt(token):
        barrier.wait()
        try:
            self.capacity.allocate(token, {'cpu': 3})
            return 'accepted'
        except RuntimeError:
            return 'rejected'
    with ThreadPoolExecutor(2) as pool:
        results = list(pool.map(attempt, ['a', 'b']))
    self.assertEqual(sorted(results), ['accepted', 'rejected'])
```

Fragmento de labs/case_12.py. Resultado: 9 testes passaram; ver results_12.txt.

Nove testes passaram. O fragmento cobre demanda multidimensional, repetição, conflito e concorrência. Os demais verificam liberação, recurso desconhecido, cenário e identidade após release: replay não recria reserva e payload diferente continua bloqueado. A concorrência usa barreira para aproximar duas tentativas, observando uma aceitação e uma rejeição. O histórico permanece somente em memória.

Isso não é teste de carga nem simulação de um provedor. A validação de produção precisaria incluir controladores independentes, persistência, resposta perdida e reconciliação de recursos. O resultado local estabelece uma propriedade: margem previamente comprometida não pode ser prometida duas vezes. O mecanismo de coordenação pode mudar sem abandonar essa propriedade.

ANÁLISE E DECISÃO

Dias até o limite só importam junto do prazo de ação



Um painel de utilização percentual é fácil de entender, mas pode esconder urgência. Oitenta por cento de uma quota estável pode ser confortável; sessenta por cento com crescimento acelerado e expansão lenta pode exigir ação imediata. Eu compararia tempo projetado até o limite seguro com tempo necessário para obter capacidade, incluindo aprovação, provisionamento, validação e eventual migração de carga.

Também acompanharia incerteza da previsão. A diferença entre cenário base e pico informa quanta margem estamos comprando contra variabilidade. Não transformaria uma regressão linear em uma data exata de falha. As fontes precisam registrar unidade, escopo e idade; uma mudança de definição deve aparecer como evento de metadados para evitar interpretar uma quebra de série como queda real de consumo.

No nível de produto, relacionaria capacidade a jornadas e campanhas relevantes. A resposta operacional pode ser aumentar quota, reduzir consumo, redistribuir carga ou adiar uma entrega. Cada alternativa possui custo e tempo. O alerta deve expor essas opções e um responsável, em vez de apenas pintar uma célula de vermelho. Mediria incidentes por limite, pedidos urgentes e tempo de resolução, mas evitaria afirmar incidentes evitados sem um contrafactual razoável. Preparação tem valor mesmo quando seu efeito não cabe em uma contagem simples.

ANÁLISE E DECISÃO

Implantar primeiro como parecer, depois como controle



A primeira versão do inventário deveria informar decisões, sem bloquear automaticamente provisionamentos. Isso permite comparar quotas, corrigir unidades e identificar lacunas de ownership antes de criar uma nova dependência crítica. Eu escolheria poucos recursos de alto impacto e manteria acesso ao dado original. Quando a classificação divergir do que os times observam, a equipe precisa conseguir explicar o motivo e corrigir a fonte.

Depois de estabilizar o dado, a admissão poderia atuar em um escopo controlado, com política clara para indisponibilidade do controlador. Falhar aberto preserva velocidade, mas pode comprometer margem; falhar fechado protege limite, mas pode impedir uma recuperação urgente. A escolha depende do tipo de operação e deve ser discutida antes da falha. Um caminho excepcional precisa ser auditável e restrito, sem virar a rotina silenciosa.

O rollout incluiria ensaios de coleta interrompida, reserva órfã e quota alterada fora da ferramenta. A reversão deve preservar registros de decisões para posterior reconciliação. Desligar a verificação não apaga recursos já criados nem restaura capacidade consumida. Por isso, a mudança é um processo operacional, não apenas uma flag. Ampliaria somente quando os times entenderem o contrato e conseguirem resolver exceções sem depender permanentemente do autor do sistema.

ANÁLISE E DECISÃO

Isolamento tem custo e precisa de uma razão



Separar contas e clusters pode reduzir o alcance de falhas e permitir evolução independente, mas aumenta inventário, configuração e esforço operacional. Compartilhar infraestrutura melhora utilização, porém cria contenção e exige governança. A decisão deve considerar perfil de carga, sensibilidade, requisitos de disponibilidade e capacidade da equipe. Um padrão único aplicado a todos os domínios simplifica o desenho no papel e pode piorar o custo total.

No Kubernetes, requests, limites e políticas de distribuição operam em uma camada diferente das quotas do provedor. Ter permissão para criar um pod não garante endereço de rede, capacidade física ou dependência externa disponível. Eu manteria esses escopos explícitos e testaria a cadeia de provisionamento completa. Também observaria como retries, autoscaling e recuperação de falhas interagem, pois automações independentes podem amplificar uma disputa por recurso.

Segurança exige acesso mínimo às fontes e às ações de mudança. Um coletor de métricas não precisa necessariamente de permissão para aumentar quotas ou criar infraestrutura. Separar leitura, recomendação e execução reduz impacto de erro e facilita auditoria. A plataforma deve proteger tanto contra excesso de consumo quanto contra bloqueios indevidos. O objetivo é permitir crescimento previsível com limites compreendidos, sem confundir controle centralizado com responsabilidade operacional bem distribuída.

ANÁLISE E DECISÃO

ADR 12 · Capacidade como contrato multidimensional



A decisão é representar capacidade por escopo e dimensão, incluindo consumo, reservas, margem e atualização da fonte. O contexto é crescimento de uma plataforma onde equipes independentes competem por recursos compartilhados. A proposta busca antecipar limites e evitar promessas incompatíveis, mantendo separadas recomendação, admissão e execução. O laboratório implementa apenas a regra local de reserva.

Uma alternativa é operar por alertas percentuais sem reserva. Ela custa menos, mas não protege contra demandas simultâneas já planejadas. Outra é reservar de forma central toda capacidade futura, reduzindo autonomia e possivelmente retendo recursos por intenções incertas. O contrato multidimensional permite um meio termo: compromissos com identidade e validade, acompanhados de reconciliação. Isso exige operar o ciclo de vida e revisar previsões, não apenas escrever um algoritmo de disponibilidade.

Eu reavaliaria o controle quando a qualidade das fontes for insuficiente ou quando a fricção imposta superar o risco que ele reduz. Também mediria quantas exceções são abertas e por quê. Muitas exceções podem indicar política inadequada, não falta de disciplina dos times. A arquitetura deve ajudar a tomar decisões sobre tempo, custo e confiabilidade. Se virar uma barreira burocrática sem informação confiável, perderá adesão e será contornada justamente quando o risco estiver maior.

ANÁLISE E DECISÃO

Execução EM: negociar margem antes do compromisso



Como Engineering Manager, conectaria planejamento de lançamentos ao calendário de capacidade. A conversa precisa ocorrer antes de anunciar datas que dependem de recursos ainda não disponíveis. Definiria um responsável por cada quota crítica e um canal de revisão com Plataforma, domínios e FinOps. A equipe de produto contribui com demanda e prioridade; a plataforma contribui com limites, alternativas e prazo realista de expansão.

O valor pode incluir redução de urgências, menor desperdício e menor exposição a indisponibilidade. Esses componentes exigem cuidado: uma quota aumentada não representa economia, e um incidente que não ocorreu não pode ser contado automaticamente como receita preservada. Eu usaria cenários de perda esperada, custo de capacidade e esforço operacional, mantendo hipóteses visíveis. A decisão pode favorecer margem adicional mesmo sem retorno imediato, quando protege uma jornada crítica.

A pergunta Staff seria: como evitar contar a mesma capacidade como reserva pendente e consumo observado durante a reconciliação? A resposta deve ligar identidade, estado e fonte. A pergunta de liderança seria quem decide entre adiar uma entrega e consumir a margem de contingência. Esse trade-off não deve ser escondido em uma configuração. Uma organização preparada torna a escolha explícita, registra o risco e dá às equipes condições para cumprir o compromisso assumido.

ANÁLISE E DECISÃO

13 · O recurso continua existindo depois que o time muda

13



How Nubank distributes infrastructure ownership to operate more than 21 thousand databases with a lean team

Captura da publicação oficial consultada em 23/10/2026.

O Nubank publicou um caso de ownership distribuído para uma operação com mais de 21 mil bancos de dados. O relato liga provisionamento, inventário e ciclo de vida à responsabilidade dos times de produto. A equipe central citada não representa todo o esforço organizacional. A análise deste capítulo se concentra em uma transição específica: o que acontece quando a estrutura responsável pelo recurso muda.

Um campo owner preenchido na criação pode ficar correto sintaticamente e errado operacionalmente meses depois. O time pode ter sido encerrado, dividido ou substituído, enquanto alertas, custos e dependências continuam ativos. Atualizar uma etiqueta não garante que alguém aceitou a responsabilidade, recebeu acesso ou sabe restaurar o serviço. Ownership precisa ser uma relação verificável entre recurso, jornada e equipe capaz de agir.

A proposta combina transferência com aceite, controle de revisão e barreiras de arquivamento. O laboratório usa um registro imutável local e testes de decisões; não cria, migra ou remove bancos. Um trecho Datomic ilustra comparação de revisão em transação, sem afirmar que seja o mecanismo interno usado no caso oficial. A pergunta Staff não é como apagar mais recursos, mas como manter contexto suficiente para mudar responsabilidade ou encerrar uso sem perder segurança, retenção e capacidade de recuperação.

How Nubank distributes infrastructure ownership to operate more than 21 thousand databases with a lean team

ANÁLISE E DECISÃO

Ownership é aceite, capacidade e contexto



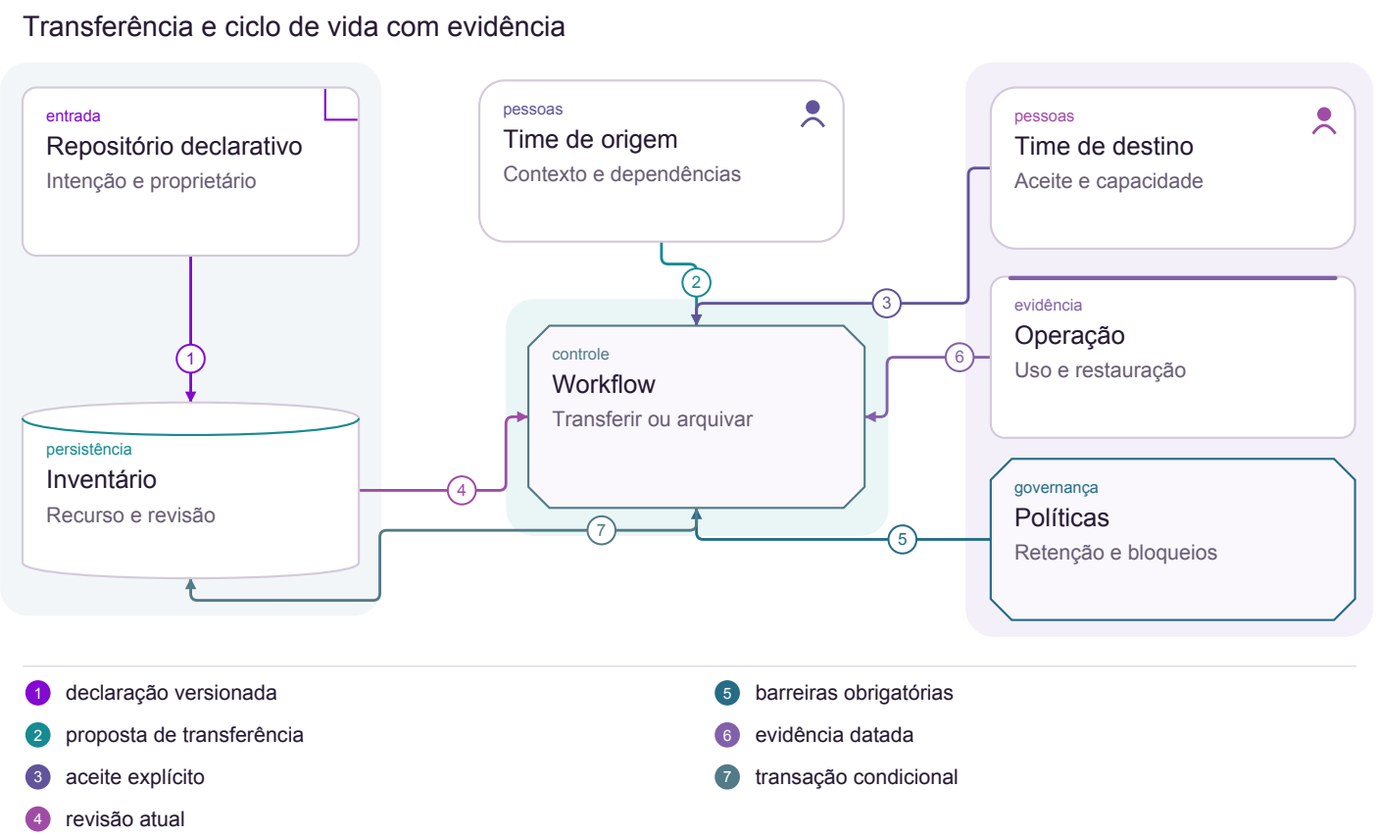
O estado mínimo de um recurso inclui identidade, responsável, revisão, situação operacional e restrições de ciclo de vida. Para transferência, o destino precisa aceitar explicitamente o compromisso. A origem não pode simplesmente escrever o nome de outro time e considerar a obrigação resolvida. O aceite deve estar associado ao contexto recebido: runbook, dependências, acesso, alertas e compromissos relevantes.

Revisão protege contra decisões tomadas sobre um estado antigo. Uma transferência aprovada antes de uma nova restrição ou mudança de dependência não deve sobrescrever o inventário atual sem reavaliação. O laboratório usa uma comparação de revisão dentro de um lock. Em uma base compartilhada, a mesma regra precisa ser atômica na fronteira de persistência. Ler, conferir em memória e escrever depois deixa uma janela para decisões concorrentes.

Arquivamento é outro workflow. Inatividade é apenas um sinal; ela não prova ausência de consumidores sazonais, obrigações de retenção ou utilidade de recuperação. O modelo exige quatro evidências sintéticas e ausência de bloqueio. Mesmo assim, a ação apenas marca o registro como arquivado. Não demonstra exclusão segura. A distinção é essencial: a governança deve separar detectar candidato, revisar contexto, preservar recuperação e executar uma ação, cada qual com autoridade e evidência apropriadas.

ARQUITETURA PROPOSTA · NÃO INTERNA

Transferência e ciclo de vida com evidência



Arquitetura externa proposta. Caixas representam responsabilidades; setas explicitam contratos.

A proposta trata inventário como um conjunto de relações, não somente uma planilha de nomes. Uma decisão usa a revisão atual e evidências com origem. A transferência depende do aceite do destino; o arquivamento depende de políticas e observações operacionais. O workflow não deve transformar silêncio de um time em aprovação automática quando há risco de perda ou indisponibilidade.

As setas representam responsabilidades que podem ser implementadas por serviços e ferramentas diferentes. Datomic é uma opção coerente com a stack publicamente documentada, mas o diagrama não exige um produto específico. A propriedade central é que a mudança condicional e sua revisão ocorram juntas. Dados de uso ajudam a encontrar candidatos, enquanto retenção e restauração respondem se uma ação é permitida e recuperável. Separar essas perguntas evita que uma otimização de custo assuma autoridade sobre compromissos de negócio que não aparecem na métrica de tráfego.

CÓDIGO COM ESCOPO EXPLÍCITO

Arquivar exige mais do que ausência de tráfego

PYTHON / PROPOSTA AUTORAL

```python
def set_hold(self):
    with self.lock:
        self.resource = replace(self.resource, hold=True,
                                revision=self.resource.revision + 1)

def archive(self, expected, *, inactive, retention_elapsed,
            no_dependencies, restore_verified):
    evidence = (inactive, retention_elapsed,
                no_dependencies, restore_verified)
    if any(type(value) is not bool for value in evidence):
        raise ValueError('evidências devem ser booleanas neste modelo')
    with self.lock:
        current = self.resource
        if expected != current.revision:
            raise ValueError('evidência de revisão antiga')
        if current.hold or not all((inactive, retention_elapsed,
                                    no_dependencies, restore_verified)):
            raise PermissionError('barreiras não satisfeitas')
        if current.archived:
            return current
        self.resource = replace(current, archived=True,
                                revision=current.revision + 1)
        return self.resource
```

Fragmento literal de labs/case_13.py. Arquivamento apenas lógico em memória; nenhum banco real é alterado.

O fragmento executado rejeita evidências de tipo inadequado, revisão antiga, bloqueio ativo ou qualquer barreira não satisfeita. A mudança preserva o registro e incrementa sua revisão. Não existe comando de exclusão no laboratório. Essa escolha permite testar a regra sem produzir uma demonstração destrutiva ou fingir que quatro booleanos bastam para remover infraestrutura real.

Em produção, cada evidência precisaria de origem confiável, validade temporal e escopo. Uma restauração testada para outro backup não comprova recuperação do recurso atual. O modelo reduz essa riqueza a entradas controladas; sua função é expor a conjunção de condições e o ponto de comparação de revisão.

CÓDIGO COM ESCOPO EXPLÍCITO

Datomic: comparar revisão e transferir no mesmo conjunto

CLOJURE / PROPOSTA AUTORAL

```clojure
(ns portfolio.resource-transfer)

(defn transfer-tx
  [resource-eid expected-revision new-owner accepting-team]
  (when-not (= new-owner accepting-team)
    (throw (ex-info "Destination must accept"
                    {:kind :missing-acceptance})))
  (when-not (and (integer? expected-revision)
                 (pos? expected-revision))
    (throw (ex-info "Invalid revision"
                    {:kind :invalid-revision})))
  [[:db/cas resource-eid :resource/revision
    expected-revision (inc expected-revision)]
   [:db/add resource-eid :resource/owner new-owner]])

;; Proposta: :resource/revision é cardinalidade one.
;; Os atributos devem existir em um schema revisado.
;; Submeter o vetor em UMA transação Datomic.
;; Aceite deve vir de identidade autenticada.
;; A checagem local não substitui autorização.
;; Restrições adicionais pertencem à mesma decisão.
;; O trecho constrói dados; não conecta a uma base.
```

Trecho Clojure/Datomic de referência não executado; schema e função propostos. :db/cas é operação documentada.

O exemplo usa a operação compare-and-swap documentada pelo Datomic para associar a mudança à revisão esperada. Se outra transação alterou a revisão, a comparação falha e a proposta precisa ser reavaliada. A alteração de owner e o incremento pertencem ao mesmo conjunto transacional; executá-los separadamente perderia a propriedade desejada.

Os atributos resource são um schema proposto, não nomes internos do Nubank. A verificação de aceite mostrada recebe valores como parâmetros e não implementa autenticação. O serviço real precisaria obter identidade confiável e incluir todas as restrições necessárias. O fragmento não foi executado em Datomic; o laboratório Python testa a semântica equivalente local.

Datomic · Transaction Data

CÓDIGO COM ESCOPO EXPLÍCITO

Evidência de ciclo de vida tem validade e origem

JSON / PROPOSTA AUTORAL

```json
{
  "resource_id": "synthetic-db-17",
  "expected_revision": 8,
  "proposed_action": "archive",
  "owner": "demo-payments-team",
  "evidence": {
    "usage_window": {
      "start": "2026-09-01",
      "end": "2026-10-01",
      "source": "synthetic-observability"
    },
    "dependencies_review": "demo-review-12",
    "retention_policy": "demo-policy-v3",
    "restore_test": "demo-restore-4"
  },
  "holds": [],
  "approval": {
    "actor": "synthetic-authorized-owner",
    "expires_at": "2026-10-24T12:00:00Z"
  },
  "deletes_data": false
}
```

Contrato proposto de governança; não representa política de retenção do Nubank nem autorização de descarte.

O envelope mostra o que os booleanos do laboratório escondem: evidência precisa de referência, janela e política aplicável. Ausência de uso durante um mês pode ser insuficiente para um processamento anual. Uma política de retenção pode variar por produto e jurisdição; não deve ser deduzida de um prazo genérico presente em um artigo.

A aprovação expira porque contexto muda. Uma nova dependência ou bloqueio deve invalidar a decisão anterior por revisão. O exemplo não fornece política jurídica nem autoriza exclusão. Ele documenta um comando proposto de arquivamento lógico, com dados sintéticos, para discutir quais provas uma implementação real teria de obter e verificar.

ANÁLISE E DECISÃO

Reorganização é uma mudança de sistema



Quando um time se divide, não basta distribuir nomes em uma planilha. Dependências, alertas, chaves de acesso e custos podem seguir caminhos diferentes. O recurso pode continuar funcionando por meses, escondendo que ninguém consegue restaurá-lo ou interpretar um incidente. Eu trataria reorganização como evento de governança que abre transferências pendentes, com responsável temporário e prazo, sem retirar ownership antes do aceite do destino.

A corrida mais importante ocorre entre revisão de ciclo de vida e mudança de contexto. Um candidato a arquivamento pode receber novo consumidor depois da coleta de métricas. Uma nova restrição pode chegar enquanto a aprovação está em andamento. Por isso, a ação precisa verificar a revisão atual e as barreiras no ponto de execução. Guardar um parecer aprovado não permite ignorar eventos posteriores que alteram sua validade.

Outro risco é o consumidor invisível. Métricas de leitura podem não capturar acesso sazonal, exportação indireta ou dependência operacional rara. A proposta combina sinais de uso com mapa de consumidores e capacidade de restauração. Quando a confiança é insuficiente, a resposta pode ser isolamento ou observação adicional, não exclusão. Reduzir custo é desejável, mas não justifica apagar contexto justamente onde a organização admite que ainda não compreende o recurso.

CÓDIGO COM ESCOPO EXPLÍCITO

Testar a barreira e a revisão, não só o caminho feliz

PYTHON / PROPOSTA AUTORAL

```python
def test_destination_must_accept(self):
    with self.assertRaises(PermissionError):
        self.registry.transfer(1, 'team-b', 'team-a')
    self.assertEqual(self.registry.resource.owner, 'team-a')

def test_stale_transfer_cannot_override(self):
    self.registry.transfer(1, 'team-b', 'team-b')
    with self.assertRaises(ValueError):
        self.registry.transfer(1, 'team-c', 'team-c')
    self.assertEqual(self.registry.resource.owner, 'team-b')

def test_hold_blocks_archive(self):
    self.registry.set_hold()
    with self.assertRaises(PermissionError):
        self.registry.archive(2, **self.evidence)

def test_each_missing_barrier_blocks(self):
    for key in self.evidence:
        args = dict(self.evidence, **{key: False})
        with self.subTest(key=key), self.assertRaises(PermissionError):
            self.registry.archive(1, **args)
```

Fragmento de labs/case_13.py. Resultado: 8 testes passaram; ver results_13.txt.

Oito testes passaram no laboratório completo. O fragmento examina aceite do destino, revisão obsoleta, bloqueio ativo e cada barreira ausente. Os demais verificam evidência antiga após transferência, preservação do registro arquivado, impedimento de transferência posterior e rejeição de evidência sem tipo correto. São decisões locais sobre um único recurso.

O resultado não comprova inventário completo, backup recuperável ou conformidade de retenção. Esses fatos dependem de sistemas e responsáveis externos. A validação real deveria ensaiar transferência entre equipes, simular mudança de contexto e restaurar recursos representativos. Um teste de booleanos é útil para a regra de decisão, desde que não seja apresentado como prova das evidências fornecidas.

ANÁLISE E DECISÃO

Ownership válido precisa ser observável








Eu mediria recursos com responsável ativo, transferências aguardando aceite e tempo em responsabilidade temporária. Uma etiqueta preenchida não equivale a um time capaz de agir. O inventário deveria permitir verificar acesso, runbook, rota de alerta e dependências relevantes, sem transformar cada item em uma certificação burocrática que envelhece no dia seguinte. A melhor evidência pode ser um exercício recente de operação.

Para ciclo de vida, acompanharia candidatos identificados, decisões tomadas, motivos de bloqueio e resultados de restauração. Taxa alta de arquivamento pode ser positiva ou indicar revisão superficial; precisa ser interpretada com incidentes e reversões. Também diferenciaria custo removido, custo preservado por retenção e capacidade recuperada. Apresentar toda redução de recurso como economia imediata ignora contratos, backups e custos fixos.

Alertas deveriam chegar ao responsável atual e possuir um caminho quando ele não existe. Enviar notificações para um canal abandonado satisfaz uma regra técnica e falha operacionalmente. Eu testaria esse fluxo periodicamente, incluindo mudanças organizacionais. O objetivo é detectar perda de contexto antes de um incidente. A governança torna-se útil quando reduz o tempo para encontrar quem decide, preserva evidência e ajuda a equipe a operar, não quando apenas aumenta a quantidade de campos obrigatórios.

ANÁLISE E DECISÃO

Automação começa pela qualidade do inventário



A primeira implantação seria observacional: reunir declarações e recursos encontrados, identificar divergências e confirmar ownership com os times. O sistema deve distinguir recurso desconhecido de recurso sem uso. Essa etapa produz aprendizado sobre nomes, dependências e fontes, evitando automatizar decisões sobre um inventário que ainda não representa a realidade. A equipe central precisa de apoio dos domínios para interpretar os casos.

Depois, automatizaria partes reversíveis, como abrir proposta de transferência, solicitar aceite e atualizar metadados após validação. Arquivamento lógico viria com escopo limitado, evidências verificáveis e restauração ensaiada. Não usaria ausência de resposta como aprovação para ações destrutivas. Um processo pode ser automático sem remover responsabilidade humana: o sistema reúne provas, aplica barreiras e registra quem possui autoridade para decidir.

A expansão dependeria de qualidade e carga operacional. Se o volume de falsos candidatos consumir mais tempo do que o trabalho manual anterior, ajustaria sinais e critérios. Reversão precisa preservar histórico de decisões e permitir retomar o recurso no estado adequado. Também verificaria quem mantém a própria plataforma de governança. Um inventário sem dono, projetado para corrigir recursos sem dono, apenas reproduz o problema em outra camada.

ANÁLISE E DECISÃO

Datomic não substitui política de dados



Um modelo de fatos e histórico pode ajudar a explicar mudanças, mas não define sozinho quem pode acessar, reter ou remover informação. A escolha de Datomic deve ser acompanhada de schema, autorização, operação e entendimento de seu ciclo de vida. O trecho deste capítulo usa comparação de revisão; não pretende implementar descarte físico nem garantir obrigações regulatórias por meio de uma transação.

Retenção exige contexto de produto, jurisdição e finalidade. Eu evitaria codificar um prazo universal a partir de um exemplo público. A política precisa ter responsável, versão e mecanismo de atualização, com bloqueios apropriados quando houver obrigação adicional. Também separaria arquivamento operacional de eliminação de dados: reduzir capacidade ativa pode manter backups e referências necessárias por outro período.

Segurança envolve a plataforma de automação. Um serviço capaz de alterar ownership ou executar ciclo de vida possui poder relevante. Separaria leitura de inventário, proposta e execução; limitaria escopo e registraria decisões com evidência. Acesso de emergência precisa ser revisado, não usado como atalho permanente. O benefício da automação é repetir regras bem definidas com consistência. Quando a regra depende de contexto ausente, o comportamento correto é pedir investigação ao responsável, e não tentar inferir permissão a partir de silêncio ou baixa utilização.

ANÁLISE E DECISÃO

ADR 13 · Transferência com aceite e revisão



A decisão proposta é fazer ownership mudar por transferência aceita pelo destino e condicionada à revisão atual do recurso. O contexto é uma organização em evolução, na qual responsabilidades técnicas sobrevivem às equipes que as criaram. A intenção é preservar capacidade de ação, evitando que uma alteração declarativa esconda perda de conhecimento ou transfira trabalho para alguém que não concordou com o compromisso.

Uma alternativa é permitir edição livre do owner em um repositório. Ela é rápida e auditável como mudança de texto, mas não garante aceite ou prontidão. Outra é centralizar permanentemente toda responsabilidade em Plataforma. Isso reduz ambiguidade aparente, porém cria gargalo e distancia operação do contexto de produto. A transferência explícita mantém autonomia e distribui responsabilidade, ao custo de workflow, acompanhamento e revisão de pendências.

O arquivamento usa a mesma disciplina de revisão, mas não o mesmo conjunto de critérios. Retenção, dependências e recuperação pertencem a uma decisão distinta. Eu reconsideraria a implementação se o processo ficar tão pesado que os times deixem o inventário desatualizado. A resposta seria simplificar a experiência e automatizar coleta de evidências, preservando as barreiras essenciais. Governança eficaz não maximiza aprovações; torna uma decisão correta mais fácil de tomar e mais difícil de perder.

ANÁLISE E DECISÃO

Execução EM: dar tempo para assumir responsabilidade



Como Engineering Manager, incluiria transferência de ownership no planejamento de mudanças organizacionais e de produto. O time receptor precisa de tempo para conhecer dependências, revisar alertas e ensaiar recuperação. Uma reunião de passagem sem capacidade reservada não cria prontidão. Definiria responsável temporário e critérios de aceite, evitando deixar o recurso em um intervalo no qual cada equipe presume que a outra está cuidando.

O valor pode vir de custo de recursos realmente desativados com segurança, horas operacionais recuperadas e menor tempo para responder a incidentes. Esses componentes seriam medidos separadamente. Não multiplicaria a economia de um banco pelo inventário inteiro, nem confundiria arquivamento com desaparecimento de todo custo. Backups, retenção e manutenção da plataforma continuam existindo. A revisão deve considerar benefício líquido e qualidade da operação após a mudança.

A pergunta Staff seria: qual evidência deixa de ser válida quando ownership ou dependências mudam durante a aprovação? A resposta precisa conectar revisão, barreira e momento de execução. A pergunta de liderança seria como impedir que um time aceite responsabilidade apenas para encerrar uma pendência administrativa. Eu procuraria sinais práticos de capacidade: acesso, runbook, rota de alerta e exercício de recuperação. Ownership é um compromisso de operação, não apenas um nome que satisfaz um formulário.

ANÁLISE E DECISÃO

14 · BDC amplia velocidade e responsabilidade

14



Captura da publicação oficial consultada em 23/10/2026.

O Nubank documentou BDC como uma plataforma de interface guiada pelo servidor, com componentes Flutter e backend Clojure. A publicação relata adoção e controle de rollout no momento em que foi escrita. Ela não fornece um contrato público completo de componentes nem autoriza inferir lacunas atuais. A proposta deste capítulo discute governança de mudanças sobre uma plataforma desse tipo, preservando a distinção entre referência oficial e desenho autoral.

Mover composição para o servidor encurta o caminho de certas mudanças, mas não elimina compatibilidade com binários instalados. O cliente só consegue interpretar capacidades que possui. Também há combinações de tema, texto, acessibilidade, experimento e estado de jornada. Uma alteração pequena no conteúdo pode produzir um efeito amplo se a plataforma distribui a mesma revisão para milhões de sessões sem critérios de exposição.

O objetivo é publicar experiências com contrato verificável: revisão imutável, catálogo de ações, semântica acessível, negociação de capacidades e reversão. O laboratório modela essas decisões em Python e testa seleção e coorte determinística. Não renderiza Flutter nem acessa BDC ou NuDS reais. Os trechos Dart são propostas ilustrativas. A pergunta principal é como permitir autonomia aos squads enquanto a plataforma impede combinações inválidas antes que uma pessoa precise descobrir o erro no próprio aplicativo.

Backend Driven Content: why we developed a Server Driven UI framework at Nubank

ANÁLISE E DECISÃO

Protocolo, componente e conteúdo evoluem separadamente






Uma versão de protocolo define como interpretar o manifesto. Um catálogo de componentes define quais elementos e propriedades o cliente conhece. Uma revisão de conteúdo define a composição publicada. Usar apenas a versão comercial do aplicativo para representar os três conceitos dificulta compatibilidade e diagnóstico. Dois clientes com a mesma tela aparente podem ter capacidades diferentes, enquanto várias revisões de conteúdo podem compartilhar o mesmo protocolo.

No laboratório, Manifest declara protocolo, componentes, ação e nome acessível. A validação exige tupla de identificadores textuais para impedir mutação dos componentes após a publicação. O publicador recusa reutilizar uma revisão com conteúdo diferente. A seleção verifica se o cliente compreende o protocolo e possui todos os componentes necessários. Se não, tenta um fallback também compatível. Quando nenhum deles é adequado, retorna erro controlado em vez de fingir que uma composição desconhecida será interpretada com segurança.

O catálogo de ações limita o vocabulário da interface, mas não concede autorização financeira. Uma ação pode abrir uma jornada que fará sua própria verificação no serviço responsável. O manifesto não carrega código arbitrário nem regras de saldo. Essa separação protege a plataforma de se tornar uma linguagem genérica de execução disfarçada de conteúdo. A evolução precisa permitir expressividade suficiente para o produto, mantendo fronteiras que podem ser testadas, documentadas e compreendidas por equipes consumidoras.

ARQUITETURA PROPOSTA · NÃO INTERNA

Publicar, selecionar e observar uma revisão



Arquitetura externa proposta. Caixas representam responsabilidades; setas explicitam contratos.

O pipeline valida uma proposta antes de torná-la selecionável. O registro mantém conteúdo imutável para que a revisão observada pelo cliente possa ser reconstruída. O seletor combina capacidade técnica e exposição planejada; pertencer à coorte nunca supera incompatibilidade. O cliente ainda valida entrada, pois falhas e versões inesperadas podem atravessar a distribuição.

A ação segue para um serviço que aplica autorização de domínio. Isso impede que uma configuração de interface conceda um privilégio por existir na tela. Observabilidade relaciona revisão a sucesso de tarefa, falha de renderização e comportamento de fallback, sem capturar indiscriminadamente dados financeiros. O retorno operacional permite pausar expansão, mas não desfaz ações confirmadas. O diagrama propõe responsabilidades, não serviços internos específicos. Uma plataforma madura precisa tornar essas fronteiras fáceis de usar pelos squads; se cada equipe recria validação, coorte e fallback, a promessa de consistência se perde.

CÓDIGO COM ESCOPO EXPLÍCITO

Compatibilidade inclui a composição de contingência

PYTHON / PROPOSTA AUTORAL

```python
def publish(self, manifest):
    validate(manifest)
    previous = self.revisions.get(manifest.revision)
    if previous is not None and previous != manifest:
        raise ValueError('revisão é imutável')
    self.revisions[manifest.revision] = manifest

def activate(self, revision):
    if revision not in self.revisions:
        raise ValueError('revisão não publicada')
    self.active = revision

def select(self, capabilities, protocol, fallback):
    if self.revisions.get(fallback.revision) != fallback:
        raise ValueError('fallback não publicado')
    current = self.revisions.get(self.active)
    compatible = current and current.protocol == protocol
    compatible = compatible and set(current.components) <= capabilities
    if compatible:
        return current
    if (fallback.protocol == protocol
            and set(fallback.components) <= capabilities):
        return fallback
    raise ValueError('nenhuma composição compatível')
```

Fragmento literal de labs/case_14.py. Modelo Python executado; não implementa BDC/NuDS reais.

O publicador executado mantém a identidade da revisão e seleciona conteúdo compatível com o cliente. A verificação do fallback é importante: devolver uma composição antiga não garante que ela seja compreendida por qualquer cliente. Quando não existe alternativa compatível, o modelo falha de forma explícita, permitindo que a experiência trate indisponibilidade sem tentar renderizar dados desconhecidos.

O armazenamento é local e o catálogo é sintético. Não há assinatura de manifesto, persistência distribuída ou garantia de propagação do rollback. Esses mecanismos precisariam ser definidos na implantação. O trecho estabelece uma propriedade do contrato que pode ser preservada mesmo quando a infraestrutura e o formato de transporte forem diferentes.

CÓDIGO COM ESCOPO EXPLÍCITO

Flutter: interpretar um catálogo fechado

DART / PROPOSTA AUTORAL

```dart
import 'package:flutter/material.dart';

Widget renderProposedNode(
  Map<String, Object?> node,
  void Function(String) dispatch,
) {
  switch (node['type']) {
    case 'text':
      return Text(node['text'] as String);
    case 'action':
      final action = node['action'] as String;
      const allowed = {'view-invoice', 'open-support'};
      if (!allowed.contains(action)) {
        throw const FormatException('Unknown action');
      }
      return Semantics(
        button: true,
        onTap: () => dispatch(action),
        label: node['accessible_name'] as String,
        child: ExcludeSemantics(
          child: ElevatedButton(
            onPressed: () => dispatch(action),
            child: Text(node['text'] as String),
          ),
        ),
      );
    default:
      throw const FormatException('Unknown node type');
  }
}
```

Flutter/Dart autoral e não executado. dispatch e tipos de nó são contratos propostos, não APIs do Nubank.

O renderizador ilustrativo aceita tipos e ações conhecidos, sem interpretar scripts. ExcludeSemantics remove a semântica do botão descendente; por isso, o nó externo fornece explicitamente onTap com o mesmo despacho da interação visual. O nome acessível pode diferir do texto sem perder a ação. Validação de schema antes dos casts e tratamento de erro continuam sob responsabilidade da composição externa.

Em uma implementação real, componentes e tokens aprovados substituiriam widgets genéricos quando apropriado. Também seriam necessários testes de foco, navegação e texto ampliado. O trecho Dart não foi compilado; ele expressa uma fronteira de design. O único runtime verificado neste capítulo é o laboratório Python.

Flutter · Semantics
Flutter · ExcludeSemantics remove a semântica dos descendentes
Flutter · SemanticsProperties.onTap

CÓDIGO COM ESCOPO EXPLÍCITO

Manifesto: identidade estável e requisitos explícitos

JSON / PROPOSTA AUTORAL

```json
{
  "contract": "portfolio.ui.v1",
  "revision": "demo-invoice-r17",
  "protocol": 1,
  "required_components": ["text", "action"],
  "purpose": "invoice-navigation",
  "nodes": [
    {
      "type": "text",
      "text": "Consulte sua fatura"
    },
    {
      "type": "action",
      "text": "Ver fatura",
      "accessible_name": "Abrir detalhes da fatura",
      "action": "view-invoice"
    }
  ],
  "rollout": {
    "experiment": "demo-navigation-v1",
    "maximum_percent": 5
  }
}
```

Manifesto e nomes de ações autorais propostos; dados sintéticos. Não é schema oficial do BDC.

O manifesto proposto declara requisitos e composição. A revisão deve identificar exatamente o conteúdo aprovado; editar o payload mantendo o mesmo identificador destruiria rastreabilidade e tornaria comparação de coortes ambígua. O campo de rollout é uma intenção de exposição, não uma permissão para o cliente se incluir em qualquer grupo.

A seleção precisa acontecer em uma fronteira confiável e considerar compatibilidade antes de percentual. Dados financeiros não aparecem neste exemplo: a ação abre uma jornada que possui seus próprios contratos. O laboratório usa uma representação reduzida, sem árvore de nós. O JSON documenta como essa decisão poderia ser expandida, sem alegar que esse seja o formato interno do BDC.

ANÁLISE E DECISÃO

Canary precisa de coorte estável e critérios de saída



Uma exposição inicial pequena é útil quando permite comparar comportamento sem trocar participantes de maneira imprevisível. O laboratório usa hash determinístico de sujeito e sal para produzir uma coorte estável. Aumentar de cinco para vinte por cento preserva quem já estava incluído. Isso não garante equilíbrio estatístico nem substitui desenho experimental; é apenas uma propriedade operacional que facilita rastrear a experiência recebida.

A coorte precisa ser avaliada junto com capacidades. Um cliente selecionado para a experiência, mas incapaz de renderizar um componente, deve receber alternativa compatível. Essa exclusão altera a população observada e precisa aparecer na análise. Caso contrário, a versão nova pode parecer melhor porque só foi exibida em dispositivos mais recentes ou clientes com características diferentes. A comparação exige recortes e critérios definidos antes do resultado.

Critérios de saída incluem estabilidade de renderização, conclusão da tarefa, acessibilidade e ausência de efeitos adversos relevantes. Um canary sem condição de pausa apenas atrasa uma implantação total. Também é necessário decidir o que fazer com jornadas em andamento quando a revisão é revertida. A tela pode mudar, mas uma confirmação financeira já aceita não pode ser repetida ou apagada para acompanhar o conteúdo novo. Rollout de apresentação e transição de domínio precisam permanecer separados.

CÓDIGO COM ESCOPO EXPLÍCITO

O cliente antigo também faz parte do teste

PYTHON / PROPOSTA AUTORAL

```python
def test_old_client_gets_fallback(self):
    self.assertEqual(self.publisher.select({'text'}, 1, self.old), self.old)

def test_capable_client_gets_active_revision(self):
    self.assertEqual(self.publisher.select({'text', 'chart'}, 1, self.old),
                     self.new)

def test_revision_cannot_change(self):
    changed = Manifest('r1', 1, ('chart',), 'view-invoice', 'Fatura')
    with self.assertRaises(ValueError):
        self.publisher.publish(changed)

def test_missing_accessibility_rejected(self):
    with self.assertRaises(ValueError):
        validate(Manifest('r3', 1, ('text',), 'view-invoice', ''))

def test_rollback_selects_old_revision(self):
    self.publisher.activate('r1')
    self.assertEqual(self.publisher.select({'text', 'chart'}, 1, self.old),
                     self.old)
```

Fragmento de labs/case_14.py. Resultado: 11 testes passaram; ver results_14.txt.

Onze testes passaram no laboratório completo. O fragmento verifica fallback, revisão ativa, identidade imutável, semântica e rollback. Os demais cobrem coortes estáveis e monotônicas, limites, ações e fallback inválidos. Uma regressão rejeita listas mutáveis e componentes sem identificador textual, impedindo alteração posterior do conteúdo publicado. As asserções observam o manifesto efetivamente selecionado.

Não foram executados testes de widget Flutter, integração BDC ou propagação distribuída. Essas validações precisariam acrescentar árvore semântica, erro de rede, versões instaladas e jornadas iniciadas antes da reversão. O laboratório estabelece contratos pequenos que uma plataforma real poderia incorporar em níveis diferentes do pipeline.

ANÁLISE E DECISÃO

A unidade de observação é a revisão que a pessoa recebeu



Um evento de erro sem revisão de conteúdo não explica qual experiência estava em uso. Eu registraria protocolo, revisão, classe de capacidade e experimento em dimensões controladas, relacionando a jornada a falhas de renderização e conclusão de tarefa. A versão do binário continua útil, mas não identifica sozinha a composição distribuída pelo servidor. Essa distinção é central para investigar regressões em plataformas de conteúdo dinâmico.

Também observaria taxa de fallback e motivo. Um fallback frequente pode significar que o catálogo novo alcançou poucos clientes, que a negociação está errada ou que o plano de depreciação foi otimista. Não deveria ser tratado automaticamente como sucesso silencioso. A experiência antiga pode permanecer correta, mas impedir o benefício esperado ou esconder que uma parte da base está sendo excluída da mudança.

Para decidir expansão, combinaria métricas técnicas com comportamento de tarefa e amostragem qualitativa. Contar cliques em um botão mais destacado não demonstra melhor compreensão. Telemetria deve minimizar dados e respeitar cardinalidade; capturar a árvore inteira com conteúdo financeiro seria um atalho inadequado. O painel deve permitir responder qual revisão, em qual população e com qual consequência, sem transformar observabilidade em uma cópia indiscriminada da experiência privada do cliente.

ANÁLISE E DECISÃO

Rollback de conteúdo não é viagem no tempo



Uma revisão imutável permite apontar a distribuição para uma composição anterior, mas isso não garante reversão instantânea em todos os clientes. Há caches, sessões em andamento e conectividade limitada. O plano precisa definir tempo esperado de propagação, comportamento offline e limites de validade do conteúdo. Se a correção depende de todos os clientes receberem imediatamente um comando remoto, a estratégia possui uma fragilidade importante.

O rollback também não desfaz ações confirmadas. Uma jornada iniciada sob uma revisão pode continuar depois da troca. Identidade de operação, estado de domínio e confirmação permanecem válidos independentemente da aparência da tela. O cliente precisa retomar a situação atual sem repetir efeitos. Esse contrato deve ser testado em fluxos relevantes, especialmente quando uma nova composição altera o número ou a ordem de passos.

Eu começaria a implantação com comparação de manifestos, testes de matriz e coortes limitadas. A equipe ensaiaria uma reversão antes de precisar dela, verificando observabilidade e comunicação com suporte. O objetivo não é tornar cada publicação lenta, mas automatizar evidências que permitam publicar com confiança. Quando uma mudança exige uma nova capacidade do binário, ela segue esse ciclo explicitamente; BDC não transforma um componente ausente em algo que o dispositivo consegue executar.

ANÁLISE E DECISÃO

Interface declarativa não pode virar autorização remota



Um manifesto pode instruir apresentação e navegação, mas não deveria carregar autoridade para executar operações financeiras. A existência de um botão não prova permissão. O serviço de ação verifica identidade, contexto e regras do domínio, mesmo quando o conteúdo veio de uma plataforma confiável. Essa separação reduz o impacto de configuração incorreta e permite revisar capacidades de interface sem alterar implicitamente controles financeiros.

Também limitaria expressividade. Aceitar código arbitrário ou URLs não controladas aumenta a superfície de comportamento e dificulta validação. Um catálogo de ações conhecidas, schema restrito e limites de tamanho ajudam a manter a plataforma compreensível. A validação deve ocorrer no servidor e no cliente, pois distribuição e versões podem falhar. Conteúdo inválido precisa de uma resposta segura e legível, não de uma tentativa genérica de continuar a qualquer custo.

Acessibilidade pertence ao contrato, mas não se reduz a um campo obrigatório. Nome acessível vazio é uma falha detectável automaticamente; nome confuso, ordem de foco inadequada e interação inacessível exigem avaliação mais ampla. Eu combinaria regras automatizadas com testes de componentes e pesquisa em jornadas críticas. Transparência, animação e efeitos visuais precisam respeitar legibilidade e preferências do dispositivo. Uma plataforma avançada amplia qualidade repetível; ela não usa expressividade visual como justificativa para remover clareza ou autonomia.

ANÁLISE E DECISÃO

ADR 14 · Conteúdo imutável com seleção por capacidade






A decisão proposta é publicar revisões imutáveis e selecionar composição conforme protocolo, capacidades e coorte. O contexto é uma plataforma que permite alterar interface sem distribuir um binário para toda mudança. A intenção é manter rastreabilidade e compatibilidade enquanto squads ganham autonomia. O catálogo de ações permanece separado da autorização de domínio, e o fallback precisa ser validado como qualquer outra composição.

Uma alternativa é distribuir apenas interfaces estáticas no aplicativo. Ela reduz parte da variabilidade de conteúdo, mas mantém dependência de atualização para mudanças elegíveis. Outra é uma camada remota muito expressiva, próxima de execução arbitrária. Ela aumenta liberdade, porém amplia risco, combinações e custo de validação. O contrato declarativo restrito oferece um equilíbrio: acelera composições conhecidas e explicita quando uma nova capacidade exige evolução da plataforma ou do binário.

Eu reavaliaria o desenho se a matriz de compatibilidade crescer sem controle ou se os squads contornarem o catálogo por falta de componentes adequados. A resposta pode ser melhorar abstrações e depreciação, não apenas adicionar mais exceções. A arquitetura deve medir custo de adoção e qualidade entregue. Revisões imutáveis são um mecanismo; o resultado desejado é conseguir explicar o que cada cliente recebeu, por que era compatível e como interromper a mudança quando a evidência recomenda.

ANÁLISE E DECISÃO

Execução EM: plataforma que multiplica qualidade



Como Engineering Manager, organizaria a plataforma em torno das equipes consumidoras e das jornadas que elas entregam. Definiria contratos e ferramentas que tornem publicação correta mais simples do que caminhos paralelos. O trabalho incluiria documentação, diagnóstico, componentes acessíveis e apoio à adoção, além do runtime. A liderança precisa reservar capacidade para essas atividades, porque uma plataforma sem experiência de uso adequada perde adesão mesmo quando sua arquitetura é elegante.

O valor seria medido por tempo de entrega elegível, esforço de implementação e correção, qualidade e adoção sustentada. Não usaria porcentagem de telas migradas como objetivo isolado. A migração pode aumentar custo sem melhorar uma tarefa. O cálculo de capacidade recuperada descontaria manutenção da plataforma e suporte aos squads. Ganhos de conversão precisariam de experimento próprio, separado da economia de desenvolvimento, evitando contar o mesmo benefício duas vezes.

A pergunta Staff seria: como garantir que uma revisão compatível e visualmente correta não introduz uma ação inadequada quando dois experimentos se combinam? A resposta envolve composição, catálogo, autorização e observabilidade. A pergunta de liderança seria como priorizar uma melhoria transversal cujo benefício aparece em vários times, mas cujo custo fica na plataforma. Eu buscaria compromissos claros, pilotos com consumidores e resultados observáveis. Multiplicar qualidade exige tanto contratos técnicos quanto uma forma sustentável de decidir e aprender em conjunto.

ANÁLISE E PROPOSTA INDEPENDENTE

Disponibilidade percebida também é comportamento

15



Platform engineering, Observability, and scaling Mobile Development

Captura da publicação oficial consultada em 23/10/2026.

O Nubank descreveu, em abril de 2025, uma plataforma mobile com releases progressivos, automação e observabilidade. A publicação apresenta uma capacidade existente, não uma lacuna que este estudo pretende revelar. Minha pergunta adicional seria como uma decisão de expansão relaciona estabilidade técnica e tarefas concluídas. Uma sessão sem crash pode conter um botão inoperante, uma espera sem resposta ou uma mensagem que o cliente não compreendeu.

Eu escolheria uma única jornada, como consultar uma fatura, antes de discutir um gate para todo o aplicativo. Definiria início, conclusão e resultado desconhecido junto a Produto, Design e à equipe responsável. Um evento de sucesso não deveria significar apenas que uma tela abriu; precisaria representar a tarefa acordada. Também distinguiria cancelamento voluntário de falha e interrupção por perda de conectividade, evitando atribuir todo abandono ao release.

Este capítulo propõe um laboratório determinístico de decisão, com dados inventados e regras deliberadamente simples. Ele demonstra como uma regressão pode impedir a expansão mesmo quando o processo não caiu. Não estima desempenho do Nubank, não testa Flutter e não substitui análise estatística. Seu valor é tornar discutível o contrato: qual informação falta, quem decide diante dessa falta e quais sinais precisam permanecer estáveis antes de expor mais pessoas à mudança.

Platform engineering, Observability, and scaling Mobile Development

ANÁLISE E PROPOSTA INDEPENDENTE

A unidade é uma tentativa de jornada



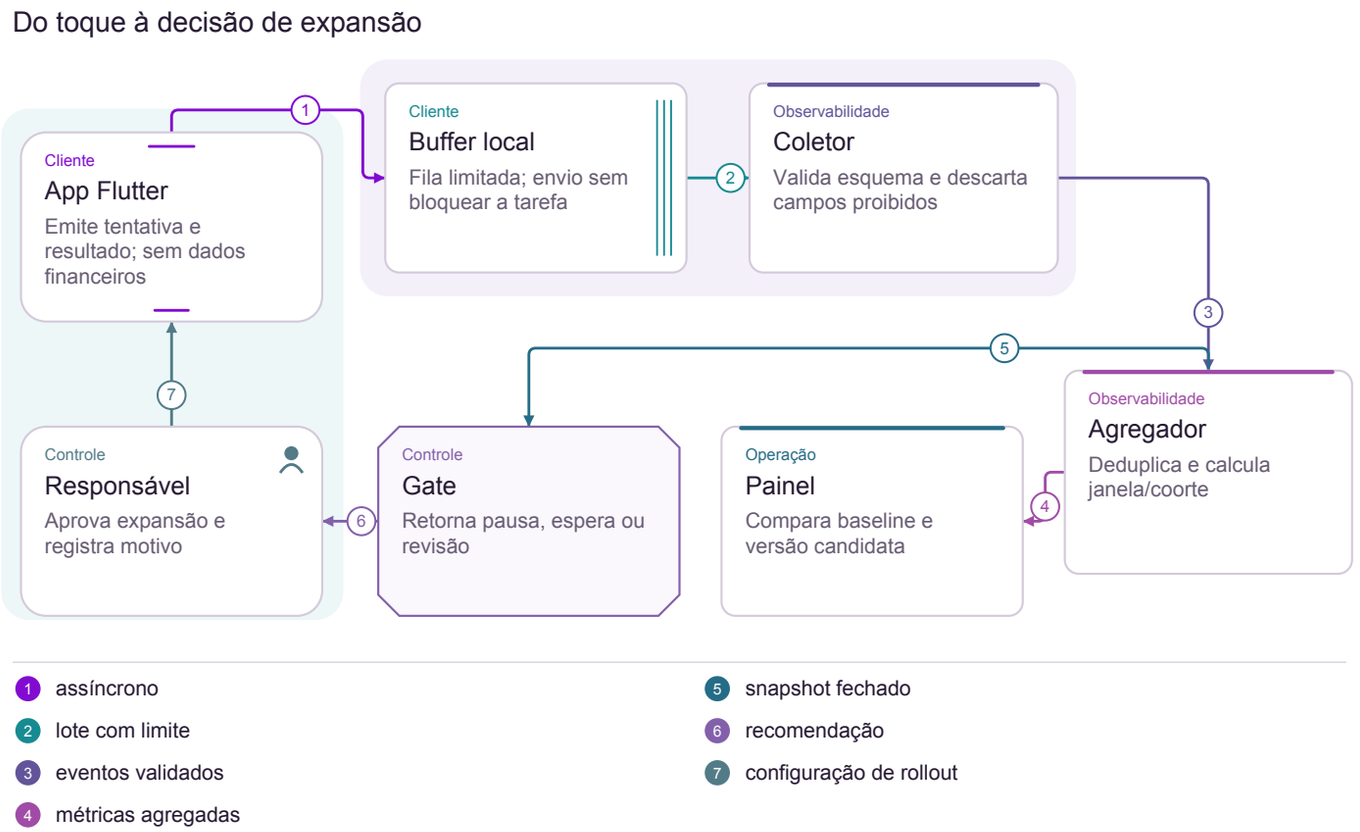
Uma tentativa começa quando a pessoa expressa a intenção definida para a tarefa. Um toque repetido durante carregamento não precisa criar outra tentativa; uma nova consulta depois de retornar à tela pode criar. Essa distinção depende do produto e deve ser registrada antes da coleta. Se cada time conta de forma diferente, uma queda aparente de conversão pode refletir apenas uma mudança de instrumentação e consumir dias de investigação.

O modelo local associa uma sessão sintética, uma coorte e uma versão a um resultado terminal. Essa simplificação permite demonstrar deduplicação, mas não representa identidade de cliente. Uma implementação real precisaria de identificador de tentativa independente, política de expiração e estados intermediários. A mesma pessoa pode realizar várias operações válidas na sessão; agregá-las indevidamente apagaria problemas e distorceria o denominador. Eventos tardios também exigiriam reconciliação dentro de uma janela explícita.

Eu manteria três estados separados: concluída, não concluída e observação incompleta. O terceiro não vira sucesso nem fracasso automaticamente. Ele informa a qualidade da medição e pode suspender a decisão de expansão. As coortes seriam comparáveis em jornada, dispositivo e conexão, com limites de cardinalidade definidos. O objetivo não é catalogar pessoas, mas evitar que diferenças no público exposto sejam confundidas com efeito da versão que estamos avaliando.

ARQUITETURA DE REFERÊNCIA

Do toque à decisão de expansão



Proposta externa. Flutter e observabilidade são contexto público; nomes e fronteiras deste desenho são autorais.

A fronteira principal separa a operação financeira da coleta de observabilidade. Um coletor indisponível não pode impedir a consulta ou o pagamento; o buffer precisa ter tamanho e descarte previsíveis. O caminho de decisão opera sobre um snapshot agregado, identificado por versão e intervalo, para que uma aprovação não mude de significado enquanto alguém a revisa. O gate recomenda, e o responsável decide dentro do processo estabelecido. Esse desenho evita atribuir autoridade de lançamento a um gráfico isolado. O laboratório implementa somente agregação e decisão, sem rede, armazenamento persistente ou controle remoto do app. Em uma integração real, eu avaliaria se os componentes já existem e acrescentaria apenas o contrato ausente. A confirmação de expansão deveria registrar qual população foi exposta, qual baseline foi usado e qual condição obriga pausa. Não existe uma conexão direta entre receber telemetria e autorizar uma ação financeira do cliente.

CÓDIGO · CONTRATO PROPOSTO

Um gate que sabe dizer “ainda não”

PYTHON / PROPOSTA AUTORAL

```python
def gate(baseline, candidate, telemetry_complete=True, min_n=50):
    if type(min_n) is not int or min_n <= 0:
        raise ValueError("min_n must be positive integer")
    if type(telemetry_complete) is not bool:
        raise ValueError("telemetry_complete must be boolean")
    validate_stats(baseline)
    validate_stats(candidate)
    if not telemetry_complete:
        return "HOLD_TELEMETRY"
    if baseline["n"] < min_n or candidate["n"] < min_n:
        return "HOLD_SAMPLE"
    if candidate["rate"] < baseline["rate"] - 0.03:
        return "PAUSE_SUCCESS"
    if candidate["p95"] > baseline["p95"] * 1.25:
        return "PAUSE_LATENCY"
    return "ELIGIBLE_FOR_REVIEW"

baseline = {"n": 100, "rate": 0.98, "p95": 300}
candidate = {"n": 100, "rate": 0.90, "p95": 300}
assert gate(baseline, candidate) == "PAUSE_SUCCESS"
assert gate(baseline, baseline) == "ELIGIBLE_FOR_REVIEW"
assert gate(baseline, baseline, False) == "HOLD_TELEMETRY"
small = {"n": 10, "rate": 1.0, "p95": 100}
assert gate(baseline, small) == "HOLD_SAMPLE"
```

Trecho de case_15.py; execução Python verificada. Regra ilustrativa, sem inferência estatística. Tipos, imports e validadores estão no arquivo completo.

A função executada no laboratório produz uma decisão explicável. Telemetria incompleta e amostra pequena retornam espera; regressões de sucesso e latência retornam pausa. Os valores de 50 observações, três pontos percentuais e 25% são hipóteses sintéticas para testar o mecanismo, não limites recomendados para pagamentos. O resultado favorável ainda solicita revisão: ele não representa significância estatística, aprovação de risco ou autorização automática. Em produção, os parâmetros precisariam ser versionados e calibrados pela variabilidade da jornada, pela tolerância ao dano e pelo desenho do experimento. Uma alteração do limiar também é mudança de política e merece registro. Tipos e domínios inválidos geram ValueError antes de qualquer resultado favorável.

Python 3.12: unittest

CÓDIGO · CONTRATO PROPOSTO

Instrumentação desacoplada em Dart

DART / PROPOSTA AUTORAL

```dart
enum JourneyOutcome { completed, failed, unknown }

class JourneySignal {
  final String attemptId;
  final String release;
  final JourneyOutcome outcome;
  final int elapsedMs;
  const JourneySignal(this.attemptId, this.release,
      this.outcome, this.elapsedMs);
}

abstract class BoundedTelemetry {
  bool offer(JourneySignal signal);
}

Future<T> observed<T>(
  Future<T> Function() task,
  BoundedTelemetry telemetry,
  String attemptId,
  String release,
) async {
  final watch = Stopwatch()..start();
  try {
    final result = await task();
    telemetry.offer(JourneySignal(attemptId, release,
        JourneyOutcome.completed, watch.elapsedMilliseconds));
    return result;
  } catch (_) {
    telemetry.offer(JourneySignal(attemptId, release,
        JourneyOutcome.unknown, watch.elapsedMilliseconds));
    rethrow;
  }
}
```

Dart ilustrativo, não executado. Interfaces e nomes propostos; não são SDK interno do Nubank.

Este fragmento Dart é uma proposta de interface e não foi compilado. BoundedTelemetry teria a obrigação de não lançar exceções e de retornar imediatamente quando sua fila estiver cheia; essa obrigação precisa ser testada na implementação. O wrapper mede duração local, mas não sabe se um servidor concluiu uma operação cuja resposta se perdeu. Por isso, o erro é inicialmente desconhecido, e não pagamento recusado. A confirmação de conclusão também deve ser adaptada ao significado real de task, não apenas ao retorno de uma requisição. Dados financeiros, tokens e descrições livres ficam fora do contrato proposto.

Flutter: testing

ANÁLISE E PROPOSTA INDEPENDENTE

Persistir a decisão, não apenas o gráfico



Um painel é uma visualização mutável. O conjunto de eventos pode crescer, a regra pode mudar e o baseline pode ser recalculado. Se uma expansão depende desses elementos, eu registraria uma decisão imutável contendo versão do release, identificador da regra, intervalo de observação, população e resultado. Esse registro torna possível entender por que uma decisão parecia razoável naquele momento, inclusive quando dados tardios revelam algo diferente depois.

Eventos brutos não precisam ser mantidos indefinidamente para sustentar essa rastreabilidade. É possível preservar agregados suficientes e metadados de qualidade, conforme política de retenção. Uma correção produziria outro snapshot vinculado ao anterior, sem reescrever silenciosamente a aprovação histórica. O dono da decisão deveria conseguir distinguir uma mudança na realidade de uma mudança no cálculo. Essa distinção também ajuda a investigar alertas inconsistentes entre equipes e a identificar problemas da própria instrumentação.

No laboratório, duas observações idênticas da mesma chave contam uma vez, enquanto resultados conflitantes provocam erro explícito. Essa política é conservadora e local: em sistemas reais, eventos de evolução de estado são legítimos e precisam de número de sequência ou regra de precedência. Eu não transportaria o dicionário em memória para produção. Usaria o armazenamento e a semântica já disponíveis, preservando o requisito: conflito conhecido não pode virar estatística aparentemente confiável.

ANÁLISE E PROPOSTA INDEPENDENTE

Quando o experimento mede a coisa errada



Imagine que a versão candidata passe a emitir dois eventos de conclusão para a mesma tentativa. O gráfico pode mostrar melhora sem que nenhum cliente tenha concluído uma tarefa adicional. A deduplicação reduz esse risco, mas depende de identidade estável. Trocar o identificador no meio da jornada ou reutilizá-lo entre operações produz erros opostos. Eu trataria mudanças de instrumentação como parte do contrato de release, revisadas junto ao comportamento visível.

Outro caso é a conexão ruim. Um cliente pode concluir a tarefa e enviar a telemetria muito depois, enquanto o baseline transmite imediatamente. Comparar janelas abertas criaria uma regressão artificial. Precisamos de uma janela de fechamento, uma política para eventos tardios e um indicador de cobertura. Se uma falha do coletor afetar preferencialmente a versão candidata, a ausência de dados não pode ser lida como ausência de problemas. O gate local mostra essa recusa em decidir.

Também há falhas fora da aplicação: backend lento, mudança de regra, campanha ou feriado. Eu evitaria atribuição causal com um simples antes e depois. A comparação precisaria considerar exposição, período e coorte, com participação de quem desenha experimentos. Se a versão piora uma jornada essencial, contudo, não é necessário esperar uma narrativa estatística perfeita para conter dano plausível. A política deve distinguir decisão conservadora de proteção e comprovação posterior de benefício.

ANÁLISE E PROPOSTA INDEPENDENTE

Dezesseis testes, três tipos de limite



O arquivo case_15.py executou dezesseis testes locais com sucesso. As verificações cobrem comportamento saudável, regressões de conclusão e latência, falta de telemetria, amostra pequena, coleção vazia, duplicidade, conflito, duração negativa, cálculo do percentil, fronteira exata dos limiares e separação entre versões. O resultado descreve funções sobre estruturas em memória; não mede bateria, tempo de renderização, qualidade de conexão ou estabilidade de um aplicativo Flutter real. A revisão acrescentou rejeição de NaN, infinito, tipos inválidos e política inconsistente; amostra vazia permanece em espera.

Na simulação principal, o baseline contém cem observações, 98% de conclusão e P95 de 300 milissegundos. Uma candidata com 90% de conclusão recebe PAUSE_SUCCESS. Esses números foram escolhidos para tornar o ramo da regra evidente e não devem aparecer como resultado de negócio. O caso saudável retorna ELIGIBLE_FOR_REVIEW, preservando a diferença entre um teste passar e uma decisão de lançamento ter justificativa suficiente. O arquivo de resultados acompanha o código para inspeção.

A próxima camada de validação precisaria de testes de widget e integração, cenários de background e foreground, redes degradadas e dispositivos representativos. Eu também ensaiaria perda de eventos e recuperação do coletor. Por fim, validaria a interpretação dos estados com pessoas, pois um teste técnico pode provar que uma mensagem apareceu sem mostrar se alguém entendeu o que fazer. O avanço entre camadas dependeria do risco da jornada, não da quantidade de testes isolada.

Python 3.12: unittest
Flutter: testing

ANÁLISE E PROPOSTA INDEPENDENTE

Observabilidade que explica uma decisão



Eu começaria com poucas métricas que tenham um uso operacional definido: conclusão por tentativa elegível, tempo percebido, observação incompleta e exposição acumulada. Cada métrica precisa informar qual decisão pode alterar. Uma taxa agregada para todo o app pode esconder regressão em um dispositivo ou em uma conexão específica; granularidade ilimitada, por outro lado, gera ruído, custo e risco de identificação. A escolha das dimensões seria parte da revisão de privacidade e operação.

Traces podem conectar etapas técnicas, mas não substituem o evento de negócio da jornada. Eu associaria o resultado agregado a referências de diagnóstico com acesso controlado, sem colocar dados financeiros em nomes de span ou atributos livres. A amostragem para troubleshooting pode ser diferente da coleta usada no denominador do experimento. Misturar essas bases sem registrar a diferença produziria métricas bonitas e decisões frágeis. A disponibilidade da telemetria também merece medição própria.

Um alerta útil diria qual versão, qual coorte, qual jornada e qual regra mudou de estado, além de indicar um responsável e um procedimento. Não enviaria a mesma notificação a todas as squads. Acompanharia tempo para reconhecer o sinal, conter exposição e confirmar recuperação. Se o alerta não muda uma ação ou não encontra um dono, é preciso ajustar seu desenho. O sucesso da observabilidade aparece no tempo e na qualidade da resposta, e não apenas no volume de dados coletados.

OpenTelemetry: traces

ANÁLISE E PROPOSTA INDEPENDENTE

Rollout como compromisso reversível



Eu iniciaria a avaliação em uma jornada de risco delimitado, sem ligar o gate a expansão automática. Durante a primeira etapa, a regra observaria decisões humanas já existentes e produziria divergências para revisão. Isso revelaria se os limiares estão mal calibrados, se os dados chegam tarde ou se as equipes usam definições diferentes de conclusão. A fase de observação também precisa de prazo: permanecer indefinidamente em shadow produz custo sem necessariamente produzir aprendizado.

Depois, escolheríamos uma pequena população de exposição com critérios definidos antes do release. A janela seguinte só começaria após avaliar a anterior, com responsabilidade explícita por pausar e comunicar. Reverter uma configuração de interface não equivale a desfazer dados ou operações já concluídas. O plano de contenção precisa dizer o que realmente volta, o que continua em versões instaladas e quais efeitos precisam de reconciliação. Uma atualização distribuída pela loja não pode ser retirada de todos os dispositivos instantaneamente.

Como EM, protegeria tempo da equipe para responder às primeiras coortes e alinharia parceiros sobre disponibilidade. Como Staff, revisaria a compatibilidade entre versões e a separação de configuração, protocolo e autorização. Eu não usaria ausência de reclamação como sinal suficiente de segurança. A liberação final exigiria qualidade de medição, resultado aceitável e capacidade de sustentar a operação. Quando essas condições não estiverem presentes, manter a exposição limitada pode ser a decisão mais responsável.

ANÁLISE E PROPOSTA INDEPENDENTE

Custo de observar sem custo para pagar



Uma coleta detalhada demais pode consumir rede, bateria e CPU em dispositivos que já operam sob restrição. Por isso, eu definiria limites de eventos por tentativa, bytes por lote, tempo de retenção local e frequência de envio. A fila não poderia crescer sem controle durante um incidente. Ao atingir o limite, descartaria dados segundo política conhecida e registraria perda agregada, preservando o funcionamento da jornada. O comportamento sob pressão precisa ser tão especificado quanto o caminho saudável.

No servidor, nomes de versão, jornadas e classes de dispositivo são dimensões controladas; identificadores pessoais não devem virar labels de métricas. Acesso a dados de diagnóstico seria separado do acesso ao painel de rollout. Uma pessoa autorizada a acompanhar qualidade não ganha automaticamente acesso ao conteúdo de uma transação. O desenho também deveria permitir revogar ou reduzir coleta quando surgir um risco de privacidade, sem exigir uma atualização emergencial para toda a base instalada.

A proteção da decisão pede controles adicionais. Configuração assinada, registro de alteração e limitação de escopo impedem que um erro operacional exponha uma versão a uma população maior que a aprovada. Nenhum desses controles é implementado pelo laboratório. Eles delimitam o trabalho que falta antes de uma integração real. Eu estimaria custo de coleta por jornada elegível e compararia com utilidade do diagnóstico, removendo atributos que ninguém consegue justificar com uma pergunta concreta.

ANÁLISE E PROPOSTA INDEPENDENTE

ADR: métrica de jornada como gate adicional







Decisão proposta: complementar o controle de release com uma avaliação explícita de jornada, mantendo a autoridade de expansão no processo da equipe. A alternativa de olhar apenas crashes é barata e útil, mas não captura falhas silenciosas de tarefa. A alternativa de automatizar toda a liberação reduz tempo de decisão, porém exige confiança muito maior na medição e na política. Para o primeiro piloto, eu escolheria recomendação explicável com revisão humana e prazo de avaliação.

O custo dessa decisão é aumentar o trabalho de definir eventos, manter contratos e investigar dados incompletos. Também há risco de bloquear releases por ruído, atrasando correções importantes. Por isso, cada pausa precisaria de motivo e caminho de resolução. A política não deveria incentivar times a reduzir a coleta de erros para melhorar a aparência do resultado. Métricas de cobertura e revisão independente da instrumentação ajudam a evitar essa distorção e precisam estar incluídas no escopo.

Eu reconsideraria a decisão se a jornada não tivesse volume suficiente, se o efeito fosse dominado por fatores externos ou se o custo de instrumentar superasse o benefício plausível. Nesse caso, pesquisa moderada, testes de integração e controles existentes poderiam responder melhor à pergunta. Um ADR útil registra essas condições de revisão. Ele não transforma a solução escolhida em padrão universal; preserva o motivo pelo qual uma equipe decidiu experimentar esse mecanismo naquele domínio e naquele momento.

ANÁLISE E PROPOSTA INDEPENDENTE

Entrega de equipe, valor e conversa Staff



O primeiro incremento poderia ser uma jornada com contrato de eventos, baseline validado, painel e gate em modo de observação. Eu distribuiria o trabalho entre mobile, backend e observabilidade, com parceria de Produto para definir conclusão. A liderança precisaria negociar capacidade para correções rápidas e impedir que o piloto virasse mais uma responsabilidade sem dono. O aceite incluiria simulação de telemetria ausente e um exercício de pausa, não apenas um dashboard funcionando em uma demonstração.

O modelo de valor começaria por exposições a falhas efetivamente evitadas e contatos de suporte associados, sem presumir que todo abandono teria gerado atendimento. Horas de investigação poupadas representam capacidade potencial; economia de caixa exige mudança verificável de despesa. Custos de instrumentação, armazenamento e manutenção entram no cálculo. Eu manteria esses resultados separados de ganho de conversão, que depende de outro desenho causal e pode se sobrepor ao mesmo grupo de clientes.

Minha pergunta de Staff seria: qual garantia estamos usando para promover uma versão e como percebemos que essa garantia deixou de ser observável? Minha pergunta de EM seria: quem pode interromper a expansão e tem tempo para agir quando isso acontece? As duas se encontram na mesma responsabilidade. Melhorar o gate só vale a pena se o time consegue interpretar o resultado e proteger a experiência de quem recebe a próxima versão.

ANÁLISE E PROPOSTA INDEPENDENTE

Roteamento é uma hipótese sobre a necessidade

16



Presenting Precog, Nubank’s Real Time Event AI

Captura da publicação oficial consultada em 23/10/2026.

Em junho de 2023, o Nubank apresentou o Precog, que usa representações de eventos para apoiar modelos de intenção. O artigo relata aumento superior a 50% no volume de chamadas corretamente direcionadas a especialistas sem entrada do cliente, comparado ao modelo anterior. Esse resultado pertence à implementação e ao período publicados. Não significa ganho de cinquenta pontos percentuais, não mede automaticamente resolução e não é uma meta que este laboratório possa prometer reproduzir.

A oportunidade de estudo está na continuidade dessa qualidade quando produtos, eventos e motivos de contato mudam. Uma previsão pode ser tecnicamente confiante e estar errada sobre o que a pessoa precisa agora. Também pode acertar o produto e encaminhar para uma fila indisponível. Eu avaliaria o roteamento como decisão operacional com possibilidade de correção, e não como uma classificação que encerra a compreensão do cliente. O suporte precisa conseguir contestar a sugestão e preservar contexto útil.

O laboratório deste capítulo usa pesos sintéticos em eventos, sem embeddings e sem treinamento. Ele testa janelas temporais, deduplicação, suporte mínimo, abstenção e prioridade declarada. A intenção é tornar essas fronteiras verificáveis antes de discutir modelos melhores. Uma distribuição de pesos não é probabilidade calibrada. O código demonstra política, não qualidade preditiva, e não oferece autorização para bloquear conta, transferir recursos ou tomar qualquer decisão financeira em nome de alguém.

Presenting Precog, Nubank’s Real Time Event AI

ANÁLISE E PROPOSTA INDEPENDENTE

Intenção, urgência e disponibilidade são coisas diferentes



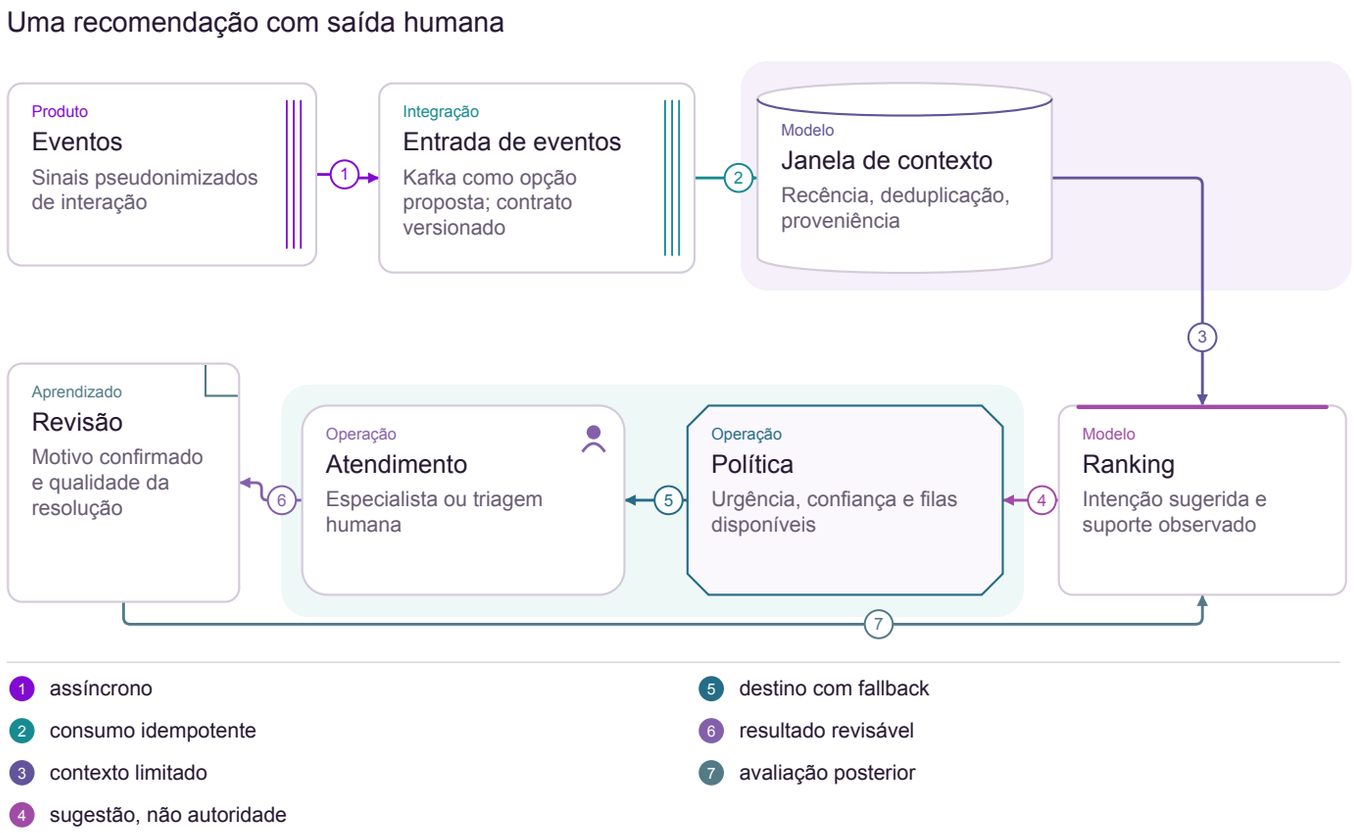
O domínio precisa distinguir o motivo provável do contato, a urgência declarada e a capacidade de atendimento. Uma pessoa que acabou de tentar pagar pode ligar por outra razão. Um sinal de risco pode exigir triagem especializada mesmo quando o produto provável é cartão. E uma fila com bons especialistas pode não estar disponível naquele horário. Combinar tudo em um único score dificulta explicar o resultado e esconderia decisões que pertencem a políticas diferentes.

Eu modelaria a recomendação como um registro contendo intenção sugerida, evidência utilizada, versão da política, motivo de abstenção e destino operacional. A pessoa e o atendente precisam de um caminho simples para corrigir o motivo. Essa correção não deve ser automaticamente tratada como rótulo perfeito: o cliente pode apresentar vários assuntos, o atendente pode escolher uma categoria conveniente e a resolução pode ocorrer em outra equipe. A qualidade da taxonomia exige trabalho conjunto com Operações.

No laboratório, uma urgência declarada leva a uma fila humana dedicada, sem depender do ranking. Eventos insuficientes ou intenção sem fila disponível levam à triagem geral. Essas decisões são escolhas didáticas, não políticas do Nubank. Uma implementação real exigiria validar capacidade, horários, idiomas e necessidades de acessibilidade. O contrato importante é separar a decisão de reconhecer um padrão da decisão de como prestar ajuda; nenhuma delas deveria retirar a possibilidade de explicar o problema.

ARQUITETURA DE REFERÊNCIA

Uma recomendação com saída humana



Arquitetura autoral; nomes de filas e fronteiras propostos. Não reproduz sistemas internos.

O artigo público descreve consumo de eventos e serviços Clojure/Python, mas o uso de Kafka neste desenho é uma escolha de referência, não uma afirmação sobre o Precog. A fronteira entre ranking e política evita que disponibilidade de uma fila seja confundida com confiança do modelo. O destino escolhido precisa carregar um motivo legível para Operações e permitir correção sem recomeçar a história. Resultados voltam para avaliação de forma assíncrona, depois de considerar o desfecho do contato. Um rótulo corrigido não reescreve a decisão original: registra o que era conhecido e o que foi aprendido. O laboratório implementa janela, ranking por pesos e política em memória. Não inclui embeddings, telefonia ou fila real. Na produção, eu preferiria integrar capacidades existentes e medir o custo adicional de uma recomendação antes de introduzir um novo serviço no caminho de atendimento.

CÓDIGO · CONTRATO PROPOSTO

Abster-se também é uma decisão válida

PYTHON / PROPOSTA AUTORAL

```python
def route(scores, available, urgent=False, threshold=0.75, min_weight=4):
    if type(urgent) is not bool:
        raise ValueError("urgent must be boolean")
    if urgent:
        return {"queue": "human_urgent", "reason": "declared_urgency"}
    validate_ranking(scores, threshold, min_weight)
    total = sum(scores.values())
    if total < min_weight:
        return {"queue": "human_general", "reason": "insufficient"}
    ranked = sorted(scores.items(), key=lambda x: (-x[1], x[0]))
    intent, weight = ranked[0]
    confidence = weight / total
    if confidence < threshold or intent not in available:
        return {"queue": "human_general", "reason": "abstention"}
    return {"queue": intent, "reason": "ranked", "share": confidence}

clear = route({"card": 8, "pix": 1}, {"card"})
uncertain = route({"card": 5, "pix": 5}, {"card"})
assert clear["queue"] == "card"
assert uncertain["queue"] == "human_general"
assert route({}, set())["reason"] == "insufficient"
assert route({}, set(), True)["queue"] == "human_urgent"
```

Trecho executado de case_16.py. Ranking por pesos inventados; não implementa Precog. Tipos, imports e validadores estão no arquivo completo.

A função aplica urgência antes do ranking, exige suporte mínimo e evita uma fila inexistente. O campo share representa participação do peso da intenção vencedora, não probabilidade de acerto. O limiar de 0,75 é sintético. Uma produção baseada em ML precisaria calibrar confiança por intenção e acompanhar a qualidade das abstenções, sem copiar esses números. O fallback humano é uma saída funcional, não um erro escondido. A regra também não administra capacidade: o conjunto available é uma entrada já resolvida. Se ele estiver desatualizado, o destino precisa recusar com um caminho de recuperação definido. Tipos e domínios inválidos geram ValueError antes de qualquer resultado favorável.

Python 3.12: unittest

CÓDIGO · CONTRATO PROPOSTO

Política pura em Clojure, efeito em outra fronteira

CLOJURE / PROPOSTA AUTORAL

```clojure
(ns independent.support-policy)

(defn choose-route
  [{:keys [urgent? available candidates min-support]}]
  (let [ordered (sort-by (juxt (comp - :weight) :intent)
                         candidates)
        total (reduce + (map :weight ordered))
        best (first ordered)
        share (if (pos? total)
                (/ (:weight best 0) total)
                0)]
    (cond
      urgent?
      {:queue :human-urgent :reason :declared-urgency}

      (< total min-support)
      {:queue :human-general :reason :insufficient}

      (or (< share 3/4)
          (not (contains? available (:intent best))))
      {:queue :human-general :reason :abstention}

      :else
      {:queue (:intent best)
       :reason :ranked
       :share share})))

;; O adaptador persiste e encaminha fora desta função.
;; Contrato proposto: candidate weights >= 0.
;; Empates recebem ordem determinística, não mais confiança.
```

Clojure ilustrativo não executado; namespaces, filas e política são propostas externas.

A função proposta recebe dados imutáveis e retorna uma recomendação, sem chamar telefonia, registrar efeitos ou atualizar estado compartilhado. Isso facilita testar a política separadamente da integração. Clojure é parte do contexto técnico publicado pelo Nubank; este trecho é independente e não foi executado. A validação de pesos e a leitura consistente de disponibilidade pertencem ao adaptador. Se um atom for usado em um protótipo, isso não fornece persistência nem transação com sistemas externos. Funções passadas a mecanismos que podem repetir avaliação devem continuar sem efeitos. O encaminhamento precisa de identidade própria e reconciliação fora do cálculo.

Clojure: atoms

ANÁLISE E PROPOSTA INDEPENDENTE

Tempo do evento e tempo de conhecimento



Um evento pode ocorrer antes do contato e chegar depois. Se reconstruirmos a decisão usando todos os eventos que hoje conhecemos, criaremos uma avaliação artificialmente favorável. Eu registraria separadamente quando o fato ocorreu e quando estava disponível para o roteador. O replay precisa respeitar ambos. Essa fronteira é especialmente importante em redes móveis, onde sinais podem ficar retidos no dispositivo e ser transmitidos em lote depois que a pessoa recupera conexão.

A janela local descarta eventos futuros, eventos fora do período e recebimentos antigos, além de rejeitar duplicatas conflitantes. Os limites de 180 e 30 unidades são parâmetros sintéticos, não minutos recomendados para o Nubank. O experimento precisa declarar a unidade e o relógio utilizado. Um relógio de cliente impreciso pode exigir validação adicional ou uso de outra referência; não basta comparar números como se todas as máquinas compartilhassem o mesmo tempo exato.

Uma chave de evento estável ajuda a deduplicar tentativas de entrega, mas não identifica duas interações diferentes do mesmo tipo. Eu evitaria descartar eventos apenas porque têm atributos semelhantes. Na integração proposta com Kafka, commit de offset não confirma encaminhamento no sistema de atendimento. Esses efeitos pertencem a fronteiras distintas. O projeto precisaria de registro durável da decisão e reconciliação para saber se uma tentativa de encaminhamento desconhecida pode ser repetida sem transferir o cliente novamente.

Apache Kafka: design

ANÁLISE E PROPOSTA INDEPENDENTE

O erro mais caro pode ser uma fila plausível



Um modelo pode encaminhar corretamente a maioria dos contatos e falhar justamente nos temas mais urgentes. A média agregada esconderia esse padrão. Eu estratificaria avaliação por intenção, urgência, produto novo e casos sem histórico, sempre com tamanho de amostra e incerteza visíveis. Também observaria quando um especialista corrige o destino, quanto tempo essa correção consome e se a pessoa precisa repetir informações sensíveis. A qualidade não termina no acerto inicial da fila.

Uma mudança de nomenclatura de eventos pode produzir perda gradual de contexto sem derrubar o serviço. Outra falha ocorre quando a intenção prevista deixa de ter equipe disponível. O contrato precisa prever uma saída geral e preservar o motivo sugerido como hipótese. Não é aceitável manter a pessoa em um circuito de transferências porque cada componente considera sua própria decisão correta. O sistema deve reconhecer repetição de encaminhamento e escalar para uma resolução humana com responsabilidade clara.

Eu testaria também eventos manipulados ou repetidos, mas sem transformar o capítulo em instrução sobre controles internos. O princípio é simples: telemetria comportamental é evidência limitada, nunca credencial de acesso. Uma pessoa não adquire autorização financeira por visitar certas telas. Mudanças em destino, prioridade ou contexto precisam respeitar políticas de atendimento e privacidade. O laboratório demonstra abstenção e deduplicação; testes de abuso, telefonia e operações reais continuam fora de seu alcance.

ANÁLISE E PROPOSTA INDEPENDENTE

Quinze verificações de política, nenhuma promessa de acurácia



Os quinze testes de case_16.py passaram localmente. O conjunto verifica intenção dominante, ambiguidade, ausência de evidência, urgência, fila indisponível, duplicidade, conflito, eventos futuros, ocorrência antiga recebida agora, recebimento antigo, peso negativo e suporte insuficiente. Cada teste faz uma afirmação sobre política e janela, não sobre capacidade de entender a intenção humana. O resultado não valida embeddings, não mede atendimento e não comprova que o limiar escolhido é apropriado para qualquer produto. As regressões adicionais rejeitam pesos não finitos, fracionários e booleanos, além de política e relógios inválidos.

Na demonstração, pesos oito para cartão e um para Pix geram encaminhamento para cartão, com participação aproximada de 0,889. Pesos iguais geram abstenção e triagem geral. Esses exemplos mostram por que uma decisão pode ser reproduzida, mas também expõem o limite do método. Pesos inventados podem produzir uma recomendação convincente e completamente errada sobre uma pessoa. A leitura correta do resultado é que o mecanismo respeitou o contrato sintético, não que alcançou determinada qualidade preditiva.

Uma avaliação real exigiria rótulos revisados, conjunto temporal separado, tratamento de múltiplas intenções e análise de contatos repetidos. Eu incluiria especialistas do atendimento na discussão dos erros, em vez de usar apenas uma matriz de confusão. Precisaríamos medir resolução e recontato, pois um roteamento aparentemente rápido pode apenas deslocar o trabalho para depois. A decisão de avançar dependeria desses resultados e da capacidade do suporte para absorver os caminhos de fallback.

Python 3.12: unittest

ANÁLISE E PROPOSTA INDEPENDENTE

Métricas que chegam até a resolução



A primeira métrica seria roteamento correto segundo uma definição acordada e revisável. Ela precisa especificar se o destino estava apto a resolver o assunto, se houve transferência e se múltiplos temas tornam a classificação ambígua. Em seguida, eu acompanharia tempo até atendimento útil, recontato e resolução no primeiro contato, sem concluir causalidade apenas porque essas medidas se movem juntas. Mudanças de equipe, escala ou política podem alterar o resultado independentemente do modelo.

A abstenção merece um painel próprio. Taxa baixa não é necessariamente melhor: pode significar confiança excessiva. Taxa alta pode ser uma escolha responsável para casos novos, mas também pode revelar eventos ausentes. Eu mostraria cobertura de evidência, recência e motivos de fallback por intenção, preservando grupos suficientemente agregados. O painel deve permitir distinguir problema de coleta, de modelo, de política e de disponibilidade operacional. Sem essa separação, todas as investigações acabam na mesma fila de dados.

Cada alerta precisaria apontar uma ação. Perda de recência pode orientar investigação do pipeline; aumento de transferências em uma intenção pode suspender sua automação; fila indisponível exige decisão de Operações. O dono do modelo não resolve sozinho a terceira situação. Eu registraria versões de modelo e política em cada decisão para comparar mudanças com contexto. Logs não devem conter transcrição livre ou identificadores desnecessários para responder à pergunta operacional que motivou sua coleta.

ANÁLISE E PROPOSTA INDEPENDENTE

Shadow com desfecho, depois exposição limitada



A primeira avaliação poderia executar a recomendação sem alterar o encaminhamento. Para cada contato elegível, compararíamos a sugestão com o destino e o desfecho observados, reconhecendo que o fluxo existente também pode errar. O desenho de revisão deve incluir contatos fáceis e difíceis, não apenas casos com conclusão clara. Uma amostra que exclui abandonos, recontatos ou categorias desconhecidas pode produzir uma avaliação otimista justamente onde o mecanismo mais precisa de supervisão.

Depois, um piloto controlado poderia automatizar apenas intenções com evidência suficiente, mantendo saída humana e correção imediata. A população, os limites e as condições de pausa seriam definidos antes da exposição. Eu não começaria por temas urgentes nem por cenários em que uma transferência errada possa atrasar proteção necessária. Esses caminhos precisam de políticas específicas, construídas com os responsáveis, e não de uma exceção improvisada depois do primeiro incidente relevante.

Como EM, alinharia capacidade de atendimento, treinamento e disponibilidade para revisar erros nas primeiras semanas. Como Staff, garantiria replay da decisão, versões dos contratos e compatibilidade entre eventos. O aprendizado do piloto deveria produzir uma escolha: ampliar a intenção, mudar o limiar, melhorar contexto ou retornar à triagem. Não trataria cobertura crescente como objetivo isolado. O benefício depende de encaminhar melhor e resolver com menos esforço, preservando o direito de a pessoa explicar uma necessidade que não cabe na previsão.

ANÁLISE E PROPOSTA INDEPENDENTE

Privacidade e separação de autoridade



O contexto de atendimento pode revelar hábitos e dificuldades financeiras. Eu aplicaria minimização desde a definição dos sinais, evitando levar descrições livres ou conteúdo de transações para um pipeline que só precisa reconhecer categorias de interação. Pseudonimizar identificadores reduz exposição, mas não torna o conjunto anônimo automaticamente. Acesso, retenção e finalidade precisam permanecer explícitos, com participação dos responsáveis por privacidade e segurança. A avaliação de benefício não elimina essas obrigações de desenho.

A recomendação de fila não deve modificar limites, liberar operações ou funcionar como prova de identidade. Essa separação aparece no código e precisa existir também nas permissões de serviço. O componente que classifica uma intenção pode ter acesso a sinais limitados, enquanto a execução de uma ação financeira exige outro contrato e outra autoridade. Aproximar os dois por conveniência criaria um caminho em que uma inferência comportamental ganha poder que nunca deveria possuir.

Desempenho também entra nessa fronteira. Se o ranking ultrapassar seu orçamento de tempo, o atendimento precisa continuar por uma rota conhecida. A fila geral não pode depender do mesmo recurso que acabou de falhar. Eu estimaria custo de inferência por contato elegível e monitoraria armazenamento de contexto, sem escolher retenção apenas para facilitar experimentação. Um modelo sofisticado que atrasa atendimento ou amplia coleta sem benefício comprovado pode ser inferior a uma política simples bem operada.

ANÁLISE E PROPOSTA INDEPENDENTE

ADR: automatizar com abstenção explícita



A decisão proposta é separar cálculo de intenção, política de encaminhamento e execução operacional, oferecendo abstenção como resultado normal. Uma regra fixa por última tela é simples, mas pode confundir navegação recente com necessidade atual. Um modelo mais expressivo pode ampliar cobertura, porém exige avaliação, manutenção e interpretação de erros. O laboratório escolhe uma regra pequena para tornar fronteiras examináveis; não recomenda substituir o Precog nem apresenta uma alternativa superior ao trabalho publicado.

A vantagem da separação é poder mudar uma política de fila sem retreinar o modelo e avaliar um modelo novo sem alterar a integração de atendimento. O custo é manter contratos e versões entre componentes, além de evitar inconsistência quando uma fila muda de nome ou fica indisponível. A decisão persistida deveria registrar qual catálogo de destinos estava vigente. Se essa complexidade não resolver um problema observado, o mecanismo pode permanecer dentro de um serviço existente, em vez de virar três novos serviços.

Eu revisaria o ADR se as correções humanas indicarem taxonomia inadequada, se a cobertura útil for pequena ou se o custo de manter contexto ultrapassar o ganho. Também consideraria uma pergunta curta ao cliente como alternativa legítima: em alguns casos, perguntar é melhor do que inferir. A escolha deve comparar esforço total e qualidade da resolução, e não apenas quantos contatos conseguem evitar uma interação inicial. A pessoa que busca ajuda continua sendo a fonte principal sobre sua própria necessidade.

ANÁLISE E PROPOSTA INDEPENDENTE

Capacidade útil e responsabilidade pela fila



O primeiro entregável seria uma intenção com contrato de eventos, política de abstenção, revisão de rótulos e acompanhamento de desfecho. Operações participaria desde a definição do problema, pois o significado de um encaminhamento correto depende de como o atendimento funciona. Como EM, eu reservaria tempo para revisão conjunta de erros e treinamento, evitando colocar todo o custo de uma experiência técnica sobre os agentes que atendem. Também definiria quem pode suspender a automação de uma intenção.

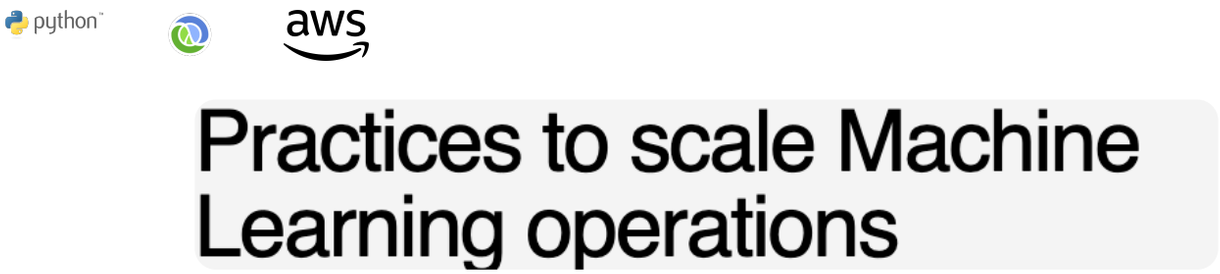
O valor econômico seria estimado por transferências evitadas, minutos de trabalho realmente recuperáveis e custo incremental do mecanismo. O fator de realização é essencial: uma economia média de segundos não se converte automaticamente em redução de despesa. Recontatos, maior duração em outra fila e trabalho de revisão precisam entrar na comparação. Tempo poupado pelo cliente seria apresentado separadamente, como resultado de experiência. Não somaria redução de transferência e de contato quando descrevem o mesmo esforço evitado.

Minha pergunta de Staff seria: conseguimos reproduzir o que o roteador sabia no momento da decisão, incluindo filas disponíveis e versão da política? Minha pergunta de EM seria: quando a previsão erra, existe uma pessoa ou equipe com responsabilidade e capacidade para resolver sem reiniciar a jornada? A resposta conecta arquitetura e liderança. O objetivo é oferecer ajuda útil com menos repetição, mantendo espaço para corrigir uma hipótese que parecia plausível.

ANÁLISE E PROPOSTA INDEPENDENTE

Proteção dentro do tempo da experiência

17



Captura da publicação oficial consultada em 23/10/2026.

O Nubank publicou, em abril de 2025, práticas para operar modelos em tempo real, incluindo Model Server com Clojure e Python, obtenção de features, otimização e shadow mode. O texto oferece contexto técnico, mas não revela políticas de decisão, limiares de risco ou perdas por produto. Este estudo não identifica uma vulnerabilidade nem pretende deduzir esses controles. Parte de uma pergunta de engenharia: como preservar uma decisão responsável quando contexto e tempo são limitados?

Em uma jornada financeira, esperar indefinidamente também é uma escolha com consequências. Permitir tudo quando uma dependência falha é outra escolha, frequentemente inadequada. Entre esses extremos há decisões por contexto, recuperação e revisão. Eu discutiria com Risco quais evidências são indispensáveis, quais podem faltar e como a pessoa continua quando uma operação não pode ser autorizada imediatamente. Essa conversa não pode ser delegada a um score sem explicação de sua finalidade e de seus limites.

O laboratório usa uma função logística com parâmetros inventados e fixos. Não existe treinamento, calibração ou conjunto de dados do Nubank. O objetivo é demonstrar separação entre cálculo, política e custo de erro, incluindo falsos positivos. Dezesseis testes verificam esse contrato pequeno. Não demonstram capacidade antifraude, redução de perdas ou adequação regulatória. Os exemplos são uma base para discutir como eu transformaria uma decisão ambígua em um experimento seguro junto aos especialistas do domínio.

Practices to scale Machine Learning operations

ANÁLISE E PROPOSTA INDEPENDENTE

O score não é a decisão



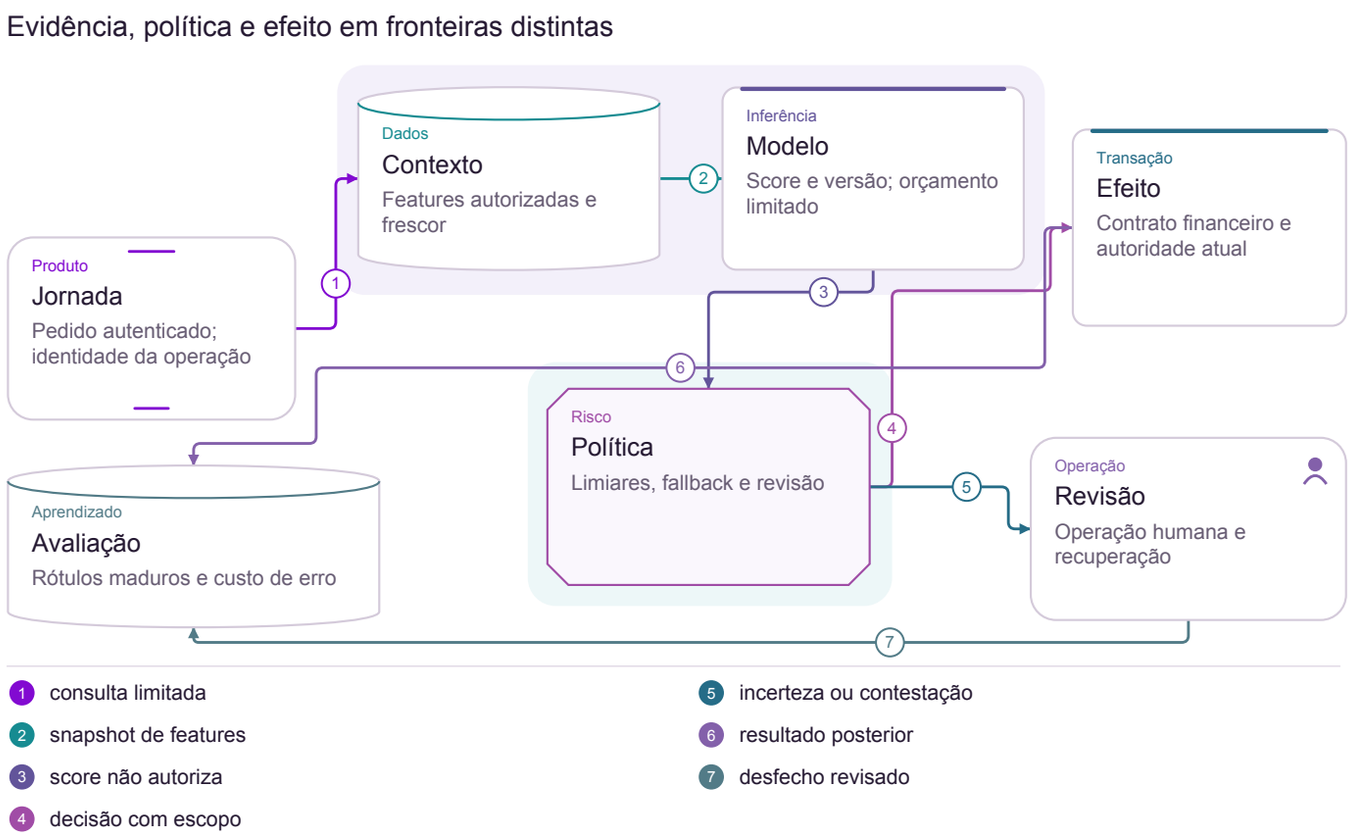
Um número entre zero e um pode parecer uma probabilidade, mas essa aparência não garante calibração. Antes de usá-lo, precisamos saber o que ele tenta prever, em qual horizonte, com quais dados e sob quais condições. O modelo local apenas transforma sinais sintéticos em um valor ordenável. Chamá-lo de probabilidade de fraude esconderia uma propriedade que nunca foi medida. Essa distinção é central para evitar que uma apresentação matemática dê autoridade indevida à política.

A política do laboratório escolhe permitir, revisar ou bloquear, com resultados específicos para dados ausentes e antigos. Os limiares são parâmetros da decisão, não características naturais do modelo. Uma mudança de limiar pode alterar volume de revisão, fricção e perdas mesmo quando o modelo permanece idêntico. Eu versionaria esses elementos separadamente para que um incidente pudesse ser atribuído à mudança correta. Também registraria os limites da ação: revisar não significa condenar e bloquear uma tentativa não necessariamente bloqueia a conta.

A revisão precisa ser uma capacidade real. Se não houver equipe, prazo ou caminho de recuperação, um estado intermediário vira fila invisível. Como proposta, cada saída teria mensagem apropriada, próximo passo e observabilidade, sem revelar mecanismos sensíveis. O contrato técnico deve sustentar uma experiência compreensível. A qualidade do modelo importa, mas a confiança do cliente também depende de como o sistema lida com a incerteza e permite corrigir uma decisão equivocada.

ARQUITETURA DE REFERÊNCIA

Evidência, política e efeito em fronteiras distintas



Arquitetura independente. Não descreve thresholds, políticas ou componentes internos do Nubank.

A decisão de risco não substitui autenticação, autorização nem consistência financeira. O componente de efeito deve validar a autoridade apropriada na fronteira da operação, mesmo quando a política recomenda permitir. Features chegam com versão e temporalidade, para que ausência não seja confundida com valor normal. O caminho de revisão exige capacidade e prazo próprios; não é um depósito de casos que o modelo não soube classificar. A avaliação recebe resultados depois que os rótulos amadurecem, evitando chamar de legítima uma transação apenas porque ainda não houve contestação. O laboratório implementa somente score, política e contabilidade sintética de erros. Não executa autorização, detecção produtiva ou atendimento. Python e Clojure são tecnologias citadas na fonte pública; a organização deste desenho é uma proposta autoral que precisaria ser comparada às fronteiras existentes antes de qualquer integração.

CÓDIGO · CONTRATO PROPOSTO

Modelo didático e política explícita

PYTHON / PROPOSTA AUTORAL

```python
def toy_score(features):
    validate_features(features)
    amount = min(features.amount_cents / 100000, 4)
    velocity = min(features.velocity, 10)
    z = -4 + 0.5 * amount + 0.7 * velocity
    z += 1.2 * int(features.device_change)
    return 1 / (1 + math.exp(-z))


def decide(features, allow_below=0.30, block_from=0.85):
    if not 0 <= allow_below < block_from <= 1:
        raise ValueError("invalid thresholds")
    if features is None:
        return "REVIEW_MISSING"
    validate_features(features)
    if features.age_seconds > 60:
        return "REVIEW_STALE"
    probability_like = toy_score(features)
    if probability_like >= block_from:
        return "BLOCK"
    if probability_like >= allow_below:
        return "REVIEW"
    return "ALLOW"
```

Trecho executado de case_17.py. Modelo logístico de brinquedo, fixo, não treinado. Tipos, imports e validadores estão no arquivo completo.

Os coeficientes foram escolhidos para produzir caminhos distintos nos testes; não foram aprendidos e não representam risco real. O valor calculado é denominado probability_like para lembrar esse limite. Dados antigos ou ausentes levam a revisão sintética, enquanto entradas negativas são rejeitadas. Os limiares 0,30 e 0,85 não devem ser transportados para operação. Uma implementação real precisa de avaliação de discriminação, calibração, estabilidade e custo de erro, além de decisão formal sobre fallback. O código preserva a separação entre produzir um score e escolher uma ação. Não contém controles internos, regras de fraude reais nem integração financeira. Tipos e domínios inválidos geram ValueError antes de qualquer resultado favorável.

Python 3.12: unittest

CÓDIGO · CONTRATO PROPOSTO

Separar perda, falso positivo e revisão

PYTHON / PROPOSTA AUTORAL

```python
def realized_cost(rows, fp_cents=500, review_cents=100):
    if any(type(x) is not int or x < 0 for x in (fp_cents, review_cents)):
        raise ValueError("costs must be non-negative integer cents")
    totals = {"missed_fraud": 0, "false_positive": 0, "review": 0}
    for action, fraud, amount_cents in rows:
        if type(amount_cents) is not int or amount_cents < 0:
            raise ValueError("amount must be non-negative integer cents")
        if type(fraud) is not bool:
            raise ValueError("fraud label must be boolean")
        if action == "ALLOW" and fraud:
            totals["missed_fraud"] += amount_cents
        elif action == "BLOCK" and not fraud:
            totals["false_positive"] += fp_cents
        elif action in {"REVIEW", "REVIEW_MISSING", "REVIEW_STALE"}:
            totals["review"] += review_cents
        elif action not in {"ALLOW", "BLOCK"}:
            raise ValueError("unknown action")
    totals["total"] = sum(totals.values())
    return totals

rows = [("ALLOW", True, 10000),
        ("BLOCK", False, 10000),
        ("REVIEW", True, 10000)]
assert realized_cost(rows)["total"] == 10600
```

Função e cenário executados em case_17.py; custos sintéticos, sem projeção do Nubank. Tipos, imports e validadores estão no arquivo completo.

Dinheiro permanece em centavos inteiros, e as parcelas ficam identificáveis. No exemplo, uma fraude permitida custa 10.000 centavos, um bloqueio incorreto custa 500 e uma revisão custa 100, totalizando 10.600. São custos arbitrários de uma simulação. A função não modela recuperação, perdas residuais depois da revisão, reputação ou efeitos de longo prazo. Em especial, revisão não garante que a fraude será impedida; seu custo completo depende do desfecho. A decomposição evita esconder falsos positivos atrás de uma métrica agregada de acurácia, mas só se torna útil para investimento quando cada parcela possui definição e evidência próprias.

Python 3.12: decimal

ANÁLISE E PROPOSTA INDEPENDENTE

Um snapshot de features precisa ter significado



Duas features chamadas velocidade podem medir coisas diferentes: número de tentativas, operações confirmadas ou dispositivos observados. Mesmo quando o tipo é inteiro, a semântica pode divergir. Eu exigiria uma definição compartilhada com janela, unidade, origem e tratamento de ausência. O modelo não deveria receber um zero produzido por timeout como se fosse ausência de atividade. Essa confusão altera a interpretação do cliente sem necessariamente provocar um erro técnico visível no serviço.

Frescor também é específico ao domínio. Uma informação cadastral pode tolerar uma idade diferente de um sinal ligado à tentativa atual. Um único campo age_seconds, usado pelo laboratório, é apenas uma simplificação para testar o ramo de dado antigo. Em produção, eu avaliaria frescor por feature e por decisão. Dados indispensáveis precisam de uma política de indisponibilidade; dados opcionais precisam de evidência de que o modelo consegue operar sem eles nas condições avaliadas.

O snapshot usado na inferência deveria ser rastreável dentro das restrições de privacidade, sem exigir conservar dados sensíveis indiscriminadamente. Versionar definições e registrar metadados pode ser suficiente para algumas investigações; outras exigem amostras governadas. A escolha depende da finalidade e do período de confirmação. A equipe precisa conseguir responder quais informações sustentaram uma decisão e quais estavam ausentes, sem inventar retrospectivamente um contexto que só chegou depois da operação.

ANÁLISE E PROPOSTA INDEPENDENTE

Falha parcial não deve virar permissão global






Uma dependência lenta pode aumentar o tempo da jornada e consumir recursos de outras solicitações. Definir timeout limita esse dano, mas não decide o que fazer depois. Eu separaria o orçamento técnico da política de risco: primeiro, quanto tempo podemos esperar; depois, qual decisão é aceitável sem aquele contexto. Essa segunda resposta pertence ao domínio e pode variar por operação, evidência restante e exposição. Um fallback permissivo genérico não é uma conclusão automática de engenharia.

Também há risco em revisar tudo. Um incidente amplo pode superar a capacidade da fila humana, atrasando recuperação de clientes legítimos. O plano precisa reconhecer saturação e definir ações previamente acordadas, com mensagem clara e prioridade apropriada. Revisão não pode ser uma palavra usada para ocultar indisponibilidade. O monitoramento deveria mostrar entradas, saídas, tempo de espera e idade dos casos, além da taxa de decisões degradadas que está alimentando essa demanda adicional.

Eu testaria dados ausentes, atrasados, inconsistentes e recuperados após a decisão, sem presumir que todos pedem a mesma resposta. Também ensaiaria o efeito de uma versão nova do produtor de features com um modelo antigo. O laboratório cobre uma pequena parte desses cenários com estruturas locais. Ele não simula capacidade de fila nem concorrência distribuída. Esses limites são importantes: passar nos testes de política não demonstra que o sistema continuará seguro durante uma falha operacional em escala.

ANÁLISE E PROPOSTA INDEPENDENTE

Dezesseis testes e um resultado que cabe em uma tabela






O laboratório executou dezesseis testes com sucesso. Há verificações para score baixo, alto e intermediário; dados ausentes, antigos e futuros; entrada negativa; ordem inválida de limiares; monotonicidade de velocidade; decomposição de custo; ações corretas sem custo modelado; e ação desconhecida. A monotonicidade é uma propriedade dos coeficientes escolhidos, não evidência de que velocidade deva sempre aumentar risco na realidade. O conjunto serve para tornar a demonstração consistente e reproduzível. A revisão acrescentou rejeição de números não finitos, centavos fracionários, flags ambíguas e ações de revisão desconhecidas.

A saída econômica soma 10.000 centavos de fraude permitida, 500 de falso positivo e 100 de revisão. O total de 10.600 centavos não representa economia nem perda de uma empresa. É uma observação de fixture, útil para verificar se a implementação soma categorias diferentes sem perder unidade. O cenário de ações corretas retorna zero porque o modelo de custo não inclui despesas fixas, infraestrutura ou atrito residual. Isso precisa ficar explícito para que o resultado não pareça eficiência perfeita.

A validação posterior exigiria dados autorizados, separação temporal de treino e avaliação, maturação de rótulos e comparação com baseline. Eu avaliaria segmentos e cenários degradados, além da média. O custo de revisão dependeria da operação real e do que acontece depois da triagem. Só então poderíamos discutir limiares. O artefato local demonstra disciplina de contrato; a qualidade de uma política antifraude precisa de evidências de outra natureza e de participação dos especialistas responsáveis.

Python 3.12: unittest

ANÁLISE E PROPOSTA INDEPENDENTE

Medir proteção e fricção no mesmo painel



Eu evitaria escolher uma métrica única para declarar sucesso. Fraude confirmada, transações legítimas bloqueadas, revisão e latência precisam aparecer juntas, com seus denominadores e períodos de maturação. Uma política que bloqueia tudo pode aparentar ótima prevenção e oferecer um produto inutilizável. Uma política que permite tudo pode parecer excelente em conclusão imediata e acumular perdas depois. A decisão exige compreender a troca entre esses resultados, e não celebrar um deles isoladamente.

A qualidade dos rótulos interfere diretamente nessa leitura. Uma transação sem contestação ainda pode ser fraude não descoberta; uma contestação pode ser revertida. Eu registraria a janela usada e revisaria séries quando houvesse atualização, preservando versões anteriores. Também analisaria viés de seleção: decisões bloqueadas podem não revelar o que teria acontecido se fossem permitidas. Comparar grupos sem considerar essa diferença produziria conclusões causais frágeis e poderia levar a limiares inadequados.

No plano operacional, observaria tempo por dependência, ausência de features e proporção de decisões degradadas. A correlação desses sinais com resultado de cliente orienta investigação, mas não prova causa por si só. Alertas deveriam chegar a responsáveis capazes de agir sobre modelo, dados ou política. Um aumento de falsos positivos pede contexto diferente de uma falha de infraestrutura. Eu manteria esse mapa de responsabilidade junto ao painel, para que a observabilidade ajude a resolver o problema identificado.

ANÁLISE E PROPOSTA INDEPENDENTE

Shadow não precisa autorizar dinheiro para ensinar



Um modelo candidato pode produzir scores em paralelo ao fluxo vigente, sem alterar a decisão real. Essa etapa permite comparar saídas e detectar incompatibilidades de features, mas não elimina todos os riscos: coleta adicional consome recursos, processamento pode atrasar dependências e retenção precisa ser governada. Eu definiria orçamento de execução e isolamento para que o experimento não prejudique a operação que pretende melhorar. Shadow é um modo de avaliação, não uma dispensa de responsabilidade operacional.

A divergência entre candidato e baseline seria analisada por tipo de caso e desfecho, com participação de Risco e Operações. Não assumiria que discordar significa melhorar. Poderíamos descobrir que o modelo novo identifica padrões úteis, mas piora justamente uma população pouco representada; ou que o ganho vem de informação indisponível no momento real. A decisão de avançar exigiria reproduzir as condições temporais e validar o custo da política, não apenas comparar curvas de desempenho offline.

Uma exposição limitada precisaria de critérios de contenção, autoridade para pausar e plano de atendimento aos clientes afetados. Eu também avaliaria reversão de modelo e política separadamente. Restaurar uma versão não desfaz bloqueios ou atrasos já ocorridos; esses efeitos podem precisar de recuperação específica. Como EM, garantiria capacidade para esse trabalho. Como Staff, manteria contratos e rastreabilidade suficientes para saber o que foi decidido, com quais dados e em qual versão.

ANÁLISE E PROPOSTA INDEPENDENTE

Menos dados, fronteiras mais claras



Acesso a uma feature não deve ser concedido apenas porque ela pode melhorar um modelo. Precisamos justificar finalidade, disponibilidade e proteção, preferindo representações que reduzam exposição quando preservam utilidade. Logs de decisão não devem virar uma cópia irrestrita do contexto financeiro do cliente. Eu discutiria com Segurança e Privacidade quais informações são necessárias para investigar, por quanto tempo e sob qual acesso. O desenho precisa permitir avaliação sem distribuir dados sensíveis entre todas as equipes envolvidas.

Também separaria permissões de leitura de contexto, publicação de modelo, alteração de política e execução financeira. Uma falha em um componente não deveria entregar todas essas capacidades ao mesmo processo. O laboratório não contém identidade, rede ou credenciais, por isso não demonstra esse isolamento. Em uma integração real, eu usaria as ferramentas e padrões existentes, avaliando explicitamente quem pode alterar um limiar e como essa alteração é revisada, registrada e revertida quando necessário.

A escolha tecnológica deve respeitar o ambiente. Python é adequado ao núcleo didático e aparece na arquitetura pública do Nubank, enquanto Clojure aparece na integração descrita. Isso não significa que cada proposta deva criar um novo servidor de modelos. Pode ser mais útil melhorar um contrato de feature, um teste de compatibilidade ou uma métrica na plataforma atual. O investimento deve resolver uma lacuna observada, com custo operacional proporcional ao benefício esperado e à capacidade de sustentação do time.

ANÁLISE E PROPOSTA INDEPENDENTE

ADR: política versionada fora do score






A decisão proposta é manter o modelo como produtor de um sinal e a política como responsável pela ação, com versões independentes. A alternativa de embutir limiares e fallback dentro do artefato de modelo simplifica a implantação, mas mistura mudanças de natureza diferente. A alternativa de distribuir regras em cada consumidor oferece flexibilidade e pode produzir divergências difíceis de rastrear. Eu começaria por um contrato explícito no lugar onde a decisão já ocorre, evitando criar uma plataforma nova sem necessidade demonstrada.

O benefício é conseguir explicar se uma mudança de resultado veio do sinal, do contexto ou da política. O custo é manter compatibilidade e impedir combinações não avaliadas. Um modelo novo não deve ser promovido com qualquer política antiga apenas porque o formato de saída é igual. O registro de versões precisa incluir condições de uso e dados esperados. A operação também precisa saber quais combinações são suportadas durante uma migração e qual delas constitui o caminho de retorno.

Eu revisaria a decisão se a separação aumentasse latência sem oferecer utilidade, ou se o domínio exigisse uma implantação conjunta para preservar consistência. Separação lógica não implica chamada remota entre serviços. O mesmo processo pode manter módulos e contratos distintos. A pergunta não é quantas caixas existem no diagrama, mas se uma equipe consegue avaliar, alterar e operar a decisão sem perder sua história, seus limites e a responsabilidade sobre os efeitos causados.

ANÁLISE E PROPOSTA INDEPENDENTE

Resultado econômico com responsabilidade humana



O primeiro incremento poderia ser um contrato de features críticas e uma avaliação de política em shadow, com revisão de divergências. Como EM, alinharia Engenharia, Ciência de Dados, Risco e Atendimento em objetivos que incluam proteção e experiência. Também reservaria capacidade para análise de falsos positivos e recuperação, evitando que o backlog de melhoria ignore os clientes afetados. A liderança precisa transformar essa discussão em prioridades claras, com responsáveis e critérios para interromper o experimento.

O modelo econômico compararia perdas por fraude em condições equivalentes, custo de transações legítimas bloqueadas, custo de revisão e infraestrutura incremental. Não usaria acurácia como substituto de valor. Custos de fricção exigem definição: atendimento adicional, tempo operacional ou perda de uso demonstrada são medidas diferentes. Benefícios que compartilham o mesmo evento não devem ser somados duas vezes. Na ausência de evidência, eu apresentaria intervalo de hipótese e sensibilidade, sem chamar o número de economia realizada.

Minha pergunta de Staff seria: qual informação pode faltar sem invalidar a decisão e quem aprovou esse limite? Minha pergunta de EM seria: temos capacidade de corrigir a decisão quando a pessoa legítima é impedida de seguir? Essas perguntas conectam o contrato técnico ao trabalho de equipe. O objetivo não é um classificador impressionante em isolamento. É um sistema de proteção que responde no tempo da jornada e permanece responsável pelos erros que inevitavelmente precisa reconhecer e tratar.

ANÁLISE E PROPOSTA INDEPENDENTE

Mesmo nome, decisões diferentes

18



Dealing with Train-serve Skew in Real-time ML Models: A Short Guide

Captura da publicação oficial consultada em 23/10/2026.

O guia público do Nubank sobre train-serve skew, de junho de 2023, discute diferenças entre dados usados no treinamento e na inferência. Entre os exemplos estão janelas temporais distintas, ausência representada como zero e fontes incompatíveis. São exemplos didáticos de uma classe de problema, não evidências de incidentes específicos ou perdas de clientes. Este capítulo usa esse contexto para explorar um contrato de feature que possa ser comparado sem reconstruir o passado com informação futura.

Uma feature pode ter nome, tipo e intervalo numérico corretos e ainda representar outra coisa. O treinamento pode contar compras confirmadas durante sessenta dias, enquanto o serviço conta tentativas durante trinta. Os dois produzem inteiros plausíveis. Nenhuma exceção precisa ser lançada para que o modelo passe a receber informação fora do significado que orientou sua construção. O problema exige observação semântica, e não apenas validação de formato ou disponibilidade do endpoint.

O laboratório implementa soma em janela com dois relógios: ocorrência e conhecimento. Também gera identidade de contrato e compara valores, ausência e versão. Tudo utiliza eventos inventados e centavos inteiros. Quinze testes verificam limites temporais, duplicidade, informação tardia e incompatibilidade de contrato. O objetivo não é construir uma feature store, mas demonstrar o mínimo necessário para uma pergunta precisa: conseguimos reproduzir a informação que estava disponível quando a decisão realmente aconteceu?

Dealing with Train-serve Skew in Real-time ML Models: A Short Guide

ANÁLISE E PROPOSTA INDEPENDENTE

O contrato contém mais do que o tipo



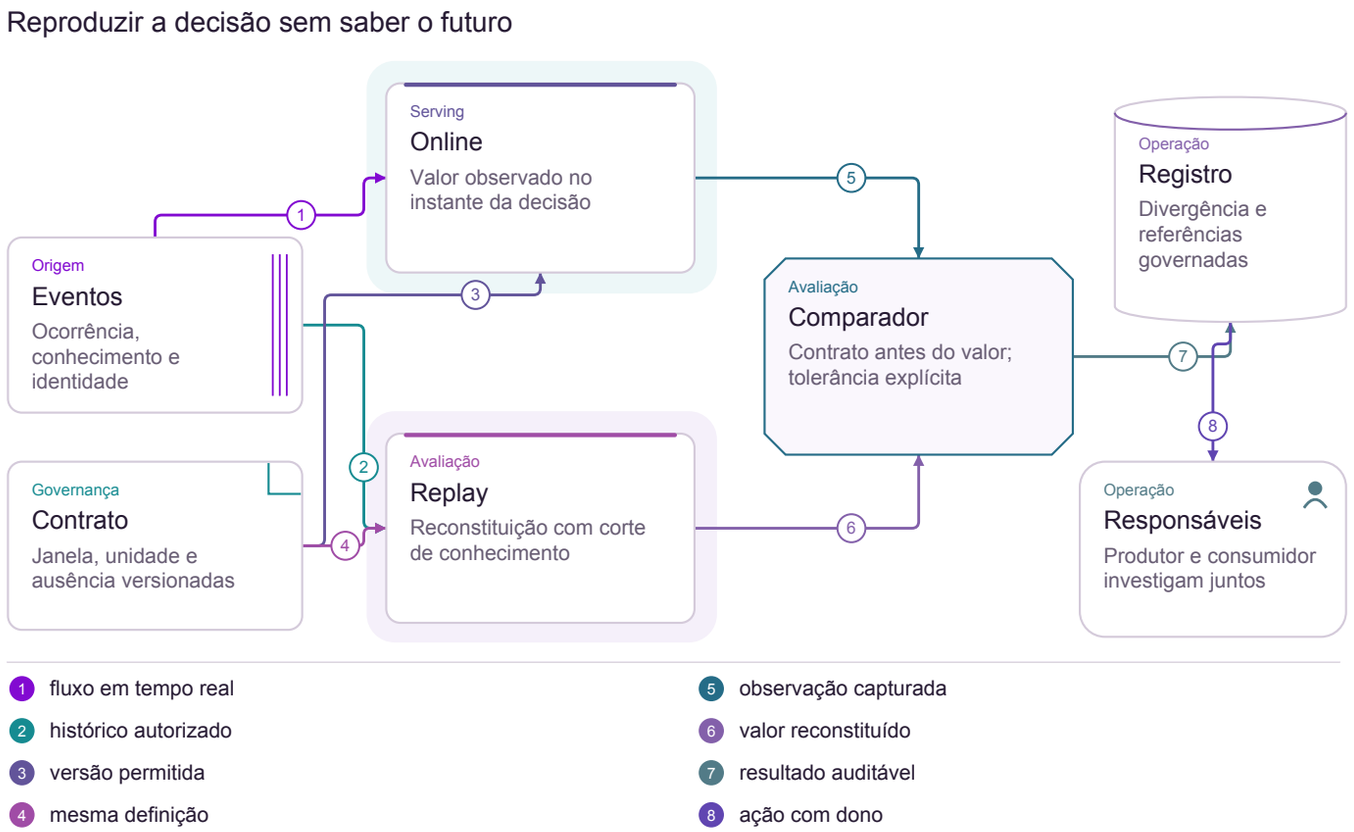
Eu definiria uma feature por nome, unidade, entidade, janela, fronteiras temporais, estado dos eventos incluídos, origem e regra de ausência. Também registraria quais correções posteriores alteram o valor e como elas afetam replays históricos. Sem essas escolhas, duas equipes podem implementar corretamente especificações diferentes e descobrir a divergência somente quando o modelo muda de comportamento. Uma descrição textual ajuda, mas precisa ser acompanhada de exemplos executáveis que revelem os casos de fronteira.

No laboratório, a janela é aberta à esquerda e fechada à direita: um evento exatamente no início fica fora; um evento no instante da decisão pode entrar, desde que já fosse conhecido. A unidade é centavo, e a ausência da fonte produz None. Uma fonte disponível sem eventos produz zero. Essa diferença é pequena no código e importante para a decisão: zero descreve observação vazia, enquanto None descreve falta de observação. O modelo pode precisar tratar esses estados de maneiras diferentes.

A identidade do contrato deriva de uma representação canônica de seus parâmetros. Ela facilita detectar combinações incompatíveis, mas não prova que a implementação obedece ao significado declarado. Um serviço pode publicar o hash correto e calcular a janela errada. Por isso, eu combinaria verificação de contrato com comparação de exemplos e replays. A documentação define o que deve acontecer; os testes e observações mostram se produtores e consumidores realmente estão fazendo a mesma coisa.

ARQUITETURA DE REFERÊNCIA

Reproduzir a decisão sem saber o futuro



Desenho autoral. Contratos e nomes propostos; não representa uma feature store interna.

O comparador só deve confrontar valores que representam a mesma entidade, instante e definição. O replay recebe um corte de conhecimento para impedir que eventos chegados depois tornem o passado artificialmente completo. O contrato governa online e offline, mas não substitui validação da implementação. O registro de divergência guarda referências suficientes para investigar, com retenção e acesso proporcionais ao dado. Uma divergência não implica automaticamente erro do online: pode refletir correção histórica, diferença legítima de disponibilidade ou defeito no próprio replay. O laboratório representa esses conceitos em memória e não executa pipeline distribuído. Em produção, eu procuraria pontos de captura já existentes para reduzir custo e evitar outra cópia integral de eventos. A investigação precisa chegar aos responsáveis pelo significado da feature, e não somente à equipe que opera o armazenamento ou o modelo.

CÓDIGO · CONTRATO PROPOSTO

A informação que ainda não existia para o sistema

PYTHON / PROPOSTA AUTORAL

```python
def window_sum(events, at, known_at, seconds=60, available=True):
    validate_window(at, known_at, seconds, available)
    if not available:
        return None
    unique = {}
    for event in events:
        values = (event.occurred, event.known, event.amount_cents)
        if any(type(value) is not int for value in values):
            raise ValueError("integer times and cents required")
        if event.known > known_at or event.occurred > at:
            continue
        if not at - seconds < event.occurred <= at:
            continue
        if event.id in unique and unique[event.id] != event:
            raise ValueError("conflicting event")
        unique[event.id] = event
    return sum(event.amount_cents for event in unique.values())

late = [Event("x", 95, 110, 900)]
assert window_sum(late, 100, 100) == 0
assert window_sum(late, 100, 120) == 900
```

Trecho executado de case_18.py; eventos e relógios sintéticos. Tipos, imports e validadores estão no arquivo completo.

O campo occurred representa quando o evento aconteceu; known informa quando ele entrou no conjunto disponível. O replay exclui dados que não eram conhecidos no corte escolhido, mesmo quando ocorreram antes da decisão. A fronteira da janela também é explícita. A estrutura não resolve correções, exclusões ou múltiplas versões de um evento: duplicatas conflitantes geram erro para que o caso não seja agregado silenciosamente. Em uma integração real, esses estados precisariam de política própria. A demonstração usa inteiros como relógio sintético; timestamps de produção exigem unidade, fuso, precisão e validação acordados entre os componentes envolvidos.

Python 3.12: datetime

CÓDIGO · CONTRATO PROPOSTO

Comparar significado antes de comparar números

PYTHON / PROPOSTA AUTORAL

```python
def contract_id(window_seconds=60, missing="null", unit="cents"):
    contract = {"window": window_seconds, "missing": missing,
                "unit": unit, "bound": "(at-window, at]", "version": 1}
    encoded = json.dumps(contract, sort_keys=True, separators=(",", ":"))
    return hashlib.sha256(encoded.encode()).hexdigest()[:16]


def compare(batch, online, batch_contract, online_contract, tolerance=0):
    validate_comparison(batch, online, tolerance)
    if batch_contract != online_contract:
        return "CONTRACT_MISMATCH"
    if batch is None or online is None:
        return "MATCH_MISSING" if batch is online else "MISSING_MISMATCH"
    if abs(batch - online) > tolerance:
        return "VALUE_MISMATCH"
    return "MATCH"

left = contract_id(60)
right = contract_id(30)
assert compare(0, 0, left, right) == "CONTRACT_MISMATCH"
assert compare(None, 0, left, left) == "MISSING_MISMATCH"
```

Funções e exemplos executados em case_18.py. Hash identifica parâmetros, não certifica implementação. Tipos, imports e validadores estão no arquivo completo.

Valores iguais podem esconder contratos diferentes. Por isso, a comparação verifica primeiro a identidade da definição, depois o estado de ausência e finalmente a diferença numérica. O digest abreviado é conveniente para a demonstração e não deve ser tratado como assinatura de segurança. Em produção, a versão precisa estar ligada a um registro governado e à implementação publicada. A tolerância também pertence ao domínio: uma soma monetária exata pode exigir igualdade, enquanto outra feature admite erro numérico limitado. Tolerância alta não corrige semântica incompatível; apenas transforma uma divergência em algo que o monitor deixa de mostrar. Valores não finitos e tolerância inválida geram ValueError, evitando coincidência falsa.

Python 3.12: unittest

ANÁLISE E PROPOSTA INDEPENDENTE

Ausência, zero e atraso não são sinônimos



Uma fonte disponível pode retornar nenhuma ocorrência porque o cliente não realizou a atividade. Isso produz zero conforme o contrato local. Uma fonte indisponível não oferece essa informação e produz ausência. Um evento atrasado pode fazer o valor atual diferir do que será conhecido depois, sem indicar defeito na implementação original. Se todos esses estados forem convertidos em zero, o modelo recebe uma simplificação que apaga exatamente o contexto necessário para interpretar a decisão.

Eu levaria essa distinção até o esquema e a avaliação. Um campo opcional não deveria existir apenas para evitar falha de serialização; seu significado precisa estar documentado e ser compatível com o treinamento. Também evitaria usar uma string especial como valor numérico sem governança. O consumidor deve saber se a feature está ausente por falta de histórico, indisponibilidade técnica ou condição de elegibilidade. Em alguns domínios, essas causas podem exigir tratamento diferente, mas a decisão precisa ser validada com especialistas.

O laboratório usa apenas fonte disponível ou indisponível, sem classificar todas as causas possíveis. Essa escolha mantém o exemplo inspecionável. Para um piloto real, eu começaria com uma feature crítica e investigaria quais estados aparecem de fato, em vez de inventar uma taxonomia completa antecipadamente. A evolução do contrato incluiria exemplos antigos e novos, para que uma melhoria de expressividade não quebre silenciosamente os consumidores que continuam em versões anteriores.

ANÁLISE E PROPOSTA INDEPENDENTE

Eventos tardios podem enganar uma avaliação correta



Considere uma compra ocorrida antes da decisão, mas recebida pela plataforma depois. Um dataset reconstruído semanas mais tarde pode incluí-la como se sempre estivesse disponível. O modelo avaliado offline parece usar contexto completo, enquanto o serviço online nunca teve essa oportunidade. Não é necessário um bug no algoritmo para surgir essa diferença. O problema está na pergunta feita ao histórico: estamos reconstruindo o mundo como era ou o mundo como passou a ser conhecido?

Outra situação envolve correções e reprocessamento. Um valor pode ser ajustado depois, uma duplicata removida ou um evento reclassificado. O replay precisa definir se a referência desejada é a visão original ou a visão corrigida, preservando ambas quando a finalidade exigir. Reescrever todos os relatórios antigos com os dados atuais destrói a possibilidade de explicar decisões passadas. Ao mesmo tempo, ignorar correções para sempre impede aprender. A solução depende de versionar referências e finalidade da comparação.

Eu também testaria fronteiras de dia e fuso, mudança de janela e eventos fora de ordem. O laboratório usa tempo inteiro para isolar o raciocínio, portanto não demonstra tratamento de horário civil. Essa camada exigiria testes adicionais com timestamps conscientes de fuso e regras acordadas. O critério não é ter muitos testes de calendário, mas verificar os instantes em que duas implementações plausíveis podem incluir conjuntos diferentes de eventos e produzir decisões incompatíveis.

Python 3.12: datetime

ANÁLISE E PROPOSTA INDEPENDENTE

Quinze testes para evitar um passado perfeito







Os quinze testes locais passaram. Eles cobrem valores equivalentes, ocorrência futura, conhecimento tardio, fronteira esquerda aberta, duplicidade, conflito, ausência diferente de zero, ausência em ambos os lados, contrato incompatível, tolerância, fonte indisponível e janela inválida. A suíte verifica a semântica declarada do exemplo. Não demonstra qualidade de um modelo, ausência de skew em um ambiente real ou correção de qualquer implementação do Nubank. Esses resultados pertencem somente ao pacote independente que acompanha o capítulo. As regressões adicionais rejeitam NaN, infinito, tolerância inválida e eventos fora do contrato inteiro de tempo e centavos.

Na demonstração, uma decisão em 100 conhece um evento de 200 centavos. Outro evento ocorreu em 95, mas só ficou conhecido em 110. Com corte de conhecimento em 100, a soma é 200; com corte em 120, a soma histórica passa a 1.100. Os dois resultados podem estar corretos para perguntas diferentes. O erro seria usar o segundo como prova de que o serviço deveria ter calculado esse valor no primeiro instante, sem reconhecer a informação indisponível.

O próximo passo seria comparar captura online e replay para um conjunto autorizado, com amostragem que inclua ausências e mudanças de versão. Eu examinaria divergências manualmente antes de escolher um limiar de alerta. Algumas representam atraso esperado; outras revelam definições diferentes ou defeito de captura. O teste mais importante do piloto é conseguir explicar a divergência e atribuir responsabilidade pela correção, sem presumir que um lado da comparação é sempre a verdade.

Python 3.12: unittest

ANÁLISE E PROPOSTA INDEPENDENTE

Monitorar divergência sem produzir outra caixa-preta



Eu separaria três famílias de sinal: incompatibilidade de contrato, incompatibilidade de ausência e diferença numérica sob o mesmo contrato. Cada uma orienta uma investigação distinta. Um hash diferente pede revisão de implantação; ausência unilateral pede olhar disponibilidade e elegibilidade; valores diferentes pedem examinar eventos e transformação. Agregar tudo em uma taxa única de skew perde essa informação e dificulta priorizar a correção que realmente pode afetar decisões de clientes.

A cobertura do comparador também precisa aparecer. Se só conseguimos reconstruir as features simples, a taxa observada não descreve todo o sistema. Eu mostraria quantas decisões elegíveis foram comparadas, quais versões ficaram de fora e por quê. As dimensões devem ser limitadas para evitar custo e exposição desnecessários. Um identificador de cliente não deve se transformar em label de uma série temporal. Casos individuais para investigação exigem outro caminho de acesso e retenção, com finalidade definida.

Um alerta útil incluiria o contrato, a janela, a magnitude, o período e os responsáveis por produzir e consumir a feature. Também indicaria se existe evidência de decisões potencialmente afetadas, sem convertê-la automaticamente em perda financeira. Eu avaliaria se a divergência muda a saída do modelo em um replay governado, quando apropriado. Essa etapa conecta qualidade de dados a impacto, mas continua exigindo cuidado causal: uma mudança de score não prova que uma decisão real foi errada.

ANÁLISE E PROPOSTA INDEPENDENTE

Compatibilidade como parte do rollout de dados



Uma mudança de janela ou fonte pode ser tão importante quanto um release de modelo. Eu trataria a evolução de feature como contrato compartilhado, com versão nova, exemplos de fronteira e consumidores identificados. O produtor não deveria substituir o significado mantendo o mesmo nome porque o formato permaneceu igual. A migração precisa permitir avaliar as duas definições em paralelo ou estabelecer uma troca coordenada, conforme custo e restrições da plataforma existente.

O piloto começaria com comparação sem bloquear decisões. Isso permite medir custo, cobertura e taxa de divergências esperadas. Depois, poderíamos criar gates para incompatibilidades inequívocas, como um consumidor requisitar contrato não suportado. Eu seria mais cauteloso ao bloquear por diferença de valor, pois atraso legítimo e captura incompleta podem gerar falsos alarmes. O nível de automação deve acompanhar a qualidade da classificação de divergência, e não apenas a disponibilidade técnica de um botão de bloqueio.

Como EM, alinharia os donos de dados e dos serviços, reservando capacidade para resolver problemas entre equipes. Como Staff, definiria o significado de compatibilidade e como manter referências históricas. O plano de retorno incluiria não apenas código, mas também dados e versões disponíveis. Se uma migração eliminar a possibilidade de reconstruir a definição anterior, o rollback pode ser nominal. O contrato precisa dizer por quanto tempo as combinações antigas continuam operáveis e quais evidências sustentam sua retirada.

ANÁLISE E PROPOSTA INDEPENDENTE

Replays úteis sem cópias indiscriminadas



Um comparador pode se tornar um consumidor poderoso de dados. Eu evitaria centralizar todo o histórico apenas porque facilita debugging. Começaria por referências, amostras governadas e agregados, escolhendo o mínimo necessário para reproduzir a feature. O acesso deve respeitar a finalidade da investigação, com retenção e trilha de consulta apropriadas. A capacidade de explicar uma decisão não exige que toda pessoa com acesso ao painel possa explorar todos os eventos financeiros que a antecederam.

O custo computacional também precisa ser limitado. Recalcular janelas extensas para cada decisão pode multiplicar consultas e prejudicar sistemas de origem. Eu avaliaria amostragem, snapshots e reuso de agregados, sem perder a semântica temporal. Uma otimização que usa dados futuros por conveniência é uma regressão de correção, mesmo que reduza o tempo do job. A avaliação deve considerar desempenho e fidelidade da reconstrução, pois os dois compõem o contrato do replay.

No laboratório, a soma percorre eventos em memória e é adequada apenas ao tamanho das fixtures. Não há índice, armazenamento distribuído ou orçamento de consulta. Uma implementação real precisaria de desenho de acesso coerente com o volume e com a plataforma existente. Eu preferiria demonstrar utilidade em poucas features críticas antes de ampliar cobertura. Se o comparador produz custo alto e divergências sem ação, o primeiro ajuste deve ser a pergunta que ele tenta responder, não a compra de mais capacidade.

ANÁLISE E PROPOSTA INDEPENDENTE

ADR: comparação por contrato e instante



A decisão proposta é comparar versões equivalentes de uma feature no mesmo instante e sob o mesmo corte de conhecimento. A alternativa de comparar médias diárias é barata e pode detectar mudanças amplas, mas não explica divergências por decisão. A alternativa de exigir igualdade exata para todas as features ignora características numéricas e atrasos legítimos. Eu combinaria contrato explícito, comparação pontual amostrada e métricas agregadas, calibrando cada parte conforme a natureza da feature escolhida.

O custo está em manter identidade de contrato, referências temporais e capacidade de replay. Isso pode exigir mudanças em produtores e consumidores, além de governança sobre dados históricos. Nem toda feature merece esse investimento. Eu priorizaria aquelas com alto impacto potencial, transformação complexa ou histórico de divergência difícil de explicar. A decisão deve considerar o benefício esperado de detectar um erro e o custo de investigar alarmes incorretos, evitando transformar toda variação em incidente operacional.

Revisaria o ADR quando a definição não puder ser reconstruída com fidelidade ou quando a comparação produzir pouco sinal útil. Nesses casos, testes contratuais no produtor, observação de distribuição ou captura governada no momento da decisão podem ser alternativas melhores. O objetivo não é provar que batch e online são sempre idênticos. É tornar explícito quando deveriam ser, quais diferenças são aceitáveis e qual ação tomar quando a realidade ultrapassa esses limites acordados.

ANÁLISE E PROPOSTA INDEPENDENTE

Um contrato compartilhado precisa de donos



O primeiro incremento poderia cobrir uma feature com definição versionada, exemplos executáveis, captura online e replay limitado. O aceite incluiria explicar pelo menos um caso de ausência, um evento tardio e uma mudança de contrato, sem reescrever o passado. Como EM, reuniria quem produz, transforma e consome o dado, com tempo reservado para corrigir as divergências encontradas. Uma plataforma não resolve sozinha desacordo semântico; alguém precisa decidir qual significado atende ao domínio e assumir sua manutenção.

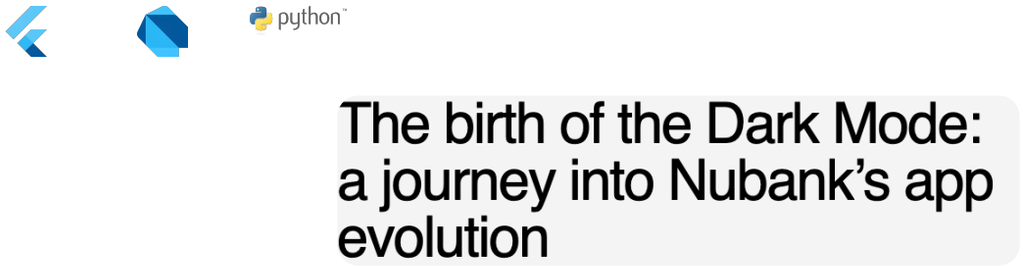
O modelo de valor começaria por tempo de investigação recuperável e incidentes de incompatibilidade efetivamente evitados. Redução de decisões inadequadas exigiria avaliação adicional, sem atribuir toda mudança de perda ao comparador. Eu separaria custo de armazenamento, processamento e revisão humana. Horas poupadas são capacidade potencial, e não economia automática. Também registraria benefícios de rastreabilidade que não têm conversão monetária simples, como explicar uma mudança de comportamento e reduzir incerteza antes de um lançamento relevante.

Minha pergunta de Staff seria: esta feature descreve o que aconteceu, o que era conhecido ou o que foi corrigido depois? Minha pergunta de EM seria: quando duas equipes discordam sobre a resposta, existe um processo rápido e respeitoso para decidir e atualizar o contrato? A qualidade emerge da combinação. Um hash e uma suíte de testes tornam a definição verificável; colaboração e responsabilidade fazem essa definição continuar correta quando o produto e a organização mudam.

ANÁLISE E PROPOSTA INDEPENDENTE

Dark Mode como evolução de um produto vivo

19



Captura da publicação oficial consultada em 23/10/2026.

O relato oficial publicado em setembro de 2023 descreve a evolução do Dark Mode do Nubank com tokens de design, adaptação de telas e colaboração entre Design e Engenharia. Também aborda fallback durante a transição e ajustes de contraste. Trata-se de uma história de desenvolvimento; ela não demonstra que a versão atual do aplicativo tenha um defeito específico. A oportunidade deste capítulo é extrair critérios para evoluir uma experiência financeira sem sacrificar sua compreensão.

O problema relevante não é apenas escolher um fundo escuro. Um valor disponível, um pagamento pendente e uma falha de transferência precisam conservar significado em temas diferentes, sob iluminação variável e com texto ampliado. Uma interface pode parecer sofisticada na apresentação e exigir esforço excessivo no uso diário. Transparência, brilho e gradientes devem servir à hierarquia, enquanto informação essencial continua distinguível.

Minha proposta independente organiza essa evolução por jornadas e estados, com verificações automáticas e avaliação humana. O laboratório mede pares sintéticos de cores e alguns sinais semânticos; ele não executa Flutter nem certifica acessibilidade. Para contratar uma liderança técnica, a pergunta produtiva é como transformar uma melhoria visual em um contrato sustentável entre componentes, produtos e atendimento. A resposta precisa combinar sistema de design, responsabilidade pelos estados financeiros e evidência de que pessoas conseguem concluir suas tarefas.

The birth of the Dark Mode: a journey into Nubank's app evolution

ANÁLISE E PROPOSTA INDEPENDENTE

O contrato do token é o significado



Um token chamado roxo-500 descreve uma aparência. Um papel chamado conteúdo-sobre-superfície descreve uma relação. Para uma experiência financeira, essa diferença permite mudar o tema sem exigir que cada squad reaprenda os pares de cores. O contrato precisa informar em qual superfície o conteúdo aparece, qual hierarquia representa e quais estados não podem compartilhar sinais ambíguos.

Eu proporia uma matriz de componentes e estados: disponível, em processamento, concluído, recusado e temporariamente indisponível. Cada célula relacionaria texto, ícone, cor, ação permitida e mensagem acessível. Uma cor de destaque não significaria sucesso por conveniência visual. O estado de domínio chegaria à interface como dado explícito, e a apresentação escolheria um papel semântico compatível. Essa separação evita que um ajuste de marca altere a interpretação de uma transferência.

A migração também precisa de regras para conteúdo legado. Um componente sem token novo pode usar uma superfície opaca conhecida, desde que texto e ações permaneçam legíveis e a exceção tenha proprietário. O fallback é uma estratégia de continuidade com prazo de revisão, não uma cor mágica que resolve qualquer composição. O ponto de controle do Staff seria a estabilidade desse contrato entre produtos. O EM organizaria a sequência de adoção para que componentes compartilhados reduzam trabalho repetido e jornadas críticas recebam verificação primeiro.

Flutter API: ColorScheme

ARQUITETURA DE REFERÊNCIA

Da intenção visual à evidência em dispositivo



Arquitetura de referência independente; nenhuma API ou ferramenta interna foi presumida.

Esta arquitetura proposta separa o significado do estado financeiro da escolha visual. Tokens são artefatos versionados; a decisão de um pagamento não depende deles. O adaptador resolve o tema localmente, e o componente produz pixels e semântica para avaliação. O registro de release mantém a configuração examinada, incluindo dispositivo, tema, escala de texto e exceções aceitas. A principal fronteira de confiança aparece entre o estado recebido e sua representação: uma alteração de cor não pode fabricar sucesso nem esconder uma pendência. Outra fronteira separa simulação e experiência real. Cálculos de contraste sobre cores declaradas são úteis, mas transparência, imagens e sobreposição exigem inspeção do resultado renderizado. O retorno da avaliação ao catálogo não é uma atualização automática em produção. Ele abre uma revisão com responsáveis de Design, Engenharia e Produto. Essa disciplina permite corrigir um componente compartilhado e verificar os consumidores antes de ampliar a mudança. O diagrama não representa a arquitetura interna do Nubank.

CÓDIGO · CONTRATO PROPOSTO

Contraste calculado antes da inspeção visual

PYTHON / PROPOSTA AUTORAL

```python
def luminance(color):
    if len(color) != 3 or any(not 0 <= x <= 1 for x in color):
        raise ValueError('Expected three channels in [0, 1]')
    linear = [x / 12.92 if x <= .04045 else
              ((x + .055) / 1.055) ** 2.4 for x in color]
    return sum(x * w for x, w in zip(linear, (.2126, .7152, .0722)))


def contrast(first, second):
    high, low = sorted((luminance(first), luminance(second)), reverse=True)
    return (high + .05) / (low + .05)

black = rgb('#000000')
white = rgb('#FFFFFF')
assert contrast(black, white) == 21
assert contrast(white, black) == 21
assert contrast(black, black) == 1
assert text_check('#777777', '#FFFFFF')['passes'] is False
assert text_check('#767676', '#FFFFFF')['passes'] is True
```

Trecho de case_19.py executado com Python; pares sintéticos, sem capturas do aplicativo. Tipos, imports e validadores estão no arquivo completo.

O trecho executado no laboratório transforma canais sRGB em luminância relativa e calcula a razão de contraste entre duas cores opacas. Ele rejeita canais fora do domínio aceito. A comparação é simétrica: trocar fundo e texto conserva a razão, embora a experiência visual e a função de cada papel continuem distintas. A WCAG usa critérios diferentes para texto normal e texto grande; o laboratório explicita essa escolha na função text_check. Esse cálculo é uma triagem determinística. Ele não verifica fonte, tamanho físico, brilho do dispositivo, compreensão do conteúdo nem toda a conformidade de uma tela financeira.

W3C: Understanding Contrast (Minimum)

Python 3.12: unittest

CÓDIGO · CONTRATO PROPOSTO

Um estado compreensível em dois temas

DART / PROPOSTA AUTORAL

```dart
import 'package:flutter/material.dart';

class PendingPayment extends StatelessWidget {
  const PendingPayment({super.key, required this.onDetails});
  final VoidCallback onDetails;

  @override
  Widget build(BuildContext context) {
    final colors = Theme.of(context).colorScheme;
    return Card(
      color: colors.surface,
      child: Padding(
        padding: const EdgeInsets.all(16),
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            Icon(Icons.schedule, color: colors.onSurface),
            const SizedBox(height: 8),
            Text('Pagamento em processamento',
                style: TextStyle(color: colors.onSurface)),
            const SizedBox(height: 8),
            Text('Confira o andamento antes de tentar novamente.',
                style: TextStyle(color: colors.onSurface)),
            TextButton(onPressed: onDetails,
                child: const Text('Ver andamento')),
          ],
        ),
      ),
    );
  }
}
```

Componente Flutter proposto; não é código do Nubank e não foi executado neste ambiente.

Este componente de referência recebe uma ação explícita e usa os papéis surface e onSurface do tema. A mensagem de processamento aparece por texto e ícone, sem depender de um tom de amarelo. A composição em coluna admite crescimento vertical, mas ainda precisa de testes com fontes ampliadas, traduções e largura pequena. O botão deve levar a uma consulta segura do andamento, conforme um contrato de produto a definir. O exemplo não declara saldo, não inicia nova transação e não consulta serviços. O Dart é ilustrativo e não foi compilado; a execução comprovada deste capítulo está no laboratório Python.

Flutter API: ColorScheme

Flutter: testing

ANÁLISE E PROPOSTA INDEPENDENTE

Vidro e transparência precisam de uma superfície previsível



Uma estética de vidro depende do que está atrás da camada. O mesmo texto pode atravessar áreas claras, escuras e saturadas quando o fundo muda, quando uma imagem carrega ou quando um painel se desloca. Portanto, aprovar apenas o valor hexadecimal do token não estabelece contraste suficiente. A unidade de avaliação deve ser o par de cores efetivamente composto na posição do conteúdo.

Para valores, avisos e ações financeiras, minha proposta privilegia uma superfície mínima previsível sob o texto. Desfoque e brilho podem permanecer nas bordas, nos elementos decorativos e nas transições, desde que não carreguem a informação essencial. Caso a composição não mantenha os critérios definidos, o fallback deve aumentar a opacidade ou usar uma superfície sólida. O usuário não deveria precisar ajustar o tema para descobrir se uma operação terminou.

O laboratório inclui composição alfa simplificada em sRGB para demonstrar por que uma cor originalmente forte perde contraste sobre um fundo. Ele não reproduz todos os espaços de cor e efeitos do renderizador. A validação final exige captura real, estado carregando, erro, conteúdo longo e preferências do sistema. Essa restrição pode melhorar o próprio design: ao reservar os efeitos para áreas adequadas, a interface mantém personalidade visual sem tornar a leitura uma competição com o fundo. Elegância também é previsibilidade durante uma tarefa importante.

W3C: Understanding Contrast (Minimum)

ANÁLISE E PROPOSTA INDEPENDENTE

Trocar o tema sem trocar o significado



Uma mudança de tema pode ocorrer enquanto a pessoa lê um comprovante, preenche um formulário ou acompanha uma operação. O risco não se resume a um frame com cor antiga. Reiniciar a tela, perder o foco, ocultar uma validação ou apagar o texto digitado transforma uma preferência visual em interrupção de jornada. O estado de domínio e o estado de edição precisam sobreviver à atualização da apresentação.

Eu definiria uma estratégia de resolução de tema estável para a árvore de componentes e testaria mudanças durante pontos críticos. O sistema não deve reinterpretar uma operação pendente como uma nova ação porque o componente foi reconstruído. Animações de transição seriam discretas e compatíveis com preferências de movimento, enquanto mensagens essenciais permaneceriam disponíveis na árvore semântica. A atualização visual não deveria disparar anúncios repetidos de um mesmo estado financeiro.

A compatibilidade com componentes antigos merece acompanhamento separado. Se parte da tela recebe o tema novo e outra permanece no anterior, a fronteira precisa conservar hierarquia e legibilidade. Uma exceção temporária pode ser aceitável; um botão que desaparece sobre a superfície não é. Para investigar regressões, registraria versão do aplicativo, versão do catálogo visual e contexto de apresentação, sem capturar valores privados. O objetivo é reproduzir a falha com dados sintéticos equivalentes e corrigir a fronteira responsável, em vez de acumular ajustes locais sem contrato.

Flutter: accessibility

ANÁLISE E PROPOSTA INDEPENDENTE

Doze testes: o que os números realmente demonstram





Os doze testes do laboratório verificam propriedades delimitadas. Preto e branco produzem razão 21:1; cores iguais produzem 1:1; a razão permanece simétrica. Há casos para os limiares de texto, composição alfa nos extremos, perda de contraste em uma transparência escolhida, entrada inválida e presença de sinais adicionais à cor. Todos passaram na execução local registrada em results_19.txt.

O resultado #777777 sobre branco foi 4,4781 e ficou abaixo do limite usado para texto normal. O par #767676 sobre branco produziu 4,5422 e passou nesse critério numérico. A pequena distância entre eles ilustra por que arredondar a razão antes da comparação seria uma decisão incorreta. A apresentação arredonda para leitura, mas a decisão usa o valor calculado. Um exemplo escuro sintético, #F6F1FA sobre #160B22, produziu 17,0857.

Esses números não são medições do aplicativo Nubank. Também não demonstram que uma pessoa compreenda o conteúdo, perceba o foco ou consiga operar o componente. O teste de sinais redundantes exige texto e identificador de ícone, porém não julga se o ícone faz sentido. Uma suíte adequada combinaria esta camada barata com testes de widget, inspeção semântica, comparação visual e tarefas observadas em dispositivos reais. A conclusão técnica é modesta e útil: certas regressões podem ser encontradas antes de chegarem à avaliação humana.

W3C: Understanding Contrast (Minimum)
Python 3.12: unittest

ANÁLISE E PROPOSTA INDEPENDENTE

Observabilidade sem transformar a tela em vigilância



Qualidade visual costuma produzir evidência diferente da observabilidade de serviços. Uma taxa de erro de API pode permanecer estável enquanto um botão fica difícil de reconhecer. Por isso, a avaliação deve relacionar versão visual e conclusão de tarefa, com uma descrição clara do que cada medida consegue indicar. Abandono, duração e contato ao suporte são sinais para investigação; nenhum deles identifica sozinho uma falha de contraste.

Minha proposta acompanharia jornadas selecionadas e registraria apenas eventos necessários, como início, conclusão, saída e categoria de erro. A análise poderia comparar temas e versões quando a coleta fosse adequada e a amostra suficiente. Preferências de acessibilidade exigem cuidado adicional: não devem virar inferências sobre deficiência, segmentação comercial ou conteúdo sensível em logs. Muitas perguntas podem ser respondidas com sessões consentidas e cenários de teste, sem telemetria individual permanente.

O painel de qualidade reuniria regressões conhecidas, cobertura dos componentes essenciais, tempo de correção e resultados de pesquisas. Uma diferença observada entre grupos motivaria reprodução e análise de composição da amostra, não uma afirmação causal imediata. Staff e EM deveriam concordar antecipadamente sobre o que faz uma mudança parar. Uma barreira crítica confirmada pode justificar suspensão mesmo sem volume estatístico alto. Essa postura mantém o indicador a serviço da experiência, em vez de usar a ausência de um alarme de backend como prova de que tudo está bem.

ANÁLISE E PROPOSTA INDEPENDENTE

Uma migração de tema é uma migração de contratos



A adoção de tokens funciona melhor quando as dependências ficam visíveis. Antes de alterar um papel compartilhado, o time precisa conhecer seus consumidores, as superfícies esperadas e as exceções. Um catálogo de exemplos com estados financeiros torna essa revisão concreta. A alteração deve vir acompanhada de evidência de compatibilidade, especialmente para componentes que aparecem em produtos diferentes e em telas compostas.

Eu começaria por componentes frequentes e jornadas essenciais, escolhendo uma sequência que produza aprendizado sem ampliar o risco de uma só vez. Cada lote teria responsável, critérios de aceite, capturas de referência e um caminho de reversão. Mudanças incompatíveis receberiam versão explícita e período de coexistência. A equipe de plataforma apoiaria a migração com ferramentas e documentação; as squads consumidoras verificariam os estados reais de seus domínios.

O critério de expansão combinaria legibilidade, manutenção de ações, compreensão das mensagens e ausência de barreiras críticas nos dispositivos avaliados. Se aparecer uma falha grave de foco ou uma informação essencial deixar de ser perceptível, a expansão para e o componente afetado volta à configuração segura. O plano não presume que a configuração remota consiga reparar qualquer versão já instalada. Quando a correção exige binário novo, o fallback precisa existir previamente. Essa diferença entre alterar dados visuais e alterar comportamento é uma decisão de arquitetura, com consequência direta para a operação.

ANÁLISE E PROPOSTA INDEPENDENTE

Texto ampliado, movimento e custo de renderização



Um componente acessível precisa continuar operável quando o texto cresce. Limites fixos de altura, truncamento de valores e botões que exigem uma única linha podem impedir a conclusão de uma tarefa. A proposta é permitir crescimento vertical, preservar a associação entre rótulo e valor e revisar a ordem de leitura quando elementos passam para outra linha. Informação crítica não deveria depender de uma elipse que a pessoa não consegue expandir.

Efeitos visuais também consomem recursos. Desfoques, sombras e sobreposições podem ampliar o trabalho de composição em dispositivos com menos capacidade. O orçamento de desempenho deve ser verificado com a configuração final, incluindo animações e estados de carregamento. Uma tela que responde lentamente pode provocar toques repetidos, dúvida sobre a ação e contato ao atendimento, mesmo quando suas cores são corretas.

Eu colocaria esses cenários na matriz de aceite: tema claro e escuro, texto ampliado, largura reduzida, movimento reduzido e dispositivo representativo de menor capacidade. Não é necessário testar todas as combinações possíveis em toda mudança; é necessário conhecer os riscos dos componentes alterados e manter uma amostra relevante. O EM protegeria tempo para essa verificação, enquanto o Staff definiria contratos de layout e instrumentação que facilitem sua execução. O resultado desejado é uma aparência consistente que continue funcionando quando o contexto do usuário é menos conveniente que o ambiente de demonstração.

Flutter: accessibility

Flutter: testing

ANÁLISE E PROPOSTA INDEPENDENTE

ADR: papéis semânticos e fallback opaco



A decisão proposta é usar papéis semânticos versionados para componentes financeiros e prever superfície opaca quando a composição translúcida não satisfizer os critérios de leitura. O contexto é uma interface que precisa evoluir visualmente sem deslocar para cada squad a responsabilidade de redescobrir contraste, hierarquia e significado de estados. A consequência aceita é limitar alguns efeitos em áreas de informação essencial.

Uma alternativa seria definir cores localmente em cada tela. Ela acelera um ajuste isolado, mas torna difícil propagar correções e distinguir uma escolha de produto de uma exceção técnica. Outra alternativa seria bloquear toda mudança até a migração completa do legado. Essa estratégia oferece uniformidade aparente, porém pode atrasar melhorias úteis e esconder a ausência de um plano de compatibilidade. A adoção gradual exige mais disciplina, mas produz fronteiras explícitas.

A decisão seria reavaliada se o catálogo deixasse de representar os produtos ou se o fallback causasse barreiras próprias. O registro incluiria exemplos antes e depois, limitações verificadas, responsáveis e data de revisão. A escolha não afirma que a WCAG cubra sozinha toda a experiência móvel. Ela estabelece um conjunto de verificações dentro de uma abordagem mais ampla. Para o Staff, o teste da arquitetura é se um componente novo consegue seguir o contrato sem copiar ajustes frágeis. Para o EM, é se a adoção tem capacidade, dono e critérios de conclusão.

ANÁLISE E PROPOSTA INDEPENDENTE

O valor de uma interface que exige menos esforço



A hipótese de valor é que uma apresentação mais legível e consistente reduza esforço de interpretação e favoreça conclusão de tarefas. Ela precisa ser medida com baseline e pesquisa; não há neste capítulo um resultado financeiro atribuído ao Nubank. Tempo de tarefa, erros de entendimento, necessidade de ajuda e regressões recorrentes são medidas complementares. Nenhuma deve transformar inclusão em competição por um único número.

Como EM, eu reuniria Design System, squads de produto, acessibilidade e atendimento em torno de uma jornada concreta. O primeiro compromisso seria esclarecer os estados, observar dificuldades e definir o que deve melhorar. A agenda incluiria participantes com necessidades diversas e orçamento para correção. O acompanhamento separaria entrega de componentes, adoção pelos produtos e resultado observado nas tarefas, evitando declarar sucesso apenas porque a biblioteca ganhou novos tokens.

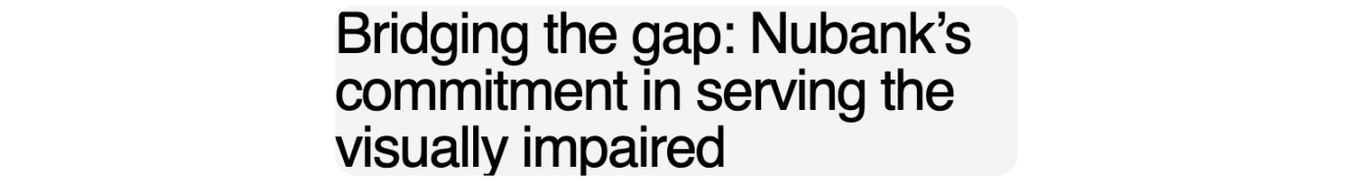
Como Staff, eu começaria pelos contratos entre domínio, apresentação e semântica. A contribuição seria tornar a forma correta de construir mais simples de repetir: pares de cores conhecidos, estados explícitos, exemplos testáveis e fallback documentado. A pergunta de entrevista seria: como mudar um token usado em dezenas de jornadas sem quebrar informação essencial? Uma boa resposta combina inventário, compatibilidade, teste de risco, release e reversão. Essa é a ligação entre qualidade visual e engenharia de sistemas: preservar confiança enquanto o produto continua mudando.

ANÁLISE E PROPOSTA INDEPENDENTE

NuBraille: uma iniciativa física com uma jornada digital

20





Captura da publicação oficial consultada em 23/10/2026.

O caso oficial de julho de 2023 apresenta o desenvolvimento do NuBraille com participação de pessoas cegas, avaliação de protótipos e colaboração com produção e atendimento. O cartão incorpora nome e últimos quatro dígitos em braille para apoiar identificação e autonomia. A publicação é evidência de uma iniciativa específica; ela não certifica a acessibilidade de toda a experiência nem garante disponibilidade universal do produto.

A principal oportunidade de engenharia está na continuidade. Identificar o cartão não conclui a jornada se as instruções forem difíceis de acessar, se o desbloqueio exigir uma etapa sem alternativa operacional ou se a ajuda perder o contexto. O sucesso depende de uma sequência que atravessa objeto físico, aplicativo, autenticação e atendimento. Cada fronteira pode introduzir uma barreira mesmo quando seus componentes parecem adequados isoladamente.

Este capítulo propõe um contrato de jornada assistiva, com estados claros e caminhos de recuperação que conservam as exigências de segurança. O laboratório sintético verifica transições e algumas propriedades de rótulos e ordem. Ele não emula leitor de tela, não implementa autenticação e não mede a experiência de uma pessoa cega. Minha contribuição como candidato seria conectar pesquisa, arquitetura e operação para que a autonomia observada em uma etapa continue nas seguintes. A pergunta deixa de ser se existe um recurso acessível e passa a ser se a pessoa consegue concluir o que veio fazer.

Bridging the gap: Nubank’s commitment in serving the visually impaired

ANÁLISE E PROPOSTA INDEPENDENTE

Estados de jornada antes de componentes de tela



A jornada proposta começa na identificação do cartão e segue por instruções, verificação de identidade, ativação e confirmação. Quando a câmera não está disponível, a pessoa recebe uma rota assistida com informação de continuidade. Essa rota não equivale a autorização financeira. A ativação continua condicionada à comprovação exigida, produzida por um mecanismo confiável a definir com Segurança e Operações.

O contrato precisa distinguir indisponibilidade técnica, necessidade de ajuda e identidade verificada. Se todos virarem um estado genérico de erro, o app não consegue explicar o próximo passo e o atendimento tende a reiniciar a conversa. Se a assistência for confundida com comprovação, a solução enfraquece a segurança. A modelagem explícita permite construir um caminho inclusivo preservando essa fronteira.

Cada estado teria uma mensagem compreensível, ações disponíveis, forma de recuperação e contexto mínimo para continuidade. O tempo de espera também merece descrição: a pessoa deve saber se a solicitação foi recebida e o que ainda falta, sem anúncios incessantes. O laboratório usa nomes internos simples para tornar essas transições verificáveis. No produto, a linguagem seria validada com participantes e profissionais de conteúdo. O trabalho do Staff é manter os contratos coerentes entre os canais. O do EM é garantir que os responsáveis por cada etapa tenham capacidade e um objetivo compartilhado de conclusão da jornada.

ARQUITETURA DE REFERÊNCIA

Autonomia através de fronteiras físicas e digitais



Proposta independente de jornada; setas não descrevem integrações existentes do Nubank.

A arquitetura de referência mostra uma jornada com múltiplos responsáveis. O artefato físico inicia a identificação, mas não autoriza a ativação. A interface apresenta instruções e envia uma intenção; a prova de identidade cruza uma fronteira de confiança separada. O caminho assistido precisa chegar ao mesmo requisito de autorização, com um procedimento acessível definido pelas áreas responsáveis. O registro de continuidade preserva o mínimo necessário para retomar o processo. Ele não deve conter cópias indiscriminadas de documentos, áudio ou dados do cartão. A pessoa recebe confirmação do estado efetivo, inclusive quando o atendimento continua de forma assíncrona. A distinção entre estado visível e efeito financeiro evita uma promessa prematura de conclusão. Uma falha de rede depois da ativação requer consulta do resultado, não uma repetição cega da operação. O laboratório testa apenas uma pequena máquina de estados local. O desenho completo exigiria contratos de idempotência, autorização, retenção e recuperação que não foram inferidos do sistema interno do Nubank.

CÓDIGO · CONTRATO PROPOSTO

A rota assistida preserva a exigência de prova

PYTHON / PROPOSTA AUTORAL

```python
def accept_proof(session, receipt, consumed):
    if session.stage not in ('VERIFYING', 'ASSISTED'):
        raise ValueError('Verification is not expected')
    if receipt.get('verified') is not True or receipt.get('card') != session.card:
        raise ValueError('Proof does not authorize this card')
    if not isinstance(receipt.get('id'), str) or not receipt['id'].strip():
        raise ValueError('Non-empty receipt identifier required')
    if receipt['id'] in consumed:
        raise ValueError('Receipt already consumed')
    consumed.add(receipt['id'])
    return replace(session, stage='VERIFIED')


def activate(session):
    if session.stage != 'VERIFIED':
        raise ValueError('Verified identity required')
    return replace(session, stage='ACTIVE')

session = advance(ready(), 'camera_unavailable')
assert session.stage == 'ASSISTED'
consumed = set()
verified = accept_proof(session, proof(), consumed)
assert verified.stage == 'VERIFIED'
assert activate(verified).stage == 'ACTIVE'
assert 'fixture-1' in consumed
```

Trecho de case_20.py executado localmente; modelo didático, não implementação de autenticação. Tipos, imports e validadores estão no arquivo completo.

O laboratório aceita uma comprovação somente nos estados de verificação esperados, vinculada ao cartão sintético e ainda não consumida. A ativação exige o estado VERIFIED. Uma falha de câmera permite chegar ao atendimento, porém não satisfaz essa condição. O recibo é uma fixture confiável de teste: o código não verifica assinatura, identidade ou autorização real. Também não fornece persistência transacional para concorrência. Essas limitações são deliberadas e explicitam o contrato que uma integração de produção precisaria cumprir. Os testes rejeitam comprovante inválido, cartão diferente e reapresentação do mesmo identificador. Tipos e domínios inválidos geram ValueError antes de qualquer resultado favorável.

Python 3.12: unittest

CÓDIGO · CONTRATO PROPOSTO

Semântica que explica estado e próximo passo

DART / PROPOSTA AUTORAL

```dart
import 'package:flutter/material.dart';

class AssistedRecovery extends StatelessWidget {
  const AssistedRecovery({super.key, required this.onHelp});
  final VoidCallback onHelp;

  @override
  Widget build(BuildContext context) {
    return FocusTraversalGroup(
      child: Column(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
          Semantics(
            header: true,
            child: const Text('Continue a ativação com ajuda'),
          ),
          const SizedBox(height: 12),
          Semantics(
            liveRegion: true,
            child: const Text(
              'A câmera está indisponível. Seu cartão ainda não '
              'foi ativado. Há outra forma de continuar.',
            ),
          ),
          const SizedBox(height: 12),
          FilledButton(
            onPressed: onHelp,
            child: const Text('Obter ajuda para continuar'),
          ),
        ],
      ),
    );
  }
}
```

Flutter/Dart ilustrativo e independente; verificar com TalkBack, VoiceOver e usuários antes de qualquer uso.

O exemplo propõe um título reconhecível, uma mensagem de estado e uma ação explícita de ajuda. O botão padrão fornece sua própria semântica; acrescentar um rótulo redundante ao redor poderia duplicar a leitura. A região dinâmica deve ser avaliada para não repetir anúncios durante reconstruções. FocusTraversalGroup organiza a navegação de foco, mas não comprova a ordem de exploração de todos os leitores de tela. O fragmento precisa ser integrado a um contêiner que comporte texto ampliado e rolagem. Ele não foi compilado nem validado em dispositivo. A disponibilidade da rota assistida é um contrato de produto, não uma integração pronta.

Flutter API: Semantics

Flutter API: FocusTraversalGroup

ANÁLISE E PROPOSTA INDEPENDENTE

A interface fala em uma ordem que precisa fazer sentido



A árvore semântica é uma representação do que a interface significa para tecnologias assistivas. Ela deve oferecer estrutura, nomes e estados coerentes com as ações possíveis. Um ícone de ajuda sem rótulo pode ser visualmente familiar e continuar indecifrável na navegação falada. Por outro lado, rotular cada fragmento decorativo cria ruído e dificulta encontrar a informação importante.

Eu começaria pela sequência da tarefa: reconhecer o contexto, compreender o estado, escolher uma ação e receber confirmação. Títulos orientam a pessoa, grupos relacionam informação e controles, e mensagens de erro explicam como avançar. O foco deve permanecer previsível quando um diálogo abre, fecha ou recebe resultado. Uma atualização não deve devolver a pessoa ao início sem necessidade, principalmente após preencher dados ou ouvir instruções longas.

No laboratório, o verificador identifica rótulos ausentes, identificadores repetidos, ordem duplicada e telas sem ação disponível. São regras sintáticas úteis para um contrato de exemplo. Elas não determinam se um texto é claro, se a ordem real do leitor de tela corresponde à lista ou se o gesto exigido é acessível. Essas perguntas dependem de execução no dispositivo e participação humana. A contribuição técnica está em combinar verificações rápidas com avaliações que alcançam o significado. Automatizar o que é verificável libera tempo para observar aquilo que uma lista de propriedades não consegue decidir.

Flutter API: Semantics
Flutter: accessibility

ANÁLISE E PROPOSTA INDEPENDENTE

Recuperar o acesso sem apagar o caminho percorrido



Interrupções fazem parte da jornada: câmera indisponível, sessão expirada, conexão instável, troca de dispositivo ou necessidade de atendimento. A recuperação precisa informar o que foi preservado e o que deve ser repetido. Exigir novamente uma etapa sensível pode ser necessário por segurança, mas a explicação deve ser compreensível e o caminho de execução precisa continuar disponível.

Minha proposta separa o estado de orientação do estado de autorização. Instruções lidas e preferências locais podem ajudar a retomar a tarefa, enquanto uma comprovação vencida exige nova validação. O app não deveria manter uma aparência de conclusão se o servidor ainda não confirmou o efeito. Quando há incerteza após o envio, consultar o estado com uma referência segura é preferível a iniciar uma nova ativação sem contexto.

O atendimento receberia apenas o necessário para entender a etapa atual, conforme permissões e retenção definidas. A pessoa não deveria ser obrigada a narrar repetidamente uma barreira já registrada, nem a compartilhar credenciais para conseguir ajuda. O laboratório demonstra que a alternativa assistida preserva a exigência de prova, mas não implementa identidade, sessão ou persistência. Em produção, essas fronteiras precisariam de revisão conjunta. A medida de qualidade seria a capacidade de retomar com segurança e autonomia, não apenas a existência de um botão que transfere a dificuldade para outro canal.

ANÁLISE E PROPOSTA INDEPENDENTE

Catorze testes protegem limites, não certificam a jornada





Os catorze testes executados cobrem o caminho direto, o caminho assistido, a exigência de comprovação, a correspondência com o cartão e a rejeição de reutilização do recibo. Também verificam transições inválidas, retomada da câmera e quatro classes de propriedades semânticas. Todos passaram na execução local registrada. O resultado mostra ASSISTED antes da fixture de comprovação e ACTIVE somente depois da aceitação dessa prova sintética. As regressões adicionais exigem verified exatamente verdadeiro e identificador não vazio; entradas rejeitadas não consomem recibos.

Outro resultado é deliberadamente negativo: uma tela com a mensagem Câmera indisponível, mas sem nenhuma ação, produz screen:no_action. Isso evidencia uma barreira estrutural que pode ser encontrada cedo. Ainda assim, a existência de um botão não demonstra que o atendimento esteja disponível, que o rótulo faça sentido ou que a pessoa consiga acioná-lo. A diferença entre propriedade de interface e resultado da jornada permanece explícita.

O laboratório usa um conjunto em memória para recibos consumidos. Ele não resolve concorrência entre processos, falhas durante a gravação ou autenticação de mensagens. Um sistema real exigiria persistência e operação atômica apropriadas. Os testes não usam leitores de tela, não simulam deficiência e não substituem pesquisa. Sua utilidade é tornar verificáveis alguns limites do contrato antes de investir na integração. Essa honestidade técnica permite discutir o que ainda falta sem desvalorizar o que já foi demonstrado de forma reproduzível.

Python 3.12: unittest

ANÁLISE E PROPOSTA INDEPENDENTE

Pesquisar autonomia com participantes, não com suposições



Uma avaliação de jornada assistiva precisa incluir pessoas que utilizam tecnologias e estratégias diferentes. Pessoas cegas e de baixa visão não formam um grupo uniforme: experiência com o aplicativo, familiaridade com braille, leitor de tela, dispositivo e condições de uso alteram a tarefa. O recrutamento deve procurar diversidade relevante para o produto e explicar com clareza o propósito da pesquisa.

Eu proporia sessões remuneradas com consentimento, dados sintéticos e possibilidade de interromper a atividade. O roteiro incluiria identificar o cartão, localizar instruções, iniciar ativação, lidar com uma falha e retomar o processo. O observador registraria barreiras e necessidades de assistência sem interpretar rapidez como única medida de competência. A pessoa poderia utilizar sua configuração habitual sempre que isso fosse viável e seguro para a sessão.

Os resultados deveriam separar dificuldade observada, hipótese de causa e mudança proposta. Se um participante se perde após um anúncio, a conclusão imediata é uma barreira naquele contexto; a causa pode envolver conteúdo, foco, repetição ou estrutura. A equipe reproduziria o cenário antes de escolher a correção. O EM garantiria tempo e recursos para incorporar o aprendizado. O Staff transformaria padrões recorrentes em contratos reutilizáveis. A pesquisa não é uma cerimônia final para aprovar o trabalho pronto: ela ajuda a decidir o que construir e quais suposições precisam ser abandonadas.

ANÁLISE E PROPOSTA INDEPENDENTE

Medir continuidade sem coletar uma identidade sensível



O objetivo de medição é entender onde a jornada deixa de funcionar e se a correção produz autonomia. Conclusão sem assistência, retomada após falha, recontato e barreiras críticas observadas podem compor a avaliação. Cada indicador precisa de definição: uma tarefa concluída após intervenção do pesquisador não pertence à mesma categoria de uma conclusão independente, embora ambas tragam aprendizado.

A coleta deve evitar transformar o uso de tecnologia assistiva em um perfil pessoal permanente. Quando possível, pesquisas consentidas e testes de qualidade respondem às perguntas com menos exposição. Eventos operacionais podem registrar etapa e categoria de falha sem conteúdo de documento, áudio, número completo de cartão ou credenciais. A análise deve respeitar finalidade, acesso e retenção, com responsáveis claros por qualquer dado adicional.

Também é necessário observar o atendimento. Uma rota disponível na interface pode fracassar se a fila não reconhece o contexto ou se o procedimento exige um canal inacessível. A medição proposta acompanha a continuidade até a resolução, incluindo espera e repetição de etapas. Pequenas amostras de pesquisa não sustentam projeções universais; revelam problemas e orientam novos testes. Uma barreira grave reproduzida pode justificar correção imediata mesmo sem estimativa de prevalência. O critério principal é oferecer um caminho funcional e seguro, apoiado por evidência proporcional ao alcance da mudança.

ANÁLISE E PROPOSTA INDEPENDENTE

Lançar uma jornada que inclui a operação



O lançamento de uma melhoria assistiva precisa abranger os canais envolvidos. Atualizar o componente no app sem preparar instruções, produção física e atendimento cria uma experiência inconsistente. A proposta é construir um checklist de dependências específico da jornada, com responsáveis e evidência de prontidão. Esse documento não substitui a experiência real; ele evita que uma etapa essencial seja esquecida entre áreas.

Antes de ampliar a mudança, eu verificaria a rota completa com participantes e profissionais que prestarão suporte. O piloto precisaria de capacidade operacional, mensagens claras de disponibilidade e um caminho seguro quando a alternativa não puder ser concluída naquele momento. O plano incluiria recuperação de sessão, dispositivos representativos e versões do aplicativo ainda suportadas. A reversão visual não pode eliminar uma rota de que pessoas já dependem sem oferecer continuidade.

O critério de parada seria uma barreira crítica reproduzida, uma confirmação incorreta de ativação ou quebra dos requisitos de autorização. O critério de expansão combinaria resolução dessas falhas, avaliação da experiência e capacidade de atendimento. O EM coordenaria a prontidão dos responsáveis; o Staff revisaria os contratos entre estado, confirmação e recuperação. O trabalho não termina quando o código entra em produção. A equipe precisa acompanhar se a pessoa atravessa as fronteiras com o mesmo nível de clareza que encontrou no primeiro passo.

ANÁLISE E PROPOSTA INDEPENDENTE

ADR: alternativa assistida com autorização equivalente





A decisão proposta é oferecer um caminho operacional acessível para continuar a jornada quando a captura por câmera não estiver disponível, preservando o requisito de comprovação de identidade. O contexto é uma tarefa que atravessa aplicativo e atendimento e não deve depender de uma única interação técnica sem recuperação. O mecanismo exato de comprovação seria definido e revisado pelas áreas competentes; este capítulo não inventa um procedimento de segurança do Nubank.

Uma alternativa seria tratar toda indisponibilidade como tentativa a repetir. Isso simplifica o fluxo, mas pode tornar a tarefa impossível para parte das pessoas. Outra seria permitir a ativação após qualquer contato de suporte. Essa opção confunde assistência com autorização e é rejeitada no modelo. A decisão aceita maior esforço de coordenação entre canais para conservar simultaneamente acesso e segurança.

As consequências incluem estado persistente, contexto mínimo compartilhável, treinamento e capacidade de atendimento. A implementação precisaria comprovar que uma prova pertence à pessoa e ao produto corretos, não pode ser reutilizada indevidamente e gera confirmação consistente. O laboratório verifica apenas algumas dessas condições em memória. A decisão seria revisitada se a alternativa não produzisse continuidade ou exigisse exposição desnecessária de dados. Um ADR útil registra esses limites e os responsáveis por resolvê-los, permitindo que a equipe discuta escolhas concretas em vez de usar acessibilidade ou segurança como palavras que encerram a conversa.

ANÁLISE E PROPOSTA INDEPENDENTE

Contratar para unir autonomia, engenharia e responsabilidade





O benefício esperado precisa ser dividido em dimensões compreensíveis. Para a pessoa, interessa concluir a tarefa com autonomia, recuperar-se de interrupções e entender o estado do produto. Para a operação, pode haver redução de contatos repetidos e de etapas refeitas. Qualquer estimativa financeira usaria contatos evitados e custo incremental validado, descontando a solução. Minutos recuperados pelo cliente permaneceriam uma medida separada, sem conversão arbitrária em dinheiro.

Como EM, eu começaria reunindo Produto, Engenharia, Pesquisa, Segurança e Operações em torno de uma jornada observável. Estabeleceria quem decide, quem executa e quem acompanha as barreiras. O plano reservaria capacidade para participantes, correções e treinamento. Uma entrega só seria considerada concluída quando o fluxo relevante pudesse ser demonstrado e as dependências operacionais estivessem prontas. Isso dá consequência prática ao compromisso com inclusão.

Como Staff, eu trabalharia na continuidade dos contratos: estado, semântica, autorização, idempotência e recuperação. Minha experiência em sistemas financeiros e antifraude serviria à discussão das fronteiras; eu aprenderia com especialistas e usuários onde minhas suposições não alcançam a experiência real. A pergunta de entrevista seria: como oferecer um caminho acessível sem relaxar a autorização? A resposta esperada conecta desenho de jornada, alternativas equivalentes, comprovação confiável, testes e observação. Esse é o valor de uma liderança que transforma uma intenção de produto em uma experiência sustentada por engenharia e operação.

LIDERANÇA E EXECUÇÃO

Primeiros 30 dias: aprender o sistema.

Eu começaria acompanhando uma jornada completa, incluindo atendimento e operação. O objetivo não seria produzir um mapa perfeito da organização, mas identificar onde o time encontra decisões recorrentes, quais riscos mais preocupam as pessoas e quais compromissos já existem com clientes e parceiros. A primeira entrega seria um problema delimitado, descrito na linguagem de quem o vive.

Nas conversas individuais, buscaria expectativas, pontos de atrito, autonomia percebida e oportunidades de desenvolvimento. Com Produto e Design, entenderia as métricas utilizadas e os motivos do roadmap atual. Com Segurança, Risco e Atendimento, investigaria como exceções chegam à engenharia e quanto contexto se perde nesse caminho. O diagnóstico deve ser compartilhado e corrigível.

Entrega	Como produzir	Sinal de utilidade
Mapa de jornada	Acompanhar app, serviço e suporte	Time reconhece fronteiras e lacunas.
Baseline	Definir eventos e denominadores	Número pode ser reproduzido.
Primeiro problema	Escolher com o time	Dono e critério de conclusão claros.

Critério de passagem

Só propor uma mudança estrutural quando a causa estiver suficientemente compreendida e as pessoas afetadas tiverem participado da formulação.

LIDERANÇA E EXECUÇÃO

Dias 31-60: tornar a hipótese testável.

Com contexto inicial, eu escolheria um piloto que produza evidência em poucas semanas e tenha reversão possível. Por exemplo, melhorar a consulta de uma operação já existente pode ser mais controlável que alterar todo o fluxo de pagamento. O escopo precisa incluir observabilidade, suporte e comunicação, porque uma mudança invisível à operação transfere trabalho para outro lugar.

A equipe definiria uma hipótese principal, duas ou três métricas de proteção e um limite de exposição. O plano deve especificar quais eventos encerram o piloto, quem pode pausá-lo e o que acontece com clientes que já entraram na jornada. Antes de lançar, eu pediria um ensaio de falha com o caminho de rollback, incluindo compatibilidade entre versões.

Decisão	Artefato	Responsável proposto
Podemos iniciar?	Contrato e baseline revisados	Tech lead + Produto + Risco.
Podemos expandir?	Amostra e guardrails	Dono da jornada.
Precisamos interromper?	Gatilho observável e runbook	Responsável operacional.

Como eu contribuiria

Como EM, removeria dependências e protegeria foco. Como Staff, aprofundaria contratos e falhas. Em ambos, buscaria que o time conseguisse explicar a decisão sem depender de mim.

LIDERANÇA E EXECUÇÃO

Dias 61-90: decidir o que merece escala.

Ao fim do piloto, o trabalho não termina com uma apresentação de resultados. É preciso decidir se a hipótese foi sustentada, se a operação consegue manter a solução e se o custo de expansão continua fazendo sentido. Uma melhoria localizada pode revelar problemas de dados, segmentos não atendidos ou dependências que impedem multiplicá-la sem novo investimento.

Eu apresentaria a decisão em três alternativas: ampliar, revisar ou encerrar. Ampliar exige plano de capacidade, ownership, suporte e compatibilidade. Revisar exige uma dúvida específica a resolver. Encerrar preserva os aprendizados e remove o custo permanente de uma hipótese que não se confirmou. Nenhuma dessas saídas deveria ser tratada como fracasso pessoal.

Alternativa	Evidência necessária	Próxima ação
Ampliar	Benefício consistente e riscos controlados	Expandir por coorte com observação.
Revisar	Sinal promissor, dúvida delimitada	Novo experimento com prazo.
Encerrar	Benefício insuficiente ou custo excessivo	Retirar experimento e registrar aprendizado.

Resultado de 90 dias

Um time com mais contexto compartilhado, uma melhoria verificável ou uma decisão informada de não expandir, e práticas que tornem o próximo ciclo mais eficiente.

LIDERANÇA E EXECUÇÃO

Capacidade é uma restrição de projeto.

Planejar como se todo o tempo do time estivesse disponível para funcionalidades torna qualquer incidente um atraso inesperado. Eu separaria capacidade de evolução, operação, dívida relevante e aprendizagem, usando dados do fluxo real para ajustar a distribuição. A proporção inicial é uma hipótese de gestão, não uma regra fixa imposta a todos os times.

A discussão de capacidade precisa considerar concentração de conhecimento. Se apenas uma pessoa consegue operar uma integração crítica, a equipe tem um risco que não aparece em pontos de história. Pareamento, documentação e rotação progressiva podem reduzir essa dependência, mas precisam de espaço no plano e acompanhamento da qualidade, não apenas de uma cerimônia.

Sinal	Interpretação a investigar	Ação possível
Muitas interrupções	Entrada de suporte sem triagem	Canal com contexto e severidade.
Trabalho envelhecido	Dependência ou escopo não resolvido	Reduzir lote e decidir bloqueio.
Uma pessoa em tudo	Conhecimento concentrado	Pareamento com entrega real.

Como EM, eu tornaria essas restrições visíveis ao negociar prazo. Como Staff, ajudaria a reduzir a complexidade que gera interrupções. Prometer velocidade sem tratar o sistema de trabalho tende a consumir as pessoas mais experientes e deixar o time menos capaz no ciclo seguinte.

LIDERANÇA E EXECUÇÃO

Delegação com fronteiras explícitas.

Delegar bem não significa entregar um problema e desaparecer. Significa combinar resultado esperado, espaço de decisão, riscos que precisam ser discutidos e pontos de acompanhamento. Uma pessoa em desenvolvimento pode liderar uma mudança importante quando o escopo e o suporte são adequados. O líder continua responsável por criar as condições para que ela aprenda sem assumir riscos invisíveis.

Eu usaria decisões reais como oportunidade de crescimento: alguém lidera o contrato, outra pessoa conduz o teste de falha, outra apresenta alternativas a stakeholders. O feedback deve apontar comportamento observável e efeito produzido. “Ser mais estratégico” é vago; “explicar a consequência de cada alternativa antes de escolher” permite prática e avaliação.

Delegação	Autonomia	Quando escalar
Componente local	Implementação dentro do contrato	Mudança de estado ou dado sensível.
Contrato entre times	Proposta e negociação	Incompatibilidade ou autoridade em disputa.
Rollout	Execução dentro dos limites	Guardrail ou amostra insuficiente.

Evidência de liderança

O sucesso aparece quando mais pessoas conseguem formular, explicar e executar decisões importantes com qualidade, e não quando todas as decisões passam pelo gestor.

LIDERANÇA E EXECUÇÃO

Um ADR resolve uma dúvida concreta.

Um registro de decisão arquitetural deve preservar o raciocínio que será necessário depois. Eu evitaria documentos que apenas descrevem a solução escolhida e omitem o problema. A primeira página precisa explicar contexto, restrições, alternativas e consequência de errar. O leitor deve conseguir discordar de uma premissa antes de discutir a ferramenta.

Parte	Pergunta	Exemplo no livro
Contexto	Qual decisão está pendente?	Como representar resultado desconhecido.
Alternativas	O que podemos fazer?	Consulta autoritativa, projeção ou espera.
Escolha	Por que esta opção agora?	Menor risco com capacidade atual.
Revisão	Quando reabrir a decisão?	Atraso, custo ou carga além do limite.

O ADR deve ter um dono e uma data de revisão condicionada a eventos, não apenas ao calendário. Se a decisão foi tomada sob pressão de prazo, isso deve aparecer. Se uma alternativa foi descartada por falta de capacidade operacional, a justificativa precisa sobreviver à saída das pessoas que participaram da conversa.

Como Staff, eu usaria o documento para criar alinhamento e expor trade-offs. Como EM, protegeria o tempo necessário para decidir e evitaria que o ADR virasse uma etapa burocrática obrigatória para mudanças triviais. O grau de formalidade deve acompanhar alcance, reversibilidade e risco.

LIDERANÇA E EXECUÇÃO

Incidentes: restaurar, compreender, melhorar.

Durante um incidente, a prioridade é reduzir impacto com coordenação clara. Eu separaria comando do incidente, investigação técnica e comunicação para que a mesma pessoa não precise executar tudo. A atualização deve dizer o que se sabe, o que ainda está incerto e qual ação está em curso, sem prometer um prazo que a evidência não sustenta.

Depois da estabilização, a revisão precisa olhar condições do sistema: sinais ausentes, decisões razoáveis com informação incompleta, dependências e mecanismos de recuperação. Buscar uma pessoa culpada produz silêncio; ignorar responsabilidade impede melhoria. O caminho útil é transformar o que aconteceu em mudanças verificáveis, com responsáveis, prioridade e teste de eficácia.

Fase	Pergunta principal	Saída
Resposta	Como reduzir dano agora?	Mitigação e comunicação coordenadas.
Reconciliação	Há efeitos pendentes ou divergentes?	Operações verificadas e casos tratados.
Aprendizagem	O que torna a repetição menos provável?	Ação testável com dono.

Teste de eficácia

Uma ação de post-mortem só está concluída quando conseguimos demonstrar que ela detecta, previne ou reduz o impacto do cenário que a motivou.

LIDERANÇA E EXECUÇÃO

Influência entre times sem centralizar tudo.

Problemas de jornada raramente cabem em um único repositório. A interface pode depender de dados de outro domínio, enquanto o suporte recebe o impacto de um contrato pouco claro. Eu começaria tornando visíveis os incentivos e limites de cada equipe. Uma solução que melhora a métrica de um time e transfere custo para outro precisa ser renegociada.

A influência técnica vem de reduzir o custo de decidir: um protótipo pequeno, dados reproduzíveis, alternativas comparáveis e uma proposta de migração que respeite a capacidade dos envolvidos. Em desacordo, eu buscaria distinguir fato, preferência e restrição. Nem todo conflito precisa de consenso; precisa de um decisor legítimo e de uma explicação compreensível.

Fronteira	Acordo necessário	Como verificar
Produtor / consumidor	Versão e compatibilidade	Fixtures e teste de contrato.
Engenharia / atendimento	Estado e próxima ação	Simulação de caso realista.
Produto / risco	Benefício e limite de exposição	Guardrails e revisão de coorte.

O resultado que procuro é uma organização que consiga mudar com menos surpresas. Isso depende de padrões úteis, ownership claro e canais para contestar decisões quando novas evidências aparecem. O Staff facilita esse sistema; o EM garante que as pessoas e a capacidade acompanhem a ambição.

ECONOMIA DA ENGENHARIA

Valor: começar pelo denominador.

Uma estimativa útil conecta volume elegível, taxa de ocorrência, mudança esperada e valor unitário. Se qualquer um desses elementos não é conhecido, o modelo deve expor a incerteza. Uma reclamação pública ajuda a formular uma hipótese, mas não informa automaticamente quantos clientes são afetados ou quanto a correção renderia.

Variável	Cenário sintético	Interpretação
Jornadas elegíveis / mês	1.000.000	Volume hipotético, não dado do Nubank.
Contatos por 1.000 jornadas	8 → 6	Redução assumida de 2 contatos.
Custo marginal por contato	R$ 6	Premissa que precisa de validação.
Capacidade liberada estimada	R$ 12.000 / mês	2.000 contatos × R$ 6.

A multiplicação não transforma capacidade liberada em economia de caixa. Se a estrutura de atendimento permanece igual, o benefício pode ser tempo disponível para casos mais complexos, menor fila ou melhor qualidade. Para afirmar economia financeira, seria necessário demonstrar que o custo efetivamente deixou de existir ou evitou expansão planejada.

Como usar na entrevista

Eu apresentaria faixas e sensibilidade, perguntaria quais premissas já são conhecidas e proporia a menor medição capaz de reduzir a incerteza que mais afeta a decisão.

ECONOMIA DA ENGENHARIA

Custo de oportunidade também é custo.

A escolha entre duas propostas não deve ser feita apenas pelo benefício bruto projetado. Um trabalho pode exigir migração ampla, coordenação com parceiros e manutenção permanente. Outro pode produzir menos benefício isolado e, ainda assim, ser a melhor primeira entrega por reduzir risco, validar uma premissa ou liberar capacidade para o próximo passo.

Alternativa sintética	Benefício esperado	Compromisso operacional
Nova projeção global	Leituras mais rápidas	Pipeline, reconciliação e suporte contínuos.
Correção de contrato	Menos ambiguidade	Mudança coordenada e testes.
Melhoria de conteúdo	Orientação mais clara	Governança editorial e medição.

Eu compararia alternativas pela mesma janela temporal e pelo mesmo resultado de jornada. Também separaria custos reversíveis de compromissos difíceis de desfazer. Introduzir mais um serviço cria responsabilidade por disponibilidade, atualização, segurança, observabilidade e transferência de conhecimento. A arquitetura precisa carregar esse custo na discussão inicial.

Como EM, faria o trade-off com Produto e stakeholders usando capacidade real. Como Staff, buscaria reduzir o custo técnico da alternativa escolhida. A melhor solução pode ser um componente novo, uma alteração pequena em um contrato existente ou a decisão de não construir até que a evidência seja suficiente.

ECONOMIA DA ENGENHARIA

Desempenho: medir a cauda e a tarefa.

Uma média de latência pode esconder a experiência das pessoas mais afetadas. Um endpoint rápido também pode não melhorar a jornada se o app repete chamadas, espera uma dependência lenta ou apresenta estado confuso. Eu começaria por uma linha do tempo completa e mediria em qual etapa o usuário percebe espera, falha ou necessidade de repetir trabalho.

Medida	Pergunta que responde	Limite
P50 / P95 / P99	Como se distribui a espera?	Não mede conclusão sozinho.
Taxa de confirmação	A tarefa terminou?	Depende de eventos completos.
Contatos posteriores	A pessoa continuou insegura?	Há atraso e outras causas.
Custo por jornada	Qual recurso foi consumido?	Precisa de atribuição consistente.

A comparação deve usar coortes semelhantes, volume suficiente e uma janela que inclua picos relevantes. Otimização de cache pode reduzir latência e piorar atualização; paralelismo pode melhorar tempo e aumentar saturação; retries podem elevar conclusão e amplificar carga. Cada ganho precisa ser lido junto de suas métricas de proteção.

Critério de decisão

Eu expandiria uma otimização quando o benefício na tarefa estiver sustentado e a correção, a atualização dos dados e o custo de operação permanecerem dentro dos limites combinados.

ECONOMIA DA ENGENHARIA

Risco: medir os dois tipos de erro.

Uma decisão de prevenção a fraudes possui efeitos assimétricos. Permitir uma ação indevida pode gerar perda; bloquear uma ação legítima pode impedir uma necessidade real e criar custo de recuperação. A discussão não pode tratar toda redução de aprovação como melhoria de segurança, nem toda redução de fricção como melhoria de produto.

Dimensão	Medida proposta	Cuidados
Proteção	Perda em população elegível	Rótulos atrasados e subnotificação.
Fricção	Legítimos bloqueados / legítimos avaliados	Base de verdade e revisão.
Recuperação	Tempo até solução segura	Não excluir abandonos silenciosos.
Equidade operacional	Desempenho por coorte relevante	Amostra e acesso aos dados.

O modelo precisa de uma via de contestação e de revisão humana proporcional ao risco. O resultado da revisão alimenta aprendizagem, mas também pode conter vieses de seleção: quem desiste não aparece como caso resolvido. Eu procuraria medir essa ausência e discutir critérios de decisão com especialistas de risco e produto.

Minha experiência em prevenção a fraudes torna esse equilíbrio central à candidatura. Nos casos de ML e recuperação de acesso, o livro separa sinal, política, efeito e revisão. O objetivo é proteger a pessoa e a instituição com decisões que possam ser explicadas e corrigidas.

GUIA DO LABORATÓRIO

Como executar os laboratórios.

O pacote complementar contém um laboratório independente por caso. Eles usam a biblioteca padrão do Python e dados sintéticos, sem credenciais, chamadas ao Nubank ou acesso a contas. O objetivo é tornar regras e falhas examináveis em uma máquina local. Cada arquivo inclui seus testes e pode ser executado isoladamente.

BASH / PROPOSTA AUTORAL

```bash
# Dentro da pasta laboratorios do pacote extraído:
python3 case_01.py
python3 case_17.py

# Todos os casos em uma única execução:
python3 -m unittest discover -p "case_*.py" -v

# Núcleo Clojure, com Clojure 1.12.0 instalado:
clojure -M decision_core.clj
```

Comandos locais; não acessam serviços bancários. Consulte LEIA-ME.html para versões e escopo.

O laboratório Clojure foi executado com Java 17 e Clojure 1.12.0. O pacote entrega o código autoral, o comando de reprodução e o registro da execução. Exemplos de Flutter/Dart, SQL, YAML e integrações cloud são identificados como referências quando não foram executados. O livro não apresenta compilação ausente como validação.

O que testar depois

Integração com adapters reais, concorrência entre processos, persistência durável, carga, falhas de rede, autorização e operação pertencem a uma próxima etapa em ambiente apropriado.

GUIA DO LABORATÓRIO

Da prova local ao piloto integrado.

Um teste unitário aprovado é evidência de uma propriedade no contexto em que foi executado. A passagem para um piloto exige novas perguntas. O adapter mantém a semântica do núcleo? A persistência faz a comparação de versão e a escrita atomicamente? Um retry chega com a mesma identidade? O consumidor continua correto quando o processo reinicia?

Nível	Cenário de validação	Evidência a guardar
Contrato	Consumidor antigo recebe evento novo	Fixture, versão e resultado.
Integração	Falha depois do commit e antes do ACK	Estado final e trilha de recuperação.
Carga	Pico com dependência lenta	Latência, saturação e perdas.
Operação	Rollback com clientes em andamento	Procedimento e reconciliação.

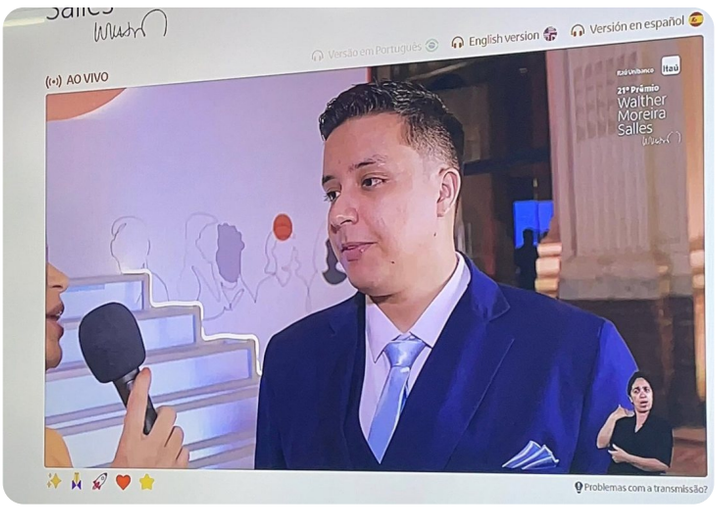
Eu manteria uma matriz que conecta risco, teste e decisão de lançamento. Não é necessário testar todas as combinações imagináveis antes de aprender algo; é necessário cobrir as propriedades que impedem dano no escopo de exposição escolhido. O piloto deve ser menor quando a evidência é mais fraca ou a reversão é mais difícil.

Critério de integridade

Nenhum resultado local deste livro é extrapolado para disponibilidade, segurança ou desempenho real do Nubank. As propostas ganham força quando os limites são conhecidos e podem orientar a próxima validação.

ENCERRAMENTO

Uma proposta de contribuição, em primeira pessoa.



Registro público do candidato em entrevista durante evento de reconhecimento, publicado em seu portfólio.

Quero trabalhar em problemas que exijam profundidade e colaboração. Gosto de descer até o contrato e o código, mas também de construir contexto para que outras pessoas possam decidir. Minha trajetória em fraude, crédito e produto técnico me ensinou que qualidade aparece tanto na exceção quanto no caminho feliz.

No Nubank, quero contribuir com equipes que transformam complexidade financeira em experiências compreensíveis. Como EM, meu foco será desenvolver pessoas e capacidade de entrega. Como Staff, será melhorar decisões técnicas que atravessam domínios. Em ambos, quero assumir responsabilidade, aprender com o contexto existente e tornar o resultado verificável.

Vamos discutir o problema real.

Wesley Vieira · LinkedIn: /in/techvieira · Portfólio: wesleyvieira99.github.io/WVPortfolio · GitHub: wesleyvieira99

BIBLIOGRAFIA RASTREÁVEL

Fontes dos casos 01-05

01 · Pix falha e a ajuda falha junto

Relato público individual em comunidade hospedada pelo Nubank. · 15/07/2026 NuCommunity - Pix fora do AR novamente?

O registro comprova a publicação da reclamação; não confirma indisponibilidade geral, duração, perda financeira ou causa técnica. O problema pode ter sido resolvido desde então.

02 · Recuperar o acesso quando a câmera frontal não funciona

Relato público individual; trecho disponível da publicação. · s/d NuCommunity - Sem acesso a minha conta, pede reconhecimento facial

O trecho não permite verificar as alternativas oferecidas pelo atendimento, o desfecho ou o comportamento atual. Não demonstra falha do mecanismo biométrico.

03 · Uma notificação deve ajudar a decidir

Relato e sugestões de um participante, acompanhados de capturas publicadas por ele. · s/d NuCommunity - Notificações Push do App Nubank esquece do valor (sugestões)

As imagens documentam a alegação do autor, sem provar a configuração de privacidade, versão do app ou situação atual. As comparações com concorrentes não foram auditadas.

04 · Open Finance precisa mostrar o que ainda não está completo

Relato público individual, com distinção entre dados visíveis e ausentes. · 03/10/2026 NuCommunity - Open Finance: Cofrinhos do Mercado Pago não aparecem no Nubank

O próprio autor não identifica em qual instituição está a limitação. A fonte não comprova obrigação de cobertura, erro de integração ou perda de dinheiro.

05 · Próximo vencimento, sem procurar entre meses

Sugestão pública detalhada com capturas anexadas pelo participante. · s/d NuCommunity - Assistente de pagamentos do Nubank precisa de melhorias

O registro descreve a experiência do autor na época. Não confirma a cobertura dos recursos atuais nem demonstra que lembretes causaram atrasos.

01 · NuCommunity - Pix fora do AR novamente?
02 · NuCommunity - Sem acesso a minha conta, pede reconhecimento facial
03 · NuCommunity - Notificações Push do App Nubank esquece do valor (sugestões)
04 · NuCommunity - Open Finance: Cofrinhos do Mercado Pago não aparecem no Nubank
05 · NuCommunity - Assistente de pagamentos do Nubank precisa de melhorias

BIBLIOGRAFIA RASTREÁVEL

Fontes dos casos 06-10

06 · Modo Rua: explicar a permissão sem enfraquecer a proteção

Relato público e sugestão com duas capturas anexadas pelo autor. · s/d NuCommunity - Modo Rua precisa de um pequeno ajuste né

A interpretação do autor sobre GPS e bateria não foi validada. Não há diagnóstico de plataforma, confirmação de defeito ou evidência de vulnerabilidade.

07 · Um centavo pode quebrar uma integração inteira

Relato público individual com exemplo textual do valor esperado e do recebido. · s/d NuCommunity - Extrato OFX com erro

Não foi obtido o arquivo original nem reproduzido o erro. A fonte não permite concluir se o defeito estava na exportação, no importador ou em uma versão específica.

08 · A experiência digital também termina na entrega do cartão

Resenha pública em loja de aplicativos, acompanhada de resposta do desenvolvedor. · 08/04/2026 Google Play - resenha de Murilo Meyrelles sobre entrega de cartão

A fonte não informa o desfecho, as tentativas da transportadora ou a causa. Não permite estimar frequência nem responsabilidade por cada etapa.

09 · Evoluir a fatura sem perder a tarefa principal

Resenha pública individual em loja de aplicativos. · 24/02/2026 Google Play - resenha de Fernando Cordeiro sobre visualização da fatura

Não há reprodução independente, versão identificada ou confirmação de comportamento atual. Uma opinião de layout não representa a preferência da base.

10 · Comprovante no contexto da compra, com significado correto

Resenha pública individual em loja de aplicativos. · 12/02/2025 Google Play - resenha de Neidslem Fernandes da Costa sobre comprovantes

Não há confirmação do tipo exato de documento solicitado nem da cobertura atual. Registro de compra, comprovante de pagamento e recibo do lojista têm finalidades diferentes.

06 · NuCommunity - Modo Rua precisa de um pequeno ajuste né
07 · NuCommunity - Extrato OFX com erro
08 · Google Play - resenha de Murilo Meyrelles sobre entrega de cartão

BIBLIOGRAFIA RASTREÁVEL

Fontes dos casos 11-15

11 · Pagamento rápido, sem trocar correção por velocidade

Relato técnico oficial com arquitetura e resultado publicado. · 2026-01-12 How we reduced critical path latency by 76% at Nubank

Resultado daquele fluxo e período; não representa a latência de todo o app nem comprova desempenho atual.

12 · Capacidade cloud como compromisso de produto

Publicação oficial de engenharia sobre capacidade e isolamento. · 2025-04-09 Managing Cloud Limits

Escala histórica declarada em abril de 2025; não é contagem atual. A publicação não fornece taxa de incidentes evitados.

13 · Ownership que sobrevive à reorganização

Relato oficial de operação, governança e automação. · 2026-07-01 How Nubank distributes infrastructure ownership to operate more than 21 thousand databases with a lean team

Cinco engenheiros não representam todo o esforço organizacional. Remoção de dados depende de retenção, dependências e contexto de negócio.

14 · BDC: velocidade com contrato de experiência

Descrição oficial da plataforma e adoção na data da publicação. · 2023-01-20 Backend Driven Content: why we developed a Server Driven UI framework at Nubank

Percentuais históricos; BDC não permite inventar componentes ausentes no binário nem elimina toda dependência de atualização do app.

15 · Release mobile guiado pela jornada real

Resumo oficial de meetup de Plataforma e desenvolvimento mobile. · 2025-04-16 Platform engineering, Observability, and scaling Mobile Development

É uma visão pública de práticas, sem acesso à implementação nem cobertura atual de cada jornada.

11 · How we reduced critical path latency by 76% at Nubank
12 · Managing Cloud Limits
13 · How Nubank distributes infrastructure ownership to operate more than 21 thousand databases with a lean team
14 · Backend Driven Content: why we developed a Server Driven UI framework at Nubank
15 · Platform engineering, Observability, and scaling Mobile Development

BIBLIOGRAFIA RASTREÁVEL

Fontes dos casos 16-20

16 · Precog: chegar ao especialista certo

Relato técnico oficial de modelo, aplicação e resultado. · 2023-06-21 Presenting Precog, Nubank's Real Time Event AI

O aumento é relativo ao volume de roteamentos corretos naquele comparativo; não significa 50 pontos percentuais, resolução garantida ou efeito atual.

17 · Antifraude que responde dentro da jornada

Prática de engenharia publicada pelo próprio Nubank. · 2025-04-07 Practices to scale Machine Learning operations

Não revela políticas de decisão, thresholds, perdas ou falsos positivos. Não autoriza inferir vulnerabilidade explorável ou eficiência antifraude atual.

18 · A mesma feature no treino e na decisão

Guia técnico oficial baseado na experiência da equipe. · 2023-06-27 Dealing with Train-serve Skew in Real-time ML Models: A Short Guide

Exemplos ilustrativos não são incidentes atribuíveis a um produto ou cliente específico. Não há magnitude de perda publicada.

19 · Dark Mode: consistência que inclui legibilidade

História oficial de desenvolvimento do app e do Design System. · 2023-09-26 The birth of the Dark Mode: a journey into Nubank's app evolution

Modo escuro não resolve acessibilidade sozinho. O relato não comprova falha de contraste na versão atual do aplicativo.

20 · NuBraille: autonomia de ponta a ponta

Caso oficial de produto inclusivo, com processo e participantes identificados. · 2023-07-18 Bridging the gap: Nubank's commitment in serving the visually impaired

É evidência de uma iniciativa específica, não certificação de acessibilidade de toda a experiência nem garantia de disponibilidade universal.

16 · Presenting Precog, Nubank's Real Time Event AI
17 · Practices to scale Machine Learning operations
18 · Dealing with Train-serve Skew in Real-time ML Models: A Short Guide
19 · The birth of the Dark Mode: a journey into Nubank's app evolution
20 · Bridging the gap: Nubank's commitment in serving the visually impaired

PROVENIÊNCIA VISUAL

Créditos de imagens e marcas

Logos e fotografias foram obtidos das páginas oficiais dos respectivos projetos, de páginas do Nubank ou do portfólio público do candidato. As marcas identificam as tecnologias discutidas. O uso editorial não indica vínculo ou endosso. O pacote inclui um manifesto com URL do arquivo, página de origem e hash do ativo. As capturas preservam o conteúdo original das fontes.

Ativo	Origem	Uso
nubank	www.datomic.com	Identificação da tecnologia
clojure	clojure.org	Identificação da tecnologia
datomic	www.datomic.com	Identificação da tecnologia
kafka	kafka.apache.org	Identificação da tecnologia
prometheus	prometheus.io	Identificação da tecnologia
flutter	flutter.dev	Identificação da tecnologia
dart	dart.dev	Identificação da tecnologia
python	www.python.org	Identificação da tecnologia
kubernetes	github.com	Identificação da tecnologia

nubank · origem
clojure · origem
kafka · origem
prometheus · origem
flutter · origem
dart · origem
python · origem
kubernetes · origem

PROVENIÊNCIA VISUAL

Créditos de imagens e marcas · continuação

Logos e fotografias foram obtidos das páginas oficiais dos respectivos projetos, de páginas do Nubank ou do portfólio público do candidato. As marcas identificam as tecnologias discutidas. O uso editorial não indica vínculo ou endosso. O pacote inclui um manifesto com URL do arquivo, página de origem e hash do ativo. As capturas preservam o conteúdo original das fontes.

Ativo	Origem	Uso
opentelemetry	opentelemetry.io	Identificação da tecnologia
postgresql	www.postgresql.org	Identificação da tecnologia
grafana	grafana.com	Identificação da tecnologia
java	dev.java	Identificação da tecnologia
portrait	wesleyvieira99.github.io	Fotografia editorial / carreira
award-0	wesleyvieira99.github.io	Fotografia editorial / carreira
award-1	wesleyvieira99.github.io	Fotografia editorial / carreira
award-2	wesleyvieira99.github.io	Fotografia editorial / carreira
aws	aws.amazon.com	Identificação da tecnologia

opentelemetry · origem
postgresql · origem
grafana · origem
java · origem
portrait · origem
aws · origem

PROVENIÊNCIA VISUAL

Créditos de imagens e marcas · continuação

Logos e fotografias foram obtidos das páginas oficiais dos respectivos projetos, de páginas do Nubank ou do portfólio público do candidato. As marcas identificam as tecnologias discutidas. O uso editorial não indica vínculo ou endosso. O pacote inclui um manifesto com URL do arquivo, página de origem e hash do ativo. As capturas preservam o conteúdo original das fontes.

Ativo	Origem	Uso
redis	redis.io	Identificação da tecnologia
typescript	www.typescriptlang.org	Identificação da tecnologia
nu-app-life	international.nubank.com.br	Fotografia editorial / carreira
nu-engineer	international.nubank.com.br	Fotografia editorial / carreira
nu-meeting	international.nubank.com.br	Fotografia editorial / carreira
nu-office	international.nubank.com.br	Fotografia editorial / carreira
nu-collaboration	international.nubank.com.br	Fotografia editorial / carreira
terraform	github.com	Identificação da tecnologia

redis · origem
typescript · origem
nu-app-life · origem
terraform · origem